GUIA VISUAL 001

# O QUE É LINUX

*Servidor desconhecido, 3h20 da manhã:* `uname -a` *e* `/etc/os-release` *respondem perguntas diferentes.*

**001**
GUIAS LINUX

## LINUX É O KERNEL — O RESTO É INTEGRAÇÃO

ESPAÇO DE USUÁRIO

**APLICAÇÕES**
navegador · nmap · Python · o seu script

**UTILITÁRIOS E SHELL**
bash · coreutils · systemd · apt

**BIBLIOTECAS**
glibc · OpenSSL · libcurl

chamadas de sistema

ESPAÇO DE KERNEL

**KERNEL LINUX**
processos · memória · arquivos · rede · drivers

**HARDWARE**
CPU · RAM · disco · placa de rede

### O KERNEL DEFINE

Drivers e suporte a hardware, escalonamento de CPU, memória, sockets. **Nenhum programa toca o hardware** — todos pedem ao kernel. Monolítico e **modular**: `lsmod` mostra o que foi carregado sem reiniciar.

### A DISTRIBUIÇÃO DEFINE

Biblioteca C, init, shell, **gerenciador de pacotes**, repositório, instalador e a política de atualização. É onde nasce o `apt` × `dnf` — e a janela de suporte.

### OS NÚMEROS NÃO CONVERSAM

Kernel `6.12.25` e distro `2025.2` não têm relação. Ler os dois é o **primeiro diagnóstico** de qualquer máquina desconhecida.

## AS DUAS PERGUNTAS DO PRIMEIRO MINUTO

kali@kali: ~

```
$ uname -a                  # qual KERNEL está rodando
Linux kali 6.12.25-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.25-1kali1 x86_64 GNU/Linux
$ cat /etc/os-release       # qual DISTRIBUIÇÃO foi instalada
PRETTY_NAME="Kali GNU/Linux Rolling"   ID=kali   ID_LIKE=debian
$ dpkg -l | grep -c '^ii'   # o tamanho do trabalho de integração
3184
```

## O TRONCO DEFINE OS COMANDOS

| FAMÍLIA | PACOTE · GERENTE | DERIVADAS | ONDE APARECE |
|---|---|---|---|
| **Debian** (1993) | `.deb` · `apt` | Ubuntu, **Kali**, Mint, Raspberry Pi OS | Servidor pequeno/médio — a família deste guia |
| **Red Hat** (1994) | `.rpm` · `dnf` | Fedora, RHEL, Rocky, Alma | Datacenter corporativo (firewalld, SELinux) |
| **Arch** (2002) | `.pkg` · `pacman` | Manjaro | *Rolling* minimalista, montado à mão |
| **Alpine** | `.apk` · `apk` | — | Contêiner: usa `musl` no lugar da `glibc` |

## O QUE SUSTENTA TUDO: AS 4 LIBERDADES

**0** Executar para **qualquer** fim

**1** Estudar e adaptar (exige o fonte)

**2** Redistribuir cópias

**3** Distribuir versões modificadas

## RESUMO RÁPIDO

- ✓ Linux, no sentido estrito, é **só o kernel**; a distro é o kernel + integração.
- ✓ `uname -a` = kernel · `/etc/os-release` = distribuição. Números independentes.
- ✓ Conceitos (processo, permissão, socket) valem em toda parte; **comandos, não**.
- ✓ Kernel sob GPLv2: quem embarca Linux num roteador devolve o fonte modificado.

## NO KALI

Debian **rolling** da OffSec, saído do ramo `testing`. O `2025.2` é marco de imagem, não versão congelada — o repositório é `kali-rolling` e `apt full-upgrade` é rotina, não evento. Kernel com sufixo `-kali1` (patches de injeção e modo monitor), usuário padrão `kali` desde 2020.1, ferramentas em metapacotes. **Não é distro de servidor:** é a máquina de quem audita, não a auditada.

# POR QUE KALI LINUX

*Instalou Kali no notebook de trabalho e o Wi-Fi morreu no `full-upgrade`? Não é defeito — é uso fora de propósito.*

**002**
GUIAS LINUX

## VINTE ANOS ATÉ O `KALI-ROLLING`

| 2004 | 2006 | 2013 | 2015 | 2020.1 | hoje |
|---|---|---|---|---|---|
| **WHOPPIX** | **BACKTRACK** | **KALI 1.0** | **KALI 2.0** | **FIM DO ROOT** | **KALI-ROLLING** |
| Live CD sobre Knoppix | Competente e bagunçado: tudo em `/pentest` | Recomeça sobre Debian: tudo vira `.deb` | Vira *rolling*, colado no `testing` | Usuário `kali` + `sudo` | Xfce, metapacotes, ~600 ferramentas |

## AS DECISÕES DE PROJETO QUE EXPLICAM O COMPORTAMENTO

### REPOSITÓRIO ÚNICO E ASSINADO

Poucos espelhos, chave GPG do projeto. **Nunca adicione repositório de outra distro** — quebra dependência e destrói a procedência.

### CURADAS, NÃO EXAUSTIVAS

~600 ferramentas que passam por triagem (é mantida? licença? duplica?). O oposto do BlackArch, que empacota milhares.

### KERNEL COM PATCHES

O sufixo `-kali1` traz **injeção de pacotes e modo monitor** em Wi-Fi. É o que você não obtém instalando as ferramentas à mão.

### METAPACOTES

`kali-linux-core` → `default` → `large` → `everything`. Só listas de dependências: VM de 2 GB ou estação de 30 GB.

**DISTRIBUIÇÃO DE SEGURANÇA ≠ DISTRIBUIÇÃO SEGURA**

Uma instalação padrão traz centenas de binários ofensivos e defaults voltados à conveniência do operador — a superfície de ataque é grande **por construção**. Se comprometem sua máquina Kali, o atacante acha a caixa de ferramentas montada e aquecida. Por isso: VM com snapshot, nada exposto à internet, e um cliente nunca encosta no outro.

**O que legitima o uso é a autorização, não a técnica.** Sem escopo por escrito e autorização de quem pode dá-la, a mesma ação é crime no Brasil (Lei 12.737/2012).

## A MÁQUINA É PARA QUÊ?

### USE KALI

- ✓ Teste de invasão **autorizado** e resposta a incidente.
- ✓ Forense digital e engenharia reversa.
- ✓ Auditoria de rede sem fio (é onde o kernel dele brilha).
- ✓ Laboratório: CTF, máquinas vulneráveis, aula prática.
- ✓ Sempre que possível: **em VM com snapshot**.

### USE OUTRA

- ✓ Servidor → **Debian estável, Ubuntu LTS, Rocky**. Rolling não tem janela de suporte.
- ✓ Desktop de trabalho → Ubuntu, Fedora, Mint.
- ✓ Só aprender Linux → Debian ou Ubuntu em VM.
- ✓ Se você não sabe o que a ferramenta faz — ou se a empresa proíbe.
- ✓ Vizinhas úteis: Parrot, BlackArch, Tails, REMnux.

### RESUMO RÁPIDO

- ✓ Debian *rolling* da OffSec para pentest, forense e auditoria — herdeiro do BackTrack.
- ✓ 2013 reconstruiu sobre Debian para ganhar **empacotamento e procedência**.
- ✓ Aprender Linux **no** Kali é ótimo; produção nele, não.
- ✓ Sem contrato e escopo por escrito, não existe teste — existe crime.

### NO KALI

Atualize sempre com `apt update && apt full-upgrade` — o `apt upgrade` se recusa a remover pacotes e deixa a máquina meio atualizada, o pior estado possível num rolling. Se o `apt update` reclamar de `EXPKEYSIG`, reinstale o `kali-archive-keyring`; **jamais** desative a verificação. Serviços (SSH, web) vêm `disabled` no boot de propósito. E `kali-tweaks` ajusta ramo, shell e o modo `kali-undercover`, que só troca o tema — engana olhar casual, não perícia.

GUIA VISUAL 003

# INSTALANDO O KALI

*VM, bare metal, live USB, WSL ou contêiner — os cinco caminhos servem a problemas diferentes.*

003
GUIAS LINUX

## ANTES DE TUDO: A IMAGEM É MESMO DO KALI?

**1 — IMPORTE A CHAVE**

`wget -qO- archive.kali.org/archive-keyring.gpg | gpg --import`

→

**2 — BAIXE AS SOMAS**

`SHA256SUMS` e `SHA256SUMS.gpg`, ao lado da imagem.

→

**3 — CONFIRA A ASSINATURA**

`gpg --verify SHA256SUMS.gpg SHA256SUMS` → tem de dizer **Good signature**.

→

**4 — CONFIRA A SOMA**

`sha256sum -c SHA256SUMS --ignore-missing` → **OK**. Só então grave.

**Conferir só a soma não prova nada** — se o espelho foi adulterado, a soma veio adulterada junto. A assinatura GPG é o elo que fecha a cadeia. O aviso `key is not certified` é normal; `BAD signature` não é: descarte e baixe de novo.

## OS CINCO CAMINHOS

| MÉTODO | ENTREGA | COBRA | USE QUANDO |
|---|---|---|---|
| **Máquina virtual** ← o padrão | Isolamento, **snapshot**, uma VM por cliente | Wi-Fi só por USB externo repassado; perde desempenho | Estudo e quase todo trabalho profissional. 4 GB RAM · 2 vCPU · **60–80 GB** |
| **Bare metal** | Desempenho total, GPU para `hashcat`, Wi-Fi interno | Sem snapshot; particionar apaga o resto | Máquina dedicada à função — nunca o notebook de uso geral |
| **Live USB** + persistência | Não escreve no disco da máquina; persistência com LUKS | Lento; gravar a mídia é a operação mais perigosa | Forense, recuperação, trabalho em máquina de terceiros |
| **WSL2** (Windows) | Rápido, integra com o Windows; `kali-win-kex` dá tela | **Kernel é o do WSL**: sem patches, USB irregular, sem modo monitor | Linha de comando, scripts, análise de arquivo. Bom complemento, mau substituto |
| **Contêiner** | Sobe em segundos, descartável, ideal em pipeline | Usa o kernel do hospedeiro; péssimo como ambiente interativo | Rodar *uma* ferramenta pontual sem instalar nada |

## NA VM, O MODO DE REDE É DECISÃO DE SEGURANÇA

### NAT · O PADRÃO

`VM` → `host` → `LAN`

Sai para a internet escondida atrás do hospedeiro; **ninguém de fora inicia conexão**. Serve para atualizar e navegar.

### BRIDGE · CUIDADO

`VM` → `LAN real`

IP próprio no roteador, visível como outra máquina. Necessário quando a VM precisa ser alcançada — e é exatamente o que a política de TI proíbe numa rede corporativa.

### HOST-ONLY · O LABORATÓRIO

`VM` ⇔ `alvo` ⇔ `sem saída`

Rede fechada entre as VMs. **Máquina-alvo vulnerável nunca em NAT nem em bridge** — senão seu laboratório vira porta de entrada.

**Gravar no dispositivo errado destrói o disco.** O `dd` escreve byte a byte, sem confirmação e sem desfazer: `of=/dev/sda` no lugar de `/dev/sdb` apaga o sistema. Identifique com `lsblk -o NAME,SIZE,TYPE,MOUNTPOINT,MODEL` e confira o tamanho. **Ensaie sem risco** escrevendo num arquivo-imagem: `truncate -s 4G /tmp/ensaio.img` e `dd if=... iso of=/tmp/ensaio.img bs=4M status=progress`. Quem está começando deve preferir o Etcher, que só lista removíveis. O mesmo vale para o instalador: **"usar o disco inteiro" destrói todas as partições**, o outro sistema incluído.

## RESUMO RÁPIDO

- ✓ Verificar tem **duas** etapas: soma prova integridade, assinatura GPG prova procedência.
- ✓ VM é o padrão: isolamento, snapshot e um ambiente por cliente.
- ✓ Modo monitor exige **adaptador USB externo** repassado — o rádio interno não chega ao convidado.
- ✓ A instalação só termina com `full-upgrade`, ferramentas de convidado e **snapshot testado**.

## NO KALI

A imagem **pré-construída de VM** é o caminho mais curto: já vem com as guest tools. Ela e a live usam `kali / kali` — troque com `passwd` no primeiro login. Na tela de software do instalador dá para trocar `kali-linux-default` por `kali-linux-headless` e poupar dezenas de GB. Depois: `apt full-upgrade`, `open-vm-tools-desktop` (ou `virtualbox-guest-x11`), `dpkg-reconfigure keyboard-configuration` para ABNT2, reboot e snapshot. Cuidado com a **senha de destruição do LUKS**: ela apaga os cabeçalhos de chave e **não existe recuperação**.

GUIA VISUAL 004

# PRIMEIRO CONTATO

*Não há "Meu Computador" nem letra de unidade. Não é falta — é outro modelo, e ele começa no firmware.*

004
GUIAS LINUX

## DO BOOT AO PROMPT: QUEM ENTREGA A TELA

**FIRMWARE** (UEFI / BIOS) → **GRUB** (escolhe o kernel) → **KERNEL** (monta a raiz) → **SYSTEMD** (PID 1, sobe tudo) → **GRAPHICAL.TARGET** (o alvo do systemd) → **LIGHTDM** (tela de login) → **PAM** (autentica) → **SESSÃO XFCE** (painel, janelas) → **TERMINAL** (o seu prompt)

**A camada gráfica é a única dispensável.** `systemctl get-default` mostra o alvo; `set-default multi-user.target` entrega um sistema completo sem tela — é o que se faz em servidor. E a saída de emergência quando o Xfce trava é **Ctrl+Alt+F3**: um console virtual ligado direto ao kernel. **Ctrl+Alt+F7** (ou F2) volta — a sessão gráfica continuou rodando o tempo todo.

## LENDO O PROMPT CARACTERE A CARACTERE

```
┌──(kali㉿kali)-[~]
└─$
```

Vermelho depois de um comando? Ele falhou — o Zsh do Kali colore o prompt pelo código de saída, poupando um **echo $?** a cada passo.

- **kali** — usuário logado
- **kali** — nome da máquina
- **~** — diretório atual = `/home/kali`
- **$** — comum · **#** = **root**, cada comando pode destruir o sistema

## XFCE NÃO É UM PROGRAMA — SÃO PEÇAS QUE SE REINICIAM SOZINHAS

| PROCESSO | O QUE DESENHA | SINTOMA DE QUE CAIU |
|---|---|---|
| `xfwm4` | Bordas, botões, qual janela fica na frente | Janelas sem barra de título → `xfwm4 --replace &` |
| `xfce4-panel` | A barra: menu, lista de janelas, relógio, bandeja | Sumiu o painel e o ícone de rede |
| `xfdesktop` | Papel de parede e ícones da área de trabalho | Fundo preto, sem ícones |
| `xfsettingsd` | Tema, fontes, cursor, teclado e mouse | Volta ao tema cinza padrão |
| `xfce4-session` | Coordena tudo e restaura o estado ao relogar | Nada restaura entre logins |

## OS QUATRO AJUSTES DO PRIMEIRO DIA

kali@kali: ~

```
$ setxkbmap -model abnt2 -layout br    # teclado ABNT2 já nesta sessão
$ sudo dpkg-reconfigure keyboard-configuration   # torna permanente
$ sudo timedatectl set-timezone America/Sao_Paulo # log com hora errada não correlaciona
$ timedatectl set-ntp true
$ mkdir -p ~/lab/{alvos,capturas,notas} ~/clientes # cliente nunca no lab
# Sistema em INGLÊS + teclado em português: erro em português quase não se pesquisa.
```

## RESUMO RÁPIDO

- ✓ Uma árvore só, começando em `/`: sem letra de unidade. `~` é onde você escreve sem `sudo`.
- ✓ Nome começando com ponto é **oculto** — `ls -a` mostra. É onde vivem as configurações.
- ✓ Maiúscula ≠ minúscula: `Documentos` e `documentos` são dois diretórios.
- ✓ `~/.config/xfce4` é o seu ambiente inteiro — copiável para outra máquina.

## NO KALI

**O menu de treze categorias numeradas é uma metodologia disfarçada**: elas seguem as fases de uma auditoria — 01 coleta → 02/03/04 análise → 05/08 exploração → 09/10 pós-exploração → 11/12 forense e relatório. Categoria vazia = ferramenta não instalada, não menu quebrado: instale o metapacote da área. Xfce é padrão desde 2019.4 e **Zsh desde 2020.4** (`chsh -s /bin/bash` troca). Serviços de rede vêm desabilitados: ferramenta que "não conecta no banco" é quase sempre o PostgreSQL parado.

# ESTRUTURA DE DIRETÓRIOS (FHS)

*Raiz com 96% de uso às 23h: se é `/var/log` a história é uma, se é `/var/cache/apt` é outra.*

005
GUIAS LINUX

## UMA ÁRVORE SÓ, SEM LETRA DE UNIDADE

| | |
|---|---|
| `/boot` | Kernel, initrd e o GRUB. Enche depois de algumas atualizações → `apt autoremove` |
| `/usr` | Programas, bibliotecas e dados estáticos. `/bin`, `/sbin` e `/lib` são só links para cá |
| `/usr/local` | O que **você** compilou. O `apt` nunca escreve aqui |
| `/etc` | Configuração, **sempre em texto**, nunca binário. Padrão Debian: fragmentos em `*.d/` |
| `/home` `/root` | Diretórios pessoais. O root fica fora de `/home` — que pode falhar ao montar |
| `/opt` | Software de terceiros autocontido, um diretório por produto |
| `/srv` | Dados servidos a terceiros (no Debian, na prática, `/var/www`) |
| `/mnt` `/media` | Pontos de montagem para mídia removível |
| `/var/log` | Logs. O primeiro suspeito no alerta de disco cheio |
| `/var/lib` | Estado persistente: base do `dpkg`, PostgreSQL, Docker |
| `/var/cache` | Recriável: `apt clean` libera espaço em 10 segundos |
| `/tmp` | Curta duração. Costuma ser **tmpfs = RAM**: some no boot |
| `/var/tmp` | Temporário que **sobrevive** à reinicialização |
| `/proc` | Processos e kernel. `/proc/self` é quem está lendo |
| `/sys` | Dispositivos e drivers: temperatura, `operstate` da placa |
| `/dev` `/run` | `null`, `zero`, `urandom`, discos · e o estado da execução atual |

em disco, estático · variável — cresce e enche · virtual — gerado pelo kernel, não ocupa disco

## A LÓGICA POR TRÁS · E ONDE INSTALAR O QUE

**ESTÁTICO · COMPARTILHÁVEL**
**`/usr`**
Pode ser montado somente-leitura ou servido por rede a várias estações.

**VARIÁVEL · COMPARTILHÁVEL**
**`/srv` · `/var/mail`**
Muda, mas faz sentido para outras máquinas.

**ESTÁTICO · LOCAL**
**`/etc` · `/boot`**
Específico desta máquina — por isso é o alvo prioritário de backup.

**VARIÁVEL · LOCAL**
**`/var/log` · `/run` · `/tmp`**
Volátil e local: exige rotação e monitoramento de espaço.

| VEIO DE ONDE | VAI PARA ONDE |
|---|---|
| Gerenciador de pacotes | `/usr/bin` — **não mexa** |
| Compilado do fonte | `/usr/local` |
| Terceiro autocontido | `/opt/<nome>` |
| Script só seu | `~/.local/bin` |
| Dados servidos | `/srv` ou `/var/www` |

## RESUMO RÁPIDO

- ✓ O caminho não diz em que disco o arquivo está — `findmnt --real` e `df -hT` dizem.
- ✓ Disco cheio: `sudo du -xh --max-depth=1 / | sort -rh | head`. O `-x` evita `/proc` e `/sys`.
- ✓ **Nunca escreva à mão em `/usr/bin`**: a próxima atualização sobrescreve sem avisar.
- ✓ `/proc`, `/sys` e `/dev` são "tudo é arquivo" na prática: leia com `cat`.

## NO KALI

Acabou o `/pentest/` do BackTrack: **ferramenta é pacote comum em `/usr/bin`** — `dpkg -S /usr/bin/nmap` diz de qual pacote veio. Os dados de cada uma ficam em `/usr/share/<ferramenta>` (scripts NSE, módulos do Metasploit) e as listas em `/usr/share/wordlists` — o `rockyou.txt.gz` vem compactado e precisa de `gunzip` uma vez. Ferramenta clonada de repositório vai para `/opt`. **Atenção ao `/tmp` em RAM:** salvar captura longa ali numa VM pequena derruba a máquina — use `~/lab/capturas`.

GUIA VISUAL 006

# ANATOMIA DE UM COMANDO

`rm arquivo com espaco.txt` *não apaga um arquivo: apaga três. Quem separou foi o shell, antes do programa existir.*

006
GUIAS LINUX

## A LINHA, PALAVRA POR PALAVRA

| `nmap` | `-sV` | `--top-ports` | `100` | `-oN` | `saida.txt` | `192.168.56.0/24` |
|---|---|---|---|---|---|---|
| COMANDO<br>o que será executado | OPÇÃO CURTA<br>um traço, uma letra | OPÇÃO LONGA<br>dois traços, palavra | VALOR<br>pertence à opção anterior | OPÇÃO<br>com valor | VALOR<br>arquivo de saída | OPERANDO<br>o alvo do trabalho |

**Para o shell, todas as palavras depois do comando são iguais.** Quem decide que `-sV` é opção e o resto é alvo é o **programa**, ao ler seu vetor de argumentos — por isso não existe sintaxe universal. Curtas sem valor se agrupam ( `ls -lah` ); `--` marca o **fim das opções** e salva de nome de arquivo que começa com traço: `rm -- -arquivo-estranho` . E há os rebeldes: `tar xzf` , `ps aux` , `dd if=` … `of=` … — sem traço nenhum.

## COMO O SHELL DECIDE O QUE EXECUTAR

1. **PALAVRA-CHAVE** — `if` , `for` , `while` — é sintaxe, não programa
2. **ALIAS** — Apelido seu ou da distro. **NÃO VALE EM SCRIPT**
3. **FUNÇÃO** — Definida na sessão ou no arquivo de init
4. **BUILTIN** — `cd` , `echo` , `export` — dentro do próprio shell
5. **PATH** — Arquivo em disco, na ordem dos diretórios

`type -a <comando>` **mostra qual das cinco venceu** — e explica por que o terminal e o script se comportam diferente. O **diretório atual não está no** `PATH` : por isso `./script.sh` . Não é inconveniência, é segurança — com `.` no `PATH` , um arquivo chamado `ls` num diretório gravável vira escalada de privilégio. **Nunca acrescente** `.` **ao** `PATH` **.**

## ZERO É SUCESSO — E É O QUE DÁ SENTIDO AO ENCADEAMENTO

| CÓDIGO DE SAÍDA | SIGNIFICA |
|---|---|
| **0** | Sucesso — o único valor bom |
| **1** | Erro genérico ( `grep` sem achar nada) |
| **2** | Erro de uso / arquivo inexistente |
| **126** | Existe, mas não é executável |
| **127** | Comando não encontrado — erro de digitação ou não instalado |
| **130** | Interrompido por `Ctrl+C` (128 + sinal 2) |

**`a && b`**
Roda **b** só se **a** deu certo. **O padrão para tudo que pressupõe o passo anterior.**

**`a || b`**
Roda **b** só se **a** falhou — o tratamento de erro.

**`a ; b`**
Roda **b** sempre, dê no que der.

**`a &`**
Roda em segundo plano e devolve o prompt na hora.

**`\rm · command rm`**
Ignora o alias e chama o programa real.

**Trocar `&&` por `;` já destruiu sistemas.** Em `cd /tmp/trabalho ; rm -rf ./*` , se o `cd` falhar por um erro de digitação, o `rm -rf` executa **no diretório em que você estiver** — que pode ser o seu `~` . Escreva sempre `cd /tmp/trabalho && rm -rf ./*` . Mesma família de erro: em `rm -rf "$DIR"/` com `$DIR` vazia, a linha vira `rm -rf /` . E `sudo echo x > /etc/arquivo` **falha**: quem redireciona é o seu shell, não o sudo — use `echo x | sudo tee -a /etc/arquivo` .

## RESUMO RÁPIDO

- ✓ Ler → separar por espaço → expandir → resolver → `fork` → `execve`. O programa só existe no fim.
- ✓ Espaço em nome quebra tudo: `"$arquivo"` entre aspas, sempre.
- ✓ `hash -r` (Bash) ou `rehash` (Zsh) quando o shell não acha o programa recém-instalado.
- ✓ O shell é só mais um programa: `ps -p $$ -o comm=` mostra qual está rodando de verdade.

## NO KALI

O shell padrão é o **Zsh** desde a 2020.4 — para tudo desta ficha ele é idêntico ao Bash; use `rehash` no lugar de `hash -r`. A distro define aliases coloridos para `ls`, `grep`, `ip`: veja com `alias` e lembre que **script nenhum os enxerga**. Ferramenta clonada em `/opt` não está no `PATH` — faça um link em `/usr/local/bin`. **Cuidado com comando copiado de artigo:** dá para esconder um comando extra num bloco formatado. Cole num editor antes de executar — o shell obedece à linha, não à sua intenção.

GUIA VISUAL 007

# OBTENDO AJUDA

*3h da manhã, link de 200 kbps, sem sinal de celular — e a resposta já está instalada na máquina.*

**007**
GUIAS LINUX

## A FONTE CERTA DEPENDE DA PERGUNTA

| A SUA DÚVIDA | ONDE PERGUNTAR | POR QUÊ |
|---|---|---|
| Sei o nome, quero a sintaxe agora | `comando --help` · `tldr comando` | Cabe na tela; `tldr` é só exemplo comentado |
| Sei o nome, quero a referência completa | `man comando` | Descreve **a versão instalada**, não a do autor do artigo |
| **Não sei o nome**, sei o que quero fazer | `apropos palavra` · `man -k palavra` | Busca na descrição de todas as páginas em disco |
| Quero o formato de um arquivo de configuração | `man 5 arquivo` | A seção 5 é dos formatos, não dos comandos |
| Por que este pacote se comporta assim? | `/usr/share/doc/<pacote>/` | `README.Debian` traz as decisões do empacotador |
| É builtin do shell (não tem arquivo) | `help cd` (Bash) · `run-help cd` (Zsh) | A doc vem do shell — confira antes com `type -a` |

## O MESMO NOME EXISTE EM SEÇÕES DIFERENTES

**1 COMANDOS DE USUÁRIO**
Tudo que se digita. `man 1 passwd` = trocar senha

**2 CHAMADAS DE SISTEMA**
A fronteira com o kernel — `open(2)`

**3 FUNÇÕES DE BIBLIOTECA**
C e glibc — `printf(3)`

**4 ARQUIVOS ESPECIAIS**
Dispositivos em `/dev`

**5 FORMATOS DE ARQUIVO**
A sintaxe da configuração. `man 5 passwd` = `/etc/passwd`

**6 JOGOS**
Herança histórica do Unix

**7 DIVERSOS E CONVENÇÕES**
Padrões e protocolos — `man 7 hier`

**8 ADMINISTRAÇÃO**
O que normalmente exige root — `man 8 mount`

**Sem o número, o `man` entrega a seção 1.** Por isso `man passwd` mostra o comando e `man 5 passwd` mostra o formato do arquivo. `whatis passwd` lista todas as seções em que o nome existe; `man -a passwd` percorre uma a uma. O número entre parênteses numa referência — `crontab(5)` — é exatamente isso: onde procurar.

## A GRAMÁTICA DO SYNOPSIS · E AS SEIS TECLAS DO `MAN`

```
ls [OPTION] ... [FILE] ...
cp [OPTION] ... SOURCE ... DIRECTORY
```

- `[colchetes]` opcional — pode omitir
- `SEM colchetes` obrigatório
- `reticências ...` pode repetir
- `{a|b}` escolha uma, obrigatória

Só com essa notação já se responde por que `ls` sozinho funciona e `cp arquivo` dá erro: o destino do `cp` não está entre colchetes.

| | |
|---|---|
| `espaço / b` | Avança / volta uma tela |
| `/texto` | **A tecla que salva:** busca para frente. Ninguém lê `man bash` inteiro |
| `n / N` | Próxima ocorrência / anterior |
| `g / G` | Início / fim da página |
| `h` | Ajuda do próprio visualizador |
| `q` | Sai |

## RESUMO RÁPIDO

- ✓ Doc local sempre casa com a versão instalada; artigo na internet, não. Divergiu? **a local está certa**.
- ✓ `tldr` para lembrar a forma usual, `man` para o detalhe e o caso de borda.
- ✓ `apropos` vazio? o índice não existe: `sudo mandb` .
- ✓ `whereis` · `dpkg -S $(which x)` · `file $(which x)` — se for script, dá para **ler**.

## NO KALI

Muita ferramenta do Kali **não tem página de manual** — são projetos de comunidade. O caminho é `-h` (e não `--help` ), depois o nome sozinho, depois **kali.org/tools**, que documenta cada ferramenta empacotada; **kali.org/docs** cobre a distribuição. Módulos do Metasploit se documentam dentro do console ( `info <modulo>` , `show options` ). Em imagem `headless` ou contêiner o `man` costuma ter sido removido: `apt install --reinstall man-db manpages` . Depois de instalar um metapacote, rode `sudo mandb` . **Comando colado sem leitura é comando não auditado** — vale para fórum, artigo e IA.

GUIA VISUAL 008

# TERMINAL E SHELL

*Você configurou o prompt a tarde toda, abriu um SSH e tudo sumiu. Nada quebrou: o que você configurou não era o terminal.*

008
GUIAS LINUX

## DA TECLA AO PROGRAMA: QUATRO CAMADAS

| | | |
|---|---|---|
| **TECLADO E TELA** | Hardware, ou a janela do ambiente gráfico. | o que você aperta |
| **EMULADOR DE TERMINAL** | `qterminal`, `xterm`, `tmux`, PuTTY. Desenha glifos, traduz teclas, guarda a rolagem. **Não sabe o que é um comando.** | fonte · cores · abas · `TERM` |
| **PSEUDOTERMINAL DO KERNEL** | Aparece como `/dev/pts/N`. Faz o eco das teclas, junta a linha até o Enter e converte `Ctrl+C` em `SIGINT`. | é o kernel — por isso o `Ctrl+C` funciona com o programa travado |
| **SHELL** | `zsh`, `bash`, `dash`. Imprime o prompt, lê a linha, expande, resolve o nome e cria o processo. | prompt · histórico · alias · autocompletar |
| **PROGRAMA** | `ls`, `nmap`, `vim`. Nasce com três descritores já abertos apontando para o PTY. | stdin · stdout · stderr prontos |

**Cada problema de terminal pertence a exatamente uma camada.** No SSH o **emulador continua sendo o seu**, na sua máquina — o que mudou foi o shell, que agora roda no servidor e não tem a sua configuração. Lixo na tela ou `Error opening terminal` é a variável `TERM` errada (chave do banco **terminfo**): `export TERM=xterm` é a saída de emergência. E `ls --color=auto | cat` sai **sem cor** de propósito — o programa perguntou se a saída era um terminal e a resposta foi não.

## QUAL ARQUIVO O SHELL LÊ — E O QUE MUDA DO BASH PRO ZSH

**1. É SHELL DE LOGIN?**
SSH, terminal virtual de texto, `su -`, `sudo -i` → lê `/etc/profile` e `~/.zprofile`. **AQUI VAI VARIÁVEL DE AMBIENTE** (`PATH`, `EDITOR`): filho nenhum precisa relê-la.

**2. É INTERATIVO?**
Tem prompt e teclado — a aba nova do qterminal **É INTERATIVA E NÃO É LOGIN**. Lê `~/.zshrc` / `~/.bashrc`. **AQUI VAI ALIAS, FUNÇÃO E PROMPT.**

**3. NENHUM DOS DOIS?**
Um script. **NÃO LÊ NADA** — e é por isso que ele nunca enxerga os seus aliases. Errar entre 1 e 2 é a causa nº 1 de configuração que "não pega".

**APLICAR SEM FECHAR O TERMINAL**
`source ~/.zshrc` executa **NO SHELL ATUAL**. Executar o arquivo criaria um filho, aplicaria nele e o jogaria fora.

| SITUAÇÃO | BASH | ZSH |
|---|---|---|
| Config interativa | `~/.bashrc` | `~/.zshrc` |
| Curinga sem par | fica literal | erro `no matches` |
| Índice de array | `0` | `1` |
| Limpar cache | `hash -r` | `rehash` |
| Uso indicado | scripts | dia a dia |

## IDENTIFIQUE CADA CAMADA EM CINCO LINHAS

kali@kali: ~

```
$ tty                    # a qual pseudoterminal este shell está ligado
/dev/pts/0
$ ps -p $$ -o comm=      # o shell que roda DE VERDADE ($SHELL só diz o cadastrado)
zsh
$ ps -p $PPID -o comm=   # quem criou o shell: o emulador
qterminal
$ echo "$TERM"; tput colors
xterm-256color
256
```

### RESUMO RÁPIDO

- ✓ Emulador desenha e traduz teclas; **shell** interpreta comandos. Prompt, histórico e alias são do shell.
- ✓ Entre os dois há o PTY do kernel ( `/dev/pts/N` ) — dono do eco e do `Ctrl+C` .
- ✓ Ambiente no arquivo de **login**; alias e prompt no arquivo **interativo**.
- ✓ **Zsh no dia a dia, script para Bash** com `#!/usr/bin/env bash` — as duas escolhas são independentes.
- ✓ Trocar de shell: `chsh -s /usr/bin/bash` , só na próxima sessão. Testar: digite `bash` e saia com `exit` .

### NO KALI

**Zsh é o padrão desde a 2020.4**, com tema próprio que muda de cor quando você é root — e o `kali-tweaks` alterna o prompt entre duas linhas, uma linha e enxuto sem editar nada à mão. O emulador é o `qterminal` : `Ctrl+Shift+T` nova aba, `Ctrl+Shift+C / V` copiar e colar — o **Shift** é obrigatório porque `Ctrl+C` já tem dono. **Colar comando de artigo é o ataque mais barato que existe**: uma página esconde texto no que você acha que selecionou. Cole num editor antes. O `~/.zshrc` já vem povoado: **acrescente ao fim**, não reescreva. E se um alias "sumiu" depois do `sudo -i` , é outro usuário, outro `~` , outra config.

GUIA VISUAL 009

# NAVEGAÇÃO E CAMINHOS

*O comando do colega funciona na máquina dele e falha na sua. Não mudou o comando — mudou de onde ele foi executado.*

009
GUIAS LINUX

## MESMO DESTINO, DOIS CAMINHOS

```
/
├─ etc
│  └─ ssh
│     └─ sshd_config
├─ home
│  └─ kali
│     └─ projetos
│        └─ recon   ← pwd
└─ var
   └─ log
```

**ABSOLUTO — COMEÇA EM `/`**

`/etc/ssh/sshd_config`

Aponta para o mesmo arquivo **de qualquer lugar**. É o que se usa em script e em cron.

**RELATIVO — PARTE DO `PWD`**

`../../../etc/ssh/sshd_config`

Digita menos, mas **só vale a partir de `recon`**. Um `cd` antes e o mesmo texto aponta para outro lugar — ou para lugar nenhum.

**O diretório atual é um campo do *processo*, não do usuário nem do terminal.** Dá para vê-lo por fora em `/proc/<PID>/cwd` — útil para achar quem segura um sistema de arquivos que você quer desmontar. Daí saem três consequências: cada aba tem o seu; **o filho herda e nunca devolve** — um script que faz `cd` move a si mesmo e morre, o seu shell fica onde estava (por isso `source`); e é por isso que `cd` precisa ser *builtin*. Já `.` e `..` são **entradas reais** do sistema de arquivos, não sintaxe — é o que faz `./script.sh` rodar sem o diretório atual estar no `PATH`.

## A LINHA DO `LS -L`, CAMPO A CAMPO

| `-` | `rw-r--r--` | `1` | `root` | `root` | `2891` | `Jul 14 09:22` | `/etc/passwd` |
|---|---|---|---|---|---|---|---|
| TIPO<br>- arquivo · d dir · 1 link | PERMISSÕES<br>dono · grupo · outros | LINKS<br>quantos nomes apontam ao inode | DONO | GRUPO | TAMANHO<br>bytes (-h converte) | MODIFICAÇÃO<br>do conteúdo | NOME |

## OS ATALHOS E OS COMANDOS QUE VIRAM REFLEXO

| SÍMBOLO | SIGNIFICA |
|---|---|
| `.` | O diretório atual |
| `..` | O diretório pai |
| `~` | Seu diretório pessoal — **não expande dentro de aspas**; em script use `"$HOME"` |
| `~kali` | O diretório pessoal do usuário `kali` |
| `-` | O diretório anterior (guardado em `OLDPWD`), só com `cd` |
| `cd` | Sem argumento nenhum: vai para o seu `~` |

**`cd -`**
Alterna entre os dois últimos diretórios. O atalho que mais rende no dia a dia.

**`pushd dir · popd · dirs -v`**
Pilha de diretórios, para quando são mais de dois pontos na investigação.

**`ls -lah`**
O padrão de trabalho: longo, com ocultos, tamanho legível.

**`ls -ltr`**
Mais recente **por último**, coladinho no prompt. O reflexo de quem lê log.

**`ls -ld /var/log`**
A linha **do próprio diretório**, não o conteúdo — é o que responde "por que não consigo entrar aqui".

**`pwd -P · readlink -f · realpath`**
Resolvem links e `..` até o caminho físico. `pwd` sozinho mostra o caminho **lógico**, por onde você chegou — e depois de entrar por um link, `cd ..` vai para o pai do *link*, não do alvo.

## RESUMO RÁPIDO

- ✓ Absoluto começa em `/` e vale sempre; relativo depende do `pwd`.
- ✓ Script que faz `cd` não move o seu shell — para mover, `source`.
- ✓ Oculto é só nome com ponto: sem `-a` o seu `~` parece vazio.
- ✓ `-F` marca o tipo (`/` dir, `*` exec, `@` link) e **sobrevive ao cano**, quando a cor não.
- ✓ Script que lê arquivo ao lado dele: `DIR="$(cd -- "$(dirname -- "$0")" && pwd -P)"`.

## NO KALI

O `~` do usuário é `/home/kali`; o do root é `/root` — depois de `sudo -i` o que você salvar **não está no seu diretório**. É a explicação de "salvei o relatório e não acho mais". O Zsh do Kali liga o `AUTO_CD`: digitar só `/var/log` já muda de diretório (não funciona em Bash nem em script), e `cd /u/l/b` + Tab expande para `/usr/local/bin`. Ferramenta clonada do GitHub vai para `/opt`, que **não está no** `PATH` — é o caso de uso nº 1 do `pushd` / `popd`. Wordlists em `/usr/share/wordlists` (a `rockyou.txt` vem `.gz` numa instalação nova). E faça de `ls -la ~/.ssh` rotina: chave com permissão frouxa faz o cliente SSH **recusar** a conexão.

GUIA VISUAL 010

# COPIAR, MOVER E APAGAR

*`mv relatorio.md backups/` — se `backups` não existir, você não moveu nada: renomeou o relatório.*

010
GUIAS LINUX

## O NOME E O CONTEÚDO SÃO COISAS SEPARADAS

### `cp a.txt b.txt`

`a.txt` → `inode 42`

`b.txt` → `inode 77`

Dois nomes, **dois conteúdos**. É a operação cara: lê e escreve tudo, ocupa o dobro, demora conforme o tamanho.

### `mv a.txt b.txt`

`b.txt` → `inode 42`

~~`a.txt`~~ `nome removido`

Só a **entrada de diretório** muda. Instantâneo e **atômico** mesmo com 40 GB — entre discos diferentes, porém, vira cópia + remoção.

### `rm a.txt`

~~`a.txt`~~ × `inode 42`

`links 1 → 0` → `blocos liberados`

Apaga um **nome**. O conteúdo só é liberado quando nenhum nome **e nenhum processo** apontam para ele — por isso apagar log em uso não devolve espaço.

## `MV ORIGEM DESTINO` — O DESTINO EXISTE?

### NÃO EXISTE

→ `renomeia para destino`

É o caso que morde: um diretório digitado errado vira um arquivo com nome de diretório, em silêncio.

### EXISTE E É DIRETÓRIO

→ `move para destino/origem`

O que você queria. **Termine o destino com barra** — `mv x backups/` — e o caso da esquerda vira erro em vez de surpresa.

### EXISTE E É ARQUIVO

→ `sobrescreve sem aviso`

Não há confirmação, não há lixeira. Antídotos: `-i` pergunta, `-n` nunca sobrescreve, `--backup=numbered` guarda o antigo.

## AS OPÇÕES QUE MUDAM O RESULTADO

| OPÇÃO | VALE PARA | O QUE FAZ |
|---|---|---|
| `-i` / `-I` | cp mv rm | Pergunta a cada arquivo / **uma vez** pelo lote |
| `-n` | cp mv | Nunca sobrescreve, e não pergunta |
| `-r` | cp rm | Recursivo — obrigatório para diretório |
| `-a` | cp | **Arquivamento**: `-r` + preserva data, dono, permissão e links |
| `-u` | cp mv | Só age se a origem for mais nova |
| `-p` | mkdir | Cria a cadeia toda e **não reclama se já existir** |

**`cmd > cfg.novo && mv cfg.novo cfg`**
Gere ao lado e mova por cima: ninguém jamais lê um arquivo pela metade.

**`rmdir vazio`**
Só remove diretório vazio — a recusa *é* o recurso de segurança.

**`rename -n 's/\.jpeg$/\.jpg/' *.jpeg`**
Renomeia em lote; o `-n` simula. Rode sempre com ele antes.

**`rsync -av --dry-run orig/ dest/`**
Árvore grande, retomável, só o que mudou. **Aqui a barra final muda tudo**: com ela copia o conteúdo, sem ela cria `dest/orig`.

**`stat arq` · `touch -d "2020-01-01" arq`**
**mtime** conteúdo · **atime** leitura · **ctime** metadados. O `ctime` não se falsifica pelo `touch`: `ctime` mais novo que `mtime` denuncia carimbo alterado.

**Não existe lixeira, e o `rm -rf` não pede licença.** Ritual antes de todo comando destrutivo: **1.** `pwd` — onde estou? **2.** `ls -la alvo/` — é isto mesmo? **3.** só então o `rm`. Ou troque o passo 2 por `echo rm -rf "$DIR"/*`: o `echo` imprime a linha **já expandida**, sem executar. Quatro linhas que já destruíram sistemas reais: `rm -rf / caminho/` (o espaço apaga a raiz) · `rm -rf "$DIR"/*` com `$DIR` vazia vira `rm -rf /*` · `cd /alvo ; rm -rf *` se o `cd` falhar (use `&&`) · `rm -rf .*`. O `--preserve-root` é padrão, mas cobre só a raiz literal — `/*` passa.

## RESUMO RÁPIDO

- ✓ `cp` duplica conteúdo · `mv` mexe só no nome · `rm` remove um nome.
- ✓ Destino com **barra final** transforma o erro silencioso do `mv` em mensagem.
- ✓ `mkdir -p` é idempotente — obrigatório em script com `set -e`.
- ✓ Backup ou evidência é sempre `cp -a`: sem ele, tudo é carimbado com a data de hoje.
- ✓ `rm -I` (maiúsculo) pergunta **uma vez** pelo lote — o meio-termo que se usa de verdade.

## NO KALI

**Não adote `alias rm='rm -i'`.** Na VM funciona; num shell de máquina-alvo autorizada o alias não existe e o reflexo falha — e `rm -f` ignora o `-i` em silêncio. Prefira `trash-cli` com um alias **novo** (`alias lixo='trash-put'`), deixando o `rm` ser o que ele é. **Coleta de evidência é `cp -a` ou `rsync -a`:** preservar data, dono e permissão faz parte da integridade do material. Para imagem de disco, nem um nem outro — é `dcfldd`, que já vem no Kali e calcula o hash durante a cópia (confira o alvo com `lsblk` antes; Vol 8). Wordlist: descompacte em `/tmp` ou no seu `~` com `gunzip -k`, **nunca dentro de `/usr/share`**, que é território do gerenciador de pacotes.

# LER O CONTEÚDO

*`cat` num log gigante arrasa a rolagem; `cat` num binário embaralha o terminal. Nenhum dos dois é acidente — o `cat` não serve para ver arquivo.*

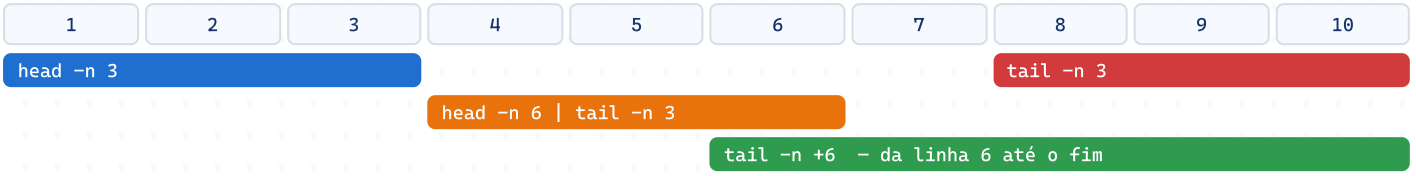
**011**
GUIAS LINUX

## RODE `FILE` ANTES DE ABRIR — E ESCOLHA A FERRAMENTA

| TEXTO CURTO | TEXTO GRANDE | SÓ AS PONTAS | CRESCENDO | COMPRIMIDO | BINÁRIO |
|---|---|---|---|---|---|
| **`cat`** | **`less`** | **`head`** **`tail`** | **`tail -F`** **`less +F`** | **`zcat zless zgrep`** | **`strings`** **`xxd`** |
| Cabe numa tela. Só aí. | Carrega sob demanda: 10 GB abrem na hora. | Primeiras ou últimas linhas. | Acompanha o arquivo em tempo real. | Lê o `.gz` sem descomprimir. | **Nunca** `cat` — conserto: `reset` . |

**O `cat` *concatena* — exibir é só o que sobra quando você dá um arquivo e não redireciona.** O `file` não confia na extensão: lê os primeiros bytes e compara com um banco de assinaturas. Num binário, os bytes que por acaso formam sequências de escape VT100 são **obedecidos** — o terminal não travou, ele mudou de fonte e de mapa de caracteres; digite `reset` às cegas e Enter. E o `cat -A` é diagnóstico de verdade: revela `^M` (fim de linha do Windows — a causa do `bad interpreter: /bin/bash^M` ), `^I` (tabulação onde você achava que havia espaços) e `$` (espaço sobrando no fim da linha).

## O SINAL DE MAIS MUDA TUDO: AS FAIXAS DE `HEAD` E `TAIL`




## DEZ TECLAS DO `LESS` E OS REFLEXOS QUE SOBRAM

| TECLA | FAZ |
|---|---|
| `Espaço` / `b` | Avança / volta uma tela |
| `g` / `G` | Início / fim do arquivo |
| `/texto` · `n` `N` | Busca; próxima e anterior ocorrência |
| `&texto` | **Mostra só as linhas que casam** — um grep interativo sem sair do arquivo ( `&` + Enter volta ao todo) |
| `F` | Acompanha o crescimento; `Ctrl+C` sai do modo **sem perder o histórico** |
| `m` +letra · `'` +letra | Marca a posição e volta a ela |
| `v` · `h` · `q` | Abre no `$EDITOR` · ajuda · sai |

**`tail -f` × `tail -F`**
O `-f` segue o **inode**: depois que o `logrotate` renomeia o arquivo, ele fica olhando o antigo para sempre. O `-F` segue o **nome** e reabre. Em servidor é sempre `-F` .

**`tail -F auth.log | grep --line-buffered "Failed"`**
Sem `--line-buffered` o grep segura kilobytes e você fica minutos achando que travou.

**`less -R` · `export LESS='-R -i -M'`**
`-R` interpreta as cores do `nmap` ; sem ele você lê `ESC[1;32m` em cada linha. É o mesmo paginador do `man` .

**`grep erro arquivo`**
Não `cat arquivo | grep` : todo comando que aceita nome de arquivo dispensa o `cat` — é um processo e uma cópia de dados a menos.

**`tac` · `batcat` · `jq .` | `less -R`**
`tac` lê do fim para o começo. `batcat` (o `bat` do Debian) colore por sintaxe — ótimo para código, dispensável para dados que seguem num cano.

## RESUMO RÁPIDO

- ✓ `file` antes de abrir: ele lê os bytes, não a extensão.
- ✓ `head -n 3` as três primeiras · `tail -n 3` as três últimas · `tail -n +3` da terceira em diante.
- ✓ Encadear `head | tail` recorta qualquer faixa do meio.
- ✓ `cat > nota.txt` grava até o **`Ctrl+D`** (fim de arquivo). `Ctrl+C` ali trunca o arquivo.
- ✓ `more` não volta e não busca para trás — use só onde não houver `less`.

## NO KALI

A `rockyou.txt` tem **mais de 14 milhões de linhas**: `cat` nela é perder o terminal na certa — `zcat rockyou.txt.gz | head`, `wc -l`, `zgrep`. Log moderno é **binário**: `cat` em `/var/log/journal` não serve para nada, use `journalctl -u ssh`, `-f`, `-p err` (Vol 7). Deixe `tail -F /var/log/auth.log` numa segunda aba enquanto trabalha: **todo `sudo` seu aparece ali** — ensina rápido o que o sistema registra a seu respeito. Relatório de `nmap -oN` numa `/16` é enorme: abra no `less` e filtre com `&`. `strings` num binário suspeito é o **primeiro** passo — em VM isolada, sem executar, e só com material que você tem autorização para analisar.

GUIA VISUAL 012

# REDIRECIONAMENTO E PIPES

*Você guardou a varredura com `>` e os erros sumiram com o buffer da tela. Trocar para `2>&1 > arq` **não** conserta.*

012
GUIAS LINUX

## TODO PROCESSO NASCE COM TRÊS CANAIS ABERTOS

**`comando`**

| | | |
|---|---|---|
| 0 · stdin | ← | teclado |
| 1 · stdout | → | tela |
| 2 · stderr | → | tela |

Resultado e erro **misturados** na mesma tela — e é só a numeração que os separa.

**`comando > saida.txt`**

| | | |
|---|---|---|
| 0 · stdin | ← | teclado |
| 1 · stdout | → | saida.txt |
| 2 · stderr | → | tela |

**Só o descritor 1 mudou de destino.** Redirecionar é reapontar um canal *antes* de o programa começar — e o programa nunca percebe.

## A MESMA DUPLA, DUAS ORDENS, RESULTADOS OPOSTOS

**`comando > log.txt 2>&1`**

1. **`> log.txt`** — o descritor **1** passa a apontar para o arquivo.
2. **`2>&1`** — o **2** copia o destino *atual* do 1: o arquivo.

✓ resultado **e** erro no arquivo

**`comando 2>&1 > log.txt`**

1. **`2>&1`** — o **2** copia o destino atual do 1: a **tela**.
2. **`> log.txt`** — só o **1** muda para o arquivo.

✗ resultado no arquivo, erro **na tela**

`2>&1` **copia o destino *daquele instante* — não é vínculo.** O shell lê os redirecionamentos da esquerda para a direita, então a regra é: **o `2>&1` vem por último**, ou use `&>`. A ordem "errada" tem um uso legítimo e elegante: `comando 2>&1 > /dev/null | grep falha` manda **só os erros** para o cano, porque o `stderr` foi copiado para o cano antes de o `stdout` ser descartado.

## AS FORMAS QUE SE USAM TODO DIA

| FORMA | O QUE FAZ |
|---|---|
| `>` / `>>` | stdout sobrescrevendo / acrescentando |
| `2>` / `2>>` | Só o stderr |
| `&>` / `&>>` | Os dois no mesmo arquivo (Bash e Zsh; em `sh` use `> a 2>&1`) |
| `<` | Lê do arquivo. `wc -l < f` imprime só o número, sem o nome |
| `<< FIM` | *Here-doc*: texto até o marcador. Entre aspas (`<< 'FIM'`) não expande variável |
| `<<<` | *Here-string*: uma linha, sem `echo` e sem cano |
| `\|` / `\|&` | Cano só do stdout / com o stderr junto |

**`a | b | c`**
Os três rodam **ao mesmo tempo**, não em sequência: o kernel põe um buffer entre cada par. É o que permite processar 100 GB com poucos MB de RAM.

**`set -o pipefail` · `${PIPESTATUS[@]}`**
Por padrão o status do pipeline é o do **último** comando: `falso | tee f` devolve **0**. Esse detalhe já mascarou backup quebrado por meses.

**`echo linha | sudo tee -a /etc/hosts > /dev/null`**
A resposta certa para `sudo ... > arquivo`, que falha porque **quem redireciona é o seu shell**, não o sudo. O `tee` grava e repassa.

**`diff <(ls /etc) <(ls /etc/apt)`**
Substituição de processo: entrega um caminho como `/dev/fd/63` onde o comando exige **nome de arquivo**. Bash e Zsh, não `sh`.

**`grep --line-buffered` · `stdbuf -oL` · `unbuffer`**
Filtro ao vivo que "trava" é *buffering*: a libc usa buffer por linha na tela e **por bloco** no cano. Não é bug — é otimização atrapalhando.

**O `>` trunca o arquivo *antes* de o comando rodar** — e trunca mesmo que o comando não exista: `comando-errado > importante.txt` deixa o arquivo com zero bytes. Proteção barata no `~/.zshrc`: `set -o noclobber` (e `>|` quando você realmente quiser forçar) — sabendo que ela não existe em servidor nem em script. **E `2> /dev/null` não conserta nada: esconde.** Use só quando você sabe *qual* erro está descartando (o `find` reclamando de diretório sem permissão, por exemplo), nunca para deixar a saída bonita.

## RESUMO RÁPIDO

- ✓ `0` entrada · `1` resultado · `2` erro. O cano leva **só o 1**.
- ✓ `2>&1` por último, ou `&>`.
- ✓ `/dev/null` descarta · `/dev/zero` e `/dev/urandom` geram · `/dev/full` falha de propósito, para testar tratamento de erro.
- ✓ `head -c 32 /dev/urandom | base64` — senha aleatória em uma linha.
- ✓ `exec 3> log` abre o arquivo **uma vez**; `echo x >&3` escreve; `exec 3>&-` fecha.

## NO KALI

**Ferramenta de varredura escreve muito no `stderr`:** `nmap`, `gobuster` e `ffuf` mandam progresso e avisos para o 2 e resultado para o 1 — guardar só com `>` perde metade. Use `&> saida.txt` ou `2>&1 | tee saida.txt`. Melhor ainda: **prefira a opção nativa** (`nmap -oA base` grava nos três formatos; `gobuster -o`, `ffuf -o`) — redirecionamento é para o que a ferramenta não formata. **`| tee` durante teste autorizado é hábito profissional**: você acompanha na tela e fica com o registro do que rodou e quando — material de evidência e de relatório (combine com `ts`, do `moreutils`, para carimbar a hora). Wordlist por cano: `zcat rockyou.txt.gz | head -1000 > /tmp/curta.txt`, sem descomprimir nem escrever em `/usr/share`.

GUIA VISUAL 013

# CURINGAS E EXPANSÕES

*O programa **nunca viu o asterisco**. Quem procurou os arquivos, montou a lista e escolheu a ordem foi o shell — antes de o comando existir.*

013
GUIAS LINUX

## OITO ETAPAS, SEMPRE NESTA ORDEM, NA MESMA LINHA

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|
| CHAVES `{a,b}` | TIL `~` | VARIÁVEIS `$VAR` | COMANDOS `$(cmd)` | ARITMÉTICA `$((1+2))` | DIVISÃO EM PALAVRAS IFS | GLOBBING `* ? [ ]` | REMOÇÃO DE ASPAS `" '` |

**A etapa 1 gera texto do nada** — não consulta o disco, e por isso `mkdir {a,b,c}` funciona com nada existindo. **A etapa 7 olha o disco** e só produz o que existe de fato. E como o globbing vem **depois** das variáveis, um padrão guardado numa variável é expandido ao ser usado *sem aspas*: `PADRAO="*.txt"` → `echo $PADRAO` lista os arquivos, `echo "$PADRAO"` imprime o texto. O programa só roda **depois das oito etapas**, com a lista final de palavras — e a conferência custa dois segundos: `echo mv *.txt arquivo/` .

## QUEM CASA COM QUÊ

| ARQUIVO NO DIRETÓRIO | *.TXT | LOG?.TXT | LOG[12].TXT | * | .* |
|---|---|---|---|---|---|
| `log1.txt` | ✓ | ✓ | ✓ | ✓ | ✗ |
| `log2.txt` | ✓ | ✓ | ✓ | ✓ | ✗ |
| `log10.txt` | ✓ | ✗ ? é **um** caractere | ✗ | ✓ | ✗ |
| `nota.md` | ✗ | ✗ | ✗ | ✓ | ✗ |
| `.oculto` | ✗ | ✗ | ✗ | ✗ o `*` **não pega oculto** | ✓ |

## CHAVES GERAM · CURINGAS ENCONTRAM · E ONDE ISSO MACHUCA

**`cp app.conf{,.bak}`**
Vira `cp app.conf app.conf.bak` . O elemento vazio produz o nome original e o com sufixo — **o backup mais curto que existe**.

**`mkdir -p proj/{doc,src}/{in,out}`**
Produto cartesiano: quatro diretórios numa linha. Chaves não consultam o disco, então criam o que ainda não existe.

**`touch teste-{01..10}.txt · echo {0..20..5}`**
Faixas com zero à esquerda preservado e com passo. `{10..1}` conta ao contrário.

**`diff arquivo.{antigo,novo}`**
Digite o nome uma vez só. E `ls *.{txt,md}` vira `ls *.txt *.md` — cada padrão passa pelo globbing depois.

**`ls *(.om[1,3]) · ls **/*(.Lm+50)`**
Qualificadores de glob do **Zsh**: os 3 mais recentes, os maiores que 50 MB. É `find` embutido na sintaxe — e não existe no Bash.

**Sem correspondência: Bash ≠ Zsh**
Bash entrega o **texto literal** ao comando; Zsh aborta com `no matches found` . Em laço Bash, `shopt -s nullglob` é obrigatório — sem ele, `for f in *.log` roda **uma vez** com `$f` valendo `*.log` .

**`cp "$f" /tmp/` — nunca `cp $f /tmp/`**
O glob entrega o nome certo; quem quebra `meu arquivo.txt` em dois é a **etapa 6**, na variável sem aspas.

**`rm -- *` · `rm ./*`**
Um arquivo chamado `-rf` no diretório transforma `rm *` em `rm -rf resto` . O `--` marca o fim das opções.

**`*` não cruza a barra e ordena como texto**
`**` desce a árvore ( `shopt -s globstar` no Bash — sem ele vira um `*` comum, **em silêncio**). E a ordem é `log1 log10 log2` : quem ordena é o shell, alfabeticamente.

**`find . -name '*.log' -print0 | xargs -0 rm`**
Para `Argument list too long` e para nomes com espaço, aspas ou quebra de linha: o byte zero é o único caractere **proibido** num nome, logo o único separador seguro. Nunca `ls | xargs` .

**Ensaie todo curinga antes de um comando destrutivo. 1.** `ls *.bak` — o que casa? **2.** `echo rm *.bak` — a linha final, já expandida, sem executar. **3.** só então `rm *.bak`. O passo 2 é o que vale: um espaço a mais, um arquivo inesperado ou um padrão que casou demais aparecem ali — e depois de expandido **não existe desfazer pelo padrão original**, porque o `rm` nunca soube que havia um asterisco.

## RESUMO RÁPIDO

- ✓ Aspas **simples** desligam tudo; aspas **duplas** mantêm variável e `$(cmd)`, mas desligam glob e chaves.
- ✓ `[!abc]` nega a lista; `[[:digit:]]` respeita a localidade e não deixa acento de fora.
- ✓ Ocultos: `shopt -s dotglob` / `setopt dotglob`, ou o portável `.[!.]*`.
- ✓ Árvore muito grande: `find`, não `**` — o shell monta a lista inteira em memória antes de executar.

## NO KALI

**O Zsh abortar em padrão sem correspondência protege você:** `rm *.pcap` num diretório sem captura nenhuma não faz nada aqui, enquanto no Bash o `rm` receberia o texto literal — atenção ao rodar script de tutorial escrito para Bash. Os **qualificadores de glob** resolvem triagem de coleta: `ls -lh *.pcap(.om[1,3])` mostra as três capturas mais recentes; `ls **/*(.Lm+50)` acha tudo acima de 50 MB na árvore de evidências. `nmap -oA base` gera os três formatos — inspecione com `ls -l base.{nmap,gnmap,xml}`. Antes de editar config de ferramenta: `sudo cp /etc/proxychains4.conf{,.bak}`. E **nome vindo de fuzzing ou download automático contém qualquer coisa** — espaço, aspas, quebra de linha: em lote é sempre `-print0` com `xargs -0`.

GUIA VISUAL 014

# HISTÓRICO E ATALHOS

*Errou o fim de uma linha de noventa caracteres? Não segure a seta nem apague tudo. E, no fim das contas, nem precisava redigitar: `!!` repete.*

014
GUIAS LINUX

## A LINHA É UM EDITOR — E O QUE CADA ATALHO APAGA DEPENDE DO CURSOR

## POR QUE O HISTÓRICO ÀS VEZES SOME

**1 — VOCÊ DIGITA**
A linha entra na sessão assim que você aperta Enter.

→

**2 — LISTA EM MEMÓRIA**
Limite: `HISTSIZE`. É daqui que vêm `Ctrl+R` e a seta para cima.

→

**3 — ARQUIVO, SÓ AO SAIR**
`HISTFILE`, limite `SAVEHIST`. **Com duas abas abertas, a última a fechar sobrescreve** — e apaga o histórico da outra.

→

**4 — A CORREÇÃO**
`setopt INC_APPEND_HISTORY SHARE_HISTORY EXTENDED_HISTORY` (Zsh) · `shopt -s histappend` + `PROMPT_COMMAND='history -a'` (Bash).

## O QUE SE APRENDE UMA VEZ E RENDE PELO RESTO DA VIDA

| ATALHO | FAZ |
|---|---|
| `Alt+B` / `Alt+F` | Anda uma **palavra** para trás / para frente |
| `Alt+.` | **Insere o último argumento** do comando anterior — repita para percorrer os anteriores |
| `Alt+D` | Apaga a palavra *depois* do cursor |
| `Ctrl+T` / `Alt+T` | Troca os dois caracteres / as duas palavras em torno do cursor |
| `Ctrl+_` | Desfaz a última edição |
| `Ctrl+L` | Limpa a tela mantendo a linha em edição |
| `Ctrl+X Ctrl+E` | Abre a linha no `$EDITOR`; ao salvar, o shell executa |
| `Ctrl+R` | **Busca reversa**. De novo = ocorrência anterior · seta = editar · `Ctrl+G` = cancelar |

**`apt update → sudo !!`**
`!!` repete a linha inteira. O idioma mais usado do histórico.

**`mkdir -p a/b/c → cd !$`**
`!$` é o último argumento · `!^` o primeiro · `!*` todos · `!nmap` o último que começou assim.

**`^sysog^syslog`**
Repete o anterior trocando a **primeira** ocorrência do texto. Conserta erro de digitação sem redigitar nada.

**`!!:p`**
**Mostra sem executar**, e deixa a linha na seta para cima. Use antes de todo `!!` que envolva `sudo` ou `rm`.

**`Ctrl+S` congela · `Ctrl+Q` descongela**
O motivo nº 1 de "meu terminal travou": a mão vai no `Ctrl+S` de salvar e o controle de fluxo para a **exibição** — o programa segue rodando. `stty -ixon` no `~/.zshrc` desliga o susto de vez. (`Ctrl+C`, `Ctrl+Z`, `Ctrl+D` e estes dois são do **driver do terminal**, não do shell — por isso funcionam com o programa ocupado.)

**`alias ll='ls -lah'` · `mkcd() { mkdir -p "$1" && cd "$1"; }`**
Alias para o que se repete; **função** quando precisa de argumento — alias não recebe. E `fzf` troca o `Ctrl+R` por busca difusa no histórico inteiro.

**Tudo o que você digita fica em texto puro no histórico — senha inclusive.** `mysql -u root -pSenha123`, `curl -u admin:senha`, `hydra`, `sshpass`: todos deixam rastro em `~/.zsh_history`. Defesas: **um espaço antes do comando** com `HIST_IGNORE_SPACE` ativo, o `HISTIGNORE='*password*:*token*'` do Bash, ou o prompt interativo da própria ferramenta. **A outra ponta também vale:** num sistema que não é seu, o histórico é **evidência** — apagá-lo é ação registrada e, fora de escopo autorizado, crime no Brasil (Lei 12.737/2012). Do lado defensivo, é um dos primeiros arquivos que a resposta a incidente examina.

## RESUMO RÁPIDO

- ✓ `Ctrl+R` é o atalho de maior retorno da ficha; `Alt+.` é o segundo.
- ✓ Sem gravação incremental, a **última aba a fechar vence** e leva o histórico das outras.
- ✓ `HISTTIMEFORMAT` / `EXTENDED_HISTORY` transformam o histórico numa linha do tempo do seu trabalho.
- ✓ **Tab é contextual:** `apt install` completa pacotes, `systemctl` unidades, `git checkout` ramos. Tab duas vezes lista.
- ✓ `watch -d -n 2 'ls -lh coleta/'` repete e **destaca o que mudou**.

## NO KALI

O Zsh do Kali já faz **busca por prefixo nas setas**: digite `nmap` e a seta para cima percorre só os comandos que começam assim — o recurso que mais economiza tempo depois do `Ctrl+R`. E aceita caminho abreviado: `cd /u/s/w` + Tab vira `/usr/share/wordlists`. O `kali-tweaks` **não** configura histórico — é edição manual, e acrescente ao **fim** do `~/.zshrc`. `EXTENDED_HISTORY` vira insumo direto de relatório num teste autorizado: o que rodou, contra quem e quando (combine com `script -T tempo.log sessao.log` para gravar a sessão inteira, saída incluída). E **`Ctrl+C` numa varredura longa perde o progresso**: comece sempre com `-oA` / `-o` — só com saída em arquivo o `--resume` funciona — e prefira `tmux` a deixar a varredura presa à aba.

# TIPOS, INODES E A ÁRVORE

*O disco tem 40% livre e o sistema insiste:* `No space left on device`.

**015**
GUIAS LINUX

## O NOME NÃO FAZ PARTE DO ARQUIVO

### DIRETÓRIO /HOME/KALI

| NOME | INODE |
|---|---|
| relatorio.txt | 917 |
| notas.md | 4432 |
| backup.txt | 917 |

Só nome e número.
Nenhum dado do arquivo aqui.

→

### INODE 917 — OS METADADOS

**inode 917**

tipo: **comum** — permissões: **rw-r--r--**
dono: **kali:kali** — tamanho: **2048**
atime · mtime — ctime · btime
**links: 2 → dois nomes, um arquivo**

Tudo sobre o arquivo — menos o nome.

→

### BLOCOS DE DADOS

`0x1A` `0x1B`
`0x2F` `0x30`

O conteúdo em si,
espalhado pelo disco.

## OS SETE TIPOS — O 1º CARACTERE DO `LS -L`

| – | d | l | c | b | p | s |
|---|---|---|---|---|---|---|
| COMUM | DIRETÓRIO | LINK SIMB. | CARACTERE | BLOCO | PIPE FIFO | SOCKET |
| /etc/passwd | ls -ld /etc | /usr/bin/vi | /dev/null | /dev/sda | mkfifo cano | /run/*.sock |

## NA PRÁTICA

kali@kali: ~

```
$ cp relatorio.txt copia.txt ; ls -li *.txt
 917 -rw-r--r-- 2 kali kali 2048 relatorio.txt
4501 -rw-r--r-- 1 kali kali 2048 copia.txt
# cp cria inode novo; mv mantem o mesmo
$ df -h /   Use% 40%   # bytes: sobra
$ df -i /   IUse% 100%  # inodes: acabaram
$ file -b foto.jpg
ELF 64-bit LSB executable # nao e foto
```

| MARCA | MUDA QUANDO |
|---|---|
| `atime` | o conteúdo é **lido** |
| `mtime` | o **conteúdo** muda |
| `ctime` | o **inode** muda: chmod, dono, nome |
| `btime` | criação — nunca muda |

**Inodes não se acrescentam depois.** A quantidade de um ext4 é fixada em `mkfs.ext4 -N` na formatação — reformatar apaga a partição inteira. Só faça com backup pronto e em disco de laboratório (arquivo-imagem via `losetup` ou VM descartável). Btrfs e XFS alocam inodes dinamicamente e não sofrem disso.

## RESUMO RÁPIDO

- ✓ O nome mora no diretório; tipo, dono, tempos e ponteiros moram no **inode**.
- ✓ Renomear é instantâneo; `cp` cria inode novo, `mv` reaproveita.
- ✓ Apagar arquivo ainda aberto não devolve espaço — o inode só morre com links = 0.
- ✓ `ctime` é *change*, não *creation*: o `touch` não o falsifica.
- ✓ Diretório vazio já tem 2 links (o nome no pai e o `.`); +1 por subdiretório.

## NO KALI

A raiz é **ext4**, de inodes fixos: varredura que grava um arquivo por alvo ( `gobuster` , `nuclei` ) esgota inodes muito antes do espaço — se `df -h` não bate com o erro, rode `df -i` . Em `/usr/share/wordlists` quase tudo é link: use `file -L` . E `stat` é forense — `ctime` de agora com `mtime` de 2015 denuncia arquivo adulterado. Ferramenta que reclama de `/dev/net/tun` ou `/dev/bus/usb` quase nunca está quebrada: é grupo do usuário.

GUIA VISUAL 016

# LINKS SIMBÓLICOS E HARD LINKS

*O `cat` diz No such file or directory — mas o arquivo está ali, dois diretórios adiante.*

016
GUIAS LINUX

## UM APONTA PARA O INODE, O OUTRO PARA UM CAMINHO

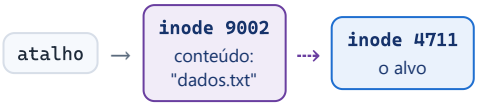
### HARD LINK

`ln dados.txt copia.txt`

- ✓ Dois nomes legítimos: **não existe "o original"**.
- ✓ Sobrevive a renomear/mover o outro nome.
- ✓ Some só quando o contador zera.
- ✗ Não atravessa sistema de arquivos.
- ✗ Não funciona para diretório.

### LINK SIMBÓLICO

`ln -s dados.txt atalho`




- ✓ Arquivo próprio, tipo `l` ; tamanho = nº de caracteres do caminho.
- ✓ Atravessa sistemas de arquivos e aceita diretório.
- ✓ Pode apontar para o que ainda não existe.
- ✗ Alvo saiu do lugar → link pendurado.
- ✗ Não conta no contador de links do alvo.

## AS TRÊS ARMADILHAS DO `LN`

### ORDEM DOS ARGUMENTOS

`ln -s ALVO LINK` — origem primeiro, como no **cp**. Se **LINK** for um diretório existente, o link nasce *dentro* dele com o nome-base do alvo.

### -SFN, NUNCA SÓ -SF

Se o link já aponta para um diretório, `ln -sf` **entra** nele e cria o link lá dentro. O `-n` trata o link como arquivo e substitui de fato.

### RELATIVO × ABSOLUTO

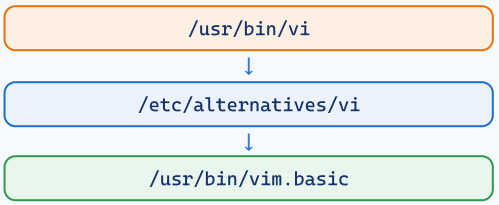
O relativo resolve a partir de **onde o link está**, não de onde você o criou. `ln -sr` calcula o `../` sozinho — ideal em árvore que será movida.

## NA PRÁTICA

kali@kali: ~

```
$ ln dados.txt copia.txt # hard link
$ stat -c '%n i=%i links=%h' dados.txt copia.txt
dados.txt i=4711 links=2
copia.txt i=4711 links=2
$ ln -s dados.txt atalho ; ls -l atalho
lrwxrwxrwx 1 kali kali 9 atalho -> dados.txt
$ find . -xtype l -printf '%p -> %l\n'
./config.yaml -> ../antigo/config.yaml
$ namei -l /usr/bin/vi # cada etapa e a permissao
```

### A CADEIA DO `VI` NO KALI




Só o elo do meio muda em `update-alternatives --config editor` . Nada em `/usr/bin` é tocado — e a atualização do pacote não desfaz a escolha.

`chown -RL` **pode arruinar o sistema.** O `-L` segue os links e leva a mudança de dono para fora da árvore — um link para `/etc` dentro de `/var/www` basta. Use `chown -Rh` (muda o dono do link, não do alvo) e, antes de qualquer `-R` em diretório que você não criou, liste o que há lá dentro: `find /var/www -type l -ls` .

## RESUMO RÁPIDO

- ✓ Hard link = outro nome para o mesmo inode; simbólico = arquivo com um caminho dentro.
- ✓ `-P` não segue · `-L` segue · `-H` só o da linha de comando.
- ✓ `cat` e `cp` seguem o link; `rm` **nunca** toca o alvo; `cp -a` preserva a estrutura.
- ✓ `find -xtype l` acha os quebrados; `namei -l` diz qual etapa do caminho barrou.
- ✓ Laço de link devolve `ELOOP` por volta de 40 níveis — é proteção, não defeito.

## NO KALI

`/usr/share/wordlists` é quase todo feito de links para os diretórios dos pacotes — `namei -l` mostra onde de fato vão parar, e um `apt autoremove` agressivo deixa links pendurados ( `find /usr/share -xtype l` ). Wordlist gigante em outra partição pede **link simbólico**: hard link não cruza sistema de arquivos. E monte a pasta de engajamento com `ln -sr` , para ela sobreviver a ser arquivada e movida.

GUIA VISUAL 017

# LOCALIZANDO ARQUIVOS

*"Quais arquivos de 100 MB mudaram no mês?" e "onde está o `nmap`?" são perguntas de ferramentas diferentes.*

**017**
GUIAS LINUX

## A LINHA DO `FIND` : QUATRO PARTES, ORDEM FIXA

| ONDE | OPÇÕES GLOBAIS | TESTES | AÇÃO |
|---|---|---|---|
| `/var/log` | `-maxdepth 2 -xdev` | `-type f -size +10M` | `-exec ls -lh {} +` |
| ponto de partida da descida | valem para a busca inteira | um **E** com curto-circuito: o 1º que falha descarta | só no que passou em todos os testes |

Teste barato primeiro é otimização real. E a regra de escolha entre as famílias: **locate** para "onde fica um arquivo com esse nome?" · **find** para tamanho, data, dono, permissão · **type** para comandos.

## OS TESTES QUE SE USA DE VERDADE

### NOME E CAMINHO

- `-name "*.conf"` — só o nome-base, com glob
- `-iname "*.CONF"` — ignora maiúsculas
- `-path "*/etc/*.conf"` — casa o caminho inteiro
- `-regex '.*cap0[12].*'` — regex sobre o caminho

### TIPO E TAMANHO

- `-type f · d · l` — arquivo, diretório, link
- `-xtype l` — link **quebrado**
- `-size +100M · -1k` — sem sufixo = blocos de 512 B
- `-empty` — arquivo ou diretório vazio

### TEMPO

- `-mtime -7` — conteúdo mudou em 7 dias
- `-mmin -15` — nos últimos 15 minutos
- `-cmin -60` — o **inode** mudou na última hora
- `-newer ref.txt` — mais recente que outro

### DONO E PERMISSÃO

- `-user kali -group sudo` — dono e grupo
- `-perm 644` — exatamente isso
- `-perm -u+w` — pelo menos esse bit
- `-perm /o+w` — qualquer bit para outros

**O sinal muda tudo:** sem sinal é valor *exato*, `+` é "mais que", `-` é "menos que" — vale para `-size`, `-mtime`, `-links` e `-perm`. E `-exec … +` agrupa numa execução; `-exec … \;` abre um processo por arquivo.

## ONDE ESTÁ UM COMANDO — E NA PRÁTICA

### ORDEM DE RESOLUÇÃO DO SHELL

1. `alias`
2. `função do shell`
3. `builtin`
4. `tabela hash`
5. `varredura do PATH`

✕ `command not found`

**`type -a`** vê as cinco · **`command -v`** idem, forma POSIX para script · **`which`** é programa externo e só enxerga o **`PATH`**: ignora alias, função e builtin — os três lugares onde um comando pode ter sido sequestrado.

kali@kali: ~

```
$ find /var/log -xdev -type f -size +10M \
    -mtime -7 -printf '%s\t%p\n' | sort -rn | head
$ find . -name node_modules -prune -o -type f -print
# -prune so funciona no idioma "-o ... -print"
$ find . -type f -print0 | xargs -0 grep -l ERRO
# -print0/-0 sempre: nome pode ter espaco
$ locate -b '\rockyou.txt'
/usr/share/wordlists/rockyou.txt.gz
$ type -a ls
ls is aliased to `ls --color=auto'
ls is /usr/bin/ls
```

**Busca destrutiva ensaia antes.** Rode a expressão com `-print`, confira a lista e só então troque pela ação. O `-delete` implica `-depth` e muda a ordem de avaliação — combinado com `-prune` já produziu remoção fora do alvo. Sem parênteses, `find . -name a -o -name b -delete` apaga só os `b`. Em expressão complexa, prefira `-print0 | xargs -0 rm --` depois de ler a lista.

## RESUMO RÁPIDO

- ✓ `find` varre a árvore; `locate` consulta um índice que envelhece ( `sudo updatedb` ).
- ✓ `-name` vê o nome-base; `-path` vê o caminho inteiro.
- ✓ `-xdev` é reflexo em busca a partir de `/` : evita `/proc` , `/sys` e discos montados.
- ✓ `type` no terminal, `command -v` em script — `which` não vê alias nem função.
- ✓ Binário recém-instalado "não existe"? `hash -r` limpa a memória de caminhos do shell.

## NO KALI

O `locate` não vem instalado: `sudo apt install plocate && sudo updatedb` — e achar wordlist é caso dele, não de `find / .` Inspeção do **próprio** sistema em uma linha: `find / -xdev -perm -4000 -type f 2>/dev/null` (SUID) e `-perm /o+w` (gravável por qualquer um); em máquina de terceiros, só com autorização escrita. E `type -a ls` denuncia o alias malicioso que o `which` jamais mostraria.

# COMPACTAR E ARQUIVAR

*Um tarbomb despeja duzentos arquivos soltos no seu `~` — e a prevenção custa dois segundos.*

018
GUIAS LINUX

## ARQUIVAR NÃO É COMPRIMIR

### TAR → GZIP

`a.txt` `b.txt` `c.txt` → **`.tar`** um fluxo contínuo → **`.tar.gz`** comprimido **inteiro**

Comprime **melhor**: repetições entre arquivos diferentes são aproveitadas. Em troca, tirar *um* arquivo obriga a descomprimir tudo até ele.

### ZIP

`a.txt` `b.txt` `c.txt` → `a.gz` `b.gz` `c.gz` → **`.zip`** índice + blocos soltos

Comprime **pior**, mas o índice lê ou troca *um* arquivo sem tocar no resto — e é o formato que todo sistema abre.

## AS LETRAS DO TAR

| `tar` | `-c` | `z` | `f` | `pacote.tar.gz` | `dir/` |
|---|---|---|---|---|---|
| arquivador | **c**reate · **t**able lista · e**x**tract extrai | gzip · **j** bzip2 · **J** xz · **a** pelo sufixo | o próximo argumento é o arquivo — sempre por último | o pacote; `-` = stdin/stdout | o que entra. Ao extrair, o tar detecta a compressão sozinho |

| `-tvf` | `-C /destino` | `--strip-components=1` | `--exclude='.git'` | `-p` |
|---|---|---|---|---|
| liste antes de extrair | extrai *em* outro diretório | descarta o nível de topo | corta o que não interessa | preserva permissões |

## QUAL COMPRESSOR — E NA PRÁTICA

| COMPRESSOR | TAXA | QUANDO USAR |
|---|---|---|
| `gzip` .gz | modesta | compatibilidade universal |
| `zstd` .zst | ótima | o dia a dia: rápido dos dois lados |
| `xz` .xz | excelente | comprime 1×, baixado muitas |
| `bzip2` .bz2 | boa | só legado; lento |

kali@kali: ~

```
$ tar -tvf desconhecido.tar.gz | head # LISTE
drwxr-xr-x kali/kali 0 projeto/ # raiz unica: ok
$ tar -xf desconhecido.tar.gz -C /tmp/inspecao
$ zgrep "Failed password" /var/log/auth.log.*.gz
$ zcat rockyou.txt.gz | head -100000 > curta.txt
$ tar -cf - dir/ | zstd -T0 | ssh host \
    'zstd -d | tar -xf - -C /dst'  # sem temporario
$ gzip -t p.gz && sha256sum p.gz > p.gz.sha256
```

**Liste antes de extrair — e não confie no `zip -e`.** Um tarball pode vir sem diretório raiz (*tarbomb*) ou com caminhos `../` que escapam do destino: confira com `tar -tvf` e extraia com `-C` num diretório vazio e descartável. A senha do `zip -e` usa *ZipCrypto*, quebrado por ataque de texto claro conhecido — para proteger de verdade, `7z a -p -mhe=on` (AES-256, nomes também cifrados) ou `gpg -c` sobre o pacote pronto.

## RESUMO RÁPIDO

- ✓ `tar` junta preservando nome, dono e permissão; o compressor só reduz o fluxo.
- ✓ `gzip` **apaga o original** por padrão — use `-k`.
- ✓ `zip -r`: sem o `-r` o pacote sai com o diretório vazio dentro.
- ✓ `zcat`, `zgrep` e `zless` leem o comprimido sem gastar disco (há versão para bz2, xz e zst).
- ✓ `-t` testa integridade em todos eles; `sha256sum` acompanha todo pacote entregue.

## NO KALI

O `rockyou.txt` vem como `.gz` em `/usr/share/wordlists`: descomprimir ali ocupa 130 MB e mexe num diretório do pacote — prefira `zcat … | head` ou `gunzip -k` para uma cópia em `~`. Log rotacionado se lê com `zgrep`, sem descomprimir. Coleta de engajamento sai com `tar -czpf` (o `p` guarda as permissões como evidência) + hash ao lado, e viaja em fluxo por SSH, sem temporário na máquina que não é sua.

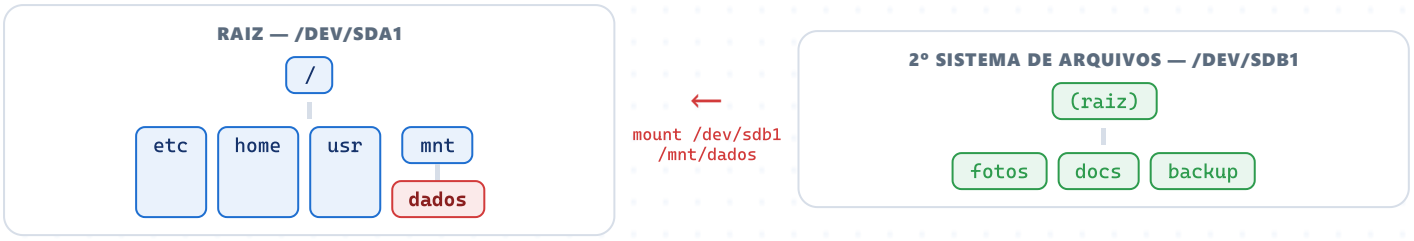
GUIA VISUAL 019

# MONTAGEM E O MOUNT

*A cópia de 2 GB terminou em três segundos, você puxou o pen drive — e do outro lado ele está vazio.*

**019**
GUIAS LINUX

## UMA ÁRVORE SÓ, SEM LETRAS DE UNIDADE




Depois do enxerto, `/mnt/dados/fotos` é caminho válido. O que houvesse em `/mnt/dados` fica **escondido** — não apagado: reaparece no `umount`. E um único caminho pode atravessar três dispositivos sem avisar — por isso existem o `find -xdev` e o `du -x`.

## OS SEIS CAMPOS DO /ETC/FSTAB

| 1 · O QUÊ | 2 · ONDE | 3 · TIPO | 4 · OPÇÕES | 5 | 6 |
|---|---|---|---|---|---|
| `UUID=1a2b-3c4d` | `/mnt/dados` | `ext4` | `defaults,nofail` | `0` | `2` |
| **Sempre UUID** ( `blkid` ): `/dev/sdb1` troca de ordem entre boots | o ponto de montagem, que precisa existir | ext4, vfat, swap, auto | as mesmas do `-o` ; **nofail** impede o boot de travar sem o disco | dump (0) e ordem do fsck: 1 na raiz, 2 no resto, 0 = não checa | |

## VER, MONTAR E DESMONTAR

kali@kali: ~

```
$ lsblk -f # o kernel enxergou o dispositivo?
sdb1  vfat  PENDRIVE  1A2B-3C4D
$ sudo mount -o ro,noexec,nosuid,nodev \
    /dev/sdb1 /mnt/inspecao
$ findmnt /mnt/inspecao # arvore de montagens
$ sudo umount /mnt/inspecao
umount: /mnt/inspecao: target is busy.
$ fuser -vm /mnt/inspecao # quem segura?
# quase sempre e o seu proprio cd: volte com "cd /"
$ sync && sudo umount /mnt/inspecao
```

### HIGIENE DE MÍDIA ALHEIA

**`-o ro`**
protege a evidência de alteração

**`-o noexec,nosuid,nodev`**
protege você do conteúdo dela

**`-o uid=,gid=,umask=`**
FAT/NTFS não guardam permissão Unix

### SEM DISPOSITIVO NENHUM

**`mount -o loop img /mnt`**
arquivo vira disco (`losetup -Pf` se tiver partições)

**`mount --bind /a /b`**
o mesmo diretório em dois lugares, com opções próprias

**`mount -t tmpfs -o size=256M`**
vive na RAM; some ao desligar

**Escrita é confirmada no cache, não no disco.** Antes de puxar qualquer mídia: `sync` e `sudo umount` — se o `umount` falhar por *busy*, não puxe: descubra o processo com `fuser -vm` . O `umount -l` é paliativo e não garante gravação. **E linha errada no `fstab` trava o boot** numa shell de emergência: copie o arquivo antes ( `cp /etc/fstab /etc/fstab.bak` ) e valide com `findmnt --verify` e `mount -a` — nunca reinicie sem testar.

## RESUMO RÁPIDO

- ✓ Montar é enxertar outro sistema de arquivos num diretório da árvore única.
- ✓ `lsblk` parte do hardware; `findmnt` parte da árvore; `/proc/mounts` é a verdade do kernel.
- ✓ "Target is busy" quase sempre é o seu próprio `cd` dentro do ponto.
- ✓ No `fstab`: UUID e `nofail` em toda mídia removível ou de rede.
- ✓ *Bind mount* não é link: funciona em `chroot` e aceita opções próprias — é a base dos contêineres.

## NO KALI

Mídia de terceiros monta **somente-leitura**: `mount -o ro,noexec,nosuid,nodev` — e em perícia nem isso basta: trabalhe sobre uma *cópia*, com `losetup -r`. O Xfce monta automático em `/media/<usuário>` pelo `udisks2`, com as opções dele: desfaça com `udisksctl unmount -b` e remonte à mão. `lsblk -f` **antes** de qualquer `mount`, `dd` ou `mkfs` — trocar `sda` por `sdb` é o erro que destrói o disco do próprio analista.

GUIA VISUAL 020

# PERMISSÕES DE ARQUIVO

*Quem pode ler, escrever e executar — e por que `chmod 777` não resolve.*

020
GUIAS LINUX

## ANATOMIA DA SAÍDA DO `LS -L`

| `-` | `r` | `w` | `-` | `r` | `-` | `-` | `r` | `-` | `-` |
|---|---|---|---|---|---|---|---|---|---|
| TIPO | DONO | | | GRUPO | | | OUTROS | | |
| arquivo | 6 | | | 4 | | | 4 | | |

`r` = ler (4) `w` = escrever (2) `x` = executar (1) → `-rw-r--r-- = 644`

## A REGRA QUE QUASE NINGUÉM APRENDE

### SÓ UMA CLASSE SE APLICA

O Linux checa na ordem: é o **dono**? senão, está no **grupo**? senão, **outros**. Para na primeira que casa.

### EM ARQUIVO

**r** lê o conteúdo · **w** altera o conteúdo · **x** executa como programa.

### EM DIRETÓRIO

**r** lista nomes · **w** cria/remove itens · **x** *atravessa* o diretório. Sem **x**, nada dentro é acessível.

## AS DUAS NOTAÇÕES DO `CHMOD`

### SIMBÓLICA

- ✓ Alvo: `u` dono · `g` grupo · `o` outros · `a` todos
- ✓ Operador: `+` adiciona · `-` remove · `=` define
- ✓ Ex.: `chmod u+x script.sh`

### OCTAL

- ✓ Soma dos bits por classe: **r=4, w=2, x=1**
- ✓ Três dígitos: dono · grupo · outros
- ✓ Ex.: `chmod 755 script.sh`

## NA PRÁTICA

```
kali@kali: ~

$ ls -l relatorio.txt
-rw-r--r-- 1 kali kali 2913 ago  1 09:14 relatorio.txt
$ chmod 640 relatorio.txt   # dono lê/escreve, grupo lê, outros nada
$ chmod u+x deploy.sh       # torna executável só para o dono
$ stat -c '%a %A %U:%G' deploy.sh
744 -rwxr--r-- kali:kali
```

| OCTAL | SIMBÓLICO | USO TÍPICO |
|---|---|---|
| `644` | `-rw-r--r--` | Arquivo comum (todos leem, só o dono edita) |
| `600` | `-rw-------` | Arquivo privado — chave SSH, segredo |
| `755` | `-rwxr-xr-x` | Script/binário e diretórios navegáveis |
| `700` | `-rwx------` | Diretório pessoal, script só seu ( `~/.ssh` ) |

**chmod 777 não é solução de nada.** Abre leitura, escrita e execução para *qualquer* usuário da máquina. Se o acesso falha, o culpado quase sempre é a falta de `x` em um diretório do caminho — não a permissão do arquivo.

## RESUMO RÁPIDO

- ✓ Três classes (dono/grupo/outros), três bits (rwx = 4/2/1).
- ✓ Só a primeira classe que casa é aplicada.
- ✓ Em diretório, `x` é o que deixa atravessar.
- ✓ `umask` define a permissão dos arquivos novos.

## NO KALI

O Kali roda muita coisa como **root**, que ignora os bits de permissão — fácil esquecer que num alvo com usuário normal `644` importa. Guarde chaves e wordlists sensíveis em `700` / `600`.

GUIA VISUAL 021

# USUÁRIOS, GRUPOS E UID

*Para o kernel, usuário é um **número** — o nome é só tradução feita em arquivo de texto.*

**021**
GUIAS LINUX

## UMA LINHA DE `/ETC/PASSWD`, CAMPO A CAMPO

| 1 · NOME | 2 · SENHA | 3 · UID | 4 · GID | 5 · GECOS | 6 · HOME | 7 · SHELL |
|---|---|---|---|---|---|---|
| `kali` | `x` | `1000` | `1000` | `Kali Live User` | `/home/kali` | `/usr/bin/zsh` |
| tradução | está no shadow | o kernel usa | grupo primário | nome real (chfn) | diretório pessoal | nologin = sem login |

## AS TRÊS BASES DE CONTAS

### /ETC/PASSWD · 644

Catálogo público de contas, 7 campos. **Legível por todos** por necessidade — sem ele nada traduz UID em nome. Não guarda senha desde os anos 90.

### /ETC/SHADOW · 640

O hash, no formato **$id$sal$hash** ($y$ yescrypt no Kali, $6$ SHA-512), mais a política de expiração. **!** = conta bloqueada · campo vazio = **sem senha**.

### /ETC/GROUP · 4 CAMPOS

`sudo:x:27:kali` — nome, senha obsoleta, GID e a lista dos membros **suplementares**. Quem tem o grupo como primário **não** aparece aqui.

## GRUPO PRIMÁRIO × GRUPOS SUPLEMENTARES

### PRIMÁRIO — SÓ UM

- ✓ Vem do **campo 4** de `/etc/passwd`.
- ✓ É o dono de **tudo que você cria**: aparece no `ls -l` de cada arquivo novo.
- ✓ Trocar na hora: `newgrp outro`.

### SUPLEMENTARES — VÁRIOS

- ✓ Vêm das linhas de `/etc/group`; só **concedem** acesso.
- ✓ São lidos no **login** — grupo novo não vale na sessão aberta.
- ✓ Adicionar: `usermod -aG docker kali` e refazer o login.

## NA PRÁTICA

```
kali@kali: ~
$ id
uid=1000(kali) gid=1000(kali) grupos=1000(kali),4(adm),27(sudo)
$ getent passwd kali   # getent pergunta ao NSS; grep so le o arquivo
kali:x:1000:1000:Kali Live User:/home/kali:/usr/bin/zsh
$ id -Gn               # grupos DESTE processo
$ groups kali          # o que os arquivos dizem — se diferir, refaca o login
```

| FAIXA DE UID | TIPO DE CONTA | EXEMPLOS E PARA QUE SERVE |
|---|---|---|
| `0` | Superusuário | `root` — o kernel testa o número, não o nome |
| `1–999` | Sistema | `daemon`, `_apt`, `www-data` — limitam o estrago de uma falha no serviço |
| `1000–59999` | Humanos | `kali` e quem você criar (limites em `/etc/login.defs`) |
| `65534` | `nobody` | Sem privilégio nenhum: NFS, sandboxes |

**Dois achados de auditoria:** `awk -F: '$3==0' /etc/passwd` tem de devolver **uma só linha** — segunda conta com UID 0 é porta dos fundos clássica; e `sudo awk -F: '$2==""' /etc/shadow` não pode devolver nada. **Cuidado com** `usermod -G` **sem o** `-a`**:** ele **substitui** a lista de grupos e tira você do `sudo`.

## RESUMO RÁPIDO

- ✓ UID e GID decidem o acesso; nome e shell são conveniência.
- ✓ Restaurou backup e o dono mudou? Os números não bateram.
- ✓ Suplementares entram no **login** — daí o *permission denied* do `docker`.
- ✓ Edite com `vipw` / `vigr` e confira com `pwck -r`.

## NO KALI

Desde 2020 o Kali **não usa root por padrão**: cria o usuário `kali` (UID 1000, senha `kali`, grupo `sudo`) — trocar a senha é o 1º comando na VM nova. Grupo `docker` **equivale a root**: quem fala com o daemon monta a raiz num contêiner. Ler `/etc/passwd` num alvo é reconhecimento (revela `postgres`, `www-data`) — só com autorização escrita (Lei 12.737/2012).

GUIA VISUAL 022

# ROOT, SUDO E ELEVAÇÃO

*Colar `sudo` na frente resolve o Permission denied — e tira a rede de proteção.*

022
GUIAS LINUX

### ROOT É O UID 0

O kernel **pula quase toda** verificação de permissão: lê e escreve qualquer arquivo, mata qualquer processo, abre porta < 1024.

### SÓ 3 LIMITES — REMOVÍVEIS

Precisa de **algum** bit x para executar · `chattr +i` trava o arquivo · SELinux/AppArmor impõem política acima. O root desfaz os três.

### MENOR PRIVILÉGIO

Rode como usuário comum e eleve **só o comando**. Como root, um `rm` com um caractere a mais não pergunta nada.

## ANATOMIA DE UMA REGRA DO `/ETC/SUDOERS`

| QUEM | EM QUAIS MÁQUINAS | COMO QUEM | QUAIS COMANDOS |
|---|---|---|---|
| `%sudo` | `ALL` | `(ALL:ALL)` | `ALL` |
| O **%** marca grupo. No Kali, `kali` já é membro. | Vale em qualquer host que receba este arquivo. | Pode virar qualquer usuário e qualquer grupo. | Trocar por uma lista de binários é o coração do menor privilégio. |

## OS DOIS CAMINHOS PARA ELEVAR

### SU — O INTERRUPTOR

- ✓ Pede a **senha do root** e abre um shell **persistente**: tudo é root até o `exit`.
- ✓ Todo admin precisa saber a mesma senha, que nunca muda.
- ✓ O log diz só "alguém virou root". No Debian/Kali a senha de root vem **bloqueada** — `su -` nem funciona.

### SUDO — A CONCESSÃO

- ✓ Pede a **sua** senha, consulta a política e eleva **um comando**.
- ✓ Cache de **15 min** por terminal — `sudo -k` invalida na hora.
- ✓ `-i` = ambiente limpo de root ( `/root` ) · `-s` = poder de root com o **seu** ambiente.

## NA PRÁTICA

```
kali@kali: ~
$ sudo -l                    # o que EU posso rodar, sem executar nada
$ sudo -u postgres psql      # sudo nao e so "virar root"
$ sudo visudo -f /etc/sudoers.d/local   # valida antes de salvar
$ sudo journalctl _COMM=sudo --no-pager | tail -1
sudo: kali : TTY=pts/1 ; PWD=/home/kali ; USER=root ; COMMAND=/usr/bin/apt update
```

| RESTRINGIR O SUDO A... | ...NÃO RESTRINGE NADA, PORQUE |
|---|---|
| `vim` , `less` , `man` | `:!bash` / `!bash` abre um shell de root de dentro do programa |
| `find` | `find . -exec bash \;` executa o que quiser |
| `awk` , `perl` , `python` | rodam código arbitrário por definição |
| `cp` , `tee` , `dd` , `tar` | sobrescrevem `/etc/sudoers` ou `/etc/passwd` |

**Nunca edite `/etc/sudoers` com editor comum** — uma vírgula fora do lugar tranca o acesso administrativo da máquina inteira. Só `visudo`, que valida a sintaxe antes de salvar. E `NOPASSWD:` para um comando que escapa (tabela acima) é **root completo sem senha**.

## RESUMO RÁPIDO

- ✓ Root é o número **0**, não o nome — e ignora os bits `rwx`.
- ✓ `su` abre shell persistente; `sudo` eleva um comando e **registra** quem foi.
- ✓ Política em `/etc/sudoers.d/`, sempre via `visudo`.
- ✓ Regra restrita só vale se nenhum comando dela rodar outro programa.

## NO KALI

Root vem **bloqueado** — mantenha assim; onde um manual antigo pede `su -`, use `sudo -i`. `nmap -sS`, `tcpdump` e `aircrack-ng` pedem root porque precisam de socket *raw*: eleve **por comando**, não abra a sessão inteira como root. Em teste autorizado, `sudo -l` é o 1º reconhecimento — a maioria das escaladas nasce de um `NOPASSWD` mal escrito (ver GTFOBins). Fora de escopo escrito, é crime (Lei 12.737/2012).

GUIA VISUAL 023

# SUID, SGID E STICKY BIT

*Você não lê `/etc/shadow` — mas troca sua senha. O `s` do `passwd` explica tudo.*

**023**
GUIAS LINUX

## O QUARTO DÍGITO OCTAL, O QUE QUASE NINGUÉM ESCREVE

MODO COMPLETO

**4 7 5 5** →

O 1º DÍGITO SOMA

- SUID = 4
- SGID = 2
- sticky = 1

→ NO `LS -L` ELE OCUPA O LUGAR DO X

**- r w s r - x r - x**

## OS TRÊS BITS, UM A UM

### SUID (4) — EM EXECUTÁVEL

O processo ganha o **UID efetivo do dono do arquivo**. O UID **real** continua o seu (para log e sinais); é o **efetivo** que o kernel checa. Por isso `passwd` (dono root) grava no `shadow`.

### SGID (2) — EM DIRETÓRIO

Todo arquivo criado ali **herda o grupo do diretório** em vez do grupo primário do autor — e subdiretórios herdam o próprio bit. É a pasta de equipe: `chgrp projeto` + `chmod 2775` + `umask 002`.

### STICKY (1) — EM DIRETÓRIO

Com `w` num diretório qualquer um apaga arquivo alheio. O sticky restringe: **só o dono do arquivo** (ou do diretório, ou root) remove. É o `t` de `/tmp` — `1777`, `drwxrwxrwt`.

## NA PRÁTICA

```
kali@kali: ~
$ find / -xdev -perm -4000 -type f 2>/dev/null   # inventario de SUID
/usr/bin/passwd  /usr/bin/sudo  /usr/bin/su  /usr/bin/mount  /usr/bin/pkexec
$ chmod 2775 /srv/projeto  &&  ls -ld /srv/projeto
drwxrwsr-x 2 root projeto 4096 ago  1 09:14 /srv/projeto
$ ls -ld /tmp
drwxrwxrwt 20 root root 4096 ago  1 09:14 /tmp
```

| NO LS -L | BIT | COMO LER |
|---|---|---|
| `s` no dono | SUID | Minúscula = o `x` também está aceso. Roda como o dono. |
| `S` no dono | SUID | Maiúscula = bit ligado **sem** `x`. Quase sempre um `chmod` incompleto. |
| `s` no grupo | SGID | Executável roda com o grupo do arquivo; diretório impõe o grupo dele. |
| `t` em outros | sticky | Só o dono apaga. Em arquivo, hoje o Linux ignora (herança do Unix antigo). |

**SUID é a maior fonte de escalada de privilégio local.** Nunca dê `chmod u+s` a um binário seu para fugir do `sudo`: você passa a garantir que ele é seguro contra *qualquer* entrada. Em **script o SUID é ignorado** pelo kernel (janela de corrida no shebang) — para privilégio pontual use regra restrita de `sudo` ou *capabilities*.

## RESUMO RÁPIDO

- ✓ Quarto dígito: **4** SUID · **2** SGID · **1** sticky.
- ✓ SUID = UID **efetivo** do dono; o real segue sendo o seu.
- ✓ SGID em diretório = herança de grupo, propagada aos subdiretórios.
- ✓ Sticky ( `1777` ) = só o dono apaga — é o que protege `/tmp` .
- ✓ `S` / `T` maiúsculo = especial ligado sem `x` : revise o `chmod` .

## NO KALI

Guarde uma **linha de base** logo após instalar: `find / -xdev -perm -4000 -type f 2>/dev/null | sort` . Um SUID que apareceu sozinho depois é dos sinais mais claros de comprometimento. Do lado ofensivo, essa mesma varredura (LinPEAS, GTFOBins) é o 1º passo da escalada local em alvo **autorizado** — `pkexec` já teve falhas graves, mantenha o `polkit` atualizado. Sem escopo escrito, é crime (Lei 12.737/2012).

GUIA VISUAL 024

# ACLS E ATRIBUTOS

*A auditora precisa ler **só este** diretório — e não cabe em dono, grupo ou outros.*

024
GUIAS LINUX

## O QUE O `GETFACL` MOSTRA

MODELO CLÁSSICO — 3 CLASSES

```
user::rwx
group::r-x
other::---
```

Os mesmos 9 bits do `ls -l`, escritos por extenso. Um usuário, um grupo, o resto do mundo.

→

DEPOIS DE `SETFACL -M U:MARCIA:RX`

```
user::rwx
user:marcia:r-x
group::r-x
mask::r-x
other::---
```

Entrada **nomeada** (verde) + a **máscara** (vermelho). E o modo no `ls -l` ganha um `+`.

### A MÁSCARA É UM TETO

A permissão real é a **interseção** entre o que a entrada pede e a máscara. `getfacl` avisa com `#effective:r--`.

### CHMOD MEXE NA MÁSCARA

Num arquivo com ACL, `chmod g-w` não altera o `group::` — baixa a **máscara**. Acesso sumiu depois de um `chmod`? É ela.

### ACL PADRÃO = HERANÇA

`setfacl -d -m u:marcia:rx dir/` não muda o diretório: define o que **os arquivos novos** ali dentro já nascem tendo. Casa com o SGID.

## UMA CAMADA ABAIXO: ATRIBUTOS COM `CHATTR`

### +I — IMUTÁVEL

- ✓ Não altera, não renomeia, não apaga, não ganha hard link — **nem pelo root**.
- ✓ Trava contra **erro** e script cego, não contra um root decidido: ele faz `chattr -i` antes.
- ✓ Uso típico: `/etc/resolv.conf` que insiste em ser reescrito.

### +A — APPEND-ONLY

- ✓ Só aceita acréscimo ao fim: `>>` passa, `>` (trunca) é negado.
- ✓ Atributo natural de log: dá para somar linhas, não para apagar o rastro.
- ✓ Leia com `lsattr`; ACL e SELinux são, por baixo, **xattrs** no inode.

## NA PRÁTICA

```
kali@kali: ~
$ setfacl -R -m u:marcia:rX relatorios/   # X: x so em diretorio
$ ls -ld relatorios/
drwxr-x---+ 2 kali analistas 4096 ago  1 09:14 relatorios/
$ getfacl relatorios/ | grep -E 'marcia|mask'
user:marcia:rwx    #effective:r-x
$ setfacl -x u:marcia relatorios/    # -b remove TODAS as ACLs
```

| COPIAR / ARQUIVAR | PRESERVA ACL E XATTR? |
|---|---|
| `cp origem destino` | Não — as ACLs somem sem aviso |
| `cp -a` · `cp --preserve=all` | Sim: ACL, xattr, dono e tempos |
| `rsync -aAX origem/ destino/` | Sim — `-A` ACL, `-X` xattr |

| COPIAR / ARQUIVAR | PRESERVA ACL E XATTR? |
|---|---|
| `tar --acls --xattrs -cf bkp.tar dir/` | Sim, e o mesmo par de flags na extração |

**Backup silenciosamente errado:** `cp` sem `-a` restaura permissões que nunca existiram. E antes de um `rm -rf` numa árvore com atributos, `sudo chattr -R -i` primeiro — um arquivo `+i` esquecido faz a remoção falhar pela metade, deixando lixo com permissão travada.

## RESUMO RÁPIDO

- ✓ ACL concede a um usuário/grupo **nomeado**, sem mexer nas 3 classes.
- ✓ `+` no `ls -l` = há ACL que o `ls` não mostra.
- ✓ Máscara = teto; `chmod` a altera e pode apagar acessos.
- ✓ `-d` dá herança; `chattr +i/+a` ficam acima das permissões.

## NO KALI

Compartilhar coleta com um colega sem alargar o grupo: `setfacl -R -m u:colega:rX evidencias/` mais `-d` para o que vier depois. `chattr +a` no log de atividades do teste protege a cadeia de custódia. E o inverso vale como **checklist defensivo**: atacante usa `+i` para travar backdoor — `lsattr -R /etc /tmp 2>/dev/null | grep -E '^....i'` denuncia. Arquive com `tar --acls --xattrs` .

GUIA VISUAL 025

# CONTAS, SENHAS E O PAM

*A senha está certa, a conta existe — e o login é recusado. Quem barrou não foi o `sshd`.*

025
GUIAS LINUX

## O PAM: OS QUATRO GRUPOS QUE TODO LOGIN ATRAVESSA

**1 — AUTH**
Prova **quem** você é: senha, chave, token. É só esta etapa que olha a senha.

→

**2 — ACCOUNT**
A conta **pode** ser usada agora? Expiração, horário, bloqueio. **É aqui que a conta expirada cai.**

→

**3 — PASSWORD**
Governa a **troca** de senha: tamanho mínimo, complexidade, histórico (`pam_pwquality`).

→

**4 — SESSION**
Prepara e encerra: monta o `/home`, aplica limites, escreve no log.

## AS DUAS CAMADAS PARA CRIAR UMA CONTA

### USERADD — BAIXO NÍVEL

- ✓ Padrão de todo Linux. Sozinho escreve **só a linha** em `/etc/passwd`.
- ✓ Sem `-m` não há `/home` nem cópia do `/etc/skel`; sem senha, não há login — daí o "não funcionou".
- ✓ Previsível: é a forma certa em **script** de provisionamento.

### ADDUSER — ALTO NÍVEL

- ✓ Script Perl do Debian/Kali que orquestra o `useradd` com defaults sensatos.
- ✓ Cria e povoa o `/home`, pede a senha, resolve o grupo primário.
- ✓ É o que você quer **na mão**. Para remover: `deluser --backup --remove-home`.

## NA PRÁTICA

```
kali@kali: ~
$ sudo useradd -m -s /bin/bash -c "Joao Silva" -G sudo joao
$ sudo passwd -S joao      # P valida | L bloqueada | NP sem senha
joao P 08/01/2026 7 90 14 -1
$ sudo chage -l joao | grep -i expira   # o que o passwd -S esconde
Conta expira: dez 31, 2026
$ sudo lastb -n 5          # logins FALHOS: enxurrada = forca bruta
```

| VOCÊ QUER... | COMANDO | EFEITO |
|---|---|---|
| Forçar troca no próximo login | `chage -d 0 joao` · `passwd -e joao` | Expira a **senha**; a conta continua utilizável |
| Senha válida por 90 dias, aviso em 14 | `chage -M 90 -W 14 joao` | Política de envelhecimento da senha |
| Desativar o acesso numa data | `chage -E 2026-12-31 joao` | Expira a **conta** — barrada em `account`. `-E -1` desfaz |
| Suspender agora / liberar | `usermod -L joao` / `-U` | Põe e tira o `!` do hash. **Não** impede login por chave SSH |

**Dois comandos que trancam o acesso.** `usermod -G docker joao` **sem o** `-a` substitui a lista inteira e tira o `joao` do `sudo` (para remover de um grupo, `gpasswd -d joao docker`). E um erro em `/etc/pam.d/` tranca o login de **todos, inclusive o root**: edite sempre com uma **sessão de root aberta em outro terminal** e teste numa sessão nova antes de fechá-la.

## RESUMO RÁPIDO

- ✓ `useradd -m` em script · `adduser` na mão; `/etc/skel` é o molde do `/home`.
- ✓ Expirar **senha** ≠ expirar **conta** — só o `chage -l` mostra as duas.
- ✓ `userdel` preserva o `/home`; `-r` apaga. Sobram órfãos: `find / -xdev -nouser`.
- ✓ Quem autentica é o PAM, não o serviço: `pam_pwquality` (senha forte) e `pam_faillock` (`faillock --reset`).

## NO KALI

Na VM, `sudo adduser analista` resolve; deixe o `useradd` explícito para provisionar. `last`, `sudo lastb` e `lastlog` são a triagem de acesso — e `/var/log/wtmp` truncado é, por si só, indício de que alguém limpou rastro. **PAM é vetor de persistência**: módulo adulterado em `/lib/*/security/` aceita senha-mestra — confira data e hash contra o pacote limpo. Auditar o próprio sistema ou alvo autorizado é defesa; mexer em PAM alheio é crime (Lei 12.737/2012).

GUIA VISUAL 026

# ROOT NO KALI, COM JUÍZO

*Até 2019 o Kali abria o terminal como root. Desde a 2020.1, não — e isso muda o seu método.*

026
GUIAS LINUX

## O FLUXO DE MENOR PRIVILÉGIO

**USUÁRIO KALI**
UID 1000, no grupo `sudo`.
A conta root fica bloqueada (`!` no shadow): `su -` nem funciona.

→ **SEM PRIVILÉGIO — O DIA INTEIRO**
Navegar, ler PDF de relatório, editar, `git`, escrever scripts. Uma falha no navegador alcança só o que o `kali` alcança.

→ **ELEVAR UM COMANDO**
`sudo nmap -sS`, `sudo tcpdump`. Cada uso vira linha de log com `TTY`, `PWD` e `COMMAND`.

→ **ELEVAR A SESSÃO INTEIRA — EVITE**
`sudo -i` devolve os 3 defeitos do modelo antigo: erro fatal, tudo com poder máximo, nada auditável. Se precisar, use e feche **CEDO**.

### CAPABILITY > SUDO

A ferramenta não quer root: quer `CAP_NET_RAW`. O `dumpcap` já vem com a capability e o grupo `wireshark` — captura **sem** sudo. Imite esse modelo.

### NUNCA CURL | SUDO BASH

Executa como root um código que você não leu, de uma origem que pode ter caído. Baixe → `less instala.sh` → só então `sudo bash`.

### TRAVE E ISOLE

`sudo -k` ao se afastar (o cache de 15 min é root para quem sentar ali). Ferramenta duvidosa em contêiner; alvo e malware em **VM com snapshot**.

## POR QUE A FERRAMENTA PEDE PRIVILÉGIO

| FERRAMENTA | RECURSO QUE O KERNEL RESERVA |
|---|---|
| `nmap -sS`, `-sU`, `masscan` | *Raw socket* para forjar pacotes SYN/UDP (`CAP_NET_RAW`) |
| `tcpdump`, `wireshark` | Modo promíscuo e captura na interface |
| `airmon-ng`, `aircrack-ng` | Modo monitor no adaptador Wi-Fi |
| `ettercap`, `responder`, `arpspoof` | Injeção na camada de enlace (`CAP_NET_ADMIN`) |
| `losetup`, montar imagem | Manipular dispositivos de bloco |

## OS PRIMEIROS CINCO MINUTOS DE UMA VM KALI

kali@kali: ~

```
$ passwd                          # kali:kali e publico. Passo zero.
$ sudo apt update && sudo apt full-upgrade -y
$ sudo passwd -S root             # tem de dizer L (locked) — deixe assim
$ awk -F: '$3==0 {print $1}' /etc/passwd    # so pode sair "root"
$ find / -xdev -perm -4000 -type f 2>/dev/null | sort > ~/suid-baseline.txt
```

**Antes de `dd`, `mkfs`, `wipefs` ou `cryptsetup luksFormat`:** como root não há confirmação — um `of=` ou `/dev/sdX` trocado destrói o disco. Ensaie num arquivo-imagem: `truncate -s 1G /tmp/teste.img` → `sudo losetup -fP /tmp/teste.img` → pratique em `/dev/loopN` → `sudo losetup -d`. E confirme o alvo com `lsblk` **imediatamente antes** do comando.

## RESUMO RÁPIDO

- ✓ Base sem privilégio; eleve **o comando**, nunca a sessão.
- ✓ Root bloqueado é o padrão do Kali — mantenha ( `sudo passwd -l root` ).
- ✓ Privilégio mínimo (*capability*, grupo) em vez de root inteiro.
- ✓ Destrutivo só depois de ensaio em `losetup` e conferência com `lsblk` .
- ✓ Baixe, **leia**, depois execute — nunca canalize script remoto para um shell privilegiado.

## NO KALI

Capturar tráfego sem `sudo` : `sudo usermod -aG wireshark kali` + **novo login** (grupo suplementar entra no login). Cuidado: entrar no grupo `docker` é **equivalente a root** no hospedeiro — o contêiner isola a ferramenta, não quem fala com o daemon. No **WSL** o `sudo` pode vir `NOPASSWD` : cheque com `sudo -l` . E `sudo -l` numa conta de alvo **autorizado** é o 1º passo da escalada — sem escopo escrito, é crime (Lei 12.737/2012).

GUIA VISUAL 027

# O QUE É UM PROCESSO

*PID, estados e a árvore — o que você realmente procura quando a máquina engasga.*

027
GUIAS LINUX

## COMO UM PROCESSO NASCE: FORK + EXEC

**bash · 2140**
O shell vai rodar
`ls`

fork() →

**cópia · 2311**
Clone do shell; o pai espera em
`wait()`

exec() →

**ls · 2311**
Troca a imagem,
**mantém o PID**

exit →

**SIGCHLD**
Pai colhe o código de saída

## A ÁRVORE ENRAIZADA NO PID 1 & O /PROC QUE A EXPÕE

```
systemd (1)
├─ sshd (729)
├─ cron (645)
└─ lightdm (701)
   └─ xfce4-terminal (1330)
      └─ bash (1342)
         └─ find (1501)
```

**PPID** é o pai; `pstree -sp PID` sobe a linhagem até o 1. **Shell filho de servidor web** = alerta clássico de invasão.

| /PROC/<PID>/ — A TABELA COMO ARQUIVOS | |
|---|---|
| `status` | Estado, PPID, UIDs, memória, threads |
| `cmdline` | Linha de comando (campos com byte nulo) |
| `exe` | Link para o binário real — não mente |
| `cwd` | Diretório de trabalho atual |
| `fd/` | Um link por descritor aberto |
| `task/` | Um subdiretório por *thread* (TID) |
| `environ` | Ambiente — inclui segredos vazados |

## A COLUNA STAT : EM QUE ESTADO ELE ESTÁ

**R** — RODANDO
Na CPU ou pronto na fila.

**S** — DORMINDO
Espera evento. O normal.

**D** — E/S TRAVADA
Ignora sinais — nem **kill -9**.

**T** — PARADO
Levou **SIGSTOP/SIGTSTP**.

**Z** — ZUMBI
Morreu; o pai não colheu.

```
kali@kali: ~
$ pgrep -a sshd            # PID + linha de comando; nunca casa consigo mesmo
729 /usr/sbin/sshd -D
$ ps -p $$ -o pid,ppid,stat,comm
  PID  PPID STAT COMMAND
 1342  1330 Ss   bash
$ ls -l /proc/1501/exe     # "(deleted)" aqui = binário apagado após subir
lrwxrwxrwx 1 kali kali 0 ago 31 10:22 /proc/1501/exe -> /usr/bin/find
$ tr '\0' '\n' < /proc/1501/cmdline   # separador é byte nulo, não espaço
```

**Cuidado com `fuser -k /caminho`:** ele sinaliza *todos* os processos que tocam o caminho — num diretório movimentado (ou por engano em `/`) derruba a sessão inteira. Rode primeiro sem `-k`, leia a lista, depois decida.

## RESUMO RÁPIDO

- ✓ Programa é arquivo em disco; **processo** é uma execução viva com PID, dono e memória próprios.
- ✓ `fork` duplica, `exec` troca a imagem **mantendo o PID** — por isso o shell sobrevive ao comando.
- ✓ Em **D** nem `kill -9` funciona: resolva a E/S. **Z** já morreu — mate o *pai*.
- ✓ Órfão não morre: é adotado pelo PID 1 (passa a exibir PPID 1).
- ✓ `pgrep` / `pstree` por nome; `lsof` / `fuser` por arquivo ou porta.

## NO KALI

`pstree -pa` é o primeiro reflexo de triagem: responde *quem chamou isso*. E **nunca passe segredo em argumento** — `mysql -psenha` fica visível em `/proc/<PID>/cmdline` para qualquer usuário, que é exatamente o que *pspy* e *LinPEAS* colhem. Uso só em laboratório próprio ou com autorização escrita (Lei 12.737/2012).

Ferramenta do Kali cria árvore profunda de propósito (NSE do `nmap`, workers do `metasploit`): antes de dizer que um processo enlouqueceu, olhe as **threads** com `ps -L` ou `H` no htop.

GUIA VISUAL 028

# MONITORAMENTO PS · TOP · HTOP

*O servidor está lento: a tela se enche de números — três deles quase sempre são lidos errado.*

028
GUIAS LINUX

### PS — A FOTOGRAFIA

Estado no instante em que rodou. É a ferramenta de **script, log e evidência**. `ps aux` (BSD) ou `ps -ef` (UNIX).

### TOP — O FILME

Redesenha a cada 3 s e revela **tendência**. Está em toda máquina: é o que se usa por SSH sem instalar nada.

### HTOP — O FILME COM INTERFACE

Cores, rolagem, árvore, filtro e ação direta sobre o processo. Investigação interativa começa aqui.

## ANATOMIA DE UMA LINHA DO `PS AUX`

| USER | PID | %CPU | %MEM | VSZ | RSS | TTY | STAT | COMMAND |
|---|---|---|---|---|---|---|---|---|
| kali | 2201 | 98.3 | 4.7 | 921648 | 385204 | pts/0 | R+ | ffmpeg -i… |

**%CPU é média de vida**, não instantâneo: quem correu a 100% por 10 s e dormiu 1 h aparece baixíssimo. Acima de 100% = várias threads.

**VSZ mente, RSS quase não.** VSZ é espaço reservado (infla fácil); RSS é o que está na RAM — mas conta bibliotecas compartilhadas em cada processo. Privado real: `Pss` em `smaps_rollup`.

**STAT tem sufixos:** `+` primeiro plano · `s` líder de sessão · `l` multithread · `<` prioridade alta. **TTY =** `?` é daemon: sem terminal, não leva SIGHUP.

top — o cabeçalho é o diagnóstico

```
top - 10:07:52 up 2 days,  2 users,  load average: 1.42, 0.98, 0.61
Tarefas: 312 total, 2 executando, 310 dormindo, 1 zumbi
%Cpu(s): 12,4 us  3,1 sy  78,2 id  6,0 wa  0,0 st
MiB Mem : 7852 total, 412 free, 3120 used, 4319 buff/cache
MiB Swap: 2048 total, 2048 free, 0 used.  4210 avail Mem
$ ps -eo pid,comm,pcpu,rss,nlwp,etime,wchan --sort=-rss | head
```

## OS CAMPOS QUE ENGANAM

- ✓ **wa alto com us baixo** = gargalo de **disco**, não de CPU. Troque de SSD, não de processador.
- ✓ **st** só existe em VM: é CPU roubada pelo hipervisor — vizinho barulhento, você não resolve de dentro.
- ✓ **buff/cache não é memória perdida** — é cache devolvido na hora. Olhe **avail Mem**, nunca **free**.
- ✓ **Load average** conta prontos *e* travados em E/S. Compare com o nº de núcleos.
- ✓ **wchan** mostra em que função do kernel o processo em **D** dorme — o atalho para a causa raiz.

## TECLAS QUE VALEM A PENA

**top:** `P` CPU · `M` memória · `1` por núcleo · `H` threads · `V` árvore · `k` sinal · `W` salva o layout

**htop:** `F4` filtro (melhor que a busca) · `F5` árvore · `F6` ordenar · `F9` sinal · `Espaço` marca vários · `l` lsof · `s` strace

**Barras do htop:** verde = usuário · vermelho = kernel · azul = *nice* positivo · laranja = E/S.

**Alternativas:** `btop` (gráficos históricos na tela) · `glances` (painel único, modo cliente-servidor) · `atop` (grava amostras em disco).

**Sem tela:** `top -b -n 1` para script · `pidstat -u 1 3` por processo · `watch -d -n1 'free -h'` destaca o que mudou · **`atop -r` é o único que volta no tempo**.

## RESUMO RÁPIDO

- ✓ **ps** fotografa · **top** filma · **htop** filma com interface. Escolha pela finalidade.
- ✓ `ps aux` e `ps -ef` são as formas canônicas; `ps -aux` só funciona por acidente.
- ✓ `ps -eo campos --sort=-campo` monta o painel exato — sempre melhor que `ps aux | grep`.
- ✓ RSS ≠ memória privada; `wa` alto = disco; `buff/cache` é cache, não uso.
- ✓ Pico que já passou só se investiga com `atop -r /var/log/atop/...`.

## NO KALI

Em máquina possivelmente comprometida, **não confie no `ps`**: rootkit esconde PID. Cruze fontes — `ps -e -o pid` contra `ls /proc | grep '^[0-9]'`; o `unhide` automatiza. Comando de `cron` vive milissegundos e nenhum `ps` manual pega — é o que o *pspy* captura, em teste autorizado (Lei 12.737/2012). E **RSS gordo em hashcat é normal**: o sinal de aperto é o **swap crescendo**, não a memória usada.

GUIA VISUAL 029

# SINAIS E CONTROLE

*O `kill -9` é o primeiro comando que se aprende e o último que se deveria usar.*

029
GUIAS LINUX

## OS SINAIS QUE VOCÊ REALMENTE USA ( `KILL -L` LISTA OS 60+)

**SIGTERM 15**
Pedido educado. Padrão do `kill` : o programa fecha arquivos e limpa.

**SIGKILL 9**
**Não capturável.**
O kernel remove o processo — nenhum buffer é gravado.

**SIGHUP 1**
Terminal caiu. Por convenção, daemon
**relê a configuração**
sem reiniciar

**SIGINT 2**
`Ctrl-C` : interrupção do usuário — vai ao
**grupo**
inteiro

**SIGSTOP / TSTP 19/20**
Congela. O
**STOP**
não é capturável; o
**TSTP**
é o `Ctrl-Z` .

**SIGCONT 18**
Descongela e retoma de onde parou.

**SIGUSR1 / USR2 10/12**
Livres: `dd` imprime progresso, `nginx` reabre logs.

**SIGPIPE 13**
Escreveu em `pipe` sem leitor — por isso `| head` encerra o produtor.

## A ESCADA CERTA & QUEM CASA O QUÊ

### ENCERRAR É SEQUÊNCIA

1. `kill PID` — SIGTERM: peça educadamente.
2. **espere** 5 a 30 s e teste com `kill -0 PID`.
3. repita o TERM **ou** confira se está em **D** (E/S travada).
4. `kill -KILL PID` — só agora, sabendo o custo.

### POR NOME, POR GRUPO

- ✓ **pkill** casa *parte* do nome: `pkill ssh` atinge sshd, ssh-agent e sshfs.
- ✓ **killall** exige o nome **exato** do comando ( `-r` habilita regex).
- ✓ `pkill -f 'python3 srv.py'` casa a linha inteira — o único jeito de separar dois scripts do mesmo interpretador.
- ✓ **PID negativo = grupo:** `kill -TERM -2201` . É o que o `Ctrl-C` faz num pipeline.
- ✓ Em script escreva `-TERM` , nunca `-15` : os nomes são universais, os números não.

**Sempre `pgrep` antes de `pkill` .** As opções são as mesmas e o `pgrep` apenas lista: `pgrep -af padrao` → depois `pkill -f padrao` . Um padrão amplo como `pkill -f python` derruba serviços do sistema; como root, `pkill -u root` encerra a sessão e pode exigir reboot forçado. E serviço do systemd se para com `systemctl stop` , nunca com `kill` direto.

## PRIORIDADE: `NICE` NA CPU, `IONICE` NO DISCO

`-20` ← mais prioridade · só root — `0` padrão — `+19` mais gentil · qualquer um

**Assimetria:** subir o *nice* é livre; **descer exige root** — nem o próprio dono desfaz o que aumentou.

**É peso, não reserva:** cada degrau vale ~25% de fatia *sob disputa*. Máquina ociosa: `nice 19` usa 100% da CPU.

**A receita completa:** `nice -n 19 ionice -c 3 tar czf bkp.tgz /dados` — cede CPU **e** disco. Em execução: `renice -n 15 -p PID` .

## RESUMO RÁPIDO

- ✓ Sinal é notificação do kernel: o processo **trata**, ignora, bloqueia ou sofre a ação padrão.
- ✓ **KILL (9) e STOP (19) não se capturam** — por isso funcionam sempre e por isso não limpam nada.
- ✓ Escada: `TERM` → esperar ( `kill -0` ) → `KILL` . É o que o systemd faz.
- ✓ `SIGHUP` recarrega config em daemon e é o sinal de terminal fechado (vide 030).
- ✓ `nice` vai de -20 a +19; `ionice -c 3` cede o disco. Nenhum dos dois limita memória — isso é *cgroup*.

## NO KALI

Varredura pesada merece gentileza: `nice -n 15 nmap …` ou `gobuster` mantém a VM usável e não custa quase nada no tempo total. E **nunca encerre ferramenta ofensiva com -9**: o *metasploit* deixa banco inconsistente e handlers órfãos na porta; o *hashcat* perde o arquivo de restauração e você recomeça horas de trabalho — ambos tratam `Ctrl-C` e `TERM` direito. Em alvo autorizado, `pkill -f java` sem conferir derruba a produção do cliente: dano real, fora do escopo da autorização.

GUIA VISUAL 030

# JOBS, PLANOS NOHUP & DISOWN

*A varredura de seis horas some junto com a conexão SSH — por causa de um sinal só.*

030
GUIAS LINUX

## A SESSÃO: QUEM RECEBE O `CTRL-C` E QUEM LEVA O `SIGHUP`

`/dev/pts/0` terminal de controle → **bash** líder da sessão PID = SID = 1342 →

- `1° plano · vim` único que lê o teclado e recebe `Ctrl-C` / `Ctrl-Z`
- `%1 · tar czf … &` escreve na tela, mas *ler* dela o para com `SIGTTIN`
- `%2 · nmap … & job = pipeline inteiro` = um grupo de processos

**Ao cair a conexão**, o kernel manda **SIGHUP** ao líder da sessão, e o bash repassa a *todos* os jobs — ação padrão: terminar. Sair digitando `exit` no bash **não** faz isso (opção `huponexit` desligada); o `zsh` do Kali se comporta diferente — não conte com ele.

## O CICLO DO JOB

- ✓ `cmd &` inicia em 2º plano e devolve o prompt: `[1] 2847`
- ✓ **Ctrl-Z** envia `SIGTSTP` : estado **T**, memória intacta, zero CPU.
- ✓ `bg` retoma em 2º plano · `fg` traz de volta ao 1º.
- ✓ `jobs -l` mostra PID; **+** é o job corrente, **-** o anterior.
- ✓ *Stopped (tty input)* = o job pediu teclado: `fg` , responda, **Ctrl-Z**, `bg` .

## REFERENCIAR E SINCRONIZAR

**%1** número · **%+** ou **%%** corrente · **%-** anterior

**%vim** começa com · **%?relato** contém

`kill %3` sinaliza o **job** 3 (o grupo inteiro).

`$!` guarda o PID do último `&` ; `wait` espera e devolve o `$?` real.

`t1 & t2 & t3 & wait` — o paralelismo básico do shell.

**jobs não atravessa terminais:** entre painéis do tmux, use PID com `pgrep` .

| DESACOPLAR DO TERMINAL | QUANDO SE USA | O QUE ACONTECE COM A SAÍDA |
|---|---|---|
| `nohup cmd > log 2>&1 &` | Antes de iniciar | Ignora o SIGHUP; sem redirecionar, despeja em `nohup.out` |
| `disown -h %1` | Depois de iniciar — "esqueci o nohup" | Tira o job da tabela do shell; a saída no terminal **se perde** |
| `setsid cmd < /dev/null` | Antes — desassociar de vez | Sessão nova, `TTY ?` ; exige redirecionamento |
| `systemd-run --user --unit=x` | Quando o certo é ser serviço | Vira unidade: log no `journald` , controle por `systemctl` |
| `tmux` / `screen` | Tarefa longa **e interativa** | **Preservada e reconectável** — você volta e vê a tela (ficha 031) |

`kill %1` **e** `kill 1` **não são a mesma coisa.** Com `%` o argumento é número de *job* do shell; sem `%` é um **PID** — e o PID 1 é o `systemd` . Como root, `kill -9 1` é o caminho mais curto para um sistema que só volta com reset físico. O `%` não é decoração.

## RESUMO RÁPIDO

- ✓ **Job é o pipeline inteiro** = um grupo de processos; por isso o `Ctrl-C` derruba todos os estágios.
- ✓ `jobs` / `fg` / `bg` são internos do shell e valem só naquele terminal.
- ✓ Terminal fechou → `SIGHUP` ao líder da sessão → repassado aos jobs.
- ✓ `nohup`, `disown` e `setsid` mantêm vivo; **nenhum preserva a interação**.
- ✓ Sem tmux, dá para espiar: `tail -f /proc/PID/fd/1`; em emergência, `reptyr PID` rouba o processo.

## NO KALI

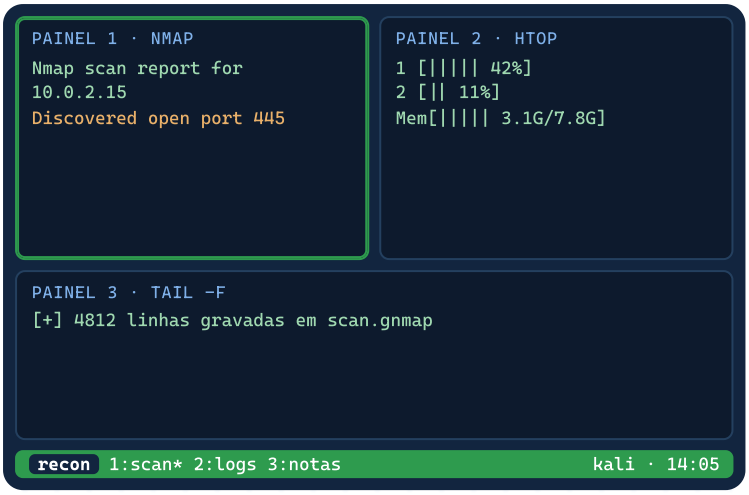
Ferramenta longa merece **tmux**, não `nohup`: você quer *ver a tela* ao reconectar. Inicie o `nmap` com `-oA scan` — sem arquivo de saída não existe `--resume`, e a interrupção obriga a recomeçar. *Listener* do metasploit ou `nc -lvnp` preso ao terminal morre com o SSH e derruba quem estiver conectado. E `nohup.out` esquecido no alvo é **vestígio**: redirecione para a sua área e limpe no encerramento. Para caçar persistência: `ps -eo pid,ppid,tty,comm | awk '$3=="?"'`.

# TMUX & SCREEN MULTIPLEXAÇÃO

*O `nohup` mantém o processo vivo; o tmux mantém a **sessão de trabalho** inteira.*

**031**
GUIAS LINUX

## SERVIDOR → SESSÃO → JANELA → PAINEL




**SERVIDOR**
Um por usuário. **SEM TERMINAL DE CONTROLE** (TTY `?`, PPID 1) — é por isso que sobrevive à queda do SSH.

**SESSÃO**
Agrupamento nomeado. É a unidade de *attach/detach*: uma por assunto.

**JANELA**
Ocupa a tela toda, como uma aba. Numerada e renomeável.

**PAINEL**
Divisão da janela; cada um com seu shell.

**Cliente** é o seu terminal: espia o servidor e pode sumir. `C-b d` e a queda da conexão têm **o mesmo efeito** — nada a salvar antes.

## PREFIXO `C-B` : SOLTE O CTRL, DEPOIS A TECLA

- `d` *detach*: sai deixando tudo rodando
- `%` divide na **vertical**
- `z` **zoom**: amplia o painel e desfaz
- `x` fecha o painel · `&` a janela
- `c` cria janela · `,` renomeia
- `"` divide na **horizontal**
- `[` rolagem e cópia (`/` busca, `q` sai)
- `espaço` percorre os layouts prontos
- `w` lista interativa de janelas
- `setas` move entre painéis · `o` próximo
- `0…9` vai direto à janela
- `?` lista todos os atalhos

**screen, o antecessor onipresente** — prefixo `C-a` : `screen -S nome` cria · `-ls` lista · `-r` reconecta · `-d -r` expulsa o outro cliente · `C-a d` destaca. Só ele tem `screen -x` : **dois usuários na mesma tela ao vivo**. Aprenda esses cinco para a máquina antiga onde o tmux não existe.

kali@kali: ~ — o essencial e o ~/.tmux.conf

```
$ tmux new -As recon    # conecta se existir, cria se não — o idioma a memorizar
$ tmux ls
recon: 3 windows (created Sun Aug 31 09:00:12 2026) (attached)
$ tmux a -d -t recon    # reconecta e desconecta os outros clientes
$ tmux send-keys -t recon:1 'nmap -sV -oA scan alvo' C-m   # monta ambiente por script
$ tmux pipe-pane -o -t recon:1 'cat >> ~/logs/painel.log'  # grava o painel
# ~/.tmux.conf — as cinco linhas que mais mudam o dia:
set -g mouse on · set -g base-index 1 · set -g history-limit 50000
set -g renumber-windows on · bind | split-window -h -c "#{pane_current_path}"
```

## RESUMO RÁPIDO

- ✓ O servidor roda **sem terminal de controle**; o seu terminal é só um cliente descartável.
- ✓ Hierarquia: **servidor** → **sessão** → **janela** → **painel**. Sessão é a unidade de attach.
- ✓ `tmux new -As nome` · `tmux ls` · `tmux a -t nome` · `C-b d`.
- ✓ O histórico de rolagem é do tmux: use `C-b [`, não a roda do emulador (ou ligue `mouse on`).
- ✓ Sessões sobrevivem a logout e à queda da conexão — **não a reboot**. Versione o script de montagem.

## NO KALI

Uma sessão nomeada **por engajamento**, com janelas *recon*, *enum*, *exploit* e *notas*. O layout de acompanhamento é ferramenta + `htop` + `tail -f`, com `C-b z` para ampliar. `pipe-pane` (ou `script -a`) grava o que foi executado — **registro que protege as duas partes** na discussão de uma indisponibilidade. No alvo é o contrário: subir tmux lá deixa servidor persistente, artefato que muda o estado do cliente. E confira `ls -ld /tmp/tmux-*` (deve ser **700**): socket frouxo entrega uma sessão de root a quem quiser.

# RECURSOS DO SISTEMA

*"O servidor está sem memória" — e o `free -h` não confirma nada disso.*

**032**
GUIAS LINUX

## MEMÓRIA: `LIVRE` É A COLUNA ERRADA

| 3,6 Gi | 11 Gi | 180 Mi |
|---|---|---|
| USADA POR PROCESSOS | BUFF/CACHE | LIVRE |

`disponível = 11 Gi` — o número que responde "quanto ainda posso alocar?"

**"O Linux comeu minha RAM" é o correto:** RAM ociosa é desperdício, então o kernel cacheia disco nela e **devolve na hora** que um processo pedir.

**Por processo, RSS superestima** (conta biblioteca compartilhada em cada um). O número honesto é o `Pss` em `/proc/PID/smaps_rollup`.

**Nem tudo é recuperável:** `tmpfs` e `/dev/shm` ocupam RAM de verdade. `Dirty` grande = escrita represada, o mesmo quadro do `wa` alto.

## LOAD AVERAGE · CPU

`0` — `N = nproc` — `2N +`

folga: há núcleo ocioso | saturado | fila de espera

**Sempre divida pelo `nproc`:** load 4,0 em 8 núcleos é metade da capacidade; em 2 núcleos é o dobro do que ela aguenta.

**A ordem dá a tendência** (1, 5 e 15 min): `5,2 2,1 1,0` é pico subindo; `1,0 2,1 5,2` é pico que passou.

**No Linux o load conta o estado D** — load alto com `us` baixo e `wa` alto é gargalo de **disco**, não de CPU.

`lscpu` separa soquete × núcleo × thread: 8 CPUs com *hyper-threading* não são 8 unidades reais.

## SWAP & O OOM KILLER

- ✓ `swappiness` (0–200) regula a *preferência* entre mover páginas e descartar cache — **não liga nem desliga** o swap.
- ✓ Sinal confiável de pressão são **si/so** no `vmstat 2`, não o total de swap usado (pode ser resíduo antigo).
- ✓ Sem memória, o kernel escolhe pelo `oom_score` e mata — **quase sempre o maior, não o culpado**.
- ✓ `sudo dmesg -T | grep -i oom` conta quem morreu e com quanto.
- ✓ `oom_score_adj` negativo protege o que não pode cair (o `sshd`); `-1000` é imunidade.

## LIMITES: TODO TETO TEM FACE FLEXÍVEL E RÍGIDA

### FLEXÍVEL × RÍGIDO

O usuário eleva o **soft** até o **hard** e baixa à vontade; **só o root eleva o hard**. `ulimit -a` mostra os de agora, `-Ha` os tetos.

### `ULIMIT -N` É O CAMPEÃO DE INCIDENTES

*Too many open files* sem outro sintoma: o padrão de **1024** descritores é baixo para serviço com carga — e para `nmap`.

### SYSTEMD NÃO PASSA PELO PAM

Mexer no `limits.conf` não afeta serviço: use `LimitNOFILE=` na unit. A verdade em vigor está em `/proc/PID/limits`.

**Dois cuidados.** `mkswap` num *dispositivo* (`/dev/sda2`) formata a partição e apaga o que houver, sem perguntar — ensaie sempre com **arquivo**: `fallocate -l 1G /swapfile` → `chmod 600` → `mkswap` → `swapon`, e desfaça com `swapoff`. E **não digite a *fork bomb*** `:(){ :|:& };:` para "testar": sem `nproc` configurado em `/etc/security/limits.d/`, ela trava a máquina até o reset físico.

## RESUMO RÁPIDO

- ✓ **livre** perto de zero é saudável; a métrica é **disponível** (`MemAvailable`).
- ✓ Load só faz sentido dividido pelo `nproc` — e inclui as tarefas travadas em E/S.
- ✓ `vmstat 2` (ignore a 1ª linha), `mpstat -P ALL`, `iostat -xz` e `pidstat` detalham a origem da carga.
- ✓ OOM killer registra tudo no `dmesg -T`; proteja o essencial com `oom_score_adj`.
- ✓ `ulimit -u` contém *fork bomb*; **cgroups** (`MemoryMax=`, `systemd-cgtop`) contêm grupos inteiros.

## NO KALI

VM apertada + `rockyou` = OOM killer: cheque `free -h` e `swapon --show` **antes** do cracking. Varredura travando com *Too many open files* se resolve com `ulimit -n 65535` — `nmap` e `masscan` abrem milhares de conexões. Load alto durante scan é rede e E/S, não CPU: aumente menos a VM e reduza mais as threads da ferramenta. No **live USB** boa parte do sistema é `tmpfs` — o que você baixa em `/tmp` come RAM real, sem cache descartável. Em alvo autorizado, `uptime` de 400 dias significa kernel sem patch há mais de um ano (Lei 12.737/2012 vale para o resto).

GUIA VISUAL 033

# APT E O MODELO DEBIAN

*A pergunta certa não é "onde eu baixo o nmap" — é "isso já está no repositório?".*

033
GUIAS LINUX

## UMA LINHA DO `SOURCES.LIST`, CAMPO A CAMPO

```
deb  http://http.kali.org/kali  kali-rolling  main contrib non-free non-free-firmware
```

**TIPO**
`deb` = binário pronto; `deb-src` = código-fonte

**URI**
Endereço do espelho. O `http.kali.org` é um **redirecionador**, não um servidor: manda para o mirror mais próximo.

**SUÍTE**
A linha de distribuição — `kali-rolling`, `kali-last-snapshot`, `kali-experimental`

**COMPONENTES**
Seções por **licença**, não por tema: livre; livre que depende de não livre; restritivo; e firmware.

O APT **concatena os campos numa URL**: `URI/dists/SUÍTE/COMPONENTE/binary-amd64/Packages.gz` — por isso o erro de origem sai como um **404 mostrando a URL inteira**, e o diagnóstico é só ler qual campo está errado. O formato **deb822** (`.sources`) diz o mesmo em chave-valor e ainda amarra a origem à sua chave com `Signed-By:`.

## COMO O APT CONFIA: UMA CHAVE ESTENDE GARANTIA A MILHARES DE ARQUIVOS

**chave GPG** (kali-archive-keyring, local) → valida a assinatura → **InRelease** (assinado; traz o hash dos índices) → valida por hash → **Packages.gz** (índice; traz o hash de cada .deb) → valida por hash → **pacote.deb** (o arquivo que será instalado)

- `NO_PUBKEY <ID>` — a chave que assinou o repositório não está instalada.
- `EXPKEYSIG` — a chave venceu; atualize o pacote de chaveiro (o Kali rotaciona a dele).
- `não é válido ainda` — relógio atrasado (VM suspensa, live USB). `timedatectl` resolve.

## DUAS CAMADAS: DPKG EMBAIXO, APT EM CIMA

- ✓ **dpkg é o operário:** desempacota, roda scripts e registra — mas **nunca sai à rede**. Falta dependência, ele aborta.
- ✓ **APT é o planejador:** lê os índices, resolve o grafo, escolhe a versão, baixa e chama o dpkg na ordem.
- ✓ "`dpkg -i` deu erro de dependência" se resolve **subindo uma camada**: `apt install -f`.
- ✓ `apt update` **não instala nada**: sincroniza índices em `/var/lib/apt/lists/` e confere assinaturas.
- ✓ `apt policy nmap` revela a decisão: **100** = instalado, **500** = origem normal, **990** = release alvo. Vence a maior.

## ROLLING: O KALI NÃO TEM "VERSÃO"

- ✓ Debian **stable** congela e só recebe correção de segurança por anos. O Kali deriva de **testing** e recebe pacote continuamente.
- ✓ As imagens numeradas (`2025.3`) são **snapshots de instalação**: a 2025.1 atualizada hoje é idêntica à 2025.3.
- ✓ A contrapartida: atualizar é operação com risco real — versão principal muda, opção some, config é substituída.
- ✓ `kali-last-snapshot` congela o ambiente (turma, laboratório reproduzível); voltar é editar uma linha.
- ✓ Índice de rolling **vence em dias**: o `404` ao instalar num Kali parado há um mês é índice velho, não pacote removido.

**Nunca misture repositórios de distribuições diferentes.** Adicionar ao Kali um PPA, o Ubuntu ou o Debian stable "porque o pacote não tem aqui" arrasta a `libc6` daquela origem — e no meio da substituição **nada mais executa, inclusive o próprio apt**, o que impede o desfazer. A única origem legítima do Kali é a do Kali; o resto se resolve compilando, com `pipx`, contêiner ou AppImage. E **jamais contorne um erro de GPG com** `[trusted=yes]`: isso não conserta o elo, desliga a cadeia inteira — em silêncio e para sempre.

## RESUMO RÁPIDO

- ✓ Repositório = diretório HTTP: `.deb` em `pool/`, índices que os descrevem e um release **assinado**.
- ✓ Cinco campos no `sources.list` viram uma URL — errar um dá 404 com a URL na tela.
- ✓ Confiança é **cadeia**: chave → InRelease → índice → pacote, cada elo por hash.
- ✓ Cache: `/var/cache/apt/archives/` cresce sem limite — `apt clean` é seguro. Histórico auditável em `/var/log/apt/history.log`.
- ✓ Família RHEL é o mesmo modelo com outros nomes: `dnf` / `rpm` e arquivos `.repo`.

## NO KALI

**Uma linha só** deve estar no `sources.list`: `deb http://http.kali.org/kali kali-rolling main contrib non-free non-free-firmware`. Qualquer origem herdada de tutorial já deixou o sistema instável mesmo parecendo funcionar. Se o `apt update` ficar lento ou entregar hash inconsistente, é espelho dessincronizado: troque por `http://kali.download/kali`, que aponta para o CDN. Chave vencida se corrige baixando o `kali-archive-keyring` atual pelo HTTPS oficial e instalando com `dpkg -i`. E **não conte com apt em live sem persistência**: listas e cache vão para `tmpfs`, comem RAM real e o `full-upgrade` pode acordar o OOM killer antes de terminar.

GUIA VISUAL 034

# INSTALAR, REMOVER E ATUALIZAR

*"Os pacotes a seguir foram mantidos" não é ruído — é meio sistema ficando para trás.*

034
GUIAS LINUX

## REMOVER NÃO É UMA COISA SÓ: OS TRÊS ESTADOS DE UM PACOTE

**NÃO INSTALADO**
nada em disco

`apt install →`
`← apt purge`

**INSTALADO**
`ii`
binários + configuração

`apt remove →`
`← apt install`

**RESIDUAL**
`rc`
só a config em `/etc`

`apt purge` também vai de `rc` direto a `não instalado` — é o único caminho que apaga a configuração.

O estado **rc** é o que faz o `apt remove` **parecer** que não funcionou: o binário some, mas o `dpkg -l` segue listando o pacote. É proposital — reinstalar recupera seus ajustes. Ache os resíduos com `dpkg -l | grep '^rc'`. **Nenhuma dessas transições toca em** `/home`: o gerenciador de pacotes nunca apaga dado de usuário.

## UPGRADE × FULL-UPGRADE: O QUE CADA UM PODE FAZER

### APT UPGRADE — PROIBIDO REMOVER

- ✓ **Pode** instalar pacote novo e subir versão. **Não pode** remover nada.
- ✓ Quando atualizar A exige remover B, ele **desiste de A** e imprime "foram mantidos".
- ✓ Serve no **Debian stable**, onde remoção é exceção e a mensagem é rara.
- ✓ **Num rolling é armadilha:** cada execução acumula pendência, o sistema fica meio antigo/meio novo — e o `full-upgrade` adiado vira uma migração gigante de uma vez.

### APT FULL-UPGRADE — CONCLUI A MIGRAÇÃO

- ✓ **Pode** instalar, atualizar **e remover** o que estiver no caminho. Nada fica pendente.
- ✓ **Exige** que você leia a lista de remoções antes de confirmar.
- ✓ É o `apt-get dist-upgrade` de antigamente — mesmo comportamento, nome menos enganoso.
- ✓ Sequência certa: `sudo apt update && sudo apt full-upgrade && sudo apt autoremove`.

## ANTES DE CONFIRMAR

### POR QUE VIERAM 30 PACOTES

`Depends` é obrigatório; `Recommends` é **instalado por padrão** (a origem do volume); `Suggests` não é. Corte caso a caso com `--no-install-recommends` — desligar global deixa faltando o firmware e o plugin que tornam a ferramenta útil.

### MANUAL × AUTOMÁTICO

`autoremove` age sobre a **marcação**, não sobre sua intenção: o que entrou como dependência vai embora quando fica órfão. `apt-mark manual <pkg>` protege o que você passou a usar direto; `showmanual` e `showauto` mostram os dois lados.

### SIMULE — E DEPOIS CONSERTE

`apt install -s` imprime o plano inteiro **sem sudo e sem tocar em nada**. Deu ruim: `apt --fix-broken install` resolve dependência quebrada e `dpkg --configure -a` conclui instalação interrompida. `apt-mark hold` congela — e `showhold` evita esquecê-lo.

**Três cuidados. (1)** `apt autoremove` depois de remover um metapacote propõe apagar **dezenas de ferramentas** — leia a lista inteira antes do `-y`. **(2)** `apt purge` apaga sua configuração em `/etc` sem volta; se houver ajuste que interessa, copie antes. **(3)** No erro `Could not get lock /var/lib/dpkg/lock-frontend`, **nunca apague o lock por reflexo**: é quase sempre o `unattended-upgrades` rodando. Confirme com `sudo lsof /var/lib/dpkg/lock-frontend` — apagar com um `dpkg` ativo corrompe o banco de pacotes, e o conserto é pior que a espera.

## RESUMO RÁPIDO

- ✓ Descobrir: `apt search --names-only`, `apt show`, `apt rdepends --installed` antes de remover biblioteca.
- ✓ `apt install ./arquivo.deb` (com o `./`) instala o local **resolvendo dependências** — melhor que `dpkg -i` + conserto.
- ✓ Em script use `apt-get`: `DEBIAN_FRONTEND=noninteractive` + `-o Dpkg::Options::=--force-confold`.
- ✓ **O APT não tem *undo*.** `/var/log/apt/history.log` conta o que houve; desfazer é reinstalar versão a versão — se ainda existir no espelho.
- ✓ No RHEL: `dnf` já remove para concluir (não há o par upgrade/full) e tem `dnf history undo`.

## NO KALI

**`apt upgrade` sozinho é o comando errado aqui.** Num rolling, pacote é renomeado e biblioteca troca de *soname*: remoção é rotina, e "foram mantidos" é alerta, não ruído. Atualize **semanalmente e por inteiro** — saltos pequenos e previsíveis; um Kali parado seis meses tenta migrar centenas de pacotes de uma vez, e reinstalar da imagem atual costuma sair mais barato que consertar. **Tire snapshot da VM antes de todo `full-upgrade` grande**: é a única rede de segurança real. E rode `apt install -s` antes de metapacote — o `kali-linux-everything` passa de **100 GB** instalado. Um `.deb` de terceiro roda script como root na instalação; por isso o `/var/log/dpkg.log` entra em toda verificação forense.

GUIA VISUAL 035

# DPKG E OS ARQUIVOS .DEB

*O `dpkg -i` falhou, o pacote ficou esquisito — e agora o APT inteiro se recusa a rodar.*

035
GUIAS LINUX

## UM `.DEB` É UM ARQUIVO `AR` COM TRÊS MEMBROS

`nmap_7.95+dfsg-3_amd64.deb` — e nada além destes três

**`debian-binary`**
Uma linha: `2.0`, a versão do **formato** do pacote.

**`control.tar.xz`**
`control` — nome, versão, arquitetura, `Depends`, descrição.
`md5sums` — hash de cada arquivo · `conffiles` — o que é configuração.
`preinst` · `postinst` · `prerm` · `postrm` — os scripts que **rodam como root**.

**`data.tar.xz`**
A árvore de arquivos **como será instalada**:
`./usr/bin/nmap`
`./usr/share/nmap/…`
`./usr/share/man/…`
Nada além de uma cópia dos caminhos de destino.

**Instalar é extrair o `data.tar` sobre a raiz e rodar os scripts do `control.tar` nos momentos certos** — `preinst` → desempacota → `postinst` na instalação; `prerm` → apaga → `postrm` na remoção. Todos **como root, sem confirmação e sem sandbox**: é assim que um pacote cria usuário, gera chave e registra serviço. Auditar qualquer `.deb` antes é uma sequência de três comandos: `dpkg-deb -I` (metadados) · `dpkg-deb -c` (arquivos) · `dpkg-deb -e` (extrai os scripts, para ler o `postinst` antes de executá-lo).

## A COLUNA DO `DPKG -L` : O QUE SE QUER × O QUE É

`ii` `nmap` `7.95+dfsg-3` `amd64` `The Network Mapper`

**1º CARACTERE — O QUE SE *QUER***

- `i` instalado
- `r` a remover
- `p` a purgar
- `h` congelado (*hold*)
- `u` desconhecido

**2º CARACTERE — O QUE *É***

- `i` instalado e configurado
- `n` não instalado
- `F` configuração falhou
- `c` só a configuração
- `U` desempacotado, sem configurar

`ii` — tudo certo: instalado e configurado. É o que se espera de quase toda linha.

`rc` — removido, configuração ainda em `/etc`. Inofensivo; `apt purge` limpa.

`iU` / `iF` — **quebrado**: desempacotado mas não configurado. **`Bloqueia o APT inteiro.`** Conserto: `sudo dpkg --configure -a` e, se faltar dependência, `sudo apt --fix-broken install`. O `dpkg --audit` lista tudo que está pendente de uma vez.

## RELAÇÕES E VERSÕES

- ✓ Além de `Depends`: `Pre-Depends` exige ordem estrita (antes até de desempacotar); `Breaks` / `Replaces` renomeiam pacote e movem arquivos sem conflito; `Provides` cria nome virtual (`mail-transport-agent`).
- ✓ Versão é `época:upstream-revisão`. A **época** conserta numeração que regrediu e nunca diminui; a **revisão** é do empacotador.
- ✓ **O til ordena antes de tudo**, inclusive do vazio: `1.0~rc1` < `1.0`, mas `1.0rc1` > `1.0`.
- ✓ Na dúvida, quem decide é `dpkg --compare-versions '1.10' gt '1.9'`.

## INSPEÇÃO — E UM PACOTE SEU

- ✓ `dpkg -L pkg` lista o que ele instalou; `dpkg -S /usr/bin/nmap` responde **de qual pacote veio um arquivo**.
- ✓ `dpkg -V` compara os hashes com o disco: saída vazia = nada alterado desde a instalação.
- ✓ Construir um `.deb` são **dois arquivos e um comando**: a árvore de destino + `DEBIAN/control`, e então `dpkg-deb --build dir pacote.deb`. As linhas extras da `Description` começam **com um espaço** — sem ele o build recusa.
- ✓ Para aplicar patch, recompile o oficial: `apt source` + `apt build-dep` + `dpkg-buildpackage -us -uc -b`, e ponha um `hold` ou revisão `-3.1local1`.

**As opções `--force-*` do dpkg quebram o sistema em silêncio.** `dpkg -i --force-depends`, `--force-overwrite` e `--force-all` não resolvem erro de dependência: instalam um pacote que **não pode funcionar** e deixam o banco de dados afirmando que está tudo bem. O estrago aparece depois, num contexto sem relação — símbolo indefinido, `apt` que se recusa a operar. Diante de erro de dependência a resposta é sempre `sudo apt --fix-broken install`. Existem para recuperação por quem sabe exatamente o que está fazendo, e só.

## RESUMO RÁPIDO

- ✓ O `dpkg` **verifica** dependências mas não as satisfaz — quem busca na rede é o APT.
- ✓ **Prefira sempre** `apt install ./pacote.deb` (com o `./`) a `dpkg -i`: mesma instalação, com as dependências resolvidas antes.
- ✓ Cada pacote instalado tem seus arquivos de controle em `/var/lib/dpkg/info/` (`.list`, `.md5sums`, scripts).
- ✓ `dpkg -l | grep -v '^ii'` mostra tudo que não está normal — o primeiro comando de um diagnóstico.
- ✓ No RHEL: `rpm -qa/-ql/-qf/-V`; o `.rpm` é `cpio` (precisa de `rpm2cpio`) e não tem o estado residual.

## NO KALI

**Todo `.deb` de terceiro merece auditoria antes de instalar:** baixe, `dpkg-deb -e` para extrair os scripts, `dpkg-deb -c` para ver os arquivos — se o `postinst` tem um `curl | bash` para um domínio qualquer, você descobre antes e não depois. **`dpkg -S` e `dpkg -V` são ferramentas de investigação**, não só de administração: "nenhum pacote é dono deste arquivo" para algo em `/usr/bin`, ou um binário do sistema com hash diferente, é achado relevante em verificação de integridade — com a ressalva de que um invasor com root pode ter reescrito os próprios hashes, então é indício, não prova. O sufixo `-0kaliN` identifica o que foi empacotado pelo Kali em vez de vir do Debian. E resolva um `iU` **na hora**: ele bloqueia o APT inteiro, e descobrir isso no meio de um teste custa tempo.

GUIA VISUAL 036

# METAPACOTES E KALI-TWEAKS

*"Quero todas as ferramentas" custa mais de 100 GB — e enche o disco no meio da instalação.*

036
GUIAS LINUX

## METAPACOTE: UM PACOTE SEM ARQUIVOS, CUJO CONTEÚDO É A LISTA DE `DEPENDS`

| | | |
|---|---|---|
| `kali-linux-core` | o mínimo absoluto: sistema base, sem ferramentas | ~1 GB |
| `kali-linux-headless` | só linha de comando, sem interface gráfica | ~3 GB |
| `kali-linux-default` | o conjunto da imagem oficial — o que a maioria quer | ~12 GB |
| `kali-linux-large` | o default mais as ferramentas menos comuns | ~25 GB |
| `kali-linux-everything` | literalmente tudo — inclusive ferramentas conflitantes | >100 GB |

▲ CADA NÍVEL CONTÉM O ANTERIOR · TAMANHOS SÃO ORDEM DE GRANDEZA, MUDAM TODA SEMANA NUM ROLLING

e por categoria, seguindo o menu do sistema: `kali-tools-top10` `information-gathering` `vulnerability` `web` `passwords` `wireless` `exploitation` `sniffing-spoofing` `post-exploitation` `forensics` `reporting`

## OUTRA SUÍTE SEM CONTAMINAR O SISTEMA

### KALI-TWEAKS — O MENU QUE ESCREVE POR VOCÊ

- ✓ **Metapackages** instala/remove os conjuntos acima mostrando o tamanho antes.
- ✓ **Network Repositories** habilita `experimental` e `bleeding-edge` **já com o pinning correto** — é por isso que se prefere o menu à edição manual.
- ✓ **Virtualization** instala o pacote de convidado (VMware, VirtualBox, QEMU) e resolve a tela travada e o copiar-e-colar.
- ✓ **Hardening** *reduz* a segurança de propósito: reabilita algoritmos SSH legados para alcançar equipamento antigo em teste autorizado. Ligue ao precisar, desligue depois.

### PINNING — PRIORIDADE NEGATIVA

- ✓ Suítes: `bleeding-edge` → `experimental` → `rolling` (a única habilitada). `kali-last-snapshot` é a cópia congelada — a resposta certa para treinamento e laboratório reproduzível.
- ✓ Regra em `/etc/apt/preferences.d/` : `Pin: release a=kali-experimental` + `Pin-Priority: -1` . Depois, `apt install -t kali-experimental <pkg>` .
- ✓ Acima de **1000** é a única prioridade que permite **rebaixar** um pacote — mais forte que o `apt-mark hold` , que só congela.

**`sem pinning`**
A suíte entra com **500**, igual ao rolling: o APT passa a preferi-la para *tudo* e o próximo `full-upgrade` migra o sistema.

**`com pinning -1`**
Existe no catálogo, mas **nunca é escolhida sozinha** — só com o `-t` , para o pacote que você pediu.

**Dois estragos clássicos. (1)** O metapacote entra como **manual** e tudo que ele puxa entra como **automático**: `sudo apt remove kali-linux-large` transforma **centenas de ferramentas em órfãs**, e o `autoremove` seguinte propõe apagar todas — inclusive as que você passou a usar direto. Antes, `sudo apt-mark manual nmap wireshark sqlmap …` , e depois **leia a lista** do `apt autoremove -s` . **(2)** Repositório de terceiro, quando inevitável, exige as três medidas **juntas**: chave em `/etc/apt/keyrings/` amarrada por `signed-by` e `Pin-Priority: 100` para que ele não substitua pacote do sistema. Nunca `apt-key add` nem `[trusted=yes]` .

## RESUMO RÁPIDO

- ✓ `dpkg -L kali-linux-default` mostra só documentação: metapacote é **lista de compras**, não programa.
- ✓ Três endereços, um repositório: `http.kali.org` (redirecionador), `kali.download` (CDN, resolve espelho dessincronizado) ou um espelho direto.
- ✓ `apt install -s` antes de qualquer metapacote — mede pacotes e disco antes de decidir.
- ✓ `apt-listchanges` avisa **antes**; `needrestart` descobre **depois** quem ainda roda com a biblioteca antiga em memória.
- ✓ No RHEL: grupos ( `dnf group install` ) e `priority=` / `excludepkgs=` no `.repo` .

## NO KALI

`kali-linux-default` **é o ponto certo para quase todo mundo** — cabe numa VM de 40 GB com folga. Suba para `large` quando faltar algo específico; `everything` não faz sentido em máquina de trabalho. Em VM apertada ou contêiner: `kali-linux-headless` + só as categorias do seu trabalho (quem faz web não precisa de rádio). **Habilite as suítes extras pelo** `kali-tweaks` , nunca à mão — sem prioridade negativa, o `full-upgrade` seguinte migra o sistema inteiro para versões experimentais. Ferramenta que não está no repositório tem **quatro caminhos legítimos**, nesta ordem: procurar outro nome, `pipx` se for Python, contêiner, ou compilar do fonte. Repositório de outra distribuição não está na lista. E o `needrestart` vale dos dois lados: garante que a correção entrou em vigor — e serviço rodando com biblioteca já corrigida em disco é achado comum em avaliação autorizada.

GUIA VISUAL 037

# COMPILANDO DO CÓDIGO-FONTE

*Seis meses depois, ninguém sabe que aquele binário existe — nem de onde veio, nem como sai.*

037
GUIAS LINUX

**1 FONTE**
`tar xzf · git clone`
**Verifique antes:** `sha256sum -c` prova *integridade*; só `gpg --verify` (ou tag assinada) prova *origem*.

→

**2 CONFIGURADO**
`./configure`
Não compila nada: **descobre o que existe nesta máquina** e escreve o Makefile. Falhou aqui = falta um pacote `-dev`.

→

**3 COMPILADO**
`make -j"$(nproc)"`
A etapa demorada, ainda dentro do diretório de fonte. Quebrou com `-j` e passou sem? Makefile com dependência mal declarada.

→

**4 INSTALADO**
`sudo make install`
Só **copia arquivos**. É a única etapa que pede `sudo` — e a única que o sistema **não registra**.

| SISTEMA | CONFIGURAR | COMPILAR | INSTALAR |
|---|---|---|---|
| **Autotools** | `./configure --prefix=/usr/local` | `make -j$(nproc)` | `sudo make install` |
| **CMake** | `cmake -B build -DCMAKE_BUILD_TYPE=Release` | `cmake --build build -j$(nproc)` | `sudo cmake --install build` |
| **Meson** | `meson setup build --prefix=/usr/local` | `ninja -C build` | `sudo ninja -C build install` |

## O PREFIXO NÃO É ESCOLHA ESTÉTICA

### /usr
TERRITÓRIO DO DPKG

Todo arquivo aqui pertence a algum pacote, e o `dpkg -S` sabe a qual.

O gerenciador pode **sobrescrever ou remover** qualquer coisa daqui numa atualização.

### /usr/local
TERRITÓRIO DO ADMINISTRADOR

O FHS reserva esta árvore para software instalado localmente. **Nenhum pacote Debian escreve aqui.**

É o padrão do `configure` justamente por isso — e o `PATH` típico já o põe **antes** de `/usr/bin`.

**Por que `--prefix=/usr` morde:** seu binário fica no mesmo caminho de um do repositório. Se o pacote oficial for instalado depois, ele **sobrescreve o seu sem aviso**; se for removido, leva junto arquivos que você achava seus. Terceira opção, sem `sudo` algum: `--prefix=$HOME/.local`, válida só para o seu usuário. E antes de deixar o `make install` agir, veja o que ele faria: `make install DESTDIR=/tmp/preview` + `find /tmp/preview -type f` é o *dry-run* que o `make` não tem.

## `MAKE INSTALL` CRU CRIA SOFTWARE INVISÍVEL — TRÊS SAÍDAS

### MAKE UNINSTALL — FRÁGIL
Só funciona **do mesmo diretório de fonte**, configurado com o mesmo prefixo. Apagou o diretório, perdeu a desinstalação — e nem todo projeto implementa o alvo.

### CHECKINSTALL — O ATALHO BOM
Intercepta o `make install`, observa os arquivos criados e empacota tudo num `.deb`. Passa a aparecer no `dpkg -l`, tem versão e sai com `apt remove`. Adivinha os metadados — aproximado, mas resolve o problema principal.

### STOW · DPKG-BUILDPACKAGE
`stow` instala cada programa sob `/usr/local/stow/` e gerencia por links: desinstalar é remover os links. A solução completa continua sendo gerar um `.deb` de verdade a partir de `apt source` + patch.

`curl -sSL … | sudo bash` **é a pior forma de instalar qualquer coisa.** Executa com privilégio total um script que você não leu, servido por um host que pode mudar de conteúdo entre a sua visita ao site e a execução — e que pode até servir conteúdo diferente para o `curl` e para o navegador. Se for o único caminho: `curl -fsSL … -o /tmp/install.sh` → `less /tmp/install.sh` (ler de verdade) → `bash /tmp/install.sh`, sem `sudo` quando possível. Lembre que **um `configure` é script shell e um `Makefile` pode conter qualquer comando**: compilar fonte não auditado na máquina que guarda credencial de cliente é escolha ruim — a VM descartável existe para isso.

## RESUMO RÁPIDO

- ✓ **Compilar é o último recurso:** repositório → `pipx` → contêiner → recompilar o `.deb` com patch → do zero.
- ✓ `libfoo3` é para **executar**, `libfoo-dev` é para **compilar**: erro em `foo.h` = instale o `-dev`. `apt build-dep` resolve tudo de uma vez.
- ✓ Biblioteca em lugar fora do padrão: arquivo em `/etc/ld.so.conf.d/` + `sudo ldconfig` — **não** `LD_LIBRARY_PATH` no `.bashrc`, que confunde meses depois.
- ✓ No CMake, `-DCMAKE_BUILD_TYPE=Release` é o que liga a otimização; build fora da árvore torna a limpeza um `rm -rf build`.
- ✓ Pacote recompilado com patch precisa de `apt-mark hold`, ou o próximo `full-upgrade` o substitui.

## NO KALI

**A maior parte da ferramenta de segurança do GitHub é Python ou Go — e nenhuma das duas pede compilação clássica:** `pipx` instala isolado e desinstala limpo; `go install` deixa um binário estático em `~/go/bin`. `make` só para C/C++ de fato. Clonar em `/opt` é convenção razoável, mas exige disciplina: um README local dizendo **de onde veio e em que commit**, e `git pull` periódico. **Adote o** `checkinstall` — cada `make install` cru é um ponto cego permanente para `dpkg -V` e `dpkg -S`. Compilar PoC é rotina de laboratório: código de exploração publicado às vezes traz erro, e ocasionalmente traz **malícia dirigida a quem compila sem ler** — leia o fonte, use VM descartável e só dentro de escopo autorizado (Lei 12.737/2012). E se o build morrer sem mensagem, olhe `dmesg -T | grep -i oom` antes do código: `-j$(nproc)` em VM de 4 GB acorda o OOM killer — caia para `-j2`.

GUIA VISUAL 038

# PIPX, FLATPAK E APPIMAGE

`externally-managed-environment` *não é obstáculo a contornar — é a proteção funcionando.*

038
GUIAS LINUX

## PYTHON: TRÊS FORMAS DE INSTALAR, TRÊS ISOLAMENTOS

### `sudo pip install`

Escreve em `/usr/lib/python3/`, o mesmo lugar dos pacotes `.deb` — e sem consultar o `dpkg`, que passa a **mentir sobre as versões**.

Uma biblioteca serve a todos os programas: um conflito quebra vários.

BLOQUEADO PELA PEP 668

### `python3 -m venv`

Um diretório por **projeto**, com seu próprio Python e suas bibliotecas. Isolamento completo; nada toca o sistema.

Exige `source .venv/bin/activate` antes de usar; remover é `rm -rf .venv`.

IDEAL PARA DESENVOLVER

### `pipx install`

Um ambiente virtual por **aplicação**, criado e gerido sozinho. Os executáveis aparecem em `~/.local/bin`, já no `PATH`.

Usar não exige ativar nada. Duas ferramentas com bibliotecas incompatíveis convivem sem se ver.

IDEAL PARA INSTALAR FERRAMENTAS

**A confusão é entre as duas da direita:** o `venv` serve a quem *escreve* Python; o `pipx` a quem só quer *usar* a ferramenta. Por baixo o pipx cria um venv — ele apenas cuida disso por você. Comandos que resolvem o dia a dia: `pipx run <ferramenta>` executa e descarta (uso único), `pipx install git+https://…` instala direto do repositório do autor sem clonar, `pipx upgrade-all` e `pipx reinstall-all` (depois de o Python do sistema mudar de versão).

## A ESCADA DA DECISÃO: DO INTEGRADO AO INVISÍVEL

1. **Está no repositório do Kali?** → `sudo apt install` — sempre a primeira opção: registrado no `dpkg` e atualizado sozinho.
2. **É uma ferramenta Python?** → `pipx install` — ou `pipx run` se o uso for único.
3. **É Go, Rust ou Node?** → `go install` · `cargo install` · `npm` com prefixo local. Binário em `~/go/bin`, `~/.cargo/bin`, `~/.local/bin` — ponha no `PATH`.
4. **Tem imagem de contêiner oficial?** → `docker run --rm` — o isolamento mais forte e a remoção mais completa; o custo é mapear volumes.
5. **É aplicação gráfica de desktop?** → `flatpak install` — conferindo antes o `flatpak info --show-permissions`.
6. **Existe AppImage do autor?** → `chmod +x` só **depois** de conferir o hash e de `--appimage-extract` para inspecionar.
7. **Nenhuma das anteriores** → `compilar` — e empacotar com `checkinstall` para não virar software invisível.

▼ SE NÃO, DESÇA UM DEGRAU — CADA DEGRAU ACRESCENTA ALGO QUE SÓ VOCÊ VAI LEMBRAR DE ATUALIZAR

### SNAP

Imagem `squashfs` + daemon `snapd` residente, confinado por AppArmor; atualiza sozinho. Não vem no Kali.

### FLATPAK

Aplicação + *runtime* compartilhado, *sandbox* por namespaces com *portals*. Atualiza sob comando; o 1º app baixa um runtime grande.

### APPIMAGE

Arquivo único executável, nada instalado. **Isolamento nenhum**; atualização e registro, nenhum.

**Dois riscos que o nome já entrega. (1)** `pip install --break-system-packages` faz exatamente o que diz: é aceitável em contêiner descartável ou imagem de build, mas numa máquina de trabalho produz um Python em que o `dpkg` afirma uma versão e o disco tem outra — e o sintoma aparece semanas depois, num `apt` que falha ao importar um módulo. Se o impulso for usá-la, o comando certo é `pipx install`. **(2)** Um **AppImage roda sem confinamento algum**, com todo o poder do seu usuário: `~/.ssh`, o chaveiro, os arquivos do cliente. `--appimage-extract` e a verificação do hash publicado pelo autor são a única defesa **antes** da primeira execução. Vale o mesmo para Flatpak que pede `--filesystem=home`: isso anula boa parte do isolamento — `flatpak override --nofilesystem=home` corrige.

## RESUMO RÁPIDO

- ✓ O bloqueio é um arquivo real: `/usr/lib/python3*/EXTERNALLY-MANAGED`. Dentro de um `venv` ativado o `pip` funciona normalmente.
- ✓ Cada ferramenta do pipx vive em `~/.local/share/pipx/venvs/`, com um link em `~/.local/bin`.
- ✓ Binário Go é **estático**: copiar o arquivo para outra máquina Linux costuma simplesmente funcionar.
- ✓ Faça o inventário do que o `dpkg` desconhece: `pipx list`, `flatpak list`, `ls ~/go/bin ~/.cargo/bin ~/.local/bin`.
- ✓ No RHEL a PEP 668 vale igual; o **Podman** substitui o Docker — sem daemon e capaz de rodar sem root.

## NO KALI

**Rode `apt search --names-only` antes de qualquer `pipx install`:** o Kali empacota centenas de ferramentas Python como `.deb`, às vezes com nome diferente do projeto no GitHub. Quando não estiver, `pipx install git+https://…` resolve sem clonar e desinstala limpo. **Snap não é a escolha natural aqui** — exige o `snapd` residente e o confinamento AppArmor conflita com o que ferramenta de segurança precisa fazer: capturar pacote, modo monitor, socket bruto. **Contêiner é ótimo para análise e ruim para rede:** analisar artefato ou processar dados isolado é ideal, mas captura e *spoofing* exigem `--net=host` e capacidades extras — o que devolve boa parte do que ele isolava; aí instalar direto é mais honesto. E o que você instala fora do `apt` **não aparece em inventário nenhum**: em teste autorizado você precisa responder por cada ferramenta que rodou no ambiente do cliente.

GUIA VISUAL 039

# O PROCESSO DE INICIALIZAÇÃO

*Do POST ao PID 1 — e como descobrir em qual dos cinco elos o sistema parou.*

039
GUIAS LINUX

## A CADEIA DE CINCO ENTREGAS — E A TELA DE CADA FALHA

**1 FIRMWARE**
POST, NVRAM. Lê a ESP e chama um `.efi`.
`no bootable device`

→

**2 GRUB**
Menu: escolhe kernel e parâmetros.
`grub rescue>`

→

**3 KERNEL**
Descompacta `vmlinuz`, detecta hardware.
`Kernel panic`

→

**4 INITRAMFS**
Carrega módulos, abre LUKS/LVM, monta a raiz.
`(initramfs)`

→

**5 PID 1**
`switch_root` e o systemd assume.
`logs travados numa unit`

## OS DOIS FIRMWARES

### BIOS (LEGADO)

- ✓ Executa os primeiros 512 bytes do disco — o **MBR**.
- ✓ Só 446 bytes úteis: o GRUB precisa se espalhar em estágios.
- ✓ Tabela de partições `dos`, quatro entradas primárias.

### UEFI

- ✓ Lê FAT32: a **ESP** em `/boot/efi`, com arquivos `.efi` por fornecedor.
- ✓ Ordem de boot em variáveis da NVRAM — `efibootmgr -v`.
- ✓ Teste definitivo: existe `/sys/firmware/efi`? Então é UEFI.

## O `GRUB.CFG` É GERADO · O QUE O KERNEL RECEBE

**`/etc/default/grub`**
timeout, resolução, parâmetros do kernel

**`/etc/grub.d/*`**
scripts que geram cada entrada do menu

▼

`sudo update-grub`

▼

**`/boot/grub/grub.cfg`**
gerado — editar aqui some na próxima atualização de kernel

| PARÂMETRO DO KERNEL | FUNÇÃO |
|---|---|
| `root=UUID=…` | Qual partição é a raiz (não renumera) |
| `ro quiet splash` | Raiz só-leitura, sem logs, tela gráfica |
| `nomodeset` | Contorna driver de vídeo / tela preta |
| `systemd.unit=rescue.target` | Sobe em modo mínimo |
| `init=/bin/bash` | Shell no lugar do PID 1 — último recurso |

## NA PRÁTICA

kali@kali: ~

```
$ ls /sys/firmware/efi && echo UEFI || echo BIOS   # o teste que vale
$ cat /proc/cmdline
BOOT_IMAGE=/boot/vmlinuz-6.12.13-amd64 root=UUID=8f3a... ro quiet splash
$ systemd-analyze
4.512s (firmware) + 2.883s (loader) + 3.201s (kernel) + 11.744s (userspace)
$ sudo efibootmgr -o 0002,0000,0001   # reordena o boot, nao apaga nada
$ sudo update-initramfs -u            # reparo padrao apos trocar disco
```

**Cuidado com `efibootmgr -b NNNN -B` :** apaga a entrada da NVRAM para sempre e pode deixar o outro sistema do dual boot inacessível. Salve antes ( `efibootmgr -v > ~/efi-antes.txt` ) e prefira `-o` , que só reordena. Gravar imagem com `dd` em `/dev/sda` destrói o GRUB em BIOS.

## RESUMO RÁPIDO

- ✓ Diagnosticar "não inicia" é decidir qual dos **cinco elos** foi o último a rodar.
- ✓ Reinstalar o Windows não apaga o GRUB — só reescreve a `BootOrder` .
- ✓ O initramfs existe para romper a dependência circular: o driver do disco raiz mora dentro do disco raiz.
- ✓ Tecla **e** no menu do GRUB edita a entrada só para este boot; **Ctrl+X** inicia. Nada é gravado.

## NO KALI

O **live** monta um `squashfs` só-leitura com *overlay* na RAM — por isso tudo some ao desligar; persistência exige partição rotulada `persistence` . O **modo forense** difere só na linha do kernel: não monta nada automaticamente. E **Secure Boot** não trava o boot, trava o driver Wi-Fi: `Required key not available` em módulo DKMS é isso — confira com `mokutil --sb-state` .

GUIA VISUAL 040

# SYSTEMD: UNITS E TARGETS

*Em vez de um script que faz, um arquivo que descreve — e o PID 1 supervisiona.*

040
GUIAS LINUX

## ANATOMIA DE UM ARQUIVO `.SERVICE`

**[Unit]**

```
Description=Coletor
After=network.target
Wants=network.target
```

*Quem sou e de quem dependo. Vale para todo tipo de unit.*

**[Service]**

```
Type=simple
ExecStart=/usr/bin/coletor
User=coletor
Restart=on-failure
RestartSec=5s
```

*Como executar e supervisionar. Só existe em **.service**.*

**[Install]**

```
WantedBy=multi-user.target
```

*Lida **só** no **enable**: diz onde criar o link que faz o serviço subir no boot. Sem ela: No installation config specified.*

TUDO É UNIT: `.service` processo · `.target` marco · `.socket` ativa sob conexão · `.timer` agenda · `.mount` montagem · `.path` vigia caminho · `.device` udev · `.slice` cgroup

## REQUISITO × ORDEM — DOIS EIXOS INDEPENDENTES

### "B PRECISA DE A?" — REQUISITO

- ✓ `Wants=` puxa A; se A falhar, B sobe assim mesmo. **O padrão sensato.**
- ✓ `Requires=` puxa A; se A falhar ao iniciar, B é desativado.
- ✓ `BindsTo=` se A parar *a qualquer momento*, B para junto.
- ✓ `Conflicts=` iniciar B para A.

### "B COMEÇA DEPOIS DE A?" — ORDEM

- ✓ `After=` / `Before=` — e nada mais.
- ✓ Sozinho, `Requires=` **não ordena**: os dois sobem em paralelo.
- ✓ Erro clássico: funciona na máquina lenta, quebra na rápida.
- ✓ Quase sempre se quer o par: `Requires=db.service` + `After=db.service`.

## TARGETS SÃO MARCOS · E QUEM VENCE ENTRE OS DIRETÓRIOS

| TARGET | MARCO QUE REPRESENTA |
|---|---|
| `sysinit` | FS, dispositivos, `/tmp` |
| `basic` | Mínimo pronto; sockets e timers ativos |
| `network` | Rede configurada — **não** é "tem internet" |
| `network-online` | Conectividade de fato (custa boot) |
| `multi-user` | Sistema completo em modo texto |
| `graphical` | Inclui o multi-user + login gráfico |
| `rescue` / `emergency` | Shell mínimo / só a raiz somente leitura |

**`/run/systemd/system/` ↑ vence**
tempo de execução — some no reboot

**`/etc/systemd/system/`**
do administrador — o seu lugar

**`/lib/systemd/system/`**
dos pacotes — o APT sobrescreve

**`…service.d/override.conf`**
o *drop-in*: muda só a chave desejada e continua recebendo as correções do pacote. Crie com `systemctl edit` — evite o `--full`.

## NA PRÁTICA

kali@kali: ~

```
$ systemctl cat ssh.service      # o arquivo + todos os drop-ins
$ systemctl show ssh -p Type -p Restart   # config efetiva, com os padroes
$ systemd-delta                  # o que neste sistema foge do padrao
$ sudo systemctl edit ssh.service         # cria o drop-in no lugar certo
$ sudo systemctl daemon-reload   # OBRIGATORIO apos editar unit na mao
$ sudo systemctl start tunel@servidor1    # template: %i = servidor1
```

### RESUMO RÁPIDO

- ✓ `Type=` diz **como o systemd sabe que subiu**: `simple` (primeiro plano), `forking` + `PIDFile=` (daemon clássico), `oneshot` (roda e acaba).
- ✓ Sem `daemon-reload`, a edição "não faz efeito" — o systemd opera com a versão em memória.
- ✓ *start repeated too quickly* é sintoma, não causa: o serviço falha na partida por outro motivo.
- ✓ Units de usuário em `~/.config/systemd/user/` dispensam `sudo` — substituem o `@reboot` do crontab.

### NO KALI

Serviços de rede vêm instalados e **desabilitados** de propósito — máquina de teste vive em rede hostil. Habilitar é ato consciente: `systemctl enable --now ssh`. O par `Requires= + After=postgresql.service` é o caso do Metasploit. E `Type=forking` sem `PIDFile=` é o erro nº 1 ao empacotar ferramenta: se ela tiver `--no-daemon`, use-a com `Type=simple`.

GUIA VISUAL 041

# GERENCIANDO SERVIÇOS

*"Parei o serviço e ele voltou no reboot" — todo serviço tem **dois** estados independentes.*

041
GUIAS LINUX

## RODANDO AGORA × SOBE NO BOOT

| | `ENABLED` · SOBE NO BOOT | `DISABLED` · NÃO SOBE |
|---|---|---|
| `ACTIVE` RODANDO | **Serviço em produção**<br>É o que se quer no fim.<br>`systemctl enable --now` | **Roda agora, some no reboot**<br>Serve para teste.<br>`systemctl start`<br>*o padrão do Kali* |
| `INACTIVE` PARADO | **Parado agora — volta no reboot**<br>A armadilha clássica.<br>`systemctl stop`<br>*só metade do serviço foi desligada* | **Desligado de fato**<br>Nem agora, nem depois.<br>`systemctl disable --now`<br>*`mask`*<br>*se nem dependência pode ativar* |

## LENDO O `SYSTEMCTL STATUS` DE VERDADE

kali@kali: ~

```
$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; preset: disabled)
     Active: active (running) since Sat 2026-08-01 09:14:02 -03; 2h 31min ago
   Main PID: 1187 (sshd)
     CGroup: /system.slice/ssh.service
             └─1187 "sshd: /usr/sbin/sshd -D [listener]"
# ago 01 09:14:02 kali sshd[1187]: Server listening on 0.0.0.0 port 22.
```

### LOADED

Caminho do arquivo (pacote ou `/etc`), habilitação e o **preset** da distro.

### ACTIVE

Estado e **há quanto tempo**. Tempo curto demais = laço de reinício.

### CGROUP

A árvore que **pertence** à unit. Por isso `stop` não deixa órfão.

### CUIDADO

**running ≠ funciona.** Confirme pelo efeito: `ss -tlnp`, uma requisição.

## QUAL VERBO USAR

**`restart`** — mata e sobe outro. Todas as conexões caem. Sempre funciona.

**`reload`** — manda `SIGHUP` : relê a config sem morrer. Só existe se a unit tiver `ExecReload=` .

**`reload-or-restart`** — tenta o melhor caso e cai para o outro. O verbo certo em script.

**`--failed` / `reset-failed`** — listar e limpar. O estado `failed` é marca deliberada: não some sozinho.

| QUERO... | COMANDO |
|---|---|
| Parar agora, talvez voltar depois | `stop` |
| Que não suba no boot | `disable` |
| Nem agora nem depois | `disable --now` |
| Que nenhuma dependência ative | `mask` |
| Ver o que está carregado | `list-units` |
| Ver o que está instalado em disco | `list-unit-files` |

**Dois comandos que custam a máquina remota:** `systemctl restart ssh` com `sshd_config` inválido mantém a sessão atual viva e recusa toda conexão nova — valide antes com `sudo sshd -t`, prefira `reload` e abra uma segunda sessão antes de fechar a primeira. E `systemctl isolate rescue.target` derruba rede e SSH junto. Mascarar `dbus` ou `systemd-logind` "para economizar" quebra o sistema com erros que não citam a unit mascarada.

## RESUMO RÁPIDO

- ✓ `is-active` e `is-enabled` retornam **código de saída** — é assim que script consulta estado, não com `grep` no `status`.
- ✓ `static` não é defeito: a unit é puxada por outra, não habilitada sozinha.
- ✓ `systemctl --failed` abre qualquer diagnóstico. Máquina sadia: *0 loaded units*.
- ✓ `daemon-reload` relê os arquivos; `daemon-reexec` reinicia o próprio PID 1.

## NO KALI

`preset: disabled` é política, não anomalia. Ao ligar o SSH, **troque as chaves de host**: a imagem vem com chaves iguais em toda máquina clonada — `rm /etc/ssh/ssh_host_*` e `dpkg-reconfigure openssh-server`. Rode `systemctl --failed` depois de todo `apt full-upgrade` no rolling. Em limites de laboratório use `set-property --runtime`: some no reboot, sem drop-in esquecido.

GUIA VISUAL 042

# LOGS: JOURNALD E SYSLOG

*Caiu às três da manhã e voltou sozinho: onde está o registro — e se sobreviveu ao reboot.*

042
GUIAS LINUX

## QUATRO FONTES, UM JOURNAL

- **kernel** — buffer do `dmesg`
- **serviços** — saída padrão e de erro
- **syslog** — socket `/dev/log`
- **API nativa** — `sd_journal_send()`

→ **`systemd-journald`** — Acrescenta metadados e indexa por campo →

- **/var/log/journal/** — binário, indexado, verificável
- **rsyslog → /var/log/syslog** — texto puro, se instalado — é o caminho do envio remoto

## UM REGISTRO É UM CONJUNTO DE CAMPOS — E CADA CAMPO FILTRA

### CAMPOS COM _ NÃO MENTEM

`_PID`, `_UID`, `_COMM`, `_SYSTEMD_UNIT` vêm do kernel, não do programa. É o que dá valor forense ao journal.

### SERVIÇO NÃO ABRE ARQUIVO

Basta escrever na saída padrão: o journald carimba com unit, PID, cgroup e boot. Quem grava log próprio perde a correlação.

### FILTRO EXATO

`journalctl _COMM=sudo` traz só o que o binário sudo emitiu — sem falso positivo textual.

`-P ERR` TRAZ →

| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|
| emerg | alert | crit | err | warning | notice | info | debug |

## NA PRÁTICA — OS FILTROS SE COMBINAM COM E LÓGICO

kali@kali: ~

```
$ journalctl -u ssh -f                # acompanha esta unit em tempo real
$ journalctl -b -1 -p err             # erros do boot ANTERIOR
$ journalctl -u ssh -p warning --since "2 days ago" -o short-iso
$ journalctl --list-boots             # 0 = agora, -1 = o anterior
$ journalctl -u gvmd -o json | jq -r '.MESSAGE'   # log componivel
```

## O LOG DE ONTEM EXISTE?

### VOLÁTIL (PADRÃO SEM O DIRETÓRIO)

`/run/log/journal/` em **tmpfs**: consome RAM e some no reboot. `journalctl -b -1` não acha nada.

### PERSISTENTE

Basta existir `/var/log/journal/`. Controle explícito: `Storage=persistent` e `SystemMaxUse=500M` em `journald.conf` — sem limite ele usa até **10% do disco**.

### O SYSLOG NÃO MORREU

Texto puro para ferramentas externas, appliances que não falam journal, roteamento por regra e — o principal — **envio remoto** a um coletor central.

`journalctl --vacuum-*` **apaga registros de forma irreversível** — num sistema sob investigação, destrói a própria evidência. Exporte antes ( `-o export` ) e prefira limitar por `SystemMaxUse=` , que é previsível.

## RESUMO RÁPIDO

- ✓ `-p err` mostra `err` **e tudo acima**; para faixa exata, `-p warning..err`.
- ✓ `-o short-iso --utc` é o par sensato para qualquer coisa que vá para relatório.
- ✓ `--verify` detecta corrupção; o **FSS** (`--setup-keys`) detecta adulteração — não impede, torna visível.
- ✓ Remover a unit não apaga o log dela: o registro do que aconteceu permanece.

## NO KALI

Em **live**, todo log vive em RAM: exporte durante a sessão (`-o export`) ou perde. Para depurar ferramenta que roda como serviço — GVM, Metasploit, BeEF — `journalctl -u <unit> -p err -b` responde mais rápido que caçar arquivo em `/opt`. Em pentest autorizado, o journal exportado é **parte do produto**: delimita a janela do que foi feito. Do lado defensivo, FSS + envio remoto respondem ao apagamento de rastros.

# AGENDAMENTO: CRON, AT E TIMERS

*O script roda no terminal e falha às 3h da manhã — a culpa quase nunca é do agendador.*

**043**
GUIAS LINUX

## ANATOMIA DE UMA LINHA DO `CRONTAB`

| 30 | 3 | * | * | 1-5 | /usr/local/bin/backup.sh |
|---|---|---|---|---|---|
| MINUTO 0–59 | HORA 0–23 | DIA DO MÊS 1–31 | MÊS 1–12 | DIA DA SEMANA 0–7 (0 e 7 = dom) | COMANDO com caminho absoluto |

`*` qualquer `,` lista (1,15) `-` intervalo (1-5) `/` passo (*/10) **`@daily` `@hourly` `@reboot`** atalhos

**A pegadinha dos dois campos de dia:** quando *dia do mês* e *dia da semana* são ambos diferentes de `*`, o cron aplica **OU**, não E. `0 0 13 * 5` roda todo dia 13 **e** toda sexta — não só nas sextas-feiras 13.

## POR QUE FALHA EM SILÊNCIO: O AMBIENTE

### PATH MÍNIMO
Só `/usr/bin:/bin`. Nada de `~/.bashrc`. Sempre **caminho absoluto** dentro do script.

### SHELL=/BIN/SH
Não é bash: `[[ ]]`, arrays e `source` quebram. Use shebang `#!/bin/bash` ou declare `SHELL=`.

### % É ESPECIAL
No crontab, `%` vira nova linha: `date +%Y` é truncado. Escape com `\%` — melhor, mova para um script.

### A SAÍDA VIRA E-MAIL
Sem MTA, o erro **desaparece**. Sempre `>> /var/log/x.log 2>&1`.

## CRON × TIMER DO SYSTEMD — QUANDO TROCAR

### CRON — PESSOAL, RÁPIDO, PORTÁTIL

- ✓ Uma linha resolve; existe em todo Unix.
- ✓ Tabelas do **sistema** ( `/etc/crontab` , `/etc/cron.d/` ) têm um campo a mais: **o usuário**.
- ✓ Nome com ponto em `/etc/cron.d/` é ignorado *sem aviso*.
- ✓ **anacron** recupera o que não rodou com a máquina desligada.

### TIMER — QUANDO A TAREFA IMPORTA

- ✓ **Dois arquivos**: `.timer` + `.service` . Habilita-se o **timer**; o serviço fica `static` .
- ✓ Log automático no journal e falha em `systemctl --failed` .
- ✓ **Nunca sobrepõe execuções** — a unit já está ativa, a 2ª é descartada.
- ✓ `Persistent=true` recupera atraso · `RandomizedDelaySec=` evita rajada · `CPUQuota=` / `MemoryMax=` limitam.

## NA PRÁTICA

kali@kali: ~

```
$ env -i /bin/sh -c '/usr/local/bin/backup.sh'  # reproduz o ambiente do cron
$ systemd-analyze calendar "Mon..Fri 09:00"     # valida ANTES de ativar
Normalized form: Mon..Fri *-*-* 09:00:00   Next elapse: Mon 2026-08-03 09:00:00
$ sudo systemctl enable --now backup.timer      # o TIMER, nao o service
$ systemctl list-timers --all    # NEXT e LAST: "isso esta mesmo rodando?"
$ echo "/opt/coleta.sh" | at now + 2 hours       # uma vez so; atq / atrm
```

`crontab -r` **apaga a tabela inteira sem confirmar** — e a tecla `r` fica ao lado do `e` . Não há desfazer. Mantenha a tabela num arquivo versionado e instale com `crontab arquivo.cron` ; no mínimo, `crontab -l > ~/crontab.bak` antes de editar.

## RESUMO RÁPIDO

- ✓ Falhou de madrugada? Teste com `env -i /bin/sh -c 'script'` — dez segundos que poupam uma noite.
- ✓ `at` preserva o ambiente e o diretório do momento do agendamento; exige o `atd` ativo.
- ✓ Critério: se alguém vai **notar** quando a tarefa falhar, ela merece um timer. Senão, cron está certo.
- ✓ `systemd-analyze calendar` é o teste que o cron nunca ofereceu.

## NO KALI

Ao receber uma máquina de terceiros, o inventário é de **cinco comandos**: `crontab -l`, `sudo crontab -l -u root`, `ls /etc/cron.d/`, `atq` e `systemctl list-timers --all` — é onde moram tarefas esquecidas e onde se procura **persistência de atacante**. Em teste autorizado, o escopo inclui **quando**: timer com `OnCalendar=` validado e carimbo no journal comprova a janela. Nunca automatize `apt full-upgrade` no rolling — agende só o `apt update` e decida você.

GUIA VISUAL 044

# RECUPERAÇÃO E PROBLEMAS DE BOOT

*Uma linha a mais no `fstab` e a máquina para em emergency mode. O difícil é chegar até a correção.*

044
GUIAS LINUX

## A ESCADA DA RECUPERAÇÃO — DESÇA UM DEGRAU POR VEZ

MAIS INVASIVO ↓

1. **Kernel anterior no menu do GRUB** — resolve atualização de kernel que quebrou o sistema. Por isso **dois kernels** sempre instalados.
2. **Tecla e: parâmetro temporário** — `nomodeset`, `systemd.unit=rescue.target`. Nada é gravado; `Ctrl+X` inicia.
3. **rescue.target** — roda o `sysinit`: raiz montada, **fstab aplicado**, swap, journal, shell de root. Sem rede.
4. **emergency.target** — só a raiz, somente leitura, nada montado. **É o degrau para quando o problema é o próprio fstab.**
5. **Live USB + chroot** — quando não inicia de forma alguma. Mais poder e mais chance de estragar o sistema errado.

## DO SINTOMA NA TELA À AÇÃO

| O QUE APARECE NA TELA | CAUSA PROVÁVEL | O QUE FAZER |
|---|---|---|
| Sem menu: `grub rescue>` | GRUB não achou os próprios módulos | `set root=` + `set prefix=` + `insmod normal` + `normal` ; depois `grub-install` e `update-grub` |
| `(initramfs)` + `ALERT! UUID… does not exist` | UUID da raiz mudou, ou falta o módulo do controlador | Compare `blkid` com `/proc/cmdline` ; `update-initramfs -u` via chroot |
| `You are in emergency mode` | Montagem do `fstab` falhou e derrubou o `local-fs.target` | `mount -o remount,rw /` , corrigir o `fstab` , validar com `findmnt --verify` |
| `A start job is running for…` contando 90 s | Serviço travado — quase sempre um `wait-online` | `journalctl -b -1 -p err` ; `systemd.mask=` no próximo boot; depois `disable` |
| Sobe, mas `Read-only file system` | Kernel remontou a raiz por erro de FS — ou acabaram os **inodes** | `dmesg \| grep -i ext4` ; `df -h` **e** `df -i` ; `fsck` com o FS desmontado |

DEPURAR O BOOT PELA LINHA DO KERNEL: **`systemd.unit=rescue.target`** **`systemd.mask=`**`x.service` — só neste boot **`systemd.debug_shell`** — shell no Alt+F9 **`systemd.log_level=debug`** **`rd.break`** — para dentro do initramfs

## NA PRÁTICA — O `CHROOT` DO DEGRAU 5

kali@kali: ~ (live USB)

```
$ lsblk -f && sudo mount /dev/sda2 /mnt
$ sudo mount /dev/sda1 /mnt/boot/efi   # a ESP: o passo esquecido no 1
$ for d in /dev /dev/pts /proc /sys /run; do sudo mount --bind "$d" "/mnt$d"; done
$ sudo chroot /mnt /bin/bash
# cat /etc/os-release; cat /etc/hostname; ls /home/   # qual sistema e este?
# grub-install --target=x86_64-efi --efi-directory=/boot/efi && update-grub
$ sudo findmnt --verify --verbose && sudo mount -a   # valide ANTES do reboot
```

**Dentro do `chroot` , confirme em qual sistema você está antes de qualquer comando.** Com dois sistemas ou um disco de backup ligado há mais de uma raiz plausível, e um `grub-install` ou `passwd root` no lugar errado não dá erro nenhum — o estrago é silencioso e o problema original continua. Na máquina remota, `systemctl isolate rescue.target` derruba a rede e o SSH junto.

## RESUMO RÁPIDO

- ✓ `nofail` **em toda montagem não essencial** — com `x-systemd.device-timeout=10` corta a espera de 90 s.
- ✓ `findmnt --verify` + `mount -a` antes de reiniciar: 5 segundos que substituem uma sessão de recuperação.
- ✓ `isolate` age **agora**; `set-default` age no **próximo boot**.
- ✓ Reparar não é entender: depois que voltar, `journalctl -b -1 -p err` diz por que caiu — senão cai de novo.

## NO KALI

O **live USB do Kali é a ferramenta de recuperação** que você já tem: traz `cryptsetup`, `lvm2`, `gdisk`, `testdisk`. Com disco criptografado, abra antes do chroot: `cryptsetup luksOpen /dev/sda3 raiz` + `vgchange -ay`. Cuidado com `apt autoremove`: ele leva o kernel anterior, que é o degrau 1. E **modo forense ≠ recuperação** — se a máquina puder virar objeto de investigação, imagem com hash primeiro, reparo depois.

GUIA VISUAL 045

# DISCOS, PARTIÇÕES E NOMES DE DISPOSITIVO

*A VM migrou de hipervisor e o boot para em* `Cannot open root device "sda2"`*. O disco está intacto — o que mudou foi o* ***nome****.*

045
GUIAS LINUX

## ANATOMIA DE UM NOME — QUEM BATIZA É O KERNEL, NÃO O DISCO

**SATA · SAS · USB · SCSI**

`sd` (família do driver) `a` (disco: a, b, c...) `1` (partição: 1, 2, 3...)

**sd** não é "SATA disk": vem de **SCSI disk** — SATA, SAS e USB passam todos pela mesma camada de tradução. O `/dev/hdX` do IDE não existe mais desde o kernel 2.6.19.

**NVME**

`nvme` (família) `0` (controladora — o chip) `n1` (*namespace* — o "disco") `p2` (partição)

Um nível a mais: a controladora expõe vários *namespaces*. Daí o prefixo **p** — sem ele `nvme0n11` seria ambíguo. Mesma regra em `mmcblk0p1` e `loop0p1`.

`vdX` virtio (KVM/nuvem) · `xvdX` Xen · `mmcblkXpY` SD/eMMC · `loopX` arquivo como disco · `mdX` RAID · `mapper/nome` LVM e LUKS · `srX` óptico

## A TABELA DE PARTIÇÃO: MBR × GPT

**MBR** — 1983 · setor 0

**2 TiB** por disco (setor de 32 bits) · **4 primárias** (mais que isso exige a estendida encadeada) · **sem cópia de segurança**: 512 bytes são ponto único de falha.

**GPT** — 2000 · GUID

Endereços de **64 bits** · **128 partições** equivalentes, cada uma com GUID e rótulo · **cópia integral no fim do disco**, conferida por CRC32 — é ela que o `gdisk` usa para recuperar a tabela. **Disco novo, GPT sempre.**

## IDENTIFICADORES PERSISTENTES: `/DEV/DISK/BY-*`

| DIRETÓRIO | O QUE IDENTIFICA | SOBREVIVE A | USE PARA |
|---|---|---|---|
| `by-uuid` | o **sistema de arquivos** dentro da partição | trocar de porta, mover o disco de máquina | montar — é o que o instalador põe no `fstab` |
| `by-id` | o **hardware**: modelo + número de série | reparticionar, reformatar, trocar de porta | operar sobre o disco inteiro em script |
| `by-partuuid` | a **partição** na tabela GPT | reformatar o sistema de arquivos | apontar uma partição que será reformatada |
| `by-label` | o rótulo do sistema de arquivos | mover o disco; some se o rótulo mudar | mídia removível (a persistência do Kali) |
| `by-path` | a **posição física**: a porta do barramento | trocar o disco por outro na mesma porta | "qual disco está na baia 3", em servidor |

kali@kali: ~

```
$ lsblk -o NAME,MAJ:MIN,SIZE,ROTA,PARTTYPENAME,FSTYPE,MOUNTPOINT
NAME   MAJ:MIN  SIZE ROTA PARTTYPENAME      FSTYPE MOUNTPOINT
sda      8:0    100G    0                           # ROTA 0 = SSD, 1 = mecânico
├─sda1   8:1     512M   0 EFI System        vfat   /boot/efi
├─sda2   8:2    97,5G   0 Linux filesystem  ext4   /
└─sda3   8:3       2G   0 Linux swap                # declarado, mas ainda sem mkfs
$ sudo fdisk -l /dev/sda | grep -iE 'disklabel|sector'   # gpt ou dos? 512e?
$ ls -l /dev/disk/by-id/ && sudo smartctl -H /dev/sda    # série + saúde
```

**Nunca escreva `/dev/sdX` em fstab, crypttab, linha do kernel ou script.** A letra vem da ordem de detecção no boot: trocar de porta SATA, somar um disco ou esquecer um pendrive espetado reordena tudo. O melhor caso é um volume que não monta; o pior é um `mkfs` no disco errado porque as letras trocaram entre o `lsblk` e a execução. Use `UUID=` para montar e `by-id` para operar. E lembre: **clonar com `dd` duplica o UUID** — no clone, `tune2fs -U random` (ext4) ou `xfs_admin -U generate`.

## RESUMO RÁPIDO

- ✓ Tudo que guarda dado é **dispositivo de bloco** (o `b` do `ls -l`), roteado pelo par *major/minor* — por isso disco, RAID, LVM e LUKS se empilham.
- ✓ **`by-uuid` para montar, `by-id` para operar** sobre o disco inteiro.
- ✓ **Código de tipo é declaração, não formatação:** `PARTTYPENAME` é o que a tabela diz, `FSTYPE` é o que existe. Só `EF00` (boot UEFI) e `8E00` / `FD00` (varredura de LVM/RAID) mudam comportamento.
- ✓ **Primeira partição no setor 2048 = 1 MiB.** Fora do alinhamento, o disco 512e faz ler-modificar-escrever e a escrita cai várias vezes — sem aparecer em log nenhum.

## NO KALI

**No live USB a nomenclatura é outra**: o próprio pendrive costuma pegar o `sda` e empurrar o disco interno para `sdb`. Antes de qualquer escrita, confirme por **modelo e número de série** (`lsblk -o NAME,SIZE,MODEL,SERIAL`), nunca pela letra. Em perícia, primeiro `blockdev --setro /dev/sdX` e confira com `--getro`: montar em leitura-e-escrita já altera *timestamps* e o *journal*. A persistência do Kali é achada pelo **rótulo** `persistence` — imune à reordenação. E todo o laboratório do Vol 8 roda em `losetup` sobre um arquivo de 1 GiB: erro custa um `rm`, não os seus dados.

GUIA VISUAL 046

# PARTICIONAMENTO FDISK · GDISK · PARTED

*Gravou a partição, mas o `lsblk` não mostra e o `mkfs` falha:* ***a tabela no disco e o mapa que o kernel tem são coisas diferentes.***

046
GUIAS LINUX

## QUAL FERRAMENTA — E A DIFERENÇA QUE SALVA DADOS

### INTERATIVAS — PERDOAM ATÉ O `w`

**`fdisk`**
MBR e GPT; o padrão do dia a dia

**`gdisk`**
só GPT; converte MBR → GPT e recupera a tabela

**`cfdisk`**
menus; bom para aprender

**`parted`**
**grava na hora** — sem `w`, sem desfazer

### PROGRAMÁVEIS

**`sgdisk`**
o gdisk sem menu: `-n n:ini:fim`, `-t`, `-c`

**`sfdisk`**
`--dump` salva e restaura a tabela

### RECUPERAÇÃO

**`testdisk`**
reconstrói tabela perdida

**`wipefs`**
sem `-a` só **lista** assinaturas — seguro

DENTRO DO FDISK: `m` ajuda · `p` imprime a tabela em memória · `n` nova · `d` apaga · `t` tipo · `g` GPT nova · `o` MBR nova · `v` verifica · `w` grava e sai · `q` sai sem gravar

## REDIMENSIONAR: A ORDEM NÃO PODE SER INVERTIDA

### AUMENTAR — DE FORA PARA DENTRO

**1. a partição**
`parted resizepart 1 100%`

→

**2. o sistema de arquivos**
`resize2fs · xfs_growfs`

Crescer a partição primeiro garante espaço para a 2ª etapa. O **ext4 cresce montado**; sem argumento, o `resize2fs` preenche a partição inteira.

### REDUZIR — DE DENTRO PARA FORA

**1. o sistema de arquivos**
`umount · e2fsck -f · resize2fs 300M`

→

**2. a partição**
`resizepart 1 350MiB`

Desmontado e com `e2fsck -f` antes, sempre. Deixe **folga** entre os dois números — arredondamento entre unidades trunca. **XFS só cresce**: reduzir exige `xfsdump`, recriar e restaurar.

**A regra:** o sistema de arquivos **nunca** pode ser maior que a partição que o contém. Com **LVM**, `lvextend -r` e `lvreduce -r` coordenam as duas camadas na ordem certa sozinhos.

## NA PRÁTICA — DO `W` AO `DEVICE OR RESOURCE BUSY`

kali@kali: ~

```
$ sudo sfdisk --dump /dev/sdb > ~/tabela-sdb.txt   # 2 KB que desfazem o acidente
$ sudo fdisk /dev/sdb          # g → n → <Enter> <Enter> +400M → p (confira!) → w
$ sudo mkfs.ext4 /dev/sdb1
mkfs.ext4: No such file or directory  # o kernel ainda tem o mapa antigo
$ sudo partprobe /dev/sdb      # ou partx -u · blockdev --rereadpt
Error: Partition(s) 2 on /dev/sdb are being used.
$ lsblk /dev/sdb; sudo fuser -mv /dev/sdb2; swapon --show; sudo dmsetup ls; cat /proc/mdstat
$ sudo umount /dev/sdb2 && sudo partprobe /dev/sdb && lsblk /dev/sdb
# libere de cima para baixo: umount → swapoff → vgchange -an → cryptsetup close
```

**Toda ferramenta desta ficha reescreve a tabela de partição.** As programáveis não têm etapa de confirmação: `sgdisk --zap-all /dev/sdX` destrói GPT principal, secundária e MBR **na hora**, e uma letra errada custa o disco inteiro. O `parted` aplica cada comando imediatamente — `rm 2` apaga a partição naquele instante, sem `q` que desfaça. Antes de tocar em disco real: confira **modelo e número de série**, salve com `sfdisk --dump` **fora do disco** e tenha backup restaurável. Ensaie tudo em `losetup` sobre arquivo-imagem.

## RESUMO RÁPIDO

- ✓ **`fdisk`, `gdisk` e `cfdisk` editam uma cópia em memória**: nada vai ao disco até o `w`, e o `q` descarta. `parted` e `sgdisk` são o oposto.
- ✓ **Gravar a tabela ≠ o kernel enxergá-la.** `partprobe` pede a releitura, e ela falha enquanto houver partição montada, swap ativa ou mapeamento de LVM, LUKS ou RAID sobre o disco.
- ✓ **Aceite os padrões do `fdisk`** (setor 2048) e informe tamanho por sufixo — `+400M`, `+20G`. Valor na mão é onde nasce o desalinhamento.
- ✓ `wipefs` sem `-a` só lista: é o primeiro comando seguro num disco de procedência desconhecida. Em SSD, `shred` não apaga de verdade — o controlador remapeia blocos; use *secure erase* do firmware.

## NO KALI

No instalador, **particionamento guiado com LVM criptografado** é a escolha para máquina de trabalho: EFI + `/boot` em claro + LUKS com raiz e swap dentro. Reserve **1 GiB de `/boot`** — no *rolling* os *initramfs* antigos enchem a partição e travam o `apt` no meio da atualização. O uso real mais comum do `fdisk` aqui é criar a 3ª partição do pendrive de persistência (rótulo `persistence`, ext4) — e é onde a troca de letras formata o disco interno. Em perícia, **nunca particione a mídia**: copie com `dd` / `dcfldd`, confira o *hash* e trabalhe sobre `losetup -rP` (o `-r` é somente leitura). E depois de expandir o disco da VM no hipervisor, o trabalho de dentro é seu: `resizepart` e só então `resize2fs`.

GUIA VISUAL 047

# SISTEMAS DE ARQUIVOS EXT4 · XFS · BTRFS

*Depois do apagão o sistema sobe em segundos, mas o documento salvo há pouco voltou vazio:* ***o journal protege a estrutura, não o conteúdo.***

047
GUIAS LINUX

## DENTRO DA PARTIÇÃO — DE BLOCO NUMERADO A ARQUIVO

| superbloco | descritores | mapa | mapa | tabela de inodes | blocos de dados |
|---|---|---|---|---|---|
| tamanho, UUID, rótulo, estado — com cópias espalhadas | de grupo | de blocos | de inodes | dono, permissões, datas e ponteiros — **menos o nome** | o conteúdo dos arquivos |

**JOURNAL** — Registra a **intenção** de alterar metadados antes de aplicá-la: após uma queda o kernel refaz ou descarta o que ficou pela metade, em segundos, sem varrer o disco. **O padrão** `data=ordered` **grava os dados antes dos metadados** — nenhum arquivo aponta para lixo, mas a última escrita pode ter se perdido (alocação atrasada). Só `fsync()` garante dado no disco.

## TRÊS CANDIDATOS — E A DECISÃO

### EXT4
PADRÃO DO DEBIAN E DO KALI

- **Maduro e previsível**, com o melhor ferramental de recuperação.
- *Extents*: intervalos contíguos no lugar de ponteiro por bloco.
- **Inodes fixos na formatação** — dá para acabar com o disco pela metade; o diagnóstico é `df -i`.
- **5% reservados ao root** mantêm a máquina administrável com a raiz cheia. Em disco de dados, `-m 1`.

### XFS
PADRÃO DA FAMÍLIA RHEL

- Grupos de alocação independentes: **escrita concorrente em paralelo**.
- Dezenas de TB, arquivos grandes; inodes alocados dinamicamente.
- **Só cresce** — e com o volume **montado**: `xfs_growfs /ponto`.
- **Não encolhe.** Reduzir = `xfsdump`, recriar, `xfsrestore`. Dimensione por baixo com LVM.

### BTRFS
INSTANTÂNEOS E INTEGRIDADE

- **Instantâneos** quase gratuitos, subvolumes e compressão `zstd`.
- **Soma de verificação nos dados**: detecta corrupção silenciosa (ext4 e XFS não).
- `df` mente aqui — use `btrfs filesystem usage`.
- **RAID 5/6 ainda instável** (*write hole*): use `mdadm` por baixo ou fique no 1 e 10.

**Cópia-na-escrita:** alterar B não sobrescreve B — grava **B'** em outro lugar e move o ponteiro do subvolume. O B original continua íntegro, e é para ele que o instantâneo aponta. **O instantâneo não copia nada**: é um segundo conjunto de ponteiros para blocos que já existem — criar leva segundos e custa quase zero, e o consumo só cresce conforme os dados divergem. O preço é fragmentação em carga de banco (daí o atributo `nodatacow`).

```
kali@kali: ~
$ lsblk -f /dev/sdb1 && sudo wipefs /dev/sdb1   # 1. o que já existe ali? (só lista)
$ sudo mkfs.ext4 -n /dev/sdb1                    # 2. SIMULA: imprime o layout e as
Backup superblock at 32768, 98304, 163840, …   #    cópias do superbloco. Não formata.
$ sudo mkfs.ext4 -L dados -m 1 /dev/sdb1         # 3. formata e rotula
$ sudo blkid /dev/sdb1                           # 4. confirma UUID e tipo
$ sudo tune2fs -l /dev/sdb1 | grep -i state      # clean = desmontado direito
Filesystem state:         clean
$ sudo umount /dev/sdb1 && sudo e2fsck -b 32768 -y /dev/sdb1  # superbloco danificado
$ sudo fstrim -av   # TRIM periódico pelo fstrim.timer — nunca 'discard' contínuo
```

**O `mkfs` não pergunta se você tem certeza** — destrói o conteúdo sem confirmação, e a única proteção embutida (recusar volume montado) some justamente quando você desmonta para trabalhar. Antes: `lsblk -f` e leia a saída, `mkfs.ext4 -n` para simular, e em disco real refira-se por `/dev/disk/by-id/`. E **nunca rode `fsck` num volume montado com escrita**: ele altera estruturas que o kernel tem em cache e produz exatamente a corrupção que se queria evitar. No Btrfs, `check --repair` é último recurso — tente `-o ro,rescue=all` e copie os dados para fora primeiro.

## RESUMO RÁPIDO

- ✓ **O *journal* protege metadados, não conteúdo.** Arquivo com o tamanho certo e cheio de zeros é o custo da alocação atrasada, não bug.
- ✓ **Disco cheio sem estar cheio:** `df -h` mostra espaço e o sistema recusa criar arquivo → acabaram os **inodes**, veja `df -i`.
- ✓ **ext4** para quase tudo · **XFS** para volume enorme (não encolhe) · **Btrfs** para instantâneo e integridade · **FAT32** na EFI (obrigatório) · **exFAT** só para trocar com Windows/macOS.
- ✓ `mkfs.ext4 -n` num volume existente imprime **onde estão as cópias do superbloco**; `e2fsck -b <bloco>` recupera a partir de uma delas.

## NO KALI

O instalador usa **ext4**, e para trabalho de laboratório é a escolha certa. Mas **Btrfs num sistema *rolling* é apólice barata**: um `btrfs subvolume snapshot` antes do `apt full-upgrade` leva segundos e evita reinstalar quando a atualização quebra o vídeo (`snapper` / `timeshift` automatizam). Em perícia, monte **sempre** `-o ro,noatime,noexec,nodev,nosuid`: preserva as datas de acesso, que são evidência, e impede que binário ou bit SUID da mídia examinada tenha efeito na sua máquina. **exFAT no pendrive de trabalho, ext4 no de persistência** — o exFAT não guarda dono nem permissão, e a persistência formatada nele não sobe. E não zere os 5% da raiz: uma captura longa de `tcpdump` enche a `/` e é a reserva que mantém a máquina administrável.

GUIA VISUAL 048

# LVM VOLUMES LÓGICOS

*A `/var` está em 95% e sobra espaço no disco — só que **depois** da `/home`. Sem LVM, crescer significa empurrar o vizinho.*

**048**
GUIAS LINUX

## AS TRÊS CAMADAS — UM RESERVATÓRIO DE *EXTENTS* DE 4 MIB

Crescer um LV é apenas **receber mais *extents***, venham de onde vierem: não existe vizinho a empurrar, e `dados` tem 60 GiB numa máquina em que nenhum disco chega a 60. O nome real é `/dev/mapper/vg0-dados` ( `/dev/vg0/dados` é link) — **estável**, porque vem do nome que você escolheu. Em troca: **perder um PV derruba todos os LVs que o tocam**, mesmo os que estão em discos sadios — LVM não é redundância.

## MONTAR A PILHA E CRESCER COM O SISTEMA NO AR

kali@kali: ~

```
$ sudo pvcreate /dev/sdb1 /dev/sdc1 && sudo vgcreate vg0 /dev/sdb1 /dev/sdc1
$ sudo lvcreate -L 20G -n raiz vg0   # -L tamanho absoluto · -l extents ou %
$ sudo lvcreate -l 80%FREE -n dados vg0  # %FREE = o que sobrou · %VG = o total
$ sudo mkfs.ext4 -L dados /dev/vg0/dados && sudo mount /dev/vg0/dados /mnt/dados
$ sudo lvextend -L +200G -r /dev/vg0/dados   # -r = resize2fs/xfs_growfs junto,
  Size of logical volume vg0/dados changed … Filesystem resized.  # na ordem certa
$ sudo pvcreate /dev/sdd1 && sudo vgextend vg0 /dev/sdd1   # VG cheio? some um disco
$ sudo lvextend -l +100%FREE -r /dev/vg0/dados && df -h /mnt/dados   # sem reiniciar
$ sudo pvs; sudo vgs; sudo lvs -o +devices,lv_attr   # o trio de diagnóstico
```

## INSTANTÂNEO

- ✓ `lvcreate -L 5G -s -n snap /dev/vg0/dados` congela uma visão coerente **em um segundo**.
- ✓ Uso clássico: **backup consistente** — monte o snap `-o ro`, rode o `tar` e remova; a aplicação nunca para.
- ✓ `lvconvert --merge` reverte o volume ao estado do instantâneo (aplica no próximo boot).
- ✓ **Tamanho fixo:** guarda os blocos do original antes de cada alteração — acompanhe `lvs -o +data_percent`.

## TROCAR DISCO NO AR

- ✓ **1.** SMART acusa setores realocados no `sdc1` — ele ainda funciona, mas vai falhar.
- ✓ **2.** `pvcreate /dev/sdd1` + `vgextend vg0 /dev/sdd1`.
- ✓ **3.** `pvmove -i 10 /dev/sdc1` — copia *extent* a *extent*, montado, e é **interrompível e retomável**.
- ✓ **4.** `vgreduce vg0 /dev/sdc1` + `pvremove`: o disco sai sem janela de manutenção.

**Reduzir continua perigoso, mesmo com LVM.** `lvreduce -r` coordena a ordem, mas exige desmontar, `e2fsck -f` antes e backup restaurável — e **em XFS simplesmente falha** (XFS não encolhe). Sem o `-r`, o `lvreduce` corta o volume por baixo do sistema de arquivos: perda imediata. Quando a pergunta de confirmação surpreender, a resposta é `n`. ***Thin pool* que enche corrompe tudo que depende dele** — monitore `data_percent` e `metadata_percent` (este esgota antes com mais frequência) e configure o `thin_pool_autoextend_threshold`.

## RESUMO RÁPIDO

- ✓ **`-r` é a opção que importa.** Sem ela o LV cresce, o sistema de arquivos fica para trás e o `df` não muda — parece que o comando não funcionou.
- ✓ **`-L` é tamanho absoluto; `-l` é *extent* ou porcentagem.** `100%FREE` ≠ `100%VG` — confundir os dois é o erro clássico no segundo volume.
- ✓ **Não aloque 100% do VG.** Sem espaço livre não há instantâneo nem extensão: reserve de 10 a 20% sem alocar.
- ✓ **`vgcfgrestore` desfaz erro de operação:** cada alteração é arquivada em `/etc/lvm/archive/`, e um `lvremove` errado volta de lá — desde que nada tenha sido escrito por cima. Inclua `/etc/lvm/` no backup.
- ✓ **Ordem:** montar de baixo para cima (`pvcreate → vgcreate → lvcreate → mkfs`), desmontar de cima para baixo. Disco com LVM que recusa reparticionar quase sempre só precisa de `vgchange -an`.

## NO KALI

O instalador oferece **LVM criptografado**, e a ordem é sempre **LUKS embaixo, LVM em cima**: uma senha abre tudo e todo volume fica protegido. Num *rolling*, a `/var` cresce sem avisar (cache do APT, logs de ferramenta) e `lvextend -l +100%FREE -r` resolve em segundos, sem reiniciar. Instantâneo antes do `apt full-upgrade` é hábito barato — mas **dimensione pelo tamanho da atualização**: um *full-upgrade* grande enche um instantâneo pequeno e o **invalida no pior momento**. Em perícia, cuidado com a ativação automática: ao espetar mídia com metadados LVM o udev pode subir os volumes e alterar a evidência — `blockdev --setro` antes, exame por `losetup -rP`, `vgchange -an` no que subiu sozinho e um filtro em `/etc/lvm/lvm.conf` na estação de análise.

GUIA VISUAL 049

# RAID POR SOFTWARE MDADM

*Um disco falha no sábado e o serviço nem pisca. No mesmo arranjo, um `rm -rf` errado é replicado em todos os discos:* ***RAID é disponibilidade, não backup.***

**049**
GUIAS LINUX

## OS NÍVEIS — CAPACIDADE, TOLERÂNCIA E VELOCIDADE NÃO SOMAM

### RAID 0 DISTRIBUIÇÃO

mín. 2 discos · 100%

| A1 | A2 | A3 | A4 |
|---|---|---|---|

Capacidade e velocidade totais, **zero tolerância**: um disco morre, tudo morre — e com 4 discos o risco é 4×. Só para dado recriável.

### RAID 1 ESPELHO

mín. 2 discos · 50%

| A1 | A1 | A2 | A2 |
|---|---|---|---|

Metade da capacidade; sobrevive a **um disco de cada par**. É o nível da raiz, da `/boot` e de arranjos pequenos.

### RAID 5 PARIDADE DISTRIBUÍDA

mín. 3 discos · (n−1)/n

| A1 | A2 | A3 | Ap |
|---|---|---|---|
| B1 | B2 | Bp | B4 |

**Ap** é o ou-exclusivo da faixa: o bloco perdido é recalculado. Tolera **1 disco** gastando 1 — mas reconstruir **lê todos os outros** de ponta a ponta.

### RAID 10 ESPELHO + DISTRIBUIÇÃO

mín. 4 discos · 50%

| A1 | A1 | A2 | A2 |
|---|---|---|---|
| A3 | A3 | A4 | A4 |

Metade da capacidade, rápido na escrita e **reconstrução curta**: lê só o disco espelhado, em horas, sem tocar no resto do arranjo.

**RAID 6** (mín. 4 · (n−2)/n) tolera **duas** falhas — é o nível para arranjos grandes. **Acima de 2 TB por disco, RAID 6 ou 10, nunca RAID 5.**

## O CICLO DE UMA FALHA — E A JANELA DE RISCO

**1 — CLEAN [UUU]**
Redundância cheia. `/proc/mdstat` com um `U` por membro.

→

**2 — DEGRADED [UU_]**
Disco fora. **Tudo funciona e nada protege** — o `_` é a marca do membro ausente.

→

**3 — RECOVERING**
`--add` do disco novo (ou o *spare* entra sozinho) e a reconstrução começa.

→

**4 — CLEAN [UUU]**
Sincronização concluída; a redundância volta sem ninguém parar o serviço.

Entre 2 e 4 está a **janela sem tolerância** — encurtá-la é o objetivo de tudo: um **disco reserva** ( `--add-spare` ) assume em segundos, o `mdmonitor` por correio evita descobrir dias depois, e **RAID 10 ou 6** impedem que a reconstrução dure dias em discos grandes.

```
kali@kali: ~
$ sudo mdadm --create /dev/md0 --level=10 --raid-devices=4 /dev/sd{b,c,d,e}1
$ sudo mdadm --detail --scan | sudo tee -a /etc/mdadm/mdadm.conf   # NÃO é opcional
$ sudo update-initramfs -u    # sem os dois, no reboot vira /dev/md127 — ou não sobe
$ sudo mkfs.ext4 -L arranjo /dev/md0   # md é bloco comum: mkfs, ou pvcreate p/ LVM
$ cat /proc/mdstat
md0 : active raid10 sde1[3] sdd1[2] sdc1[1] sdb1[0]
      2094080 blocks super 1.2 512K chunk 2 near-copies [4/3] [UU_U]  # degradado!
$ sudo mdadm --manage /dev/md0 --fail /dev/sdd1 --remove /dev/sdd1   # retirada
$ sudo mdadm --manage /dev/md0 --add /dev/sdf1   # entra e reconstrói sozinho
$ sudo mdadm --examine /dev/sdb1   # --detail pergunta ao kernel; --examine lê o disco
$ grep -q "_" /proc/mdstat && echo "ARRANJO DEGRADADO"   # o monitor de uma linha
```

**RAID não é backup** — exclusão acidental, corrupção de software e *ransomware* são replicados com perfeição em todos os membros. E há três armadilhas: **RAID 5 com discos grandes** lê todos os membros na reconstrução, justamente quando não há mais redundância, e um erro irrecuperável ali (≈1 setor a cada 12 TB lidos) mata o arranjo — some a isso discos do mesmo lote falhando na mesma janela. `mdadm --grow` **reescreve o arranjo inteiro**, leva horas ou dias e é a operação com mais partes móveis: backup real antes. E ao desfazer um arranjo, `--stop` e `--zero-superblock` em cada disco — sem isso o kernel remonta um arranjo fantasma ( `/dev/md127` ) no próximo boot.

## RESUMO RÁPIDO

- ✓ `[3/3] [UUU]` **é saúde;** um `_` no lugar do `U` é membro ausente. `grep -q "_" /proc/mdstat` cabe em qualquer script.
- ✓ **Arranjo degradado funciona normalmente — e é por isso que é perigoso.** Sem monitoramento ele passa semanas sem redundância até o segundo disco cair.
- ✓ **RAID embaixo, LVM em cima:** o LVM enxerga um dispositivo já redundante e a falha de disco se resolve na camada de baixo.
- ✓ ***Scrub* mensal** ( `echo check > /sys/block/md0/md/sync_action` ) acha o bloco corrompido antes da reconstrução; no Debian e no Kali já vem no `/etc/cron.d/mdadm` .
- ✓ **Prefira o `md` ao "fake RAID" da placa-mãe:** um arranjo `md` remonta em qualquer Linux com `--assemble --scan` ; o proprietário pode exigir a mesma controladora.

## NO KALI

RAID aqui é **nicho — e o nicho é guardar evidência**: imagens de disco de centenas de GB precisam de espaço e de tolerância, porque reimagear a mídia nem sempre é possível. Use **RAID 10**, não 5: reconstrói lendo só o espelho e mantém a escrita rápida enquanto você grava uma imagem de 4 TB. Em perícia, a **montagem automática é o risco**: discos apreendidos trazem superblocos `md` e um *assemble* automático escreve metadados de sincronização na evidência — `blockdev --setro` antes, e `mdadm --examine` , que só lê, para descobrir nível e ordem dos discos. **Reconstruir a partir das imagens** é procedimento real: `losetup -rP` em cada uma e `mdadm --assemble --readonly` sobre os *loop*. Num sistema sem correio configurado, ponha o `grep -q "_" /proc/mdstat` no `.bashrc` .

GUIA VISUAL 050

# CRIPTOGRAFIA DE DISCO LUKS E A PERSISTÊNCIA

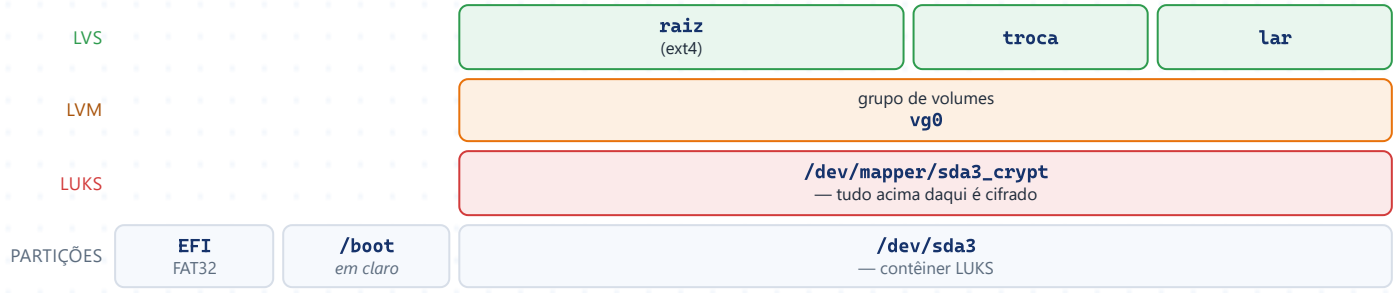
*O notebook fica no táxi. Com um live USB, ler tudo leva cinco minutos:* ***permissão é conferida pelo kernel que está rodando*** *— outro kernel, outro root.*

050
GUIAS LINUX

## DUAS CHAVES — POR QUE TROCAR A SENHA É INSTANTÂNEO

**Dados do volume.** Cada senha só decifra uma **cópia** da mestra num *slot* — daí trocar a senha reescrever poucos KB em vez de recifrar o disco, e várias pessoas terem senhas distintas para o mesmo volume. LUKS1 tem 8 *slots*; **LUKS2, 32**, com Argon2id (caro em tempo **e** em memória). E daí também: **apagar o cabeçalho destrói todas as cópias de uma vez**.

## A PILHA DE UMA INSTALAÇÃO CIFRADA — RAID → LUKS → LVM




**LUKS embaixo, LVM em cima:** uma senha abre um contêiner e protege todos os volumes lógicos — a ordem inversa pediria uma senha por volume, sem ganho. A **EFI e a `/boot` ficam fora por necessidade**: alguém tem de carregar o kernel e o *initramfs* que vão pedir a senha. É a superfície que sobra — com acesso físico repetido dá para alterar o *initramfs* e capturar a senha (a "camareira maligna"). Defesa: *Secure Boot* com chaves próprias ou selar a chave no TPM (`systemd-cryptenroll --tpm2-device=auto`).

```
kali@kali: ~
$ sudo cryptsetup luksFormat /dev/sdX3   # confirme YES em maiúsculas — apaga tudo
$ sudo cryptsetup luksHeaderBackup /dev/sdX3 --header-backup-file ~/hdr.img  # AGORA
$ sudo cryptsetup open /dev/sdX3 cofre && sudo mkfs.ext4 -L persistence /dev/mapper/cofre
# mkfs e mount sempre em /dev/mapper/<nome> — no dispositivo cru, mata o cabeçalho
$ echo "/ union" | sudo tee /mnt/cofre/persistence.conf   # persistência do Kali
$ sudo cryptsetup luksAddKey /dev/sdX3 ~/chave.bin   # exige uma senha que já funcione
$ sudo cryptsetup --test-passphrase open /dev/sdX3   # testa sem mapear nada
$ sudo blkid /dev/sdX3          # UUID do CONTÊINER  → /etc/crypttab
$ sudo blkid /dev/mapper/cofre  # UUID do SISTEMA DE ARQUIVOS → /etc/fstab
cofre  UUID=8f2e…  none  luks,discard,tries=3   # crypttab: nofail em disco externo
$ sudo update-initramfs -u   # quem pede a senha da raiz é o initramfs. Não é opcional.
```

**Perder o cabeçalho é perder os dados — não existe recuperação.** Ele ocupa poucos MB no início do volume e guarda **todas** as cópias da chave mestra: um `dd` nos primeiros blocos, um `wipefs -a` ou uma tabela de partição reescrita com outro deslocamento e nenhuma senha do mundo abre o volume. Faça `luksHeaderBackup`, guarde **fora do disco protegido**, trate como material sensível — e **teste a restauração** antes de precisar dela; note que um cabeçalho antigo **revive senhas já removidas**. O `luksAddKey` exige uma senha válida: **removeu o último *slot*, acabou**. E o `reencrypt`, que troca a chave mestra com o sistema no ar, é a operação mais longa e arriscada — backup completo antes, sem exceção.

## RESUMO RÁPIDO

- ✓ **Permissão não protege contra acesso físico.** Ela é conferida pelo kernel em execução contra o usuário autenticado — outro sistema, iniciado como `root`, ignora tudo.
- ✓ **Opere sempre em `/dev/mapper/<nome>`**, nunca no dispositivo cru: formatar o original destrói o cabeçalho.
- ✓ **Dois UUIDs:** o do contêiner vai no `crypttab`, o do sistema de arquivos vai no `fstab`. Trocá-los trava o boot esperando um dispositivo que nunca aparece.
- ✓ **Com AES-NI a criptografia não é o gargalo** — o `cryptsetup benchmark` entrega mais do que um SSD SATA sustenta. Em ARM sem aceleração, compare com `xchacha20`.
- ✓ `discard` em volume cifrado revela **quais blocos estão em uso**: bom para SSD, ruim quando o próprio volume de dados é sensível.

## NO KALI

**Persistência cifrada é o padrão para pendrive.** Crie a 3ª partição depois das duas da imagem, `luksFormat`, formate em **ext4** com rótulo `persistence` e grave `/ union` no `persistence.conf`. O *initramfs* acha a partição **pelo rótulo** — imune à troca de letras a cada máquina — e exFAT/FAT32 não servem: não representam dono nem permissão, e o sistema não sobe. Num equipamento de trabalho, criptografia **não é opcional**: ele carrega capturas, credenciais de teste e dados de cliente. O `cryptsetup` do Kali traz as **chaves *nuke*** (`luksAddNuke`): digitada no boot, a senha apaga todos os *slots* em vez de abrir — protege confidencialidade contra acesso não autorizado, mas destruir dados para obstruir investigação legalmente autorizada é crime, e sem backup do cabeçalho é definitivo também para o dono. Em perícia, a chave mestra vive na **memória** enquanto o volume está aberto: **máquina ligada com volume aberto não se desliga** antes da captura de memória.

# DO COMANDO AO SCRIPT

*Seis comandos toda manhã, sempre na mesma ordem. Enquanto a sequência só existe nos dedos de quem digita, ela é um **hábito** — não um procedimento. O script é o terminal gravado.*

**051**
GUIAS LINUX

## DO ENTER AO PROCESSO — O QUE ACONTECE EM `./ROTINA.SH`

**1 · VOCÊ**
`./rotina.sh`
digita e tecla Enter.

→

**2 · SHELL**
`fork() + execve()`
cria o filho e pede ao kernel para executar o arquivo.

→

**3 · KERNEL**
`#!`
lê os **dois primeiros bytes**. Não é binário: é *shebang*.

→

**4 · KERNEL**
`/bin/bash`
carrega o interpretador da linha e lhe entrega o arquivo.

→

**5 · BASH**
`linha a linha`
lê *enquanto* executa — não valida o arquivo antes.

O passo 3 é a razão de o bit `x` ser obrigatório: sem ele o `execve()` falha **antes** de qualquer leitura. Sem *shebang* algum, o kernel devolve `Exec format error` e o Bash interpreta por cortesia histórica — cortesia que o `cron` e o `systemd` não têm. E, como o passo 5 lê por posição de byte, **erro de sintaxe na linha 50 não impede as linhas 1 a 49 de rodarem**: valide com `bash -n` antes da primeira execução.

## EXECUTAR × `SOURCE` — A DIFERENÇA QUE EXPLICA O `.BASHRC`

### ./SCRIPT.SH

- ✓ **Processo novo.** O filho recebe uma *cópia* do ambiente; ao terminar, a cópia é descartada.
- ✓ Exige bit `x` e *shebang*. `bash script.sh` dispensa os dois.
- ✓ `cd`, `export` e funções **não sobrevivem**: o shell que chamou fica como estava.
- ✓ Depurar: `bash -x script.sh` mostra cada linha expandida.

### SOURCE SCRIPT.SH

- ✓ **Mesmo processo.** As linhas são lidas como se tivessem sido digitadas — `$$` não muda.
- ✓ Forma POSIX: `. script.sh`. Não exige bit `x`.
- ✓ Altera a sessão em curso: PWD, variáveis e funções passam a valer.
- ✓ É o mecanismo do `.bashrc` e do `/etc/profile.d/`. Por isso `bash ~/.bashrc` "não faz nada".

## O *SHEBANG* CERTO E OS TRÊS ERROS CLÁSSICOS

| PRIMEIRA LINHA | O QUE SIGNIFICA |
|---|---|
| `#!/usr/bin/env bash` | Procura o `bash` no `PATH`. **Padrão portável** — roda em Debian, macOS, BSD. |
| `#!/bin/bash` | Caminho fixo e absoluto. Use quando você controla o sistema e quer garantia de qual binário roda. |
| `#!/bin/sh` | No Debian/Kali é o **dash**: sem `[[ ]]`, arrays ou `local`. Em RHEL é o Bash em modo POSIX — o mesmo arquivo funciona lá e falha aqui. |

**`Permission denied`**
Achou o arquivo, falta o bit `x`. → `chmod +x`

**`command not found`**
O arquivo está fora do `PATH`. → `./` na frente, ou instale em `~/bin`.

**`/usr/bin/env^M: bad interpreter`**
Editado no Windows: fim de linha CRLF. → `dos2unix` ou `sed -i 's/\r$//'`

```
kali@kali: ~/lab-scripts
$ cat > rotina.sh <<'FIM'      # aspas em FIM: nada é expandido agora
#!/usr/bin/env bash            # 1. quem interpreta
# rotina.sh - conferencia diaria | Uso: ./rotina.sh   # 2. o que faz e como usar
set -euo pipefail              # 3. -e para no 1o erro · -u variavel indefinida · pipefail
DESTINO="$HOME/relatorios"     # 4. config no topo, SEMPRE entre aspas
mkdir -p "$DESTINO"
df -h /  >  "$DESTINO/sis.txt" # 5. > cria/zera · >> acrescenta
free -h  >> "$DESTINO/sis.txt"
FIM
$ bash -n rotina.sh              # valida a sintaxe sem executar NADA
$ chmod +x rotina.sh && ./rotina.sh
```

**Nunca acrescente `.` ao `PATH`.** O `PATH="$PATH:."` no `.bashrc` troca segurança por conveniência: basta plantar um arquivo chamado `ls`, `git` ou `sudo` num diretório compartilhado para que o próximo comando digitado ali execute código alheio. A solução correta é `mkdir -p ~/bin` — o `.profile` do Debian/Kali já o acrescenta ao `PATH` se ele existir. E **nunca edite um script longo enquanto ele roda**: o Bash lê por posição de byte e continuará a partir do deslocamento antigo, agora no meio de outra linha. Edite uma cópia e troque com `mv`.

## RESUMO RÁPIDO

- ✓ **Script é o terminal gravado.** Mesma sintaxe, mesmo interpretador — monte o pipeline no terminal, confira a saída, e só então copie para o arquivo (`history 20 | cut -c 8-` é o rascunho).
- ✓ **O *shebang* diz quem interpreta**, e o bit `x` autoriza o kernel a sequer olhar o arquivo. São dois requisitos, não um.
- ✓ **Executar cria processo; `source` não.** Só `source` muda a sessão atual.
- ✓ **Esqueleto decente em 5 itens:** *shebang* · cabeçalho de comentário · `set -euo pipefail` · variáveis de config no topo · toda expansão entre aspas.
- ✓ `bash -n` antes da primeira execução. Custa nada e o shell não valida sozinho.

## NO KALI

**O `~/bin` já está previsto — mas só se você criá-lo.** O `.profile` só o acrescenta ao `PATH` *se o diretório existir*, e a checagem roda no login: você cria a pasta, põe o script lá e nada acontece até `source ~/.profile`. Para ferramenta própria de uso geral, a convenção é `sudo install -m 755 meu.sh /usr/local/bin/meu` — diretório que nenhum pacote do APT toca. **O shell padrão do Kali é o Zsh desde 2020**, mas isso não muda nada em script: com `#!/usr/bin/env bash` no topo ele roda em Bash. O erro é testar a linha no Zsh interativo, ver funcionar e supor que valerá no script — confira com `bash -c '...'`. Script que embrulha ferramenta ofensiva precisa ser **mais** explícito: valide a lista de alvos, exiba-a e exija confirmação antes de agir — agir fora do escopo autorizado é crime (Lei 12.737/2012) e "o script leu o arquivo errado" não é defesa. Para aprender convenção lendo gente experiente: `less $(which kali-tweaks)`.

GUIA VISUAL 052

# VARIÁVEIS E O AMBIENTE

*O script de backup roda em quatro segundos no terminal. No `cron`, às duas da manhã, falha todas as noites com `nmap: command not found`. Nada no script mudou — mudou **o ambiente**.*

052
GUIAS LINUX

## A SINTAXE IMPLACÁVEL · E O QUE O FILHO HERDA

✓ `nome="kali"`
✗ `nome = "kali"` vira comando
✗ `nome= "kali"` roda `kali`
✓ `${nome}_2024` chaves = fronteira
✗ `$nome_2024` outra variável, vazia
✓ `MAIÚSCULA` ambiente/constante
✓ `minúscula` local do script

**SHELL PAI · PID 4021**

`TOKEN="abc"`
variável do shell — só aqui

`export CAMINHO="/opt"`
variável de ambiente

`printenv` só as exportadas · `set` tudo · `declare -p` com atributos

→ `fork + exec`
cópia do ambiente
✗ nunca de volta

**FILHO · PID 4088**

`$CAMINHO` → `/opt` herdada
`$TOKEN` → **vazia** não exportada
`CAMINHO="/tmp"`
altera só a cópia — o pai não vê

Prefixo temporário não suja a sessão: `LC_ALL=C date`

## QUAIS ARQUIVOS CADA SHELL LÊ — E POR QUE O `CRON` NÃO LÊ NENHUM

**SHELL DE LOGIN**
console, ssh, su -
`/etc/profile`
`/etc/profile.d/*.sh`
`~/.bash_profile`
`~/.profile` (que chama o `.bashrc`)

**INTERATIVO, SEM LOGIN**
nova aba do terminal
`/etc/bash.bashrc`
`~/.bashrc`
e mais nada — o resto vem **herdado**

**NÃO INTERATIVO**
script, cron, systemd
nenhum arquivo lido
PATH mínimo (/usr/bin:/bin)
sem TERM, sem aliases

A coluna vermelha é a resposta ao mistério: o `.bashrc` **nunca foi lido**, logo o `PATH` que você acrescentou lá não existe. Correções, em ordem: **1.** caminho absoluto para cada comando · **2.** `export PATH=...` explícito no topo do script · **3.** `PATH=` no próprio *crontab*. Chamar o `.bashrc` de dentro do script é a saída tentadora e **errada** — ele tem código de sessão interativa e pode abortar cedo. Para a máquina inteira: `/etc/environment` (pares simples, lido pelo PAM) ou um fragmento em `/etc/profile.d/`.

## VAZIO × NÃO DEFINIDO · AS ESPECIAIS QUE O BASH PREENCHE

| EXPANSÃO | COMPORTAMENTO |
|---|---|
| `${VAR:-padrao}` | **Não definida OU vazia** → usa o padrão. É a que se usa 90% das vezes. |
| `${VAR-padrao}` | Sem os dois-pontos: só se **não definida**. Vazia continua vazia. |
| `${VAR:=padrao}` | Usa **e atribui** o padrão à variável. |
| `: "${VAR:?msg}"` | **Aborta com a sua mensagem** se faltar. O `:` é o comando nulo, só força a avaliação. |
| `${VAR:-}` | Sob `set -u`, a forma de dizer "sei que pode não existir". |

| | |
|---|---|
| `$?` | saída do último comando |
| `$$` | PID do shell · `$PPID` do pai |
| `$0` | nome do script |
| `$#` | quantos argumentos |
| `$@` | cada argumento uma palavra |
| `$*` | todos como uma só |
| `$!` | PID do último em 2º plano |
| `$LINENO` | linha atual — para mensagem de erro |
| `$SECONDS` | `$((SECONDS-inicio))` cronometra |
| `$RANDOM` | 0–32767, **não** criptográfico |

```
kali@kali: ~/lab-vars
$ LOCAL="so-aqui"; export GLOBAL="vai-junto"
$ bash -c 'echo "no filho: LOCAL=[$LOCAL] GLOBAL=[$GLOBAL]"'
no filho: LOCAL=[] GLOBAL=[vai-junto]   # sem export, o filho nao ve
$ env -i /bin/bash ./backup.sh    # simula o cron: ambiente vazio, falha na hora
$ echo "$SHELL"; ps -p $$ -o comm= # o de LOGIN x o REALMENTE em execucao
/usr/bin/zsh
bash
$ declare -i n=0; n="3*7"; echo "$n"   # -i: atribuicao vira conta -> 21
$ readonly CONF=/etc/app.conf          # erro de digitacao vira erro na hora
```

**Segredo em variável de ambiente é segredo exposto.** O ambiente de um processo vive em `/proc/PID/environ`, legível pelo dono e pelo root — e é **herdado por todo filho**, inclusive o instalador de pacote que resolveu mandar telemetria. Por argumento (`--senha s3nh4`) é pior ainda: aparece no `ps aux` para qualquer usuário da máquina. Em ordem: **1.** arquivo `600` lido com `source` · **2.** entrada padrão (`--password-stdin`) · **3.** gerenciador (`pass`, `secret-tool`) · **4.** prefixo de um comando só, nunca `export` na sessão. E **nunca** segredo dentro de script versionado: apagar a linha num commit posterior não apaga nada do histórico.

## RESUMO RÁPIDO

- ✓ **Sem espaço em volta do `=`.** Com espaço, `nome` vira nome de comando — daí o `command not found` que engana quem vem de outra linguagem.
- ✓ **`export` é a fronteira** entre variável do shell e de ambiente. A herança é por **cópia** e de **mão única**: o filho altera só o retrato que recebeu.
- ✓ **Shell não interativo não lê arquivo nenhum.** `PATH` explícito no topo ou caminho absoluto — a correção do `cron`.
- ✓ `SHELL` é o shell de **login** do `/etc/passwd`, não o em execução (`ps -p $$ -o comm=`). E `LANG` muda a saída dos comandos: script que interpreta saída fixa `LC_ALL=C`.
- ✓ **Vazio ≠ não definido.** `${VAR:-x}` cobre os dois; `${VAR:?msg}` aborta cedo com mensagem sua.

## NO KALI

**O `sudo` limpa o ambiente — e isso quebra script.** O `env_reset` está ligado no `/etc/sudoers`: um `export CHAVE=v` seguido de `sudo meu.sh` chega ao script **sem** a chave. Saídas: `sudo CHAVE=v comando`, `sudo -E` (preserva *tudo* — cuidado) ou `env_keep` via `visudo`. **O Kali põe `/usr/sbin` e `/sbin` no `PATH` de qualquer usuário**, porque quase toda ferramenta de rede vive ali — por isso `tcpdump` e `iptables` completam com Tab sem `sudo`, e depois falham com `Operation not permitted`; compare `printenv PATH` antes e depois de `sudo -i`. Conheça as variáveis das ferramentas: `PROXYCHAINS_CONF_FILE`, `HTTP_PROXY` / `NO_PROXY`, `GOPATH` — proxy configurado no `~/.profile` e esquecido é a origem clássica de "a rede parou depois do reboot". Em laboratório, `HISTCONTROL=ignorespace` mantém credencial de teste fora do `~/.bash_history` (no Zsh, `setopt HIST_IGNORE_SPACE`). E em **live USB sem persistência** tudo que você exportou some no desligamento: use `PATH` explícito e caminho absoluto, a mesma disciplina do `cron`.

GUIA VISUAL 053

# ASPAS E EXPANSÕES

*O script de limpeza tinha uma linha só e rodou oito meses sem incidente — até o dia em que* `DESTINO` *passou a valer* `/srv/Arquivos Antigos`*. Nenhum comando estava errado. Faltavam* ***duas aspas****.*

**053**
GUIAS LINUX

## A LINHA QUE VOCÊ DIGITA NÃO É A LINHA QUE RODA

| | |
|---|---|
| A LINHA DIGITADA | `cp ~/dados{1,2}.csv "$DEST"/$(date +%F)` |
| 1 · CHAVES | `cp ~/dados1.csv ~/dados2.csv "$DEST"/$(date +%F)` |
| 2 · TIL | `cp /home/kali/dados1.csv … "$DEST"/$(date +%F)` |
| 3 · VARIÁVEIS, $(( )) E $( ) | `cp /home/kali/dados1.csv … "/srv/Back up"/2026-08-03` |
| 4 · DIVISÃO EM PALAVRAS | `[cp] [/home/kali/dados1.csv] [/home/kali/dados2.csv] [/srv/Back up/2026-08-03]` |
| 5 · GLOBBING | `sem curinga nesta linha: nada muda` |
| 6 · REMOÇÃO DAS ASPAS | as aspas somem — elas nunca chegam ao comando. O `cp` recebe 3 argumentos. |

**SEM ASPAS**
`rm -rf $DESTINO/*`

`rm` `-rf` `/srv/Arquivos` `Antigos/*` → o 2º caminho é resolvido a partir do **diretório atual**

**COM ASPAS**
`rm -rf "$DESTINO"/*`

`rm` `-rf` `/srv/Arquivos Antigos/relatorio.pdf` → um argumento por arquivo real

**Ensaie todo `rm -rf` com `echo` na frente.** `echo rm -rf "$DESTINO"/*` mostra exatamente os argumentos que o comando receberia, sem apagar nada. O `/*` fica **fora** das aspas de propósito: é ele que precisa da etapa 5 para virar a lista de arquivos. **Aspas em volta do caminho, curinga fora** — é o que separa o *dado* (o valor da variável) da *sintaxe* (o curinga que você escreveu).

As etapas **4 e 5 são as perigosas**, e as duas acontecem **depois** de a variável virar valor. O shell não distingue o espaço que você digitou daquele que veio de dentro do valor — para ele é tudo a mesma linha. Aspas duplas não impedem a substituição da etapa 3; impedem que o resultado dela seja fatiado na 4 ou reinterpretado como curinga na 5.

## AS FORMAS DE CITAR · E AS OUTRAS EXPANSÕES

| FORMA | O QUE DESLIGA | O QUE SOBRA LIGADO |
|---|---|---|
| sem aspas | nada | tudo — inclusive a divisão e o curinga |
| `'simples'` | **tudo** | nada: cada byte é literal |
| `"duplas"` | divisão em palavras e *globbing* | `$var`, `$(cmd)`, `$((..))`, `\` |
| `\c` | só o próximo caractere | o resto da linha |
| `$'texto'` | como as simples, mas... | interpreta `\n`, `\t`, `\x41` |

**`$(( a + b ))`**
Aritmética **só inteira**: `10/4` vale **2**. Decimal é trabalho de `bc`, `awk` ou `python3`.

**`{1..5}` · `{a..e}` · `{0..20..5}`**
Primeira etapa, logo **não enxerga variável**: `{1..$n}` sai literal — use `seq 1 "$n"`. Idioma clássico: `cp conf{,.bak}`.

**`~` · `~root` · `~+`**
Não expande dentro de aspas: `"~/dados"` é literal. Em script, `"$HOME/dados"`.

**`<(comando)`**
Entrega a saída como se fosse arquivo: `diff <(sort a) <(sort b)`. E `while … done < <(cmd)` é o laço **sem** subshell.

kali@kali: ~/lab-aspas

```
$ curinga="*"; echo $curinga      # lista TODOS os arquivos: a etapa 5 vê o VALOR
$ echo "$curinga"                 *
$ dono="$(stat -c %U "$(readlink -f /bin/sh)")"  # $( ) aninha; a crase `` nao
$ n=0; printf 'a\nb\nc\n' | while read -r _; do n=$((n+1)); done; echo $n
0  # o pipe roda o while num SUBSHELL — o contador voltou zerado
$ n=0; while read -r _; do n=$((n+1)); done < <(printf 'a\nb\nc\n'); echo $n
3  # com substituicao de processo, o laco fica no shell atual
$ find /tmp -name '*.log' -delete  # a lista NAO passa pelo shell: o jeito certo
```

**O `eval` transforma dado em sintaxe — é a via curta para injeção de comando.** Com `entrada="/tmp; rm -rf ~"`, o `eval "ls $entrada"` executa também o `rm`: o ponto e vírgula que estava no *dado* virou sintaxe. Defesas, em ordem: **1.** não use `eval` — quase sempre há array (`"${opcoes[@]}"`), `${!var}` ou `case` · **2.** valide por **lista de permitidos**, nunca removendo caracteres proibidos · **3.** passe como argumento e encerre as opções: `comando -- "$valor"` · **4.** `printf %q` quando montar linha for inevitável. O mesmo vale para `bash -c "$linha"`, `find -exec sh -c` e, sobretudo, `ssh host "cmd $VAR"` — que atravessa **dois** shells: o local expande, o remoto reinterpreta o resultado inteiro.

## RESUMO RÁPIDO

- ✓ **Seis etapas fixas:** chaves → til → variáveis/aritmética/ `$( )` → divisão em palavras → *globbing* → remoção das aspas. Quase toda surpresa do shell é uma etapa fora do lugar que você imaginou.
- ✓ **Aspas duplas em toda expansão, sempre**, salvo motivo articulável. As simples tornam tudo literal; `$'…'` interpreta `\n` e `\t`.
- ✓ **`$(cmd)` no lugar da crase:** aninha, dispensa escapar barra e é inequívoco. Ele **corta as quebras de linha finais** e roda num subshell.
- ✓ **Nome de arquivo pode ter espaço, `;` e até quebra de linha.** Nunca analise a saída do `ls`: use *globbing* (`for f in *.txt`) ou `find -print0 | xargs -0`.
- ✓ `shellcheck script.sh` acha sozinho SC2086, SC2046, SC2006 e SC2045 — o hábito de maior retorno do volume.

## NO KALI

**Nome com espaço é rotina em perícia, não exceção.** Evidência vinda de máquina Windows chega com `Área de Trabalho`, `Novo Documento (2).docx`, acentos e parênteses. Script de triagem sem aspas falha **parcialmente**: processa alguns arquivos, ignora outros e o relatório sai incompleto *sem erro visível*. Nesse contexto a combinação é obrigatória: `find -print0 | xargs -0` ou `find -exec … {} +` — o byte nulo é o único caractere que não pode aparecer num nome. **A saída de ferramenta do Kali não é contrato estável:** `nmap`, `nikto` e `gobuster` mudam formato entre versões e, num *rolling*, isso acontece sem aviso — um `grep`/`cut` na saída de tela quebra no próximo `full-upgrade`. Use saída estruturada (`nmap -oX`, `-oG`, `--json`) e `jq` em cima. E cuidado com quem *consome* as aspas: em `proxychains nmap -sT "$ALVO"` quem as come é o shell local, mas em `ssh host "cmd $ALVO"` um alvo com `;` no meio vira comando na máquina remota. Em teste autorizado é a diferença entre um script e um incidente — agir fora do escopo contratado é crime pela Lei 12.737/2012, ainda que por acidente de aspas.

# READ, ECHO E PRINTF

*O relatório saía com três colunas alinhadas na máquina de quem escreveu. Chamado pelo `cron` com `sh inventario.sh`, virou `-e Servidor\tCPU\tMemoria`. O culpado é o `echo` — o único comando do shell cuja portabilidade é problema real.*

**054**
GUIAS LINUX

## O ECHO MENTE · O PRINTF NÃO

**`echo -e "a\tb"`**
Três implementações discordam: o **dash** (o `/bin/sh` do Kali) *sempre* interpreta a barra e imprime o `-e` literal; o **Bash** faz o inverso; com `shopt -s xpg_echo`, o Bash vira dash.

**`valor="-n"; echo "$valor"`**
Imprime **nada** — o valor virou opção. Qualquer valor vindo de fora que comece com hífen é engolido sem aviso.

**`printf "$entrada"`**
Dado **nunca** entra na cadeia de formato: um `%d%d%d` no valor consome argumentos que não existem. O certo é `printf '%s\n' "$entrada"`.

**`printf '%b\n' "$txt"`**
O substituto seguro de `echo -e`: interpreta a barra invertida do *argumento*, não do formato.

`%` `-` `10` `.2` `f`

**`-` `sinalizador`** — alinha à esquerda (padrão: direita)
**`10` `largura`** — ocupa 10 colunas, completa com espaço
**`.2` `precisão`** — 2 casas decimais
**`f` `conversor`** — ponto flutuante

**A cadeia é reaproveitada ciclicamente** até acabarem os argumentos: `printf '%-8s %5.1f\n' raiz 42.75 var 8.5 home 120.4` imprime **três linhas** sem laço nenhum — e é por isso que `printf '%s\n' "$@"` põe cada argumento na sua linha.
`%s` texto · `%d` inteiro · `%.2f` decimal · `%x` hex · `%o` octal (permissões!) · `%b` com barras · `%q` escapado para o shell · `%%` literal.

## O IDIOMA CANÔNICO DO LAÇO DE LEITURA — DECORE O BLOCO INTEIRO

`while` `IFS=` `read` `-r` `linha` `|| [[ -n "$linha" ]]` `; do … done` `< "$arq"`

**`IFS=`**
Esvazia o separador **só nesta linha**. Sem ele, o `read` apara espaços e tabulações das pontas — e a linha indentada perde a indentação.

**`-r`**
Desliga o tratamento da barra invertida. Sem ele, `C:\Users` vira `C:Users` e uma linha terminada em `\` é colada na seguinte.

**`|| [[ -n "$linha" ]]`**
Salva a **última linha** quando o arquivo não termina em quebra — caso em que o `read` lê o conteúdo e mesmo assim retorna falha.

**`< "$arq"`**
Redirecionamento, não *pipe*: `cat arq | while` funciona, mas cria **subshell** e as variáveis alteradas no laço voltam vazias.

**As três formas erradas mais comuns:** `for linha in $(cat arq)` quebra em **espaços**, não em linhas · `cat arq | while read linha` subshell + perde indentação · `while read linha` come barra invertida e espaços. E para dividir campos, o `IFS` trabalha a favor: `while IFS=: read -r usuario _ uid _ _ _ shell; do … done < /etc/passwd` — o `IFS=:` vale só para aquele `read`, o do script fica intacto.

kali@kali: ~/lab-io

```
$ erro() { printf 'ERRO: %s\n' "$*" >&2; }   # erro no descritor 2: o usuario
$ aviso(){ printf 'aviso: %s\n' "$*" >&2; }  # redireciona a saida util e ainda ve
$ [[ -t 1 ]] && V="$(tput setaf 2)" N="$(tput sgr0)" || { V=; N=; }
# sem o teste -t 1, redirecionar grava ^[[32m cru no arquivo; e no cron o TERM nem existe
$ printf '%-18s %8s %10s\n' SERVICO ESTADO PID   # tabela: largura fixa alinha
$ while IFS= read -r host; do ssh -n "$host" uptime; done < alvos.txt
# o -n e OBRIGATORIO: sem ele o ssh le o resto do arquivo e o laco roda 1 vez so
# vale para ssh, ffmpeg, mysql, apt. Alternativa: read -u 3 ... done 3< alvos.txt
```

**Senha lida com `read -s` continua exposta.** O `-s` resolve o ombro do colega e mais nada: o valor fica numa variável do shell (legível pelo root), **aparece inteiro no rastreio se o script rodar com `set -x`**, e some para o `ps aux` de qualquer usuário se você o passar adiante como argumento. O caminho correto: `read -rs` seguido de entrada padrão — `printf '%s' "$senha" | comando --password-stdin` — ou um gerenciador como o `pass`; e `set +x` antes de qualquer linha que toque na variável. Ao confirmar ação irreversível, o padrão é **não**: `read -rp "Apagar? [s/N] " r` com `${r,,}` para comparar em minúsculas, e `rm -rf -- "$ALVO"` — o `--` encerra as opções e protege contra caminho iniciado por hífen. Sem terminal (`[[ ! -t 0 ]]`), aborte com mensagem: no `cron`, um `read` sem `-t` espera para sempre.

## RESUMO RÁPIDO

- ✓ **`echo` só para texto literal simples.** Bash, dash e `/bin/echo` discordam sobre `-e`, e valor começado com hífen vira opção.
- ✓ **`printf` é o substituto em qualquer shell**, e a cadeia se reaproveita: `printf '%s\n' "$@"` e as tabelas com `%-20s` saem em uma linha.
- ✓ **Dado nunca entra na cadeia de formato** — sempre `printf '%s\n' "$var"`.
- ✓ **Erro vai para o descritor 2** ( `>&2` ) e **cor só com `[[ -t 1 ]]`**, senão o arquivo recebe códigos de escape crus.
- ✓ **`read` sem `-r` corrompe dados** e sem `IFS=` apara as pontas. As duas são obrigatórias, e nenhuma substitui a outra.
- ✓ **Comando de dentro do laço pode comer a entrada do laço:** `ssh -n`, `< /dev/null` ou um descritor separado.

## NO KALI

**Lista de alvos é o uso nº 1 do `read` em laboratório — e o que mais quebra.** Um `alvos.txt` real tem linha em branco, comentário com `#`, espaço no fim e, se veio de planilha, terminador **CRLF**. Limpe no começo do laço: `linha="${linha%$'\r'}"` e `[[ -z "${linha// }" || "$linha" == \#* ]] && continue`. Apontar a ferramenta para um alvo com um retorno de carro colado no nome é como um teste autorizado vira acesso não autorizado — e a Lei 12.737/2012 não distingue intenção de descuido; valide e exiba a lista antes de agir. **O `tput` falha em muito contexto do Kali**: contêiner sem `ncurses-base`, `screen` mal configurado, shell reverso — `TERM` ausente ou `dumb`. O par `[[ -t 1 ]] + tput … 2>/dev/null` deixa o script em preto e branco em vez de encher o relatório de erro. Para script de configuração com várias perguntas, `whiptail --yesno/--inputbox/--menu` já está instalado (é o que o `kali-tweaks` usa) — devolve o valor pela saída de **erro**, daí a inversão `3>&1 1>&2 2>&3`. E relatório que outra ferramenta vai ler não se alinha com espaços: `printf '%s\t%s\t%s\n'` ou JSON linha a linha, para o `jq` e o `awk` trabalharem sem heurística.

# CONDICIONAIS TEST, [[ ]] E IF

*O script de rotação tinha uma guarda impecável: `if [ -f $ARQUIVO ]`. Na noite em que `ARQUIVO` ficou vazio, o teste **passou** — e a condição que existia para impedir o estrago disse "sim, o arquivo existe" sobre um nome vazio.*

**055**
GUIAS LINUX

## `[` NÃO É SINTAXE: É UM COMANDO — E ISSO EXPLICA TUDO

`[ -f $ARQUIVO ]` → `[` `-f` `]` — **1 argumento útil.** A regra do POSIX para um argumento é "verdadeiro se a cadeia não for vazia" — e `-f` não é vazia. **Passa.**

`[ -f "$ARQUIVO" ]` → `[` `-f` `` `` `]` — **2 argumentos.** Agora é mesmo um teste de arquivo, sobre um nome vazio. **Falha**, como deveria.

`[[ -f $ARQUIVO ]]` → PALAVRA-CHAVE DO SHELL — **Não é um comando.** O shell analisa o conteúdo *sem* dividir em palavras — as aspas ficam dispensáveis. **Falha**, como deveria.

Quando o `[` é finalmente executado, **não existe mais vestígio de que houve ali uma variável**: a linha já passou pelas seis etapas de expansão e a variável vazia sem aspas simplesmente sumiu antes da chamada. Daí as duas regras: **aspas em toda expansão dentro de `[ ]`**, e **`[[ ]]` sempre que o script for Bash**. Pelo mesmo motivo, os espaços em volta dos colchetes não são opcionais — `[-f arq]` é o shell procurando um comando chamado `[-f`, e o `]` é só o último argumento obrigatório. E o `if` não avalia expressão booleana: ele **executa um comando e olha o código de saída** (zero = verdadeiro), então `if grep -q '^root:' /etc/passwd`, `if systemctl is-active --quiet ssh` e `if command -v nmap >/dev/null` dispensam colchete nenhum — e são melhores que capturar a saída e testá-la.

## O MAPA DOS OPERADORES — ARQUIVO · CADEIA · NÚMERO

### ARQUIVO

`-e` existe (qualquer tipo)
`-f` arquivo comum · `-d` diretório
`-L` link simbólico · `-s` não vazio
`-r` `-w` `-x` legível, gravável, executável
`-u` `-g` `-k` SUID, SGID, *sticky*
`-b` `-c` `-p` `-S` bloco, char, fifo, socket
`a -nt b` a é mais novo · `a -ef b` mesmo inode

`-f` segue o link e responde sobre o **destino**; `-L` responde sobre o próprio link.

### CADEIA

`-z s` vazia · `-n s` não vazia
`a == b` igual — **padrão** no lado direito
`a != b` diferente
`a =~ re` casa com a expressão regular
`a < b` ordem alfabética

Em `[[ ]]`, **cite o lado direito** do == para comparar literalmente. Com =~ é o inverso: a regex **não** vai entre aspas — guarde-a numa variável e leia o resultado em `${BASH_REMATCH[1]}`.

### NÚMERO

`-eq` igual · `-ne` diferente
`-lt` `-le` menor, menor ou igual
`-gt` `-ge` maior, maior ou igual
Dentro de `(( ))`, os símbolos:
`(( a > b ))` · `(( a ≥ 1 && a ≤ 10 ))`
`(( contador++ ))` · `(( total += v ))`

**`[[ 10 > 9 ]]` é falso**: compara texto, e "1" vem antes de "9".

**A regra que não falha: número dentro de `(( ))`, o resto dentro de `[[ ]]`.** Os operadores com hífen funcionam nos dois colchetes; os símbolos só têm sentido numérico em `(( ))` — e trocar os contextos é o erro **silencioso** mais caro do capítulo: `[[ $versao > 9 ]]` aprova a versão 10 como menor que a 9 e nunca reclama. Guarde `[ ]` só para script que precise rodar sob `/bin/sh`; ali, use `=` (não `==`) e troque o obsoleto `-a` / `-o` por `[ … ] && [ … ]`.

kali@kali: ~/lab-if

```
$ re='^([0-9]{1,3}\.){3}[0-9]{1,3}(/[0-9]{1,2})?$'   # regex SEM aspas no teste
$ [[ $alvo =~ $re ]] || { erro "alvo invalido: $alvo"; exit 1; }
$ [[ $arq == *.log ]] && echo casou     # padrao glob, nao regex
$ case "$1" in
      -h|--help)      ajuda; exit 0 ;;
      *.tar.gz|*.tgz) tar xzf "$1" ;;
      [0-9]*)         echo "comeca com digito" ;;
      "")             erro "argumento vazio"; exit 1 ;;
      *)              erro "opcao desconhecida: $1"; exit 1 ;;   # o *) e obrigatorio
    esac
$ ( set -e; n=0; (( n++ )); echo "nao chega aqui" )  # ABORTA: devolveu o 0 antigo
$ ( set -e; n=0; (( ++n )); echo "n=$n" )            n=1  # ou: (( n++ )) || true
```

**Testar um arquivo não é mecanismo de segurança.** Entre `if [[ -w "$ALVO" ]]` e o `rm -rf -- "$ALVO"` da linha seguinte existe um intervalo em que outro processo troca o arquivo por um link apontando para outro lugar — é o **TOCTOU** (*time of check to time of use*), explorável quando o script roda com privilégio sobre caminho gravável por terceiros, com o `/tmp` como caso clássico. Defesas: **aja e trate o erro** em vez de testar antes ( `rm -rf -- "$ALVO" 2>/dev/null || true` não tem intervalo) · `tmp="$(mktemp -d)"` para temporário, que já nasce `700` e com nome imprevisível, nunca um nome previsível · `trap` para limpar na saída · e **nunca** opere como root sobre caminho de terceiro sem `--no-dereference` ou `-not -type l` . O teste de arquivo continua útil — para dar mensagem de erro decente antes de tentar.

## RESUMO RÁPIDO

- ✓ `[` **é um comando**: espaços obrigatórios, `]` é só o último argumento, e o conteúdo passa pelas seis etapas de expansão **antes** do teste.
- ✓ `if` **executa um comando e olha o código de saída.** Zero é verdadeiro — qualquer programa serve de condição.
- ✓ **Em Bash,** `[[ ]]` **é o padrão:** não divide em palavras, aceita `&&` / `||` , casa padrão com `==` e regex com `=~` (preenchendo `BASH_REMATCH` ).
- ✓ **Número em** `(( ))` **, o resto em** `[[ ]]` **.** `[[ 10 > 9 ]]` é falso.
- ✓ `case` **para várias alternativas**, com `*)` sempre presente — sem ele, entrada inesperada passa em silêncio.
- ✓ **Sob** `set -e` **,** `(( n++ ))` **com** `n=0` **derruba o script.** Use `(( ++n ))` .

## NO KALI

**Validar o alvo com** `=~` **antes de disparar a ferramenta é a diferença entre laboratório e incidente.** Um teste de formato — `[[ $alvo =~ ^[0-9]{1,3}(\.[0-9]{1,3}){3}(/[0-9]{1,2})?$ ]]` — pega o alvo digitado errado *antes* de a varredura sair; melhor ainda é conferir contra a lista do escopo autorizado e recusar o que não estiver nela. Varrer fora do escopo contratado é crime pela Lei 12.737/2012, e um `if` de três linhas é a barreira mais barata que existe contra um erro de digitação. **Desde 2020 o usuário padrão é o** `kali` **, não o root**: um `(( EUID ≠ 0 )) && { erro "execute com sudo"; exit 1; }` na primeira linha economiza o trabalho perdido no meio do caminho. **Num *rolling*, confira se a ferramenta existe:** `command -v nmap >/dev/null || { erro "sudo apt install nmap"; exit 1; }` — o `which` não é embutido, varia entre distribuições e devolve códigos inconsistentes. E cuidado com ponto de montagem que some: em mídia montada pelo `udisks` ou em live USB, um `[[ -d /media/kali/EVIDENCIA ]]` pode passar e o script gravar no diretório vazio do ponto de montagem em vez de na mídia — `mountpoint -q` responde à pergunta certa. Por fim, `-r` e `-w` respondem sobre a permissão **efetiva**, ACLs e *capabilities* incluídas: um `[[ -r /etc/shadow ]]` verdadeiro para usuário comum é achado de auditoria.

# CÓDIGOS DE SAÍDA

*Duas linhas corretas isoladamente: `cd /var/www/releases/atual` e `rm -rf *`. No dia em que `atual` virou link quebrado, o `cd` falhou — e o `rm` rodou no diretório em que o script tinha começado. Faltava um `&&`.*

**056**
GUIAS LINUX

## O MAPA DOS 256 VALORES — 0 É SUCESSO, O RESTO DIZ *QUAL* FALHA

| | | | | | |
|---|---|---|---|---|---|
| **0** sucesso | **1** erro genérico | **2** uso incorreto — sintaxe, opção inválida | **64–78** convenção do `sysexits.h` | **126** achou, mas não é executável | **127** comando não encontrado |
| **128+N** morto pelo sinal N | **130** Ctrl+C — SIGINT (2) | **137** SIGKILL (9) — o OOM killer | **143** SIGTERM (15) — parada de serviço | **255** fora de faixa, ou erro do `ssh` | **$?** o último — sobrescrito a cada comando |

A faixa de 126 a 143 é a que mais economiza diagnóstico: um serviço que sai com **137** não tem defeito de lógica — foi morto pelo kernel, quase sempre por falta de memória (procure no `dmesg`, não no código). Um **127** é o `PATH` sem o diretório certo; um **126** é bit `x` ausente ou *shebang* errado. **O código é um byte:** `exit 256` vira **0** e `exit -1` vira **255** — contador de erros não serve como código de saída. E `$?` é sobrescrito pelo comando seguinte, *inclusive pelo `echo` que o imprime*: guarde em `rc=$?` se for usar duas vezes. Já `exit` sem argumento propaga o `$?` do último comando — o comportamento certo no fim de um bloco de tratamento de erro.

## CURTO-CIRCUITO · E A ARMADILHA DO FALSO TERNÁRIO

**`A && B || C`**
o caso feliz: **`A` falha**

**A** falhou → **B** pulado pelo `&&` → **C** executa — o `||` viu a falha ✓

**`A && B || C`**
a armadilha: **`A` passa e `B` falha**

**A** sucesso → **B** executa **e falha** → **C** executa **também** ✗

**A forma correta quando B pode falhar:** `if A; then B; else C; fi`. O `||` não faz ideia de que o `&&` existiu — ele só observa o código do que veio *imediatamente* antes. Enquanto `B` for algo que não falha (`echo`, atribuição, `true`), a cadeia é idiomática e segura.

`;` e a **quebra de linha** significam a mesma coisa: *execute o próximo, aconteça o que acontecer*. Para agrupar, `cmd || { erro "falhou"; exit 1; }` roda no **shell atual** (espaço depois de `{` e `;` antes de `}` são obrigatórios); já `cmd || ( erro; exit 1 )` cria **subshell** — o `exit` encerra só o subshell e **o script segue**, com o erro que deveria abortá-lo não abortando nada.

## O *PIPELINE* MENTE · O `SET -E` TEM EXCEÇÕES

kali@kali: ~/lab-exit

```
$ tar czf - /naoexiste | gzip > b.gz; echo $?
0 # o backup "deu certo" e saiu vazio
$ set -o pipefail       # devolve o 1o que falhou
$ false | true; echo $?         1
$ false | true | false
$ echo "${PIPESTATUS[@]}"       1 0 1
# PIPESTATUS tambem e sobrescrito: copie antes de usar
$ readonly EX_USO=2 EX_DEP=3 EX_PERM=4
$ command -v jq >/dev/null || exit "$EX_DEP"
```

### SET -E IGNORA A FALHA EM...

```
if cmd; then … fi
while cmd; do … done
cmd || true
cmd && outro
! cmd
```

Não são caprichos: em todas elas a falha **faz parte da lógica escrita**. Duas consequências pegam quase todo mundo: **1.** uma função chamada dentro de `if` roda **inteira** sem `errexit` — só o retorno final é avaliado; **2.** o **último** comando de uma função vira o retorno dela, e um `(( n ))` com `n=0` reporta falha que nunca houve. Termine toda função com `return 0` explícito.

**Uma quebra de linha entre dois comandos é um `;` — e num script de limpeza isso é o desastre inteiro.** `cd "$DIR"` numa linha e `rm -rf ./*` na seguinte apaga o diretório *atual* se o `cd` falhar. A correção cabe em dois caracteres: `cd "$DIR" && rm -rf ./*` — melhor ainda com `set -euo pipefail` no topo e `echo` na frente no primeiro ensaio. Prefira `./*` a `*`: um arquivo chamado `-rf` ou `--no-preserve-root` deixa de virar opção. E `rm -f` engole o erro sem avisar — foi ele que fez o script do cenário de abertura terminar com cara de sucesso.

## RESUMO RÁPIDO

- ✓ **0 é sucesso, o resto diz qual falha.** `$?` guarda só o último — copie em `rc=$?` antes de reusar.
- ✓ **As faixas têm significado:** 126 achou/não executa · 127 não achou · `128+N` morte por sinal (130 Ctrl+C, 143 SIGTERM, 137 SIGKILL/OOM).
- ✓ **`&&` e `||` são curto-circuito**, e `A && B || C` **não** é um ternário: se `B` falhar, `C` roda também.
- ✓ **`{ …; }` agrupa no shell atual; `( … )` cria subshell** — e o `exit` lá dentro não encerra o script.
- ✓ **Um *pipeline* devolve o código do último.** `set -o pipefail` ou `PIPESTATUS`; sem um dos dois, um `tar | gzip` com o `tar` quebrado reporta sucesso.
- ✓ **Dê nome aos seus códigos** (`EX_USO=2`, `EX_DEP=3`): quem chama distingue "faltou dependência" de "argumento errado".

## NO KALI

**Muita ferramenta usa código ≠ 0 para "não encontrei nada" — e isso não é erro.** O `grep` devolve **1** quando não casa e **2** quando houve erro de verdade; o `diff` devolve 1 para "arquivos diferentes"; o `find` devolve 0 mesmo sem achar nada. Sob `set -e`, um `grep` que não casa **aborta o script** — corrija com `|| true` ou teste com `if`. **O `nmap` sai com 0 mesmo quando o alvo não responde**: para ele, "varri e não achei porta aberta" é sucesso, então decidir o próximo passo pelo código de saída decide errado — use a saída estruturada (`-oX`, `-oG`) analisada depois; o mesmo para `nikto`, `gobuster` e `hydra`, cujos códigos variam entre versões num *rolling*. `systemctl is-active --quiet ssh` foi feito para ser usado por código, não por texto — a palavra pode mudar de idioma e existe o estado `activating`. **O `ssh` devolve o código do comando remoto, mas usa 255 para os próprios erros** (autenticação, host inalcançável): num laço sobre hosts, trate 255 como problema de conexão, registre e siga. E registrar comando, horário e código por alvo é parte da **trilha de auditoria** — é o que sustenta depois a demonstração de que o trabalho ficou dentro do escopo contratado (Lei 12.737/2012 vista pelo lado de quem tem autorização e precisa comprová-la).

# BOAS PRÁTICAS DO SCRIPT

*`limpa.sh` : dezenove linhas escritas numa tarde de 2023, sem shebang, sem comentário, com `777` num diretório do grupo `devs` — e rodando às 4h no crontab do root de três servidores. Este capítulo é sobre não escrever esse arquivo.*

**057**
GUIAS LINUX

## O MOLDE — COMECE TODO SCRIPT DAQUI

comprime — o esqueleto que vale adotar no primeiro dia

```
#!/usr/bin/env bash
# comprime - resumo em uma linha | Uso: comprime [-v] [-n] ARQUIVO...
# Saidas: 0 ok | 1 erro | 2 uso incorreto | 3 dependencia ausente | Versao: 1.2.0
set -euo pipefail
readonly PROGRAMA="${0##*/}"        # nome sem chamar basename
readonly AQUI="$(cd -- "$(dirname -- "${BASH_SOURCE[0]}")" && pwd -P)"
DESTINO="${DESTINO:-/var/backups}"  # ajustavel pelo ambiente, padrao seguro
erro() { printf '%s: %s\n' "$PROGRAMA" "$*" >&2; }
main() {
  command -v gzip >/dev/null || { erro "gzip ausente"; exit 3; }
  (( $# )) || { uso >&2; exit 2; }
  for a in "$@"; do
    (( SIMULAR )) && { printf 'SIMULACAO: gzip -- %q\n' "$a"; continue; }
    gzip -- "$a"                    # o -- protege nome iniciado por hifen
  done
}
main "$@"
```

**`main "$@"`**
O shell lê o arquivo *enquanto* executa: com tudo em funções e a chamada no fim, ele precisa ter lido o arquivo **inteiro** antes de rodar qualquer coisa — imune a edição em execução e a *download* truncado.

**`$BASH_SOURCE`**
O diretório do **script**, não o seu: `$PWD` depende de onde o usuário estava e `$0` falha sob `source`. O `pwd -P` ainda resolve os links.

**`mktemp` + `trap`**
`tmp="$(mktemp -d)"` e `trap 'rm -rf -- "$tmp"' EXIT INT TERM`: limpa na saída normal, no erro sob `set -e` e no sinal. Execução única: `flock -n 9`.

**`-n` · idempotência**
Todo script destrutivo merece **simulação barata de digitar**, e o `%q` mostra o nome escapado como o shell o vê. Rodar duas vezes não pode estragar: `mkdir -p`, `rm -f`, `ln -sfn`, `install -d`.

## QUEM PODE REESCREVER ? · E EM QUE CAMADA VOCÊ ESTÁ

**✗ VULNERÁVEL**
`-rwxrwxr-x root:devs /opt/limpa.sh`

1. O *crontab* do root executa o script às 4h.
2. Qualquer membro de `devs` reescreve o arquivo.
3. Às 4h01, o que esse membro escreveu roda **como root**.

**✓ CORRETO**
`-rwxr-xr-x root:root /usr/local/bin/limpa`

1. Instalado com `sudo install -o root -g root -m 755`.
2. Só o root grava; os demais leem e executam.
3. Às 4h01, roda o que o root aprovou.

↑ MAIS PORTÁTIL — MAIS RECURSOS ↓

**`#!/bin/sh` — POSIX puro**
dash, busybox ash, Alpine, *initramfs*, contêiner mínimo. **Sem** `[[ ]]`, arrays, `local`, `${var,,}`, `$RANDOM`, `{1..10}`.

**`#!/usr/bin/env bash` — Bash 3**
Linux, macOS (que ainda traz o 3.2), BSD com Bash. Ganha `[[ ]]`, `=~`, arrays indexados, `local`, `(( ))`. **É a camada certa para administração de Linux.**

**Bash 4+**
`declare -A`, `${var,,}`, `mapfile`, `globstar`. Ausente no macOS de fábrica.

**+ utilitários GNU**
`sed -i`, `date -d`, `readlink -f`, `sort -V`, `find -printf`. Sintaxe diferente no BSD/macOS, ausentes no busybox.

Ao descer para POSIX: `[[ $a == b* ]]` → `case` · `(( n > 10 ))` → `[ "$n" -gt 10 ]` · `arr=(a b c)` → `set -- a b c` · `source` → `.` · `echo -e` → `printf`. Verifique com `checkbashisms`, `shellcheck -s sh` e uma execução real sob `dash script.sh`.

**SUID em script de shell não funciona — e a tentação de contornar isso é o buraco.** O kernel **ignora** o bit SUID em arquivo com *shebang* (seria explorável por corrida entre a leitura do *shebang* e a abertura pelo interpretador). A saída legítima é uma regra específica no `sudoers`, criada com `visudo -f /etc/sudoers.d/…` — e aí a permissão do arquivo vira o ponto crítico: com um `NOPASSWD` apontando para um script **gravável pelo grupo**, quem grava tem root sem senha. Evite curinga na regra (`/usr/local/bin/*`) e `sudo` de interpretador — `sudo python3`, `perl`, `awk`, `vim` equivalem a `sudo su`. `chmod 777` **não tem caso de uso legítimo.** Audite com `namei -l /usr/local/bin/cmd` (o caminho *inteiro*) e `sudo find /usr/local/bin /opt -type f -perm -o+w -ls`: a resposta esperada é vazia. A regra em uma frase — **tudo que um processo privilegiado executa ou carrega deve ser gravável apenas por quem já tem esse privilégio**: o arquivo, o diretório que o contém, todos os diretórios acima, o `.conf` que ele lê e a biblioteca que ele carrega com `source`.

## RESUMO RÁPIDO

- ✓ **Sempre o mesmo molde:** *shebang* · comentário com uso e códigos de saída · `set -euo pipefail` · constantes `readonly` · funções de mensagem · `main "$@"` na última linha.
- ✓ **A pergunta da permissão é "quem pode reescrever?"**, não "quem executa". `install -o root -g root -m 755` em `/usr/local/bin` — diretório que nenhum pacote toca.
- ✓ **Escolha a camada de portabilidade uma vez**, declare-a no *shebang* e verifique com ferramenta. "Portabilidade que não se testa é portabilidade imaginária."
- ✓ `shellcheck` **não é opcional** em código que outra pessoa executa. Silencie por *ocorrência* e com justificativa (`# shellcheck disable=SC2016`), nunca por arquivo — e ponha num *hook* de pré-commit.
- ✓ Os campeões: **SC2086** aspas · **SC2046** `$( )` sem aspas · **SC2006** crase · **SC2045** `ls` em laço · **SC2164** `cd` sem tratar falha · **SC2181** testar `$?` em vez do comando no `if`.
- ✓ **Não use** `IFS=$'\n\t'` como "modo estrito": muda comportamento global para compensar defeito local, e quebra `read`, `set --` e código de terceiros. Aspas em toda expansão resolve sem efeito colateral.

## NO KALI

**Num *rolling*, script é código vivo.** Uma ferramenta muda de opção ou de formato de saída entre duas execuções do mesmo script — as três defesas: `command -v` antes de usar, **saída estruturada** (`-oX`, `--json`) em vez de analisar texto de tela, e uma linha de versão para saber com o que ele foi testado. **Regra `NOPASSWD` apontando para script é a escalada mais encontrada em máquina de laboratório**: se você monta ambiente de treino, está criando o vetor de propósito; se administra a sua máquina, rode `sudo -l` e trate cada linha que aponte para arquivo gravável por não-root como root disponível. **Script de live USB ou de *initramfs* está na camada cinza**: ali o shell é o `busybox ash` e `[[ ]]` e arrays simplesmente não existem — escreva em POSIX desde o início e teste com `dash script.sh`, porque descobrir isso durante uma recuperação real é o pior momento possível. E **automação de ferramenta de segurança merece registro e simulação por padrão**: valide o alvo contra a lista do escopo autorizado, exiba o que fará, exija confirmação (ou um `--sim` explícito) e registre horário, comando e código de saída. Serve para depurar e serve para demonstrar depois que o trabalho ficou dentro do contratado — uso não autorizado, ainda que por erro de script, é crime pela Lei 12.737/2012.

GUIA VISUAL 058

# LAÇOS
# FOR · WHILE · UNTIL

*O expurgo da madrugada rodava havia meses até a coleta gravar `relatorio de marco.log`. Naquela noite o `gzip` foi chamado três vezes — com relatorio, de e marco.log —, o log real continuou crescendo e um mês depois a partição encheu. O erro não estava no `for` nem no `gzip`: estava na **fonte da lista**.*

**058**
GUIAS LINUX

## DE ONDE VEM A LISTA — O LAÇO ITERA PALAVRAS, NÃO ARQUIVOS

| | | |
|---|---|---|
| `for f in $(ls *.log)` | `relatorio` `de` `marco.log` | **3 iterações.** A saída virou **texto** e o `IFS` recortou nos espaços; um nome com `*` ainda se expandiria outra vez. Não existe caso em que iterar sobre `ls` seja melhor. |
| `for f in *.log` | `relatorio de marco.log` | **1 iteração.** O *globbing* entrega **itens**, não uma cadeia de caracteres: espaço e quebra de linha sobrevivem inteiros. Vale igual para `{1..10}` e `"${arr[@]}"`. |
| `while IFS= read -r -d '' f`<br>`  < <(find … -print0)` | `relatorio de marco.log` | **1 iteração**, e também em subdiretório e com `\n` no nome. O separador é o **byte nulo** — o único caractere que o kernel proíbe num nome de arquivo. |

**Glob que não casa não vira lista vazia:** em diretório sem `*.log`, o laço roda **uma vez** com `f` valendo o texto literal `*.log`. Conserte com `shopt -s nullglob` ou com `[[ -e $f ]] || continue` na primeira linha do corpo. E se o corpo é uma chamada única a um comando, dispense o laço: `find … -exec gzip {} +` ou `xargs -0`.

## QUEM TESTA O QUÊ, E QUANDO

### `for x in lista`

expande a lista
**uma única vez**

↓ sobrou item?

x = próximo
executa o corpo ↺

Acrescentar arquivos ao diretório no meio de um `for f in *` **não** muda o que já foi expandido.

### `for (( ; ; ))`

i=0 · i<n · i++
contexto aritmético

↓ enquanto verdadeiro

executa o corpo ↺

A forma certa quando o limite é **variável**: `{1..$n}` não funciona — chaves expandem antes das variáveis.

### `while comando`

o comando
saiu com **0**?

↓ sim

executa o corpo ↺

Reavalia **a cada volta**: serve para esperar estado externo — e erro na condição vira laço infinito, não mensagem.

### `until comando`

o comando
saiu com **0**?

↓ **não**

executa o corpo ↺

É o `while !` escrito de forma legível: `until systemctl is-active --quiet pg; do sleep 2; done`.

## O IDIOMA DE LEITURA — TRÊS DETALHES OBRIGATÓRIOS

while read — linha a linha, e por campos

```
# IFS= preserva espacos das pontas | -r preserva a \ | < depois do done evita o subshell
$ while IFS= read -r linha || [[ -n $linha ]]; do
>     printf '[%s]\n' "$linha"
> done < dados.txt          # o || pega a ultima linha sem \n final

# com valor, o IFS divide em campos; o resto cai na ultima variavel
$ while IFS=: read -r usuario _ uid _ _ _ shell; do
>   (( uid >= 1000 && uid < 65534 )) && printf '%-12s %s\n' "$usuario" "$shell"
> done < /etc/passwd
kali         /usr/bin/bash
```

`break n` e `continue n` **saem de *n* níveis** — `break 2` abandona os dois laços aninhados e dispensa a variável `encontrado=1`. E **redirecione o laço inteiro** (`done > saida.txt`) em vez de `>>` a cada volta: o arquivo é aberto uma vez, não cinco mil.

## A VARIÁVEL QUE SOME NO `done`

**SHELL PAI**
`total=0` continua valendo **0** depois do laço. Nenhum erro é emitido.

**SUBSHELL DO PIPE**
a cópia de `total` chega a 4 — e morre com o processo.

`cat arq | while read l; do (( total++ )); done` — cada estágio de um *pipe* roda no **próprio processo**. O sintoma é sempre o mesmo: valor certo *dentro* do laço, valor antigo depois do `done`.

- **`done < arq`** redirecionamento: sem *pipe*, sem *fork*. A forma preferida quando a origem é um arquivo.
- **`done < <(cmd)`** substituição de processo — o laço fica no pai. O **espaço entre os dois `<` é obrigatório**.
- **`shopt -s lastpipe`** o último estágio passa a rodar no pai (exige `set +m` em script).

**Um erro de lógica num `if` atinge um arquivo; num laço, atinge todos os que casaram.** Por isso todo laço destrutivo se escreve **duas vezes**: primeiro com `echo "APAGARIA: $arq"` no lugar da ação, para ler a lista inteira; só depois se troca o `echo` por `rm -f -- "$arq"`. O `--` impede que um arquivo chamado `-rf` seja lido como opção — vale para `rm`, `cp`, `mv` e `chown`. **Nunca rode laço destrutivo com lista vinda de variável não validada:** um `$DIR` vazio transforma `rm -rf "$DIR"/*` em `rm -rf /*`. Ensaie num diretório descartável de `mktemp -d` e use `rm -i` na primeira execução real.

## RESUMO RÁPIDO

- ✓ **O `for` itera sobre palavras, não sobre arquivos.** Quem decide as palavras é a expansão que gera a lista: `for f in *.log`, **nunca** `for f in $(ls *.log)`.
- ✓ **Glob sem correspondência passa literalmente** — `shopt -s nullglob` ou `[[ -e $f ]] || continue`. E `**` recursivo exige `shopt -s globstar`, desligado por padrão em toda distribuição.
- ✓ **`for (( i=0; i<n; i++ ))` quando o limite é variável**, porque a expansão de chaves ocorre antes da de variáveis e `{1..$n}` vira texto literal.
- ✓ **`while` / `until` reavaliam a condição a cada volta**: é a ferramenta de esperar por estado externo — sempre com `sleep` e com **limite de tentativas**, senão o script espera para sempre.
- ✓ **O idioma é `while IFS= read -r linha; do … done < arquivo`**, com `|| [[ -n $linha ]]` quando o arquivo puder não terminar em quebra de linha.
- ✓ **`cmd | while read` perde as variáveis no `done`** — o lado direito do *pipe* é subshell. Corrija com `< arquivo` ou `< <(comando)`.
- ✓ **Para `find`, o par seguro é `-print0 + read -r -d ''`.** Corpo com uma só chamada? Prefira `-exec … +` ou `xargs -0`.

## NO KALI

**O laço amplifica tanto o acerto quanto o engano — por isso o filtro de escopo vai *dentro* dele, não depois.** Valide cada alvo contra a faixa contratada logo na primeira linha do corpo: `[[ $alvo =~ ^10\.10\.10\. ]] || { echo "fora do escopo: $alvo" >&2; continue; }`. Um erro de digitação numa lista de 254 endereços não produz um alvo errado, produz 254 — e varrer fora do escopo autorizado é crime pela Lei 12.737/2012. **`for host in {1..254}` é conveniente e quase sempre a ferramenta errada:** `nmap`, `masscan` e `fping` já varrem em paralelo com controle de temporização, e `nmap -sn 10.0.0.0/24` leva segundos onde o laço serial leva minutos. Use o laço para **orquestrar** — iterar sobre arquivos de saída, perfis de varredura, alvos já confirmados —, não para substituir a ferramenta. **O `sleep` é controle de intensidade, não enfeite:** laço sem pausa contra um serviço de laboratório é indistinguível de uma negação de serviço acidental, e derruba justamente o alvo que você precisava analisar. **Saída de `gobuster` e `nikto` vira nome de arquivo com `%20`, `&`, `?` e espaço** — exatamente a família que quebra `for f in $(ls)`: percorra com glob ou `find -print0` e cite toda expansão. E no *rolling*, cheque a ferramenta **antes** de entrar no laço (`command -v nmap >/dev/null || exit 1`): dentro dele, o teste custa 254 chamadas e 254 mensagens idênticas.

GUIA VISUAL 059

# FUNÇÕES RETORNO E ESCOPO

*O inventário contava pacotes com `return "$n"`. Na instalação enxuta imprimia **180** — certo. Na estação com o `kali-linux-everything`, 4.412 pacotes, imprimiu **60**. Sem erro, sem aviso: o valor não coube no canal. Uma função tem **dois** caminhos para devolver informação, e o script escolheu o estreito.*

**059**
GUIAS LINUX

## OS DOIS CANAIS DE SAÍDA — DADO ≠ VEREDITO

**FUNÇÃO**
`contar_pacotes`
Um **comando novo**, indistinguível de
`ls`
para quem chama — e com precedência sobre o programa externo.

**SAÍDA PADRÃO**
`n=$(contar_pacotes)`
**Texto de qualquer tamanho.** Cabe `4412`, cabe uma tabela inteira, cabe um caminho com espaços. Custo: o `$( )` cria **subshell** — variável atribuída lá dentro morre junto — e remove as quebras de linha do fim.

**CÓDIGO DE SAÍDA**
`contar_pacotes; $?`
**Inteiro de 0 a 255, e só.** O `return` responde *deu certo?*, não *qual é o resultado?*. `return` sem argumento devolve o código do **último comando** — o que se quer numa função-predicado como `usuario_existe() { id "$1" >/dev/null 2>&1; }`.

| `return 0` | `return 1` | `return 255` | `return 256` | `return 300` | `return 4412` |
|---|---|---|---|---|---|
| **0** | **1** | **255** | **0** falha vira sucesso | **44** | **60** o bug da abertura |

### COMO DEVOLVER DADOS DE VERDADE

**`n=$(contar_pacotes)`**
**Idiomática.** A função imprime, o chamador captura. Comporta-se como função de verdade — ao custo de um processo por chamada, o que pesa em laço apertado.

**`RESULTADO=… ; contar_pacotes`**
**Global por convenção.** Sem subshell, a mais rápida. Cria acoplamento invisível: quem lê a chamada não vê que a variável mudou. Nome em MAIÚSCULAS e comentário na linha acima.

**`local -n destino="$1"`**
**Referência nomeada** (Bash 4.3+): o chamador escolhe onde o resultado vai parar. Prefixe o nome interno (`__destino`) — passar variável homônima dispara erro de referência circular.

`local x=$(cmd)` **engole o erro do comando** (SC2155): `local` é um comando, e o `$?` que sobra é o **dele** — quase sempre zero. Declare numa linha, atribua na seguinte: `local v; v=$(cmd) || return 1`. Vale igual para `declare`, `export` e `readonly`.

### ESCOPO **DINÂMICO**: VALE A PILHA, NÃO O TEXTO

**nível superior**
`segredo` **não existe** aqui — nem antes nem depois da chamada.

↓ chama

**`externa() { local segredo="abc"; … }`**
Dona da variável. Ao retornar, o valor anterior é restaurado.

↓ chama

**`interna() { echo "$segredo"; }`**
Vê `abc` sem receber nada como argumento.

Em C ou Python o escopo é **léxico** — definido por onde a função está escrita. No shell é definido por **quem chamou quem** em execução. Sem `local`, toda variável da função é global; e um `local i` sobrescreve o `i` de qualquer função chamada abaixo. Em biblioteca, nada de `i`, `n`, `tmp` ou `arquivo`.

### ENCERRAR, ENVOLVER, E QUANDO PROMOVER A FUNÇÃO A SCRIPT

as três armadilhas do dia a dia

```
# exit encerra o SCRIPT; return encerra so a FUNCAO. Biblioteca nunca chama exit.
morrer() { printf 'erro: %s\n' "$*" >&2; exit 1; }   # unica excecao legitima

# 'command' pula a funcao e chama o programa real -- sem ele, recursao infinita
rm() { command rm -i "$@"; }     # legitimo no ~/.bashrc, armadilha dentro de script

# $0 continua sendo o SCRIPT dentro da funcao; o nome da funcao esta em FUNCNAME
$ quem() { echo "$0 | ${FUNCNAME[0]} | chamado por ${FUNCNAME[1]:-<script>}"; }
./ctx.sh | quem | chamado por externa

# sob set -e, a funcao usada como CONDICAO tem o set -e suspenso dentro dela
if valida; then …   # erro interno nao aborta: exija return explicito em cada saida
```

| SITUAÇÃO | ESCOLHA | SITUAÇÃO | ESCOLHA |
|---|---|---|---|
| Alterar diretório ou variáveis da sessão | **Função** — script não consegue | Chamada por `cron` ou `systemd` | **Script** próprio |

| SITUAÇÃO | ESCOLHA | SITUAÇÃO | ESCOLHA |
|---|---|---|---|
| Corpo passou de ~50 linhas ou tem lógica própria | **Script** | Usada em três scripts diferentes | **Biblioteca** com `source` |

## RESUMO RÁPIDO

- ✓ **Uma função é um comando**, não uma função matemática: recebe posicionais ( `$1` , `"$@"` ), tem precedência sobre o programa externo e **precisa ser definida antes** da chamada. Prefira a forma POSIX `nome() { … }` — `function` não sobrevive a um `#!/bin/sh` .
- ✓ `return` **devolve código de 0 a 255, nunca um valor.** `return 4412` vira **60** e `return 256` transforma falha em sucesso. Dado sai pela **saída padrão**.
- ✓ **Sem** `local` **, toda variável da função é global** — inclusive os contadores de laço, a fonte nº 1 de colisão silenciosa.
- ✓ **Escopo dinâmico:** o `local` é visível em tudo que a função chamar abaixo, nunca acima.
- ✓ `local x=$(cmd)` **descarta o código de saída de** `cmd` **.** Declare numa linha, atribua na seguinte.
- ✓ `$0` **é o script, não a função** — use `${FUNCNAME[0]}` , e a pilha inteira em `FUNCNAME` , para o log dizer de onde o erro veio.
- ✓ `command` **evita a recursão infinita** ao envolver um comando; `builtin` força o *builtin*. Em script, dê nome próprio à função em vez de mascarar o comando.

## NO KALI

**Função de varredura não chama** `exit` **.** Num script que percorre alvos autorizados, abortar o programa inteiro porque um host não respondeu descarta o trabalho já feito nos anteriores — o relatório parcial de dezoito alvos vale mais que nenhum relatório. Devolva `return` com código por tipo de falha (**1** host inalcançável, **2** fora do escopo, **3** ferramenta ausente) e deixe o laço decidir se continua. E cuidado com o efeito colateral: se essa função for chamada dentro de `$( )` , o `exit` mata só o subshell e o script segue como se nada tivesse acontecido. **A validação de escopo pertence a uma função, e ela devolve código, não texto:** `no_escopo "$alvo" || continue` é legível e testável isoladamente; a validação escrita *inline* em quatro pontos do script garante que uma das cópias vai divergir — e varrer fora do escopo contratado é crime pela Lei 12.737/2012. **Não sobrescreva** `nmap` **,** `curl` **ou** `ping` **com função de mesmo nome:** quem lê `nmap -T4` conclui que a taxa é a do `-T4` , sem ver o `--max-rate` escondido vinte linhas acima. Chame de `nmap_lento` — o nome documenta a intenção. `local` **em função que manipula credencial não apaga nada da memória:** ele controla visibilidade, não conteúdo. E **não use** `export -f` **em script que eleva privilégio** — função exportada é código que viaja no ambiente, e foi exatamente essa forma de transporte que o Shellshock (CVE-2014-6271) explorou.

GUIA VISUAL 060

# ARRAYS INDEXADOS E ASSOCIATIVOS

*O backup guardava as exclusões numa variável de texto. Funcionou meses, até alguém acrescentar `/home/kali/Meus Documentos` : o `rsync` recebeu `--exclude=/home/kali/Meus` e, à parte, `Documentos` — que virou **mais uma origem**. Pôr aspas piora: tudo vira um argumento gigante. **Cadeia de texto não guarda onde um item termina.***

060
GUIAS LINUX

## AS QUATRO EXPANSÕES DE UM ARRAY INTEIRO

`arr=(um "dois com espaco" tres)` — a pergunta é sempre a mesma: o resultado vira **vários argumentos** ou **um texto**?

| | | |
|---|---|---|
| `"${arr[@]}"` | um · dois com espaco · tres | **3 palavras.** Um argumento por item, espaços preservados. **É a forma correta** para passar a comando, iterar num `for` ou copiar para outro array. Array vazio → zero argumentos. |
| `"${arr[*]}"` | um dois com espaco tres | **1 palavra.** Itens unidos pelo **primeiro caractere do** `IFS` . Só para exibir ou registrar em log. Outro separador: `( IFS=,; echo "${arr[*]}" )` num subshell. |
| `${arr[@]}` | um · dois · com · espaco · tres | **4 palavras.** Sem aspas, o `IFS` recorta de novo — o bug da abertura, agora com array. |
| `${arr[*]}` | um · dois · com · espaco · tres | **4 palavras**, idênticas às de cima: **sem aspas, `[@]` e `[*]` são indistinguíveis** — e ambos desfazem a estrutura que motivou o array. |

`${#arr[@]}` conta **elementos** · `${#arr[0]}` conta **caracteres do primeiro** · `${!arr[@]}` lista os **índices existentes** · `${arr[-1]}` é o último · `${arr[@]:1:3}` fatia (o **espaço** em `${arr[@]: -2}` é obrigatório, senão o `:-` vira valor padrão). O par `@` / `*` é o mesmo de `"$@"` / `"$*"` — não é coincidência: os posicionais **são** um array.

## A MESMA TABELA DE CHAVES — MUDA SÓ O TIPO DA CHAVE

`declare -a` · chave = inteiro · **esparso**

| | | |
|---|---|---|
| `[0]` | a | |
| `[2]` | c | índice 1 apagado |
| `[3]` | d | |
| `[9]` | j | saltou de 3 para 9 |

`${#arr[@]}` vale **4**, mas o maior índice é **9**. Um `for (( i=0; i<${#arr[@]}; i++ ))` perderia o `j` e leria dois índices vazios. `arr[100]=x` não aloca 97 posições — só acrescenta uma chave.

**Percorra pelos valores ou pelos índices que existem:** `for v in "${arr[@]}"` ou `for i in "${!arr[@]}"` . Reindexe com `arr=("${arr[@]}")` . Cite o `unset 'arr[1]'` — sem aspas o `[1]` pode virar glob.

`declare -A` · chave = texto qualquer

| | | |
|---|---|---|
| `[ssh]` | 22 | |
| `[http]` | 80 | |
| `[https]` | 443 | |
| `[dns rev]` | 53 | chave com espaço, citada |

A ordem de `${!mapa[@]}` é a da tabela *hash*: **nem a de inserção, nem a alfabética**, e pode mudar entre versões do Bash. Relatório sai por `printf '%s\n' "${!mapa[@]}" | sort` .

**Sem `declare -A` , o Bash trata como array indexado** e avalia a chave como *aritmética*: `ssh` e `http` valem zero, tudo vai para o índice 0 sobrescrevendo-se — **e nenhum erro é emitido**. Dentro de função, `declare -A` cria variável local: use `declare -gA` para global.

### CRIAR, CARREGAR E MONTAR A LINHA DE COMANDO

o padrão que resolve o problema da abertura em qualquer contexto

```
arr=(um dois tres)   arr+=(quatro)   arr[9]=dez   # literal, append, por indice
logs=( /var/log/*.log )               # do glob: cada arquivo e um item
mapfile -t linhas < arquivo.txt       # -t tira o \n; mais rapido e seguro que while read +=
mapfile -t users < <(cut -d: -f1 /etc/passwd)
IFS=: read -ra caminhos <<< "$PATH"   # de uma cadeia, com separador escolhido

# flags condicionais + caminho com espaco, sem eval e sem depender do IFS
argumentos=( -a --delete )
[[ -n ${VERBOSE:-} ]] && argumentos+=( -v )
[[ -n ${LIMITE:-}  ]] && argumentos+=( --bwlimit="$LIMITE" )
for dir in "${EXCLUSOES[@]}"; do argumentos+=( --exclude="$dir" ); done
$ printf 'rsync'; printf ' %q' "${argumentos[@]}"   # confira ANTES de executar
rsync -a --delete -v --bwlimit=2000 --exclude=/home/Meus\ Documentos
$ rsync "${argumentos[@]}" "$ORIGEM/" "$DESTINO/"
```

**O que arrays não fazem:** não aninham — o shell tem uma dimensão só (as saídas são chave composta `m["2,3"]` , ou aceitar que o problema pede `awk` ou Python) — e **não são passados nem devolvidos por funções**: expanda com `"${arr[@]}"` e reconstrua dentro ( `local -a itens=( "$@" )` ), ou use `local -n ref="$1"` quando a função

precisa *modificar* o array do chamador. E **arrays não são POSIX**: `arr=( ... )` falha no `dash` com `Syntax error: "(" unexpected` — se o script usa array, o *shebang* é `#!/usr/bin/env bash`, sem exceção.

## RESUMO RÁPIDO

- ✓ **Lista de caminhos, alvos ou opções é array, nunca texto.** A cadeia depende do `IFS` e quebra no primeiro item com espaço; pôr aspas na cadeia só troca um defeito por outro.
- ✓ `"${arr[@]}"` **com aspas** produz um argumento por elemento — a forma certa em quase todo uso. `"${arr[*]}"` junta numa palavra só e serve para exibir. **Sem aspas, as duas viram a mesma coisa** e desfazem o array.
- ✓ `${#arr[@]}` **conta elementos;** `${#arr[0]}` **conta caracteres.** Os índices que existem estão em `${!arr[@]}`.
- ✓ **Arrays do shell são esparsos:** `unset` deixa buraco, e iterar por índice numérico pula elementos.
- ✓ **Associativo exige** `declare -A` **antes do primeiro uso** — sem isso a chave de texto vira aritmética, vale zero, e tudo se sobrescreve no índice 0 em silêncio.
- ✓ `mapfile -t` **carrega arquivo ou saída de comando** em array, uma linha por item — mais rápido e mais seguro que acumular com `+=` num `while read`.
- ✓ **Monte a linha com** `argumentos+=( … )` **e chame** `cmd "${argumentos[@]}"`**.** Resolve flag condicional e caminho com espaço sem `eval` — e o `eval` reintroduz injeção de comando sem resolver nada.
- ✓ Sob `set -u`, array vazio em `"${arr[@]}"` era fatal **até o Bash 4.3**; do 4.4 em diante (todo Kali atual) expande para nada, corretamente.

## NO KALI

**Lista de alvos autorizados é array, nunca cadeia de texto.** `ALVOS=( 10.10.10.5 "servidor de teste.lab" )` sobrevive a nome com espaço e a lista vazia; `ALVOS="10.10.10.5 10.10.10.7"` depende do `IFS` e falha em silêncio no primeiro item atípico. Como o laço sobre alvos é justamente onde o filtro de escopo se aplica, o tipo de dado errado aqui vira **alvo varrido fora do escopo contratado** — crime pela Lei 12.737/2012. **Opções de** `nmap`**,** `gobuster` **e** `hydra` **montadas condicionalmente pedem array:** o *wrapper* que escolhe entre `-sS` e `-sT` conforme o privilégio, acrescenta `--max-rate` quando há limite contratado e insere o arquivo de saída só às vezes é exatamente o caso do `argumentos+=( … )` — com *string* e `eval`, uma senha ou um caminho com aspas na lista vira execução de comando. **Associativo é a estrutura natural para agregar resultados:** `achados["$ip:$porta"]="$banner"` dá deduplicação de graça e consulta direta, sem reprocessar arquivo — lembrando de ordenar na hora do relatório. `mapfile -t` **sobre arquivo de alvos é mais seguro que** `for alvo in $(cat alvos.txt)`, que divide por espaço e ainda expande glob numa linha de comentário; filtre com `[[ $alvo =~ ^[[:space:]]*(#|$) ]] && continue` dentro do laço. **Mas wordlist não cabe em array:** o `rockyou.txt` tem mais de 14 milhões de linhas e o `mapfile` carregaria tudo na memória do shell, com dezenas de bytes de sobrecarga por item. Wordlist se processa em fluxo com `while IFS= read -r`, ou vai direto para a ferramenta. **Array é para dezenas ou milhares de itens, não para milhões.**

# POSICIONAIS E GETOPTS

*A análise do script de coleta cresceu um `if` por vez: `if [ "$1" = "-v" ]; then … shift; fi`, três vezes. Funcionava — enquanto as opções viessem **na ordem dos `if`**, uma por vez, e nunca depois do arquivo. A análise de argumentos é a fronteira entre **o que o usuário digitou** e **o que o script executa**, e ela precisa de estrutura.*

**061**
GUIAS LINUX

## CINCO INVOCAÇÕES RAZOÁVEIS — E NENHUMA MENSAGEM DE ERRO

**`coletar -n -v dados`**
O `-v` é testado **primeiro**: não é reconhecido, e `VERBOSE` fica 0.

**`coletar -vn dados`**
Agrupadas: `"-vn"` não é igual a `"-v"`. Nada é reconhecido.

**`coletar dados -v`**
Opção **depois** do operando: simplesmente ignorada.

**`coletar -o dados`**
`SAIDA` vira `dados` e o **arquivo fica vazio** — argumento faltando.

**`coletar -- -rf`**
O `--` vira **nome de arquivo**, e o `-rf` chega ao comando interno como **opção**.

GETOPTS CAMINHANDO PELO VETOR · `WHILE GETOPTS ':VNO:H' OPCAO`

| `-vn` $1 | `-o` $2 | `saida.txt` $3 | `alvo1` $4 | `alvo2` $5 |
|---|---|---|---|---|

| | | |
|---|---|---|
| 1ª | `opcao=v · OPTARG vazio · OPTIND=1` | ainda **dentro** do `-vn` : o *getopts* desagrupa sozinho. |
| 2ª | `opcao=n · OPTARG vazio · OPTIND=2` | terminou o `-vn` ; agora aponta para `$2` . |
| 3ª | `opcao=o · OPTARG=saida.txt · OPTIND=4` | consumiu **dois** argumentos. Aceita colado também: `-osaida.txt` . |
| 4ª | `devolve código ≠ 0 · OPTIND=4` | `$4` não começa com hífen: acabaram as opções, o laço termina. |

`shift $(( OPTIND - 1 ))` — Descarta os três argumentos já analisados: agora `"$@"` contém **só os operandos** e `$#` vale 2. **Sem essa linha**, o laço seguinte processaria `-vn` , `-o` e `saida.txt` como alvos — o erro nº 1 com `getopts` , e ele não emite aviso nenhum: só um alvo chamado `-vn` .

o esqueleto que resolve os cinco casos

```
while getopts ':vno:h' opcao; do   # : inicial = modo silencioso
  case "$opcao" in                 # o : DEPOIS da letra = exige argumento
    v)  VERBOSE=1 ;;
    n)  SIMULAR=1 ;;
    o)  SAIDA="$OPTARG" ;;
    h)  uso; exit 0 ;;             # --help e uso bem-sucedido: stdout, codigo 0
    :)  erro "-$OPTARG exige um argumento"; exit 2 ;;
    \?) erro "opcao invalida: -$OPTARG"; uso >&2; exit 2 ;;
  esac
done
shift $(( OPTIND - 1 ))            # a linha que todo mundo esquece
(( $# )) || { uso >&2; exit 2; }   # erro de uso: stderr, codigo 2

# opcoes LONGAS: getopts nao faz. O laco com case faz -- e trata o --
while (( $# )); do case "$1" in
  -v|--verbose) VERBOSE=1; shift ;;
  -o|--saida)   SAIDA="$2"; shift 2 ;;
  --saida=*)    SAIDA="${1#*=}"; shift ;;
  --)           shift; break ;;    # fim das opcoes
  -*)           erro "opcao desconhecida: $1"; exit 2 ;;
  *)            break ;;           # primeiro operando
esac; done
```

| O QUE COBRE | IF | WHILE+CASE | GETOPTS |
|---|---|---|---|
| Agrupadas `-vn` | não | não | sim |
| Colado `-osaida` | não | não | sim |
| Longas `--verbose` | não | sim | não |
| Ordem livre das opções | não | sim | sim |
| Separador `--` | não | sim | sim |
| Rejeita opção desconhecida | não | sim | sim |

**O `if` encadeado não tem caso de uso:** custa o mesmo trabalho e falha em tudo. Curtas apenas → `getopts` ; qualquer opção longa → laço com `case` . O `getopt` externo do `util-linux` faz as duas coisas, mas exige o idioma `eval set -- "$OPCOES"` e não existe no BSD/macOS.

**Posicionais são um array:** `"$@"` repassa, `"$*"` só exibe, e `${10}` exige as chaves — `$10` é lido como `$1` seguido de `0` . `set -- a b c` reescreve a lista (o `--` inicial protege argumento com hífen). E `shift` **falha com código 1** se pedir mais do que existe, sem alterar nada: teste `$#` antes, ou o laço nunca termina.

**Todo caminho vindo de variável entra no comando depois de um `--`.** Se `$arquivo` valer `-rf` ou `--no-preserve-root`, o utilitário lê o nome como **opção**: escreva `rm -- "$arquivo"`, `cp -- "$origem" "$destino"`, `chown -- "$dono" "$alvo"`. Vale também para `grep -- "$padrao"` — um padrão que comece com hífen vira opção do `grep`, e se ele vem de entrada do usuário isso é uma forma de alterar o comportamento do comando à distância. Para glob em diretório arbitrário, a defesa complementar é `for f in ./*`: cada item passa a começar com `./`, nunca com hífen. E valide **antes** de qualquer efeito colateral — quantidade mínima de operandos, coerência entre opções (`-n` com `-o` é incompatível) e se o destino é gravável.

## RESUMO RÁPIDO

- ✓ **Cadeia de `if` comparando `$1` falha em silêncio** na ordem trocada, na flag agrupada, na opção depois do operando e no argumento faltando.
- ✓ **`getopts` traz a convenção POSIX de graça:** agrupamento `-vn`, argumento colado `-osaida`, `OPTARG` e parada no `--`.
- ✓ **Comece a cadeia com `:`** (`':vno:h'`) para ficar com o controle das mensagens — sem isso o Bash imprime as dele, em inglês, sem o nome do seu script, e os ramos `:` e `\?` ficam indistinguíveis.
- ✓ **`shift $(( OPTIND - 1 ))` não é opcional.** Sem ele as opções continuam em `"$@"` e viram operandos, sem aviso.
- ✓ **`getopts` não faz opções longas** — para `--verbose` e `--saida=X`, o laço `while` com `case`, que também trata o `--` e **rejeita** opção desconhecida no ramo `-*`.
- ✓ **Dentro de função, `local OPTIND`**; para analisar um segundo vetor na mesma execução, `OPTIND=1`. É variável global do shell.
- ✓ **`--help` sai na saída padrão com código 0; erro de uso sai em `stderr` com código 2.** Trocar os dois faz a ajuda poluir *pipeline* e o erro passar despercebido na automação.
- ✓ **`$0` é o caminho como foi chamado** — pode ser relativo, pode ser link, e não serve sob `source`. Use `${0##*/}` para mensagens e `${BASH_SOURCE[0]}` para achar o diretório real.

## NO KALI

**A validação de escopo entra logo depois da análise, antes de qualquer pacote sair da máquina.** Verifique cada operando contra a faixa autorizada assim que o `shift` final entrega os alvos, e saia com código próprio se algum não pertence ao escopo — deixar a checagem para dentro do laço de varredura significa que os primeiros alvos **já foram tocados** quando o problema aparece. Varredura fora do escopo contratado é crime pela Lei 12.737/2012, e a análise de argumentos é o último ponto onde impedi-la ainda é barato. **`-o` sem argumento é a falha que mais aparece em *wrapper* de ferramenta:** `coletar -o` no fim da linha deixa `OPTARG` vazio e o arquivo de saída vira o diretório atual — ou pior, o próximo argumento, que seria o alvo. Com o `:` inicial, esse caso cai no ramo `:` e o script para com mensagem clara em vez de sobrescrever algo. **Não repasse `"$@"` cru para `nmap`, `hydra` ou `sqlmap`:** se o script encaminha opções que ele mesmo não entende, o usuário ganha toda a superfície da ferramenta por baixo do seu *wrapper* — inclusive o `--script` do `nmap`, que **executa código NSE**. Aceite uma lista explícita de opções e monte a linha da ferramenta com um array. **Nome de saída de ferramenta ofensiva merece o `--`:** relatório de `gobuster` e `nikto` carrega URL no nome, e URL tem hífen com frequência. E **`getopts` dentro de função exige `local OPTIND`** — sem ele a segunda análise continua de onde a primeira parou, e o resultado é uma opção ignorada, sem erro, com um alvo processado na configuração errada.

# EXPANSÃO DE PARÂMETROS

*O laço sobre 12 mil arquivos levava **34 minutos**. Ninguém suspeitou do `mv` — o custo eram as outras cinco linhas: `dirname`, `basename`, `echo|sed` e `echo|cut` somam **seis processos por arquivo**. Setenta e dois mil forks, dois milissegundos cada. Trocados por expansão de parâmetros: **seis segundos**.*

**062**
GUIAS LINUX

## ONDE CADA OPERADOR CORTA UM CAMINHO

```
A   B          C   D         E
/ | var/log/apt / | history | .log
```

| | | |
|---|---|---|
| `${p#*/}` | `var/log/apt/history.log` | menor prefixo até a **1ª** barra — corte **B** |
| `${p##*/}` | `history.log` | maior prefixo até a **última** barra — corte **D** · é o `basename` |
| `${arq%.*}` | `history` | tira a extensão — corte **E** |
| `${p%/*}` | `/var/log/apt` | menor sufixo desde a última barra — corte **C** · é o `dirname` |
| `${p%%/*}` | `(vazio)` | maior sufixo desde a **1ª** barra — corte **A** |
| `${arq##*.}` | `log` | só a extensão |

**Dobrar o símbolo torna o corte guloso**, e o padrão é **glob**, não expressão regular: `*`, `?` e `[abc]` existem; `+` e grupos de captura, não. **Os casos-limite divergem dos comandos externos:** sem barra, `${p%/*}` devolve o caminho inteiro (o `dirname` diria `.`); terminando em barra, `${p##*/}` devolve vazio (o `basename` diria `log`); sem ponto, `${p##*.}` devolve tudo. E `${var}` só precisa das chaves para **delimitar o nome**: `$nome_final` é outra variável, `${nome}_final` é o valor mais o sufixo.

## VALOR PADRÃO · SUBSTITUIÇÃO · FATIA · CAIXA

| OPERADOR | UNSET VAR não definida | VAR="" definida, vazia | VAR="X" com valor |
|---|---|---|---|
| `${var:-p}` | p | p | x |
| `${var-p}` | p | vazio | x |
| `${var:=p}` | p · atribui | p · atribui | x |
| `${var:+a}` | vazio | vazio | a |
| `${var+a}` | vazio | a | a |
| `${var:?msg}` | aborta | aborta | x |

**A coluna do meio é a única em que o dois-pontos muda algo:** com ele, definida-e-vazia conta como ausente. Na dúvida use a forma com `:` — variável vazia costuma ser tão inútil quanto inexistente. O `:?` é o único que **interrompe**. E o `:+` resolve o "acrescente só se existir" sem `if`: `comando+="${PORTA:+ -p $PORTA}"`.

| | |
|---|---|
| `${v/a/b}` | troca a **primeira** ocorrência · `${v//a/b}` troca **todas** · `${v//a}` remove |
| `${v/#a/b} ${v/%a/b}` | ancora no **início** / no **fim**. `${ip//./-}` → `10-0-0-1` ; a barra precisa de escape: `${c//\//-}` |
| `shopt -s extglob` | acrescenta `+(x)`, `*(x)`, `@(x\|y)`, `!(x)` ao glob — ligue **no topo**, nunca no meio: o Bash analisa a sintaxe antes de executar |
| `${#s} ${s:0:5} ${s: -5}` | comprimento e fatia — **espaço antes do menos**, como nos arrays. `${s:0:-4}` corta do fim |
| `${n,,} ${n^^} ${n^}` | caixa sem chamar `tr` (Bash 4) — o jeito certo de comparar resposta: `case "${resposta,,}" in s\|sim)` |
| `${!nome} ${!PREFIXO@}` | indireção e listagem por prefixo. **Perigosa com entrada externa** — para tabela, `declare -A` |
| `${v@Q}` | o valor **citado**, com as fronteiras visíveis (Bash 4.4): `processando 'meu arquivo.txt'` — o formato certo para log |

**Onde a expansão para e o `awk` começa:** ela opera sobre **uma** variável com padrões de glob. Texto com campos e registros, regex de verdade (grupos, quantificadores) ou transformação que dependa do contexto entre linhas são trabalho de `grep`, `sed` e `awk` — e aí **chame o programa uma vez sobre o conjunto inteiro**, nunca uma vez por item.

`: "${DESTINO:?variavel DESTINO nao definida}"` **no topo do script é a linha mais barata que existe.** Sem ela, um `DESTINO` vazio por erro de digitação transforma `rm -rf -- "${DESTINO}/cache"` em `rm -rf -- /cache`. O `:` inicial é o comando nulo — não faz nada além de **forçar a avaliação** da expansão; a mensagem vai para `stderr` e o shell sai com código 1, encerrando o script não interativo ali mesmo. Ponha uma linha dessas para **cada** variável obrigatória, antes de qualquer comando com efeito colateral, e mantenha o `--` na chamada.

## RESUMO RÁPIDO

- ✓ **Expansão de parâmetros roda dentro do processo do shell.** Trocar `basename`, `dirname`, `sed` e `cut` por `${v##*/}` e companhia elimina um *fork* por chamada — a diferença entre segundos e dezenas de minutos num laço grande.
- ✓ **`#` corta do começo, `%` corta do fim; dobrado, o corte é guloso.** `${p##*/}` é o `basename`, `${p%/*}` é o `dirname` — com casos-limite diferentes.
- ✓ **`:-` usa · `:=` usa e atribui · `:+` usa só se houver valor · `:?` aborta.** O dois-pontos decide se "definida mas vazia" conta como ausente.
- ✓ **Os padrões são glob, não regex.** `shopt -s extglob` acrescenta `+(…)`, `@(…)` e `!(…)`; o resto é do `sed` e do `awk`.
- ✓ **`${v//a/b}` troca todas as ocorrências**, com `/#` e `/%` ancorando início e fim.
- ✓ **`${v,,}` e `${v^^}` normalizam caixa sem chamar `tr`** — o jeito certo de comparar resposta do usuário.
- ✓ **Portabilidade:** `:-`, `:=`, `:+`, `:?`, `#`, `##`, `%`, `%%` e `${#v}` são **POSIX**; fatia e substituição são Bash 2, caixa é Bash 4, `@Q` é Bash 4.4. `${v,,}` num `#!/bin/sh` falha no `dash`.

## NO KALI

**O caso mais comum de varredura fora do escopo não é malícia: é variável vazia.** Um `ALVO` em branco faz `nmap "$REDE/24"` virar `nmap /24` e, dependendo do *wrapper*, faz uma faixa padrão entrar no lugar — `: "${ALVO:?informe o alvo}"` transforma configuração ausente em parada imediata, com mensagem, em vez de comportamento inventado. Varrer fora do escopo autorizado é crime pela Lei 12.737/2012. **Nome de saída de ferramenta se monta com expansão, não com `sed`:** `nmap -oA "${ALVO//[^a-zA-Z0-9._-]/_}-$(date +%F)"` normaliza o alvo para um nome de arquivo seguro numa linha só, sem `/`, `:`, `?` ou espaço — a mesma técnica que sanitiza URL vinda do `gobuster`. **Laço sobre *wordlist* ou sobre saída de varredura é onde o custo do *fork* explode:** cem mil linhas com dois `$(basename …)` cada dão duzentos mil processos, e esse tempo compete com a janela contratada do teste. Use `${v##*/}` e `${v%.*}`; quando a transformação for mesmo complexa, passe o conjunto inteiro por **um** `awk`. **`${var@Q}` no log evita ambiguidade no relatório:** `alvo: 10.10.10.5` não distingue um alvo com espaço no fim; `alvo: '10.10.10.5 '` distingue — e em teste autorizado o registro do que foi tocado é parte da entrega. **Cuidado com `${!var}` sobre dado vindo de arquivo de alvos:** deixar conteúdo de terceiro escolher qual variável do seu script será lida é uma primitiva de leitura arbitrária dentro do processo. Se a intenção é uma tabela, `declare -A`.

# ARITMÉTICA NO SHELL

*O monitor rodava a cada cinco minutos e nunca alertou. O disco chegou a **97%** e o script continuou calado — sem erro de `df`, sem erro de permissão. O `percentual` valia **zero**, e valia zero desde o primeiro dia. A aritmética do shell é só de **inteiros**, e ela não avisa onde a fração se perde: não há erro, não há `NaN` — há um número redondo e errado.*

**063**
GUIAS LINUX

## ONDE A FRAÇÃO SE PERDE

`usado=41943040 · total=52428800 · ocupação real: 80%`

| | | |
|---|---|---|
| `$(( usado / total * 100 ))` | **0** | A divisão acontece **primeiro** e **0,8 vira 0**. Zero vezes 100 continua zero: o alerta nunca dispara, com o disco vazio ou cheio. |
| `$(( usado * 100 / total ))` | **80** | **Multiplique antes de dividir.** Correção de uma posição de caractere — e a que resolve quase todos os casos. |
| `$(( usado * 10000 / total ))` | **8000** | **Ponto fixo:** trabalhe em centésimos e formate na hora de imprimir — `printf '%d.%02d%%' $((c/100)) $((c%100))` → **80.00%**. |
| `$(( (a + b/2) / b ))` | **≈** | **Arredonda** em vez de truncar (soma metade do divisor). E `$(( (a + b - 1) / b ))` é o **teto** — quantos blocos, páginas ou partes de tamanho fixo cabem. |

A divisão inteira **trunca em direção a zero**: `$(( -7 / 2 ))` vale **−3**, não −4, e `$(( -7 % 2 ))` vale **−1** — é a regra do C, não a matemática modular. Toda expressão com `/` merece a pergunta: *o intermediário cabe em inteiro?* Inteiros são de **64 bits com sinal** e estouram **em silêncio**, dando a volta para negativo. Quando a fração é o próprio resultado — média, desvio, raiz —, pague o processo: `printf '%.2f'` (builtin, sem processo) · `awk 'BEGIN{printf "%.1f", s/n}'` (sempre instalado) · `bc -l` (**sem `-l` ou `scale`, o `bc` trunca igual ao shell**, e não vem instalado por padrão). **Nunca uma vez por item dentro do laço:** acumule inteiros e divida uma vez no fim.

## ZERO À ESQUERDA É OCTAL · E O COMANDO QUE DEVOLVE CÓDIGO

| mes=$(date +%m) | $(( mes )) | $(( 10#mes )) |
|---|---|---|
| 07 | 7 | 7 |
| 08 | value too great | 8 |
| 09 | value too great | 9 |
| 010 | 8 | 10 |
| 12 | 12 | 12 |

O prefixo `0` significa **octal**, como em C — e isso atinge tudo que o sistema formata com zero à esquerda: `date +%m`, `+%d`, `+%H`, `+%M`, número de série, código de agência, octeto de IP. O sintoma clássico é um script que roda o ano inteiro e **quebra em agosto e setembro**; o `010` é pior — nunca falha e sempre calcula errado. Defesa: `mes=$(( 10#$(date +%m) ))`, preferível a `${mes#0}`, que transformaria `00` em vazio. As outras bases: `$(( 0x1F ))` =31 e `$(( 2#1011 ))` =11.

**`(( x = 5 + 3 ))`**
**COMANDO.** Avalia, atribui e não imprime nada — o que ele devolve é um **código de saída**.

**`echo $(( 5 + 3 ))`**
**EXPANSÃO.** Avalia e produz o texto `8`. É POSIX; o `(( ))`, o `let`, o `**` e o `$RANDOM` são do Bash.

**A inversão:** `(( 1 ))` → `$?=0` (sucesso) e `(( 0 ))` → `$?=1` (falha). Em C zero é falso; no shell zero é sucesso, e o `(( ))` faz a tradução. **Consequência sob `set -e`:** `(( contador++ ))` com o contador em zero devolve o valor *anterior* ao incremento — zero — e **aborta o script** numa linha que parecia inofensiva. As três defesas: **`(( ++contador ))`** · **`(( contador++ )) || :`** · **`contador=$(( contador + 1 ))`**.

Dentro de qualquer contexto aritmético — `(( ))`, `$(( ))`, índice de array — as variáveis **dispensam o cifrão** e não sofrem divisão em palavras; **variável não definida vale zero**, conveniente para contador e traiçoeiro para configuração ausente. Os operadores são os de C: `+ - * / % **`, `++ --`, `<< >> & | ^ ~`, `== ≠ < >`, `&& || !`, o ternário `? :` e as atribuições compostas. **Evite os herdados:** `let` exige citar a expressão, e `expr` é um **programa externo** — um *fork* por conta, com escape obrigatório no `*`. `declare -i` avalia toda atribuição àquela variável, mas engole texto virando **0 em silêncio**.

`$RANDOM` **não é criptograficamente seguro.** É um gerador pseudoaleatório simples, com semente previsível e período curto — senha, *token*, chave e nome de arquivo temporário que precise ser imprevisível **não saem dele**. Nomear temporário com `$RANDOM` num diretório compartilhado abre **corrida de link simbólico**: quem prevê o nome cria o link antes de você. As fontes corretas: `token=$(head -c 32 /dev/urandom | base64)`, `openssl rand`, e `tmp=$(mktemp)` para temporários — nunca invente o nome. E há um viés estatístico à parte: `RANDOM % 100` favorece os valores baixos, porque 32768 não é múltiplo de 100; para sorteio uniforme, `shuf -i 0-99 -n 1`.

## RESUMO RÁPIDO

- ✓ **A aritmética do shell é só de inteiros e a fração se perde sem aviso.** `$(( a / b * 100 ))` dá zero sempre que `a < b` — **multiplique antes de dividir**.
- ✓ **`(( ))` é comando e devolve código; `$(( ))` é expansão e produz texto.** No `(( ))`, valor zero vira código de **falha**: `(( c++ ))` com `c=0` aborta um script sob `set -e`. Use `(( ++c ))`.
- ✓ **Em contexto aritmético as variáveis dispensam o cifrão** e a não definida vale zero.
- ✓ **Ponto fixo resolve a maioria dos relatórios:** escale por 100 ou 1000, calcule em inteiro e formate com `printf '%d.%02d'`. Guarde `(a + b/2)/b` para arredondar e `(a + b - 1)/b` para o teto.
- ✓ **Zero à esquerda é octal:** `$(( 08 ))` é erro e `$(( 010 ))` é 8. Force a base com `$(( 10#$valor ))`.
- ✓ **Inteiros de 64 bits com sinal estouram em silêncio**, dando a volta para negativo.
- ✓ **Ponto flutuante de verdade:** `awk` **ou** `bc -l` — e nunca uma vez por item dentro do laço.
- ✓ **`$RANDOM` não gera senha nem nome de temporário.** Use `/dev/urandom`, `openssl rand` ou `mktemp`.

## NO KALI

**Cálculo de intervalo entre requisições é aritmética inteira, e o erro tem consequência.** Um script que calcula a pausa como `$(( 1 / taxa ))` segundos dá **zero** para qualquer taxa maior que 1 — o laço passa a bater no alvo sem pausa nenhuma, o que é indistinguível de uma negação de serviço acidental e derruba justamente o serviço que você precisava analisar. Trabalhe em milissegundos (`pausa_ms=$(( 1000 / taxa ))`) ou use o `--max-rate` da própria ferramenta. **Aritmética sobre octetos de IP quebra em zero à esquerda:** alvos escritos como `010.010.010.005` — formato que aparece em planilha de escopo e em exportação de inventário — fazem `$(( octeto ))` ler `010` como **8**, e o resultado é um endereço **fora do escopo autorizado** calculado a partir de um dentro dele. Varredura fora do escopo contratado é crime pela Lei 12.737/2012: aplique `10#` em todo octeto antes de qualquer conta e valide o endereço reconstruído contra a faixa. **Contador sob `set -e` derruba script de varredura no primeiro alvo** — é justamente quando o contador está em zero que o `(( total++ ))` devolve código 1, e o script morre antes de fazer qualquer coisa útil. **Somar tamanhos de captura passa de 32 bits rápido:** um `.pcap` de algumas horas já tem centenas de milhões de bytes, e multiplicado por um fator de escala para média em kbit/s o intermediário cresce depressa — o teto de 64 bits aguenta, mas confira o resultado, porque estouro não emite aviso, só troca o sinal. E **o `bc` não vem instalado no Kali** (`sudo apt install bc`); o `awk` está sempre lá, o que faz dele a escolha segura para ponto flutuante em script que vai rodar em máquina alheia.

# EXPRESSÕES REGULARES

*O servidor estava sob força bruta e o analista rodou `grep 10.0.0.1 /var/log/auth.log`. Vieram as linhas certas — e uma com `id=10a0b0c1`, que não tem endereço nenhum. O ponto que ele digitou querendo dizer **ponto** significa, para a ferramenta, **qualquer caractere**. Na tentativa seguinte, `grep 'Failed+'` devolveu zero linhas — sem erro, sem aviso: o `grep` sem `-E` não conhece o `+`. Um padrão descreve **forma**, e há **três dialetos** dessa notação na mesma máquina.*

**064**
GUIAS LINUX

## ANATOMIA DE UM PADRÃO REAL — ENDEREÇO IPV4

`^` `(` `[0-9]` `{1,3}` `\.` `)` `{3}` `[0-9]` `{1,3}` `$`

**`^ $` âncora**
Condição sobre a **posição**: não consome caractere. Daí `^$` ser "linha vazia".

**`( )` grupo**
Delimita o item para o quantificador **e captura** o trecho como `\1`.

**`[ ]` classe**
**Um** caractere dentre os listados. `[^0-9]` nega; `[.]` desarma o ponto.

**`{n,m}` quantifica**
Repete só o item **imediatamente anterior**. Em `ab*`, o `*` governa só o `b`.

**`\.` literal**
A barra **tira** o poder do símbolo. `\.` é ponto de verdade.

Lido da esquerda para a direita: *da borda esquerda da linha, três repetições de "um a três dígitos seguidos de ponto", depois mais um a três dígitos, e a borda direita.* Note o que ele **não** diz: nada impede `999.999.999.999`. **Expressão regular reconhece forma; validar valor é outro problema** — trabalho de `awk` ou de uma linguagem. Quando o padrão passa de três linhas ou precisa *contar*, troque de ferramenta: HTML, JSON e parênteses balanceados **não são linguagens regulares** e nenhum padrão resolve o caso geral. E não confunda com o **globbing**: o glob é do **shell** e opera sobre nomes de arquivo (`*` = qualquer sequência); a expressão regular é da **ferramenta** e opera sobre linhas (`*` = zero ou mais do item anterior).

## OS TRÊS SABORES QUE CONVIVEM NA MESMA MÁQUINA

| Construto | BRE<br>grep · sed · expr | ERE<br>grep -E · sed -E · awk | PCRE<br>grep -P · perl · python |
|---|---|---|---|
| uma ou mais | `a\+` | `a+` | `a+` |
| opcional | `a\?` | `a?` | `a?` |
| repetição contada | `a\{2,5\}` | `a{2,5}` | `a{2,5}` |
| grupo | `\(ab\)` | `(ab)` | `(ab)` |
| alternância | `a\|b` | `a|b` | `a|b` |
| dígito | `[[:digit:]]` | `[[:digit:]]` | `\d` |
| preguiçoso · olhar adiante | não existe | não existe | `.*?` `(?=…)` |

A regra que resolve quase tudo: em **BRE** o construto avançado exige barra invertida e o símbolo cru é literal; em **ERE** é o contrário — a barra só serve para *tirar* o poder. **Na dúvida,** `grep -E` : é o mesmo padrão que você escreveria em `awk`, `sed -E` ou em qualquer linguagem moderna. O `egrep` e o `fgrep` ainda rodam, mas imprimem aviso de depreciação no Kali atual. E há uma quarta variante que não é expressão regular nenhuma: `grep -F` desliga a interpretação inteira, busca texto literal e é a mais rápida.

**`grep ^root /etc/passwd`**
**Padrão vai entre aspas simples, sempre.** Sem elas o shell processa `*`, `?`, `[…]`, `$` e `\` **antes** da ferramenta ver o padrão — e o que chega ao `grep` já não é o que você escreveu.

**`grep -E '^(erro|falha)'`**
**A alternância tem a menor precedência de todas.** `^erro|falha` significa "começa com erro" **ou** "contém falha" em qualquer posição — quase nunca o que se quer. Agrupe.

**O `.*` é guloso e atravessa delimitador.** Numa linha com três campos entre aspas, `grep -oE '".*"'` devolve **uma cadeia gigante**, da primeira à última aspa. Negue em vez de generalizar: **`"[^"]*"`** — em PCRE existe também o preguiçoso **`".*?"`**. **Faixa depende da localidade:** `[a-z]` muda de resultado conforme `LC_ALL`; as **classes POSIX nomeadas** são estáveis — `[[:digit:]]`, `[[:alpha:]]`, `[[:alnum:]]`, `[[:space:]]`, `[[:upper:]]`, `[[:punct:]]`, `[[:xdigit:]]`. Dentro dos colchetes as regras mudam: o `^` só nega na **primeira** posição, o `]` só é literal **logo no início** e o `-` só é literal no **começo ou no fim**. Bordas de palavra `\<pai\>` e `\bpai\b` impedem que `10.0.0.1` case dentro de `10.0.0.14`.

**Retrocesso catastrófico (ReDoS) — não rode em produção.** O motor POSIX (`grep -E`, `sed`, `awk`) usa autômato finito: casa em tempo proporcional ao tamanho do texto e **não retrocede**. O motor PCRE (`grep -P`, linguagens) retrocede — e um quantificador aninhado explode: `echo "$(printf 'a%.0s' {1..40})" | grep -qP '(a+)+b'` prende um núcleo por minutos, porque o motor tenta um número **exponencial** de divisões antes de desistir. É por isso que `(a|ab)` casa `ab` em POSIX (mais longo) e `a` em PCRE (primeira alternativa que fecha). Defesas: prefira `-E` a `-P` quando os dois servirem; em PCRE, nada de quantificador dentro de grupo quantificado, e classe negada no lugar de `.*`.

## RESUMO RÁPIDO

- ✓ **Padrão descreve forma, não texto literal.** O `.` é qualquer caractere: `grep 10.0.0.1` casa com `10a0b0c1`. Ponto de verdade se escreve `\.`.
- ✓ **Padrão vai entre aspas simples, sempre** — senão o shell consome `*`, `?`, `$`, `[` e `\` antes da ferramenta.
- ✓ **São três dialetos.** BRE exige `\+ \? \{ \( \|`; ERE usa os símbolos direto; PCRE acrescenta `\d`, `\b`, `.*?` e olhar adiante. **Na dúvida, ERE.**
- ✓ **Classe POSIX nomeada é mais estável que faixa:** `[[:digit:]]` não muda com `LC_ALL`; `[0-9]` e `[a-z]` mudam.
- ✓ **Quantificador governa só o item anterior** — para repetir mais de um caractere, agrupe. E a alternância tem a **menor** precedência: `^a|b` quase nunca é o que se quer.
- ✓ **`.*` é guloso e atravessa delimitadores.** Prefira a classe negada `[^"]*`.
- ✓ **POSIX não retrocede; PCRE sim.** Daí `(a|ab)` casar diferente e o ReDoS existir só no segundo.
- ✓ **Nem tudo é linguagem regular.** HTML, JSON e aninhamento não se resolvem com padrão: troque para `awk`, `jq` ou Python.

## NO KALI

**`grep -F -- "$ALVO"` para tudo que veio de fora.** Filtrar a saída de uma varredura por um domínio guardado em variável trata o **conteúdo** da variável como padrão: um ponto vira coringa e um `[` mal fechado derruba o comando com *Unmatched [*. Se o alvo veio de arquivo de escopo, de argumento ou de resposta de API, use `-F`; o `--` fecha a lista de opções e evita que um alvo começando com traço seja lido como *flag*. **Wordlist se filtra com ERE, e o ganho é de horas:** `grep -E '^.{8,}$' rockyou.txt | grep -E '[[:upper:]]' | grep -E '[[:digit:]]'` reduz a lista à política de senha do alvo antes de entregá-la à ferramenta — três `grep` em cadeia são mais legíveis, e em geral mais rápidos, que um padrão único com olhar adiante. **Extrair indicador de log é o uso diário**, e quem faz isso é o `-o`: `grep -oE '\b([0-9]{1,3}\.){3}[0-9]{1,3}\b'` puxa IPv4, `grep -oE '[0-9a-f]{32}'` puxa *hashes* MD5 e `grep -oPi 'href="\K[^"]+'` puxa URLs de um HTML salvo — o `\K` do PCRE descarta o que veio antes e dispensa o grupo de captura. **ReDoS é classe de vulnerabilidade, não incômodo:** uma aplicação que valida entrada do usuário com quantificador aninhado cai com uma requisição de poucos quilobytes; em teste autorizado, `grep -rnE '\(\[?[^)]*[+*]\)?[+*]' --include='*.js' --include='*.py'` levanta candidatos. Isso é **análise de código com autorização do dono** — disparar a carga contra serviço de terceiro sem contrato é indisponibilidade provocada, enquadrada na **Lei 12.737/2012**. **Localidade muda a varredura:** `LC_ALL=C` deixa a semântica previsível e acelera bastante o `grep`, mas quebra o casamento de acentos — não use em texto em português. Fora do mundo GNU (macOS, BSD) não há `-P`, nem `\<`, nem `\K`: script portátil usa ERE puro.

GUIA VISUAL 065

# GREP E A BUSCA DE PADRÕES

*Uma chave de API vazou no repositório e a pergunta era simples: em quantos lugares ela aparece? O `grep CHAVE *` devolveu `É um diretório` — o `*` é do shell e não desce um nível. O `grep -r CHAVE .` acertou o alvo e errou o alcance: dois minutos varrendo `.git`, `node_modules` e um venv, e uma parede com **Arquivo binário … coincide** e uma linha de mil e duzentos caracteres de bundle minificado. A informação estava lá, **afogada**.*

**065**
GUIAS LINUX

## AS OPÇÕES EM TRÊS ESTÁGIOS — DIAGNOSTIQUE O RESULTADO RUIM PELO ESTÁGIO

### 1 · COMO INTERPRETAR

- `-E` ERE — o dialeto do dia a dia
- `-F` literal: **nada** é especial, e é o mais rápido
- `-P` PCRE ( `\d` , `\K` , `.*?` )
- `-i` ignora caixa
- `-e` mais um padrão · e resolve padrão que começa com traço
- `-f` padrões vindos de arquivo

### 2 · QUAIS LINHAS PASSAM

- `-v` inverte: imprime as que **não** casam
- `-w` palavra inteira — `root` não casa `chroot`
- `-x` a linha **inteira** é exatamente isso
- `-m N` para depois de N linhas
- `-A -B -C` N linhas depois · antes · dos dois lados

### 3 · O QUE IMPRIMIR

- `-o` só o trecho casado — vira **extrator**
- `-c` só a contagem de **linhas**
- `-l` / `-L` só o nome do arquivo que tem / não tem
- `-n` número da linha — o que dá citação de relatório
- `-h` / `-H` nunca / sempre o nome do arquivo

**FORA DOS TRÊS ESTÁGIOS: ONDE PROCURAR**

`-r` percorre diretórios ( `-R` ainda segue link simbólico, com risco de laço — prefira `-r` ) · `--include='*.conf'` e `--exclude-dir={.git,node_modules,venv}` filtram **arquivos, não linhas**, e aceitam **glob**, nunca expressão regular · `-I` pula binário. Em árvore de código, **esta linha faz mais pela qualidade do resultado que qualquer refinamento de padrão**: `grep -rnI 'AKIA3XPLR7QF2' . --exclude-dir={.git,node_modules} --include='*.py' --include='*.env'` .

Quase todo mau resultado vem de **mexer no estágio errado**. "Veio ruído demais" costuma ser o estágio 2 (falta `-w` ) ou o bloco vermelho (falta `--exclude-dir` ); "veio linha longa demais" é o estágio 3 (falta `-o` ); **"não veio nada" é o estágio 1** — padrão escrito em ERE entregue a um `grep` em BRE. E o modelo por trás de tudo: **o `grep` decide linha por linha**. Padrão não atravessa quebra de linha, a saída sai na ordem da entrada e **não vem deduplicada** — ordenar e contar é trabalho do `sort` e do `uniq` .

## NA PRÁTICA

kali@kali: ~/lab-grep

```
# -c conta LINHAS; para contar OCORRÊNCIAS, é -o | wc -l
$ grep -c 'ip=' acesso.log
3
$ grep -o 'ip=[0-9.]*' acesso.log | wc -l
4
# o que está DE FATO em vigor na configuração
$ grep -vE '^\s*(#|$)' /etc/ssh/sshd_config
# -o + sort + uniq -c: o pipeline canônico de contagem
$ grep -oE 'ip=[0-9.]+' acesso.log | cut -d= -f2 \
    | sort | uniq -c | sort -rn
      2 10.0.0.1
      1 10.0.0.2
# o grep que casa consigo mesmo — e a classe que resolve
$ ps aux | grep '[n]ginx'
```

**Pegadinhas que custam tempo.** `grep -c` com **vários arquivos** imprime `arquivo:contagem` por arquivo, nunca um total. Com **um** arquivo o `grep` não prefixa o nome, com **dois ou mais** prefixa — o que quebra script cujo `--include` às vezes casa um, às vezes vários: force com **`-h`** ou **`-H`**. Blocos de `-A` / `-B` vêm separados por uma linha `--` , ótima para ler e péssima para processar. **Cor vaza para o *pipeline*:** um `--color=always` herdado de *alias* injeta escapes ANSI que quebram o `sort` e o `awk` seguintes — o padrão sensato é **`--color=auto`**. E `grep padrao` sem arquivo **não travou**: está lendo o `stdin` — `Ctrl+D` encerra.

**0** **Achou** pelo menos uma linha.

**1** **Não achou nada.** Não é erro — é resposta.

**2** **Erro real:** arquivo inexistente, padrão inválido.

**O status de saída é informação, não formalidade** — é ele que permite usar o `grep` como teste: `if grep -q '^PermitRootLogin yes' /etc/ssh/sshd_config; then …` . O `-q` é silencioso **e encerra na primeira linha que casa**: num arquivo de gigabytes, muda o tempo de minutos para milissegundos, e é sempre preferível a `grep … > /dev/null` .

**Desempenho, em ordem de impacto:** `LC_ALL=C` desliga a decodificação multibyte e costuma render de duas a cinco vezes em texto ASCII (não use com acento) · use o **dialeto mais fraco que resolva** — `-F` > `-E` > `-P` · **filtre barato antes de caro**: um `-F` que descarta 90% das linhas antes de um `-P` complexo vale mais que o inverso · `-q` e `-m 1` param cedo quando a pergunta é só "existe?". E `-F -f lista.txt` casa **milhares** de padrões literais numa **única** varredura: um laço com mil `grep` faz mil varreduras do mesmo log.

**Duas combinações que derrubam script ou terminal. 1)** `grep -q` sob `set -e` : "não encontrou" é status **1**, e o `set -e` lê isso como falha e **encerra o script** numa linha que só queria perguntar. Use dentro de `if` , ou acrescente `|| true` . **2)** `grep -a` trata binário como texto e pode **despejar bytes de controle** no terminal, deixando-o ilegível — se acontecer, `reset` conserta; a defesa é `-I` , que pula binário por completo. Para casar padrão que atravessa linhas, `-z` usa o byte nulo como separador de registro: `grep -zoP 'BEGIN CERTIFICATE.*?END CERTIFICATE' arquivo.pem | tr -d '\0'` .

## RESUMO RÁPIDO

- ✓ **O `grep` decide por linha e nada mais.** Padrão não atravessa quebra de linha (salvo `-z`); a saída sai na ordem de entrada e não vem deduplicada.
- ✓ **Status: 0 achou · 1 não achou · 2 erro.** Use `grep -q` em teste — mas o status 1 encerra script sob `set -e`.
- ✓ **As opções se dividem em estágios:** interpretar (`-E -F -P -i`), quais linhas passam (`-v -w -x -m -A -B -C`), o que imprimir (`-o -c -l -n -h`) e onde procurar (`-r --include --exclude-dir -I`).
- ✓ **`-c` conta linhas, não ocorrências.** Para ocorrências, `grep -o | wc -l`.
- ✓ **`-o` transforma o `grep` em extrator** — é a peça que o encaixa no pipeline com `sort`, `uniq -c` e `awk`.
- ✓ **`-w` e `-x` matam os dois falsos positivos mais comuns** sem escrever padrão nenhum.
- ✓ **Em árvore de código, `--exclude-dir={.git,node_modules}` e `-I` valem mais que refinar o padrão.**
- ✓ **Desempenho:** `LC_ALL=C`, dialeto mais fraco possível, barato antes de caro, `-q` / `-m 1` para "existe?".

## NO KALI

**Varrer um alvo baixado é o uso mais frequente.** Depois de espelhar um site com `wget -r`, um único `grep -rniI --include='*.js' -E 'api[_-]?key|secret|token|password' ./espelho/` levanta credencial exposta em segundos: o `-I` evita afogar a saída em imagem, o `-i` cobre variação de caixa e o `--include` restringe ao que interessa. Vale para material de **teste autorizado ou de escopo público** — vasculhar sistema de terceiro sem contrato é acesso não autorizado pela **Lei 12.737/2012**. **`grep -F -f iocs.txt acesso.log` é o casamento em massa:** com uma lista de IPs ou domínios maliciosos num arquivo, os dois conjuntos se comparam numa varredura só; mil chamadas num laço fazem mil varreduras — em log de gigabytes, a diferença é entre segundos e uma tarde. **Refinar wordlist antes da ferramenta economiza horas:** `grep -E '^.{8,16}$' rockyou.txt | grep -vE '^[0-9]+$' > filtrada.txt` reduz milhões de candidatos ao que a política de senha do alvo permite — menos requisições, menos ruído no log do alvo e menos risco de derrubar um serviço que o contrato manda preservar. **Auditar serviço são dois comandos:** `grep -vE '^\s*(#|$)' /etc/ssh/sshd_config` mostra o que está em vigor, e `grep -niE '^\s*(PermitRootLogin|PasswordAuthentication|X11Forwarding)'` vai direto às diretivas — o `-n` é o que transforma o achado em **referência citável no relatório**. **No shell de um alvo comprometido, `grep` é o que existe:** não conte com `rg`, `jq` ou Python num contêiner mínimo, e saiba que o `grep` do BusyBox **não tem `-P` nem `--exclude-dir`** — o plano B é `find … -print0 | xargs -0 grep`. No Kali há o `rg` (multiencadeado, respeita `.gitignore`, pula binário sem opção nenhuma); é ótimo em casa e ausente no alvo.

# SED
# O EDITOR DE FLUXO

*O banco mudou de `10.0.0.5` para `10.0.0.59` e trinta arquivos precisavam refletir isso. Um comando resolveria: `sed -i 's/10.0.0.5/10.0.0.59/' /etc/app/*.conf`. Rodou **em silêncio**. No dia seguinte três aplicações apontavam para lugar nenhum. O `sed` é a ferramenta certa para esse trabalho — foi feito para ele —, mas é **afiada e executa sem perguntar**.*

**066**
GUIAS LINUX

## O CICLO — UMA LINHA POR VEZ, NUM ESPAÇO DE TRABALHO

**LÊ A PRÓXIMA LINHA** → **ESPAÇO DE PADRÃO** → **EXECUTA O SCRIPT INTEIRO** → **IMPRIME**

**LÊ A PRÓXIMA LINHA** — Do arquivo ou do `stdin`. Nunca o arquivo inteiro na memória.

**ESPAÇO DE PADRÃO** — A linha, **sem o** `\n` — por isso `$` casa o fim do texto e `s/$/;/` acrescenta no final.

**EXECUTA O SCRIPT INTEIRO** — `s///` `d` `p` `a` `y` … **Na ordem escrita:** `'s/a/b/; s/b/c/'` leva `a` a `c`, não a `b`.

**IMPRIME** — Automático. É isso que faz `sed ''` ser um `cat` — e `-n` desliga.

↺ **próximo ciclo** — e o espaço de padrão é esquecido: só o **espaço de reserva** atravessa as linhas

### ESPAÇO DE RESERVA — VER ALÉM DA LINHA ATUAL

| | | | |
|---|---|---|---|
| `h` | padrão → reserva (sobrescreve) | `H` | padrão → reserva (acrescenta) |
| `g` | reserva → padrão (sobrescreve) | `G` | reserva → padrão (acrescenta) |
| `x` | troca os dois de lugar | | minúscula sobrescreve, **maiúscula acrescenta** |

### O QUE INTERROMPE O CICLO

| | | | |
|---|---|---|---|
| `d` | apaga e recomeça já | `q` | encerra o `sed` |
| `n` | imprime e lê a próxima | `N` | **acrescenta** a próxima ao buffer |
| `10q` | o `head -10` que **para de ler** | `$=` | imprime o número da última linha |

**Esquecer o `-n` num script com `p` é o motivo número um de saída duplicada** — uma cópia sai pela impressão automática, outra pelo `p`. Daí o idioma `sed -n '5p'`: nada sai por padrão, e a linha 5 sai por ordem explícita. Como o espaço de padrão guarda **uma linha só**, nenhum `s///` comum enxerga duas linhas ao mesmo tempo — para isso existem o `N` e o espaço de reserva, que dão idiomas como `sed -n '1!G;h;$p'` (inverte o arquivo, igual ao `tac`), `sed 'G'` (dobra o espaçamento) e `sed ':a;N;$!ba;s/\n/, /g'` (junta tudo numa linha). **Vale saber ler; não vale escrever:** quando o script pede rótulo (`:a`) e desvio (`ba`), ou um `s///` com mais de três capturas, a resposta é `awk`.

## OS TRÊS DEFEITOS DAQUELE COMANDO — E O HÁBITO QUE OS EVITA

| O defeito | O estrago silencioso | A correção |
|---|---|---|
| Pontos não escapados | casou `10a0b0c5`, um identificador de instância | `10\.0\.0\.5` |
| Sem âncora de fim | casou o **prefixo** de quem já era `10.0.0.59` → `10.0.0.599` | `…\.5$` |
| Sem endereço | agiu em **toda** linha do arquivo, não só nas de endereço | `/^ip=/ s/…/…/` |
| Faltou o `g`? | só a 1ª ocorrência da linha muda — mas **com âncora há no máximo uma** | `(de propósito)` |

kali@kali: ~/lab-sed

```
# 1. ensaie SEM o -i e leia a saída
$ sed -E '/^ip=/ s/10\.0\.0\.5$/10.0.0.59/' db.conf
# 2. confira o que MUDARIA, linha a linha
$ diff db.conf <(sed -E '/^ip=/ s/…/…/' db.conf)
2c2
< ip=10.0.0.5
---
> ip=10.0.0.59
# 3. só então, com sufixo de backup
$ sed -i.bak -E '/^ip=/ s/10\.0\.0\.5$/10.0.0.59/' *.conf
```

### ENDEREÇO: ONDE O COMANDO AGE

| | |
|---|---|
| `sed '5d'` | a linha 5 · `'$d'` a **última** · `'1,10d'` faixa |
| `sed '/^#/d'` | toda linha que casa o padrão |
| `/inicio/,/fim/` | faixa por padrão — se o 2º nunca casar, **vai até o fim**; e ela **reabre** se o 1º voltar a aparecer |
| `/^\[web\]/,+5` | 5 linhas após o cabeçalho da seção |
| `/^#/!s/=/ = /` | **negação**: "em tudo, menos nos comentários" — subutilizada e resolve muita coisa |
| `{ …; … }` | chaves agrupam comandos que **compartilham** o endereço |

### OS OUTROS COMANDOS

| | |
|---|---|
| `p d q` | imprime · apaga · encerra |
| `a i c` | acrescenta depois · insere antes · **troca a linha inteira** |
| `y/abc/ABC/` | transliteração caractere a caractere — **não aceita padrão** (é o `tr` do `sed`) |
| `r arq` · `w arq` | insere arquivo depois da linha · grava as casadas |

**Na substituição:** `&` é o trecho casado **inteiro** (`&` literal precisa de `\&`), `\1` `\2` são os grupos, `\n` `\t` valem na troca (GNU) e `\U` `\L` … `\E` mudam a caixa. **Flags:** `g` todas · `3` só a terceira · `g3` da terceira em diante · `i` ignora caixa · `p` (par obrigatório do `-n`) · `w saida.txt`. **O delimitador é qualquer caractere logo após o `s`**: com caminhos, `s|/var/log|/srv/log|` é legível e `s/\/var\/log/\/srv\/log/` não é. Use `-E` (ERE, o mesmo dialeto do `grep -E` e do `awk`); o `-r` é sinônimo só na GNU.

**O `-i` grava por cima sem perguntar — e não é o que parece.** O `sed` cria um temporário e o renomeia por cima: **o inode muda**, links físicos se quebram e um link simbólico vira arquivo comum (salvo com `--follow-symlinks`). Nunca digite `-i` sem sufixo: `-i.bak` deixa um `arquivo.bak` ao lado e é a diferença entre dois minutos de contratempo e uma restauração de *backup*. Ensaie sem `-i` → confira com `diff` → aplique. **O `sed` é a ferramenta errada — não parcialmente errada — para estrutura aninhada:** HTML/XML (`xmlstarlet`), JSON (`jq`), CSV com vírgula dentro de aspas, e qualquer coisa que exija **contar, somar ou lembrar estado** (é `awk`).

## RESUMO RÁPIDO

- ✓ **Uma linha por vez** no espaço de padrão: roda o script inteiro, imprime e esquece. `sed ''` é `cat`; `-n` desliga a impressão e faz o `p` valer a pena.
- ✓ **`s/padrão/troca/flags` é o comando central.** O delimitador pode ser qualquer caractere — use `|` quando o padrão tiver barras.
- ✓ **Sem `g`, só a primeira ocorrência da linha muda** — e com âncora, há no máximo uma.
- ✓ **Endereço restringe onde o comando age** e é a diferença entre edição cirúrgica e estrago geral: `/^ip=/`, faixas `/a/,/b/` e a negação `!`.
- ✓ **`-E` para ERE** — mesmo dialeto do `grep -E` e do `awk`, muito mais legível que BRE com barras.
- ✓ **`-i` grava por cima e muda o inode.** Ensaie sem, confira com `diff`, aplique com `-i.bak`.
- ✓ **O espaço de reserva (`h H g G x`) é o que vê além da linha atual.** Quando o script pede rótulo e desvio, troque para `awk`.
- ✓ **`sed` não entende estrutura aninhada:** JSON, XML e CSV com aspas pedem outra ferramenta.

## NO KALI

**Editar configuração em massa pede endereço, não só padrão.** Trocar `yes` por `no` sem endereço destrói o arquivo inteiro; a forma segura ancora na diretiva: `sed -i.bak -E 's/^#?\s*(PasswordAuthentication)\s+yes/\1 no/' /etc/ssh/sshd_config`, seguida de `sshd -t` para **validar a sintaxe antes de recarregar** — um `sshd` que não sobe por erro de configuração deixa você **de fora da máquina remota**. **Sanitizar relatório antes de entregar** é uso diário: `sed -E 's/(password=)[^[:space:]]+/\1REDIGIDO/g; s/([0-9]{1,3}\.){3}[0-9]{1,3}/IP-REDIGIDO/g' bruto.md > entrega.md` limpa senha em claro, *token* e endereço em massa — sempre com ensaio antes do `-i` e sempre conferindo o resultado com `grep -c` pelo que **deveria ter sumido**: redação que falha em silêncio é vazamento. **Preparar wordlist é trabalho de `sed`:** `sed 's/$/2026/'` acrescenta o ano, `sed 's/a/@/g; s/o/0/g'` faz *leetspeak* e `sed -E 's/^(.)/\U\1/'` capitaliza — derivados dos nomes da empresa alvo em segundos. É **preparação de material para teste autorizado**; a mesma lista contra sistema de terceiro sem contrato é tentativa de acesso não autorizado pela **Lei 12.737/2012**. **Normalizar a saída de uma ferramenta para servir de entrada de outra** é boa parte do encadeamento no Kali: `nmap -oG` produz linhas que não se encaixam no `cut`, e `sed -E 's/^Host: ([0-9.]+) .*Ports: /\1 /'` deixa endereço e portas prontos para o `awk`. E `sed -n '/inicio/,/fim/p'` extrai a seção de um alvo específico num relatório de `nikto` ou `enum4linux` com centenas de linhas. Fora do mundo GNU (macOS, BSD) o `-i` **exige** sufixo, o `\n` na troca não funciona e não há `\U` nem `0~N`: script portátil usa `-i.bak`, que funciona nos dois.

# AWK
# CAMPOS E REGISTROS

*Quanto tráfego cada endereço consumiu no dia? O `cut -d' ' -f1,10` saiu **quase** certo — e "quase" é o problema: ele corta em **cada** espaço, então um espaço duplo desloca tudo, e a requisição entre aspas virou três campos. Pior: o `cut` extrai, mas **não soma**. O `awk` divide sozinho, tem aritmética, tem tabela associativa e tem um lugar para rodar depois da última linha. É a ferramenta entre **"o `grep` não dá conta"** e **"vou escrever em Python"**.*

**067**
GUIAS LINUX

## REGISTRO, CAMPO E O LAÇO IMPLÍCITO

| 10.0.0.14 | - | - | [14/Jul/2026:09:22:01 | -0300] | "GET | /index.html | HTTP/1.1" | 200 | 5120 |
|---|---|---|---|---|---|---|---|---|---|
| $1 | $2 | $3 | $4 | $5 | $6 | $7 | $8 | $9 | $10 = $NF |

`$0` — o registro inteiro

**`NF` · `$NF`**
Quantos campos tem este registro, e o **último** campo — evita contar coluna na mão e sobrevive a um log com uma coluna a mais.

**`NR` · `FNR`**
Número do registro desde o início; o `FNR` **reinicia a cada arquivo** (e `FILENAME` diz qual é).

**`FS` · `OFS`**
Como dividir na entrada e como **juntar** na saída. `RS` / `ORS` fazem o mesmo para registros.

**padrão `{ ação }`**
Padrão sem ação → `print $0`. Ação sem padrão → **roda em toda linha**. Daí `awk '/ERRO/'` ser um `grep` e `awk '{print}'` ser um `cat`.

**Repare nos campos em vermelho.** A data entre colchetes tem um espaço antes do fuso, então ocupa **dois** campos — e a requisição entre aspas ocupa **três**. Nem o `awk` entende aspas ou colchetes por conta própria: ele divide por espaço em branco e mais nada. **A diferença para o `cut`** é que o `awk` trata *qualquer sequência* de espaços como um separador só, enquanto o `cut` conta cada espaço. Para campos delimitados, mude o `FS` — ele aceita **expressão regular** quando tem mais de um caractere: `-F'[,;]'`, `-F'\t'`, `-F'" "'`. Cuidado com um caso especial: `FS=" "` (o padrão) é "sequências de espaço, ignorando as das bordas"; `FS="[ ]"` é "exatamente um espaço". E o `OFS` **só entra em vigor quando `$0` é reconstruído** — daí o idioma obscuro `{ $1=$1; print }`, uma atribuição que não muda nada e força a reconstrução.

## AGREGAÇÃO COM ARRAY ASSOCIATIVO — A RAZÃO DE O AWK EXISTIR

| | | |
|---|---|---|
| `10.0.0.14` … 200 5120 | 10.0.0.14 | 5120 |
| `10.0.0.14` … 404 512 | 10.0.0.14 | 5632 |
| `192.168.1.9` … 200 8192 → | 192.168.1.9 | 8192 |
| `10.0.0.14` … 200 1024 | 10.0.0.14 | 6656 |
| `192.168.1.9` … 500 64 | 192.168.1.9 | 8256 |

```
{ soma[$1] += $NF }
```

```
END { for (ip in soma) print soma[ip], ip }
```
o mesmo par chave/valor sendo atualizado — não linhas novas

Três coisas são automáticas e é por isso que a agregação cabe em duas regras: **o array nasce sozinho**, **chave inexistente vale zero** na primeira soma e o `END` alcança tudo o que sobreviveu ao laço. O custo é **memória**: uma entrada por chave distinta — agregar por endereço num log de milhões de linhas é barato (poucos milhares de chaves); por URL completa com parâmetros pode consumir gigabytes. E **a ordem de `for (k in a)` é arbitrária**: ordene depois, com `sort -rn`.

`awk '!visto[$0]++'` **deduplica preservando a ordem** — o que o `sort -u` não faz. É padrão sem ação (logo, imprime): `visto[$0]++` devolve o valor **antes** de incrementar — zero na primeira aparição (falso, negado vira verdadeiro, imprime) e ≥1 nas seguintes. **E um detalhe que morde:** `if (tab[k] == "")` **cria** a chave como efeito colateral do acesso; para testar sem criar, `if (k in tab)`. `length(arr)` dá o número de chaves e `delete tab[k]` remove uma.

### PADRÕES: QUAIS REGISTROS INTERESSAM

| | |
|---|---|
| `$3 ~ /^ERRO/` | padrão sobre **um campo** (`!~` nega) — não sobre a linha toda |
| `$9 ≥ 400` | comparação **numérica**: não confunde status com trecho de URL |
| `NR > 1` | pula o cabeçalho · `NF == 0` pega linha vazia |
| `BEGIN { }` | antes da 1ª linha: separadores e cabeçalho. Só com `BEGIN`, nem lê a entrada — `awk 'BEGIN{print 3*7}'` é calculadora |
| `END { }` | depois da última: é onde sai o total |

### AÇÕES E FUNÇÕES

| | |
|---|---|
| `total += $9` | variável não se declara e **nova vale zero** |
| `print $1, $2` | a vírgula insere o `OFS`; **sem vírgula, gruda** (justaposição é concatenação) |
| `printf "%-15s\n"` | o mesmo do shell, **sem quebra de linha automática** |
| `substr · split` | índices começam em **1** — a pegadinha de quem alterna com array do Bash |
| `sub · gsub` | primeira · todas as ocorrências · `match` preenche `RSTART` / `RLENGTH` |
| `next · exit` | pula o registro · encerra a leitura (**mas ainda roda o `END`**) |

### FNR==NR: JUNÇÃO DE DOIS ARQUIVOS

`awk 'FNR==NR { ioc[$1]=1; next } $1 in ioc { print "SUSPEITO:", $0 }' iocs.txt acesso.log` — o `FNR==NR` é verdadeiro **apenas durante o primeiro arquivo**, o que permite carregar uma tabela e depois consultá-la. É junção sem banco de dados, e resolve uma classe inteira de problemas que viraria laço aninhado com `grep`. **Qual `awk` está instalado importa:** no Kali/Debian costuma ser o `mawk` (rápido, enxuto); no RHEL é o `gawk`, que tem `gensub()`, `IGNORECASE`, `asort()` e `RS` como expressão regular. Confira com `awk --version`; se o script depende de extensão, o *shebang* é `#!/usr/bin/gawk -f`.

**Nunca monte o programa `awk` por interpolação do shell — e cuidado com `system()`.** Escrever `awk "\$1 == \"$IP\""` funciona até o dia em que a variável tiver uma aspa: aí o valor **quebra a sintaxe**, e um valor hostil **executa o que quiser**. A forma segura existe para isso: `awk -v alvo="$IP" -v limite=400 '$1 == alvo && $9 ≥ limite'`. Pelo mesmo motivo, `system("…")` e `print | "comando"` montados a partir de dado vindo do alvo — nome de host, *banner* de serviço, cabeçalho HTTP — são **injeção de comando** com o privilégio de quem rodou o `awk`. Se precisa disparar comando por linha, **imprima a lista e passe para o `xargs`**, que não interpreta o conteúdo como shell.

## RESUMO RÁPIDO

- ✓ **Lê registro a registro e divide em campos sozinho**, tratando sequências de espaço como um separador só — o que o `cut` não faz.
- ✓ **A estrutura é `padrão { ação }`.** Padrão sem ação imprime a linha; ação sem padrão roda em todas. `BEGIN` antes da primeira, `END` depois da última.
- ✓ **`$0` é o registro inteiro, `$1..$NF` os campos, `NF` a quantidade, `NR` o número do registro** e `FNR` o número dentro do arquivo atual.
- ✓ **`-F` define o separador e aceita expressão regular.** O `OFS` só vale quando `$0` é reconstruído — daí `$1=$1`.
- ✓ **Variável não se declara e vale zero quando nova**, o que faz de `soma[$1] += $NF` uma agregação completa.
- ✓ **Todo array é associativo.** `!visto[$0]++` deduplica na ordem original; `k in tab` testa sem criar a chave; a ordem de `for` é arbitrária.
- ✓ **`FNR==NR` é o idioma de junção de dois arquivos.**
- ✓ **Valor de fora entra com `-v`**, nunca por interpolação. E `substr` / `split` contam a partir de **1**.
- ✓ **Quando precisar de array de arrays, pare:** a resposta é Python.

## NO KALI

**Somar e agrupar saída de varredura é o uso diário.** Depois de `nmap -oG - alvo/24 > varredura.gnmap`, um `awk '{ portas[$NF]++ } END { for (p in portas) print portas[p], p }' | sort -rn` diz qual porta é a mais comum na rede — **contar antes de olhar economiza horas** de leitura de relatório. **`FNR==NR` cruza escopo com resultado**, e essa é a aplicação que evita problema jurídico: com um arquivo de hosts autorizados pelo contrato e outro com o que a varredura encontrou, `awk 'FNR==NR{ok[$1]=1;next} !($1 in ok) { print "FORA DO ESCOPO:", $1 }' escopo.txt achados.txt` sinaliza cada host que não deveria ter sido tocado. **Rodar isso antes da varredura, sobre a lista de alvos, é melhor ainda** — varrer fora do escopo contratado é acesso não autorizado, com as consequências da **Lei 12.737/2012**. **Detecção de anomalia é agregação simples:** `awk '$6 ~ /Failed/ { falhas[$NF]++ } END { for (ip in falhas) if (falhas[ip] > 20) print falhas[ip], ip }' /var/log/auth.log` lista os endereços com mais de vinte falhas de autenticação — a mesma lógica de um `fail2ban` rudimentar, útil tanto para defender o próprio host quanto para **documentar o ruído que o teste gerou no alvo**. **E o `awk` é o desempacotador de formato esquisito:** saída de `enum4linux`, de `smbclient` e de ferramenta antiga raramente é CSV ou JSON — um `-F` bem escolhido mais `split()` e `substr()` transformam aquilo em colunas que o `sort` e o `uniq` entendem. Num contêiner mínimo ou em firmware embarcado, o que existe é o `busybox awk`, ainda mais limitado: atenha-se ao POSIX (`sub`, `gsub`, `split`, `substr`, `printf`) e nada de `RS` multicaractere.

GUIA VISUAL 068

# CUT · SORT · UNIQ · TR
# A CAIXA DE TEXTO

*O relatório dizia que o diretório mais pesado era o `/var/cache`, com 900 MB — mas o `/var/lib` tinha 1,5 GB e nem apareceu. O `du -sh | sort -rn` comparou os **números** 1 e 900: o `-n` só entende dígitos e **ignora a unidade**. No mesmo relatório, a contagem de endereços únicos veio baixa demais: o `uniq` compara **apenas linhas adjacentes**. Dois erros silenciosos, a mesma raiz — **supor que a ferramenta faz o que o nome sugere**.*

**068**
GUIAS LINUX

## O PIPELINE CANÔNICO DE CONTAGEM — ESTÁGIO POR ESTÁGIO

| `acesso.log` | → `awk '{print $1}'` | → `sort` | → `uniq -c` | → `sort -rn` |
|---|---|---|---|---|
| 10.0.0.1 … 200 | 10.0.0.1 | 10.0.0.1 | 1 10.0.0.1 | 3 10.0.0.2 |
| 10.0.0.2 … 404 | 10.0.0.2 | 10.0.0.2 | 3 10.0.0.2 | 2 10.0.0.3 |
| 10.0.0.3 … 200 | 10.0.0.3 | 10.0.0.2 | 2 10.0.0.3 | 1 10.0.0.1 |
| 10.0.0.2 … 500 | 10.0.0.2 | 10.0.0.2 | | |
| 10.0.0.3 … 200 | 10.0.0.3 | 10.0.0.3 | colapsa e conta os grupos | ordena pelo número, do maior |
| 10.0.0.2 … 200 | 10.0.0.2 | 10.0.0.3 | | |
| | extrai a coluna que interessa | **agrupa** as iguais lado a lado | | |

**Tire o `sort` do meio e o resultado fica errado sem avisar.** Sobre a entrada em ordem cronológica, o `uniq -c` veria **seis grupos de um** — `1 10.0.0.1`, `1 10.0.0.2`, `1 10.0.0.3`, `1 10.0.0.2` … — em vez de três endereços com 3, 2 e 1 ocorrências. Nenhum erro, nenhum aviso, só um relatório errado.

O `sort -rn` final funciona porque o `uniq -c` põe a contagem **na frente**, alinhada com espaços à esquerda — e o `-n` ignora esses espaços. Para desempatar pelo endereço, `sort -k1,1rn -k2,2`. **O mesmo resultado sai de um `awk` só** — `awk '{n[$1]++} END{for(k in n) print n[k], k}' | sort -rn` —, numa passada em vez de três, o que importa em arquivo grande. E para deduplicar **preservando a ordem de aparição**, o `sort` está fora: é `awk '!visto[$0]++'`.

## A ARMADILHA DE CADA FERRAMENTA

### `sort -k2n`

```
alfa  10 zzz
beta   9 aaa
```

"do campo 2 **até o fim da linha**". Compara

`10 zzz`

contra

`9 aaa`

como se fosse um número só — o resto da linha entra na conta. **Às vezes acerta, o que é pior que errar sempre.**

### `sort -k2,2n`

```
alfa  10 zzz
beta   9 aaa
```

"do campo 2 **ao campo 2**". Compara

`10`

contra

`9`

, e só. É o que se queria dizer em praticamente todo uso de

`-k`

. A regra: **sempre escreva o campo final.**

O delimitador vem de `-t`; sem ele o `sort` corta na *transição* de espaço para não-espaço, e o campo inclui o espaço à esquerda — daí existir o modificador `b`. **Várias chaves se acumulam e valem em ordem**, cada uma com seus modificadores: `sort -t: -k3,3n -k1,1` ordena por UID e desempata por nome; em `-k2,2nr -k1,1`, o `r` vale só para a primeira.

### SORT · AS OPÇÕES QUE DECIDEM O RESULTADO

| | |
|---|---|
| `-h` | numérica **com sufixo humano** K, M, G — **é o conserto da abertura** |
| `-V` | versão: `1.9` antes de `1.10` |
| `-u · -s · -c` | remove duplicatas · estável (empate mantém a entrada) · só **verifica** se já está ordenado |
| `LC_ALL=C` | a localidade **define a ordem**: em `pt_BR` o `sort` ignora caixa e pontuação; sob `C` é a ordem dos bytes — previsível **e mais rápida** |
| `-S · -T` | tamanho do buffer e onde ficam os temporários — importa quando `/tmp` é pequeno ou é `tmpfs` |

### UNIQ · SÓ ENXERGA O VIZINHO

| | |
|---|---|
| `-c · -d · -u` | conta · só as repetidas · só as que aparecem **uma única vez** |
| `-f2 · -w8` | ignora os 2 primeiros campos · compara só os 8 primeiros caracteres |
| `sort -u` | mais rápido que `sort \| uniq`, porque elimina **durante** a ordenação — mas `-c`, `-d` e `-u` são do `uniq` e exigem o par completo |

### CUT · RÁPIDO, PREVISÍVEL E COM TRÊS LIMITES

| | |
|---|---|
| `-d é 1 char` | não existe delimitador de dois espaços nem por expressão regular |
| `não colapsa` | espaço repetido desloca tudo — normalize antes com `tr -s ' '` |
| `não reordena` | `cut -f3,1` devolve **1,3**: o `cut` ignora a ordem pedida. Reordenar é `awk '{print $3, $1}'` |
| `-c1-8 · -s` | corta por **caractere** · suprime as linhas **sem** o delimitador (que por padrão saem inteiras) |

### TR · CARACTERE A CARACTERE, SÓ DO STDIN

| | |
|---|---|
| `tr -d '\r'` | conserta arquivo do Windows — a causa do `bad interpreter: /bin/bash^M` |
| `tr -s ' '` | **colapsa** espaços repetidos: é o que torna o `cut -d' '` viável |
| `tr -c … '\n'` | `tr -c '[:alnum:]' '\n' \| sort \| uniq -c` é o contador de palavras de uma linha |
| `não é regex` | não conhece padrão e **não substitui palavras** — e `-d` com `-s` juntos raramente faz o esperado |

### O RESTO DA CAIXA

| | |
|---|---|
| `comm -13 -23 -12` | **o que mudou entre duas listas**: só no 2º · só no 1º · em ambos |
| `paste -sd,` | lado a lado · `-s` junta **todas** as linhas numa só |
| `join -t: -1 1` | junção relacional — **exige as duas entradas ordenadas** pelo campo, ou perde pares em silêncio |
| `column -t` | alinha para ler · `nl` numera · `tac` inverte a ordem · `rev` inverte os caracteres |
| `tail -n +2` | tudo **a partir** da linha 2 — o jeito de pular cabeçalho |
| `shuf -n 100` | amostra aleatória · `split -l` fatia arquivo grande |
| `tee copia.txt` | bifurca no meio do pipeline: **a ferramenta de depuração** de pipeline longo (`tee /dev/stderr` mostra na tela e segue) |

`rev | cut -d/ -f2 | rev` pega o **penúltimo** campo de uma linha de tamanho variável sem `awk`. E `comando | head -5` **mata o comando anterior com `SIGPIPE`** — é o comportamento desejado, o que faz `zcat enorme.gz | head` ser instantâneo. Só que sob `set -o pipefail` isso aparece como **falha do pipeline**: é a causa mais comum de script que "falha" fazendo exatamente o certo.

`sort arquivo > arquivo` **esvazia o arquivo.** O shell **trunca o destino antes** de o `sort` ler qualquer coisa — o conteúdo se perde por completo, sem erro e sem confirmação. O certo é `sort -o arquivo arquivo`, que o próprio `sort` trata com segurança. A mesma regra vale para todo comando que leia e escreva o mesmo caminho num redirecionamento; quando não houver a opção `-o`, escreva num temporário e mova depois. E lembre que o `sort` guarda tudo na memória até estourar: em arquivo maior que a RAM, ele passa a usar `/tmp` — se não houver espaço lá, ele **morre no meio** do processamento.

## RESUMO RÁPIDO

- ✓ `sort -n` **ignora sufixo de unidade.** Para `1,5G` contra `900M`, é `sort -h`; para versão, `sort -V`.
- ✓ `uniq` **só compara linhas adjacentes.** Sem `sort` antes ele conta errado e não avisa. O canônico é `extrair | sort | uniq -c | sort -rn`.
- ✓ **Em `sort -k`, escreva sempre o campo final:** `-k2n` é "do campo 2 ao fim da linha"; `-k2,2n` é o campo 2.
- ✓ `cut` **é rápido e limitado:** delimitador de um caractere, sem colapsar espaços e sem reordenar. Faltou uma das três? É `awk`.
- ✓ `tr` **opera caractere a caractere** e não conhece padrão: `tr -d '\r'`, `tr -s ' '` e `tr -c '[:alnum:]' '\n'` cobrem quase todo o uso real.
- ✓ **A localidade muda a ordem do `sort`.** Fixe `LC_ALL=C` quando a saída for comparada entre máquinas — e ganhe desempenho de brinde.
- ✓ `sort arquivo > arquivo` **apaga o arquivo.** Use `sort -o arquivo arquivo`.
- ✓ `join` **e** `comm` **exigem entradas ordenadas** e falham em silêncio se não estiverem.
- ✓ `tee` **no meio do pipeline** grava o estágio intermediário sem interromper o fluxo.

## NO KALI

**A contagem de portas abertas é o pipeline canônico.** `grep -oE '^[0-9]+' portas.txt | sort -n | uniq -c | sort -rn | head -20` diz quais serviços dominam a rede varrida — e portanto por onde começar. Esquecer o `sort` do meio aqui produz um relatório que **subestima a exposição**, o tipo de engano que passa despercebido até alguém conferir na mão. `comm` **compara duas varreduras e mostra o que mudou:** `comm -13 <(sort ontem.txt) <(sort hoje.txt)` lista o que apareceu desde a anterior — host novo, porta nova, serviço que subiu. É monitoramento de superfície de ataque em duas linhas, e serve tanto para quem defende quanto para documentar a evolução do escopo num teste longo. `tr -d '\r'` **antes de qualquer coisa** em *wordlist*, lista de alvos ou script trazido de host Windows: o sintoma é uma ferramenta que **não encontra o host** `10.0.0.5` — porque o que ela recebeu foi `10.0.0.5\r`, que não resolve. **Refinar *wordlist* é** `sort`, `uniq` **e** `shuf` **combinados:** juntar várias listas e deduplicar (`LC_ALL=C sort -u l1 l2 l3 > combinada.txt` — várias vezes mais rápido que sob UTF-8), cortar as curtas demais e embaralhar para não bater sempre na mesma ordem; em lista de milhões de linhas, confira antes se há espaço em `/tmp`, ou o `sort` morre no meio. **E amostrar antes de varrer é prudência, não preguiça:** `shuf -n 50 alvos.txt` valida o comando e estima o tempo antes de disparar contra a lista inteira. Um erro de opção descoberto no alvo 3 de 50 é contratempo; descoberto no alvo 3 de 60 mil, já gerou **tráfego indevido, alarme no monitoramento do cliente e uma conversa desagradável sobre escopo** — com o pano de fundo da **Lei 12.737/2012**.

# JQ
# JSON E CSV

*O `curl … | grep '"ip"' | cut -d'"' -f4` funcionou por duas semanas. Aí a API passou a mandar a resposta **minificada**, tudo numa linha — e o `grep`, que decide por linha, viu o documento inteiro como **uma** linha. Consertado o corte, quebrou de novo num campo com **aspa escapada**; e de novo quando a API **trocou a ordem das chaves**, coisa que o JSON permite porque objeto é **mapa, não sequência**. Nenhum é bug do script: os três vêm da mesma decisão errada.*

**069**
GUIAS LINUX

## O MODELO: UM FILTRO QUE TRANSFORMA VALORES EM VALORES

**Tudo no `jq` é esse encadeamento.** O `|` interno funciona como o do shell, só que carregando **valores JSON** em vez de bytes: `.[] | select(.porta==22) | .ip` lê-se "para cada item da lista, se a porta for 22, devolva o campo `ip`". A diferença crucial é que um filtro pode emitir **zero, um ou vários** valores para cada entrada — o `.[]` emite vários, o `select` emite zero ou um, e é dessa aritmética que sai todo o poder da ferramenta. **E o que sai do `jq` ainda é JSON**: cadeias vêm entre aspas e escapadas. Para alimentar o próximo comando do pipeline é quase sempre `jq -r` (*raw*) — **a opção mais esquecida** e a causa número um de nome de arquivo com aspas no meio.

## CSV É UMA ARMADILHA — E OS SELETORES QUE RESOLVEM O RESTO

**linha bem-comportada:** `10.0.0.1,web,producao`

| | | |
|---|---|---|
| `cut -d, -f2` | `web` | correto |

**linha real:** `10.0.0.1,"servidor web, filial SP",producao`

| | | |
|---|---|---|
| `cut -d, -f2` | `"servidor web` | cortou ao meio |
| `awk -F, '{print $2}'` | `"servidor web` | mesmo erro |
| `mlr --icsv --ojson` | `"servidor web, filial SP"` | correto |

A especificação do CSV permite **vírgula dentro de campo entre aspas**, **aspas duplicadas** para representar uma aspa literal (`""`) e até **quebra de linha dentro de um campo** — o que arruína a premissa "um registro por linha" de que dependem `grep`, `cut`, `sed` e `awk` **ao mesmo tempo**. Exportação de planilha, de banco e de ferramenta comercial traz as três coisas rotineiramente. As saídas: `mlr` (Miller), `csvkit` ou uma linguagem com biblioteca de CSV; ou converta uma vez para JSON/TSV e trabalhe dali. **Para *gerar* CSV, `jq -r '… | @csv'` produz a citação correta de graça.** Só trate CSV com `cut` quando você controlar quem o gerou **e** souber que não há vírgula, aspa nem quebra de linha nos valores.

**Os outros formatos têm suas ferramentas.** `yq -o=json '.' config.yaml` converte YAML e devolve o problema ao `jq` — o `yq` moderno é, em essência, um `jq` que fala YAML, com a mesma sintaxe de filtros. **Cuidado:** existem **dois projetos com o nome `yq`** (o de Mike Farah, em Go, e o `yq` em Python), com linhas de comando **incompatíveis** — saiba qual está instalado. Para XML, **`xmlstarlet`** usa XPath e é a ferramenta certa para o `-oX` do `nmap`.

### NAVEGAR

| | |
|---|---|
| `.host.rede.ip` | campo aninhado · `.["chave com espaço"]` quando não é identificador |
| `.[] · .[0] · .[-1]` | cada elemento · o primeiro · **o último** · `.[2:5]` fatia |
| `.talvez? · ..` | não falha se o tipo não permitir · **descida recursiva**: todos os valores, em qualquer nível |
| `.inexistente` | devolve `null` **em silêncio**, não erro — um erro de digitação vira uma coluna de `null`. Exija com `has("chave")` ou `.chave // error("faltou")` |
| `a // b` | valor padrão quando o lado esquerdo é `null` ou `false` |

### FILTRAR, TRANSFORMAR, AGREGAR

| | |
|---|---|
| `select(cond)` | emite **zero ou um** · `select(.ip \| startswith("10."))` · `select(.tags \| index("prod"))` |
| `map(f) vs .[]\|f` | `map` **devolve um array;** `.[] \|` **devolve vários valores soltos.** Escolher errado é a origem de `Cannot iterate over null` e dos colchetes a mais |
| `group_by · add` | `group_by(.porta) \| map({porta: .[0].porta, n: length})` · `sort_by`, `unique_by`, `any`, `keys`, `to_entries` |
| `{a: .x} · \(…)` | constrói objeto novo · interpola em cadeia: `"\(.ip):\(.porta)"` |

### SAIR DO JSON E VOLTAR AO PIPELINE

| | |
|---|---|
| `-r · -c` | sem aspas · compacto, **um documento por linha** (NDJSON) |
| `@csv @tsv @uri` | citação e escape **corretos** de graça · `@base64` |
| `-s (--slurp)` | junta o fluxo **num array** — sem ele, cada linha do NDJSON é processada isoladamente e `length` responde por documento |
| `-R · -n` | lê texto comum (converte log em JSON) · não lê entrada, só produz |
| `-e (--exit-status)` | sai com **1** quando o resultado é `null` / `false`: é o `grep -q` do JSON, para usar em `if` |
| `--stream` | para arquivo maior que a memória — desconfortável e raro; o usual é converter para NDJSON uma vez com `jq -c '.[]'` |

**Valor do shell entra com `--arg`, nunca por interpolação — a mesma regra do `-v` do `awk`.** `jq --arg alvo "$IP" '.[] | select(.ip == $alvo)'` é seguro (e `--argjson lim 1024` passa número, booleano ou objeto). Já `jq "select(.host == \"$HOST\")"` **quebra na primeira aspa e é injeção de código na segunda** — e o valor costuma vir do próprio alvo: nome de host, cabeçalho, campo de resposta. **Dado que veio do alvo é dado hostil por definição.** Pelo mesmo motivo, **não redija relatório com `sed` sobre JSON**: é possível e falha em silêncio no primeiro campo aninhado — e redação que falha é vazamento. O certo percorre a estrutura inteira: `jq '(.. | .password? , .token? , .cookie?) |= "REDIGIDO"' bruto.json > entrega.json`.

## RESUMO RÁPIDO

- ✓ **JSON não é texto orientado a linhas.** Espaço em branco é livre, a estrutura é aninhada e os valores têm escapes — daí `grep`, `cut` e `sed` funcionarem até deixarem de funcionar, sem aviso.
- ✓ **O `jq` é uma linguagem de filtro:** recebe um fluxo de valores, aplica a expressão e emite outro fluxo. O `|` interno compõe filtros como o do shell compõe comandos.
- ✓ **`.[]` explode uma coleção em vários valores; `map(f)` devolve um array.** Escolher o errado é a origem das saídas com colchetes a mais ou a menos.
- ✓ **`-r` é quase obrigatório quando a saída alimenta outro comando** — sem ele, as cadeias saem com aspas de JSON.
- ✓ **Campo ausente devolve `null` em silêncio.** Use `has()`, `//` para padrão ou `// error(…)` para exigir presença.
- ✓ **Valores do shell entram com `--arg` e `--argjson`**, nunca por interpolação.
- ✓ **`@csv` e `@tsv` geram saída citada corretamente**, e `-e` faz o `jq` servir de condição em `if`.
- ✓ **CSV com vírgula ou aspa dentro de campo quebra `cut`, `awk` e `sed` ao mesmo tempo.** Use `mlr` ou `csvkit` — ou converta para JSON e volte ao `jq`.
- ✓ **NDJSON é o formato do ferramental moderno**, e o `jq` o processa nativamente; `--slurp` junta tudo num array quando a agregação exigir.

## NO KALI

**A saída JSON do ferramental moderno é feita para o `jq`.** O `nmap` produz XML (`-oX`), mas `nuclei`, `httpx`, `subfinder` e `naabu` emitem **NDJSON — um achado por linha**. `nuclei -jsonl -o achados.jsonl` seguido de `jq -r 'select(.info.severity=="critical") | "\(.host) \(.info.name)"' achados.jsonl` transforma centenas de linhas no resumo do que precisa de atenção imediata. Isso é **triagem de resultado de teste autorizado** — a mesma ferramenta contra alvo sem contrato não muda o enquadramento da **Lei 12.737/2012**. **Converter o XML do `nmap` uma vez economiza o resto do trabalho:** `xmlstarlet sel -t -m '//host' -v 'address/@addr' -o " " -v 'ports/port[state/@state="open"]/@portid' -n varredura.xml` produz host e portas em texto simples, pronto para `sort`, `uniq` e `awk`. **`jq -e` transforma a checagem em condição de script:** `if jq -e '[.[] | select(.severity=="critical")] | length > 0' achados.json >/dev/null; then notificar; fi` só dispara o alerta quando há achado crítico — é a forma correta de encadear a triagem com o resto da automação, **sem `grep` sobre JSON. E confira a versão antes de assumir sintaxe:** o `jq` 1.6 e o 1.7 diferem em detalhes que mordem — o 1.7 corrigiu o arredondamento de números grandes, mudou a ordenação de `keys` para chaves não ASCII e acrescentou `pick`, `abs` e `toarray`. Um filtro escrito para o 1.7 falha num host mais antigo; `jq --version` resolve a dúvida em um segundo.

# JUNTANDO TUDO PIPELINES DE DADOS

*Onze da noite, resposta a incidente: quais endereços bateram no `/admin` mais de vinte vezes na hora anterior, e com quais agentes de usuário? Noventa arquivos comprimidos, **quarenta gigabytes**. A primeira tentativa rodou **três horas** e devolveu uma lista errada por quatro motivos, nenhum deles um erro visível. O pipeline correto roda em **quatro minutos** e cabe em cinco linhas — e usa os mesmos comandos. **A diferença não é vocabulário; é método.***

**070**
GUIAS LINUX

## OS SETE ESTÁGIOS — DESCARTE CEDO, ESTRUTURE TARDE

| 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|
| **LOCALIZAR** quais arquivos | **REDUZIR** descartar o que não interessa | **EXTRAIR** isolar os campos | **NORMALIZAR** uniformizar a forma | **AGREGAR** contar, somar, agrupar | **ORDENAR** o relevante no topo | **APRESENTAR** formatar para ler |
| `find · zcat · ls` | `grep -F · -E` | `awk · cut · grep -o` | `tr · sed · awk` | `awk · uniq -c` | `sort -rn · -k` | `head · column · jq` |
| os únicos que reduzem VOLUME | | dão estrutura ao que sobrou | | resumem em resposta | | entrega |

**A ordem não é decorativa.** Tudo depois do estágio 2 paga o preço do que passou: um `grep` caro antes de um barato faz o caro processar cem vezes mais linhas do que precisava — foi um dos quatro erros da abertura. **Filtre por arquivo antes de filtrar por linha, filtre por linha antes de extrair campo, e só estruture o que sobreviveu.** Nem todo pipeline tem os sete: um relatório rápido é `grep | sort | uniq -c | sort -rn`, ou seja, os estágios 2, 5 e 6. Mas quando o resultado sai errado, a pergunta **"em qual estágio isso se perdeu?"** encurta a depuração de uma hora para dois minutos, porque cada estágio é testável isoladamente. Os outros três erros daquela noite: o `grep admin` casou `/administrativo` e `admin.css` (faltou ancorar na requisição); o `>>` dentro de um laço abriu o arquivo noventa vezes e **grudou a última linha de um arquivo na primeira do seguinte**; e metade da pergunta — os agentes de usuário — simplesmente ficou de fora.

## CONSTRUIR POR ESTÁGIO · E O QUE REALMENTE CUSTA TEMPO

kali@kali: ~/lab-pipeline

```
# 1. veja como os dados REALMENTE são
$ zcat acesso.log.gz | head -3
# 2. um estágio por vez, com head na frente: instantâneo
$ zcat acesso.log.gz | head -1000 | grep -F ':02:' | head -3
# 3. confira a CONTAGEM a cada estágio — é onde a perda aparece
$ zcat acesso.log.gz | wc -l
200000
$ zcat acesso.log.gz | grep -cF ':02:'
66701
# 4. o pipeline final: 4 minutos sobre 40 GB
$ LC_ALL=C zcat /var/log/nginx/acesso-*.gz \
    | grep -F ':02:' \
    | grep -E '"(GET|POST) /admin(/|\?| )' \
    | awk -F'"' '{split($1,c," "); print c[1] "\t" $6}' \
    | sort | uniq -c | sort -rn | head -20 | column -t -s $'\t'
```

**Para inspecionar um pipeline já pronto**, o `tee` grava o meio sem interromper o fluxo: `… | tee /tmp/e2.txt | awk … | tee /tmp/e3.txt | sort -rn`, e depois `wc -l /tmp/e[23].txt`. Para saber se um comando longo **está andando ou travado**, `pv enorme.log | grep -F 'ERRO' | wc -l` mostra vazão e progresso **sem alterar os dados**.

### DESEMPENHO: QUATRO DECISÕES, QUASE TODO O TEMPO

| | |
|---|---|
| `barato antes` | `grep -F` > `-E` > `-P` > `awk` com condição composta. Encadeados, **o barato vem primeiro** — inverter custa 10× |
| `uma passada` | `awk '/ERRO/{n[$1]++} END{…}'` substitui `grep\|cut\|sort\|uniq -c`: 2 processos em vez de 4, e de 2 a 5× mais rápido **em arquivo grande com poucas chaves** (em volume pequeno, a cadeia legível vence; e o `sort` externo é imbatível quando as chaves não cabem na memória) |
| `LC_ALL=C` | vale para `grep`, `sort` e `awk` — e muda **resultado**, não só velocidade |
| `xargs -P 8` | só ajuda quando o trabalho **se divide por unidade independente** e o gargalo é CPU. Em HD mecânico, oito processos disputando cabeçote são **mais lentos** que um |

### OS ERROS QUE NÃO APARECEM COMO ERRO

| | |
|---|---|
| `uniq sem sort` | conta em silêncio errado — e `join` sobre entrada não ordenada perde pares |
| `xargs -P` | **intercala a saída** dos processos: grave um arquivo por processo, ou use `parallel`, que serializa. **A redução final é sempre sequencial** |
| `nome com espaço` | quebra `for f in $(ls)` e `xargs` sem `-0`: use `find … -print0 \| xargs -0` |
| `sem \n final` | `>>` gruda registros. Na dúvida, `tail -c 1 arquivo \| xxd` |
| `\r do Windows` | campeão de falha silenciosa — `cat -A` mostra `^M`, `^I` e `$` |
| `entrada vazia` | produz nada e sai com **status 0**: indistinguível de "processou tudo e não achou". Se importa, conte antes |
| `subshell` | `cat x \| while read l; do ((n++)); done` deixa `n` em zero — use `done < <(cat x)` |
| `SIGPIPE` | `\| head -5` mata o produtor, o que é **correto** — mas sob `set -o pipefail` vira "falha" do pipeline |

**Pipeline que se repete vira script:** `#!/usr/bin/env bash`, `set -euo pipefail`, parâmetros nomeados com mensagem de uso (`"${1:?uso: $0 <dir> <hora>}"`), limite configurável por variável, `export LC_ALL=C` e o `--` antes dos caminhos. **E saiba quando parar:** junção de várias tabelas grandes é banco de dados (`duckdb` lê CSV e Parquet direto do disco); estrutura profunda, estado complexo entre registros, tratamento de erro por registro ou necessidade de **testes** pedem um programa. A heurística honesta: **mais de cinco ou seis estágios, mais de um `awk` não trivial, ou você vai ter de explicá-lo a alguém na semana que vem** — escreva em Python. Não é derrota: é a mesma escolha que fez o `awk` substituir o `cut`.

**Paralelizar por alvo multiplica o tráfego que você gera no alvo.** `xargs -P 8` sobre uma lista de hosts gera **oito vezes** o volume — e o contrato de um teste autorizado costuma ter limite de taxa. Estourá-lo é **indisponibilidade provocada**, mesmo sem intenção e mesmo com autorização para testar (Lei 12.737/2012). Use `-P` compatível com o combinado, `parallel --delay` para espaçar, e **prefira paralelizar o processamento local** dos resultados, que não toca o alvo. Pelo mesmo motivo, **todo pipeline que gera lista de alvos passa por um filtro de escopo** — como **estágio obrigatório**, nunca como conferência mental: `awk 'FNR==NR{ok[$1];next} $1 in ok' escopo.txt candidatos.txt` é o que impede que um `/16` digitado errado vire varredura fora do escopo.

## RESUMO RÁPIDO

- ✓ **Todo pipeline tem sete estágios:** localizar, reduzir, extrair, normalizar, agregar, ordenar, apresentar. Nomeá-los é o que permite perguntar "em qual estágio isso se perdeu?".
- ✓ **Descarte cedo, estruture tarde.** Só os dois primeiros reduzem volume; filtro barato antes de filtro caro.
- ✓ **Construa um estágio por vez, com `head` na frente**, e confira a contagem depois de cada um — é onde as perdas silenciosas aparecem.
- ✓ **Uma passada de `awk` costuma vencer `grep|cut|sort|uniq -c`** em arquivo grande com poucas chaves; em volume pequeno, a cadeia legível vence.
- ✓ **`LC_ALL=C` muda desempenho e resultado.** Fixe em qualquer script cuja saída seja ordenada ou comparada.
- ✓ **Paralelize por unidade independente e cuide da saída:** `xargs -P` intercala; a redução final é sempre sequencial.
- ✓ **Os erros que não aparecem como erro:** `uniq` sem `sort`, nome com espaço sem `-print0`, `\r` do Windows, entrada vazia com status 0, variável perdida em subshell, `SIGPIPE` sob `pipefail`.
- ✓ **Pipeline que se repete vira script** com parâmetros, uso, `set -euo pipefail` e `--`.
- ✓ **Saiba quando parar:** junção de tabelas grandes, estrutura profunda, estado complexo ou testes = escreva o programa.

## NO KALI

**A triagem de resultado de varredura é exatamente um pipeline de sete estágios:** localizar (`find . -name '*.gnmap'`), reduzir (`grep -F 'Ports:'`), extrair (`awk`), normalizar (`tr -s ' '`), agregar (contagem por porta), ordenar e apresentar (`column -t`). Escrever isso como **script parametrizado**, e não como linha decorada digitada de novo a cada teste, é o que permite rodar a mesma triagem em todo engajamento sem reconstruir o raciocínio às onze da noite. **O `tee` preserva a evidência bruta**, e isso importa juridicamente: numa resposta a incidente ou num teste com relatório, o dado intermediário é **prova**. `zcat *.gz | tee bruto-preservado.txt | grep …` guarda a entrada exata do processamento, e um `sha256sum` sobre ela fixa a **cadeia de custódia** — reprocessar depois com o pipeline corrigido só é possível se o bruto foi guardado. **A redação de dado sensível é o último estágio, e é verificada:** antes de entregar qualquer arquivo gerado, passe o `sed` / `jq` de redação e **depois confira com `grep -c` procurando o que deveria ter sumido** — um pipeline de redação que falha em silêncio entrega senha em claro num PDF de relatório. E lembre que **o ambiente muda o estágio 1**: numa máquina da família RHEL os logs de serviço estão no `journald` em formato binário, então o começo é `journalctl --since … -o json` em vez de `zcat /var/log/*.gz` — e a saída em JSON cai direto no `jq`. O `parallel` e o `pv` vêm com um `apt install` no Kali, mas exigem o EPEL no RHEL: num alvo, conte só com `xargs`.

GUIA VISUAL 071

# TRATAMENTO DE ERROS SET -EUO PIPEFAIL E TRAPS

*O script de backup rodava havia sete meses e o e-mail do `cron` chegava vazio — sinal de sucesso. No dia da restauração, o arquivo do dia anterior tinha **20 bytes**. Três meses antes o `/mnt/backup` deixara de ser montado: o `mkdir` criou o diretório no disco raiz, o `tar` encheu a partição e morreu, e o script **seguiu obediente** para a linha seguinte — apagou os dumps, imprimiu "backup concluido" e **saiu com status zero**. Nada ali está sintaticamente errado: é o padrão do Bash. **Este assunto é desligar esse padrão.***

**071**
GUIAS LINUX

## OS TRÊS PECADOS DO PADRÃO · O PRÓLOGO QUE OS DESLIGA · E ONDE O SET -E AINDA NÃO AGE

**1 ERRO NÃO INTERROMPE**
O Bash segue para a próxima linha, aconteça o que acontecer. Um **cd** que falha não impede o `rm -rf *` seguinte de rodar — **no diretório errado**.

**2 VARIÁVEL AUSENTE = STRING VAZIA**
Erro de digitação não gera erro nenhum: `rm -rf "$DESTNO/"` vira `rm -rf /`. É literalmente a origem do bug que apagou a instalação do Steam em 2015.

**3 O PIPELINE MENTE**
O status é o do **último** comando. Em `zcat corrompido.gz | wc -l`, se o `zcat` explodir o `wc` imprime `0` e sai com zero: o pipeline inteiro reporta sucesso.

```
#!/usr/bin/env bash
set -Eeuo pipefail
shopt -s inherit_errexit
```

| | |
|---|---|
| `-e` | **errexit** — morre no primeiro status ≠ 0 |
| `-u` | **nounset** — variável não definida é erro fatal |
| `pipefail` | status do **último que falhou**, não do último |
| `-E` | **errtrace** — o trap ERR é herdado por funções e subshells |
| `inherit_errexit` | fecha o buraco do `x=$(cat /nao/existe; echo ok)`, que **sobrevive** sem ele (Bash ≥ 4.4) |

| `if / while` | `&& ou \|\|` | `! comando` | `a \| b \| c` | SÓ AQUI O SCRIPT MORRE |
|---|---|---|---|---|
| condição de teste | tudo à esquerda | negação | quem não é o último | fora das quatro, status ≠ 0 encerra na hora |

**as quatro escapatórias são INTENCIONAIS — sem elas, todo `if` falso mataria o script**

**A exceção que mais engana é a função.** Chamada em contexto de condição, o `set -e` fica desligado **dentro dela inteira, recursivamente**: uma função de 40 linhas invocada como `if preparar; then` roda até o fim mesmo que a terceira linha falhe — imprime `AINDA RODEI` e ainda devolve "tudo certo". Duas regras daí: **não chame função longa em contexto de condição** (separe a checagem do trabalho) e, dentro dela, verifique explicitamente com `cd /alvo || return 1`. A segunda armadilha é aritmética: `((i++))` devolve **status 1 quando o valor anterior era zero** — um contador que começa em `i=0` mata o script. Use `((++i))` ou `i=$((i + 1))`. Sob `set -u`, o idioma de memorizar é `${VAR:-}` para leitura segura e `${1:?uso: $0 <arquivo>}` para argumento obrigatório com mensagem própria. E para saber **qual** estágio do pipeline falhou existe o array `PIPESTATUS` — volátil, sobrescrito até por um `echo`: copie na linha seguinte com `estados=("${PIPESTATUS[@]}")`.

## TRAP: A LIMPEZA QUE O SET -E NÃO FAZ

kali@kali: ~/lab-erros — limpeza.sh

```
$ cat limpeza.sh
#!/usr/bin/env bash
set -Eeuo pipefail; shopt -s inherit_errexit
# temporário com credencial: só o dono lê
umask 077
TMP="$(mktemp -d)"          # nunca /tmp/script-$$
limpar() {
    local st=$?              # PRESERVE o status ANTES de tudo
    rm -rf -- "$TMP"         # aspas + -- : se vazio, falha
    return "$st"
}
trap limpar EXIT             # roda SEMPRE (menos em kill -9)
erro() { printf '%s: erro %d na linha %d: %s\n' \
    "${BASH_SOURCE[1]##*/}" "$?" "${BASH_LINENO[0]}" \
    "$BASH_COMMAND" >&2; }
trap erro ERR                # só diagnóstico, nunca limpeza
mountpoint -q /mnt/backup || die 78 "nao montado"
$ ./limpeza.sh --falhar; echo "status: $?"
limpeza.sh: erro 1 na linha 18: false
limpando /tmp/tmp.7Kx2 (status 1)
status: 1
```

**Ctrl-C que se comunica.** Sair com `exit 1` depois de uma interrupção faz o chamador acreditar em falha genérica. A forma correta de "morrer como se não houvesse trap" é `interromper() { limpar; trap - INT; kill -INT "$$"; }` — o pai enxerga **130** (128+2) e sabe que houve `SIGINT`.

### OS QUATRO PSEUDO-SINAIS

| | |
|---|---|
| `EXIT` | o shell vai terminar **por qualquer motivo** — fim normal, `exit 1`, morte por `set -e`, `SIGTERM`, `SIGINT`. Só `kill -9` escapa. **É onde mora a limpeza** |
| `ERR` | um comando falhou, sob **as mesmas quatro exceções** do `set -e`. Só diagnóstico |
| `DEBUG` | antes de **cada** comando simples (herdado com `set -T`) |
| `RETURN` | ao voltar de uma função ou de um `source` |

### A ORDEM É FIXA

`ERR` **dispara antes do** `EXIT`, e o `EXIT` é o último código do processo a rodar — mesmo que o handler de `ERR` chame `exit`. O status que o chamador enxerga é **o que o handler de** `EXIT` **devolve**: sem o `local st=$?` na primeira linha, um `rm` bem-sucedido transforma falha em sucesso. Nas aspas do `trap`, **simples e uma função nomeada**: `trap "rm -f $ARQ" EXIT` expande **no registro**, `trap 'limpar' EXIT` expande **no disparo** (é o aviso `SC2064` do ShellCheck).

### CÓDIGOS COM SIGNIFICADO

| | |
|---|---|
| `1–63` | livre para o seu script |
| `64` | argumento inválido · `65` entrada malformada |
| `69` | dependência ausente · `78` erro de configuração (`sysexits.h`) |
| `126/7` | não executável / não encontrado — do shell |
| `128+N` | morto pelo sinal N: **130** = Ctrl-C, **143** = `SIGTERM` |

### O QUE O PRÓLOGO **NÃO** FAZ

Não detecta o erro que **não devolve status** (`grep` sem casamento devolve 1 sendo normal; `find` sem resultado devolve 0). Não valida entrada. Não garante atomicidade — só o `trap`, ou escrever num temporário e `mv` no fim. E não substitui a verificação explícita: `mountpoint -q /mnt/backup || die 78 …` é o que teria salvado o script da abertura. **É o piso, não o teto:** converte falha silenciosa em falha ruidosa, e é só isso que se pede dele.

**O caso `rm -rf` guiado por variável é a pior consequência dos pecados 1 e 2 juntos.** Sem `set -u`, `rm -rf "$DESTNO/"` com o nome digitado errado apaga a raiz; sem aspas e sem `--`, um `$TMP` vazio ou começando com `-` vira comando de destruição ou erro de sintaxe silencioso. A forma segura é **sempre** `rm -rf -- "$TMP"`, com `TMP` vindo de `mktemp -d` — nome previsível em `/tmp` é vulnerabilidade clássica de link simbólico: o atacante cria antes `/tmp/meuscript-1234` apontando para `/etc/shadow` e o seu script escreve lá com o privilégio que tiver. **Ensaie qualquer trap de limpeza com `echo rm -rf --` no lugar do `rm`** antes de rodar de verdade, e ensaie em VM descartável.

## RESUMO RÁPIDO

- ✓ **O padrão do Bash é continuar depois do erro**, tratar variável ausente como vazia e reportar só o último do pipeline. Os três juntos produzem o script que falha e diz que deu certo.
- ✓ **`set -Eeuo pipefail` + `shopt -s inherit_errexit`** é o prólogo completo: as três correções mais a herança do trap `ERR` e o subshell de `$( )`.
- ✓ **`set -e` tem quatro exceções intencionais** — condição de `if` / `while`, esquerda de `&&` / `||`, negação com `!` e comando não final de pipeline. A pior é a **função chamada em contexto de condição**, que roda inteira sem proteção.
- ✓ **`((i++))` com `i=0` devolve 1** e mata o script. Use `((++i))` ou `i=$((i + 1))`.
- ✓ **`${VAR:-}` é o acesso seguro sob `set -u`**; `${1:?mensagem}` documenta o argumento obrigatório na própria expansão.
- ✓ **A limpeza mora no `trap … EXIT`**, sempre com `local st=$?` na primeira linha do handler e `mktemp -d` em vez de nome previsível. O `ERR` cuida só do diagnóstico, e dispara antes.
- ✓ **`trap - INT; kill -INT "$$"`** é a forma correta de morrer por interrupção: devolve 130 em vez de um `1` mudo.
- ✓ **Nenhuma opção do `set` substitui verificação explícita** — `mountpoint -q`, `command -v` e `die` com código significativo continuam sendo trabalho seu.

## NO KALI

**Ferramenta cujo status 1 significa "não achei" é a briga diária com o `set -e`.** `nmap` com host fora do ar, `grep` sem casamento, `nikto` sem achado — todos devolvem status ≠ 0 em situação **normal** de um teste, e o script morre no primeiro alvo silencioso. O idioma é isolar e decidir pelo conteúdo, não pelo status: `resultado=$(nmap -Pn "$alvo") || true`. **Script de coleta que morre no meio deixa evidência pela metade:** o `trap EXIT` não deve só apagar temporários — tem de fechar o arquivo de resultado de forma consistente e **registrar no log que a execução foi interrompida**, porque um relatório truncado sem marca de interrupção é pior que nenhum: parece completo. Se o script manipula hash, token ou arquivo de senha, o `mktemp -d` vem precedido de `umask 077` e a limpeza usa `shred -u` onde o sistema de arquivos permitir. **Num script que toca alvo externo, o `trap INT` tem de parar de verdade** — e parar também os filhos: continuar gerando tráfego depois que o operador mandou parar é, na prática, exceder o escopo combinado (Lei 12.737/2012). E **verificação de escopo é `die`, não aviso**: a checagem de que o alvo está na lista autorizada encerra o script com código distinto, nunca imprime alerta e prossegue — aviso em `stderr` dentro do `cron` não é lido por ninguém. **Família RHEL:** `set -e` e `trap` são do Bash, mas `/bin/sh` ali é o Bash em modo POSIX, sem `inherit_errexit` — shebang `#!/bin/sh` perde as proteções nas duas famílias, e `#!/usr/bin/env bash` é a forma portátil. Em serviço do systemd, o status de saída é o que dispara `Restart=on-failure`: sair com zero depois de falhar impede o serviço de reiniciar.

GUIA VISUAL 072

# DEPURAÇÃO DE SCRIPTS SET -X E SHELLCHECK

*"Funciona quando eu rodo na mão." O script de rotação de logs foi testado dezenas de vezes e sempre terminou certo. Posto no `cron` para as 3h, falhou na primeira noite: `line 14: jq: command not found`. O `jq` **estava** instalado — o `which` respondia `/usr/local/bin/jq`. O que mudou não foi o script: foi o **ambiente**. Depurar shell é quase sempre isto — descobrir **qual suposição silenciosa** deixou de valer.*

072
GUIAS LINUX

## AS QUATRO CAMADAS · E O AMBIENTE QUE DERRUBOU O SCRIPT DA ABERTURA

**1 · NÃO EXECUTA** — SEGUNDOS
erro de sintaxe, aspas faltando, variável atribuída e nunca usada
`shellcheck · bash -n · shfmt`

**2 · EXECUTA E VÊ** — MINUTOS
expansão inesperada, argumento partido, caminho errado
`set -x · PS4 · bash -x`

**3 · REGISTRA** — HORAS
falha intermitente, erro só em produção, ordem dos eventos
`log em arquivo · BASH_XTRACEFD`

**4 · ISOLA** — DIAS
bug que só aparece com dados reais ou sob concorrência
`bisseção · bats · trap DEBUG`

**SEU TERMINAL — INTERATIVO, DE LOGIN**

- `PATH` completo, com `/usr/local/bin` e `/opt`
- lê `/etc/profile` e o perfil do usuário: **aliases e funções existem**
- **TTY presente** — cor faz sentido; `stdin` é o teclado
- `LANG=pt_BR.UTF-8`; `HOME` e `USER` são os seus

**SOB CRON OU TEMPORIZADOR DO SYSTEMD**

- `PATH` mínimo: `/usr/bin:/bin` — e o `jq` some
- **não lê nenhum arquivo de perfil**: alias não existe, função só se o script definir ou fizer `source`
- **sem TTY**: nada de cor nem de prompt; `stdin` fechado ou `/dev/null`
- `LANG=C` ou vazio; `HOME` pode ser `/` ou o do serviço

**A regra do funil: suba de camada só quando a de baixo não resolveu.** Pular direto para o rastro de um script de 400 linhas quando o ShellCheck acharia o defeito em dois segundos é o desperdício mais comum de tempo em depuração de shell. A passagem da camada 2 para a 3 é a mais importante: um defeito que **não reproduz** no terminal não vai ser resolvido olhando mais forte para o terminal — ele precisa de **registro persistente do ambiente em que acontece**. **As cinco defesas contra o "funciona na minha máquina":** (1) caminho absoluto, ou `PATH` declarado no topo do script; (2) nada de alias — e função só existe se o próprio script a carregar; (3) `[[ -t 1 ]]` antes de emitir cor, para não encher o log de sequências de escape; (4) `export LC_ALL=C` quando a saída for ordenada ou comparada; (5) **reproduza o ambiente antes de acusar o script**. Fora do funil ainda existe o clássico do arquivo vindo do Windows: `line 2: $'\r': command not found`, diagnosticado com `file script.sh` ("with CRLF line terminators") ou `cat -A`, e curado com `dos2unix` ou `sed -i 's/\r$//'`.

## SHELLCHECK ANTES DE RODAR · SET -X DURANTE

kali@kali: ~/lab-depuracao

```
$ shellcheck rotacionar.sh
In rotacionar.sh line 8:
for arq in $(ls /var/log/*.log); do
           ^-- SC2045: Iterating over ls output is fragile.
               ^-- SC2046: Quote this to prevent word splitting.
# o rastro mostra o que o SHELL entendeu, não o que você escreveu
$ bash -x -c 'arq="meu relatorio.txt"; rm -f $arq'
+ arq='meu relatorio.txt'
+ rm -f meu relatorio.txt   # DOIS argumentos!
# PS4 diz onde cada linha do rastro aconteceu
$ export PS4='+ ${BASH_SOURCE##*/}:${LINENO}:${FUNCNAME[0]:-main}(): '
+ backup.sh:24:validar(): mountpoint -q /mnt/backup
# rastro fora da saída, para ler com calma depois
$ exec 9>>/tmp/rastro.log; export BASH_XTRACEFD=9; set -x
# reproduza o cron sem esperar até as 3h da manhã
$ env -i HOME="$HOME" PATH=/usr/bin:/bin \
    /bin/bash --noprofile --norc ./script.sh </dev/null
shellcheck NAO encontrado neste PATH
```

**Ligue o rastro só onde ele serve.** No script inteiro produz rastro demais para ler: use `set -x; trecho; set +x` em volta do bloco suspeito, ou deixe uma chave de ambiente — `[[ -n "${DEBUG:-}" ]] && set -x` — e rode `DEBUG=1 ./script.sh` **sem editar nada**. Para o defeito que resiste: **bisseção** (um `exit` no meio; funcionou? está na outra metade — cinco iterações localizam num script de 300 linhas), `trap DEBUG` como passo a passo, ou `bats` para virar teste. E **escreva o teste que reproduz o defeito antes de corrigi-lo**: é o que impede que ele volte.

### OS AVISOS QUE SÃO CORREÇÃO, NÃO ESTILO

| | |
|---|---|
| SC2086 | expansão **sem aspas** — quebra em espaços e sofre globbing |
| SC2046 | substituição de comando sem aspas — mesma coisa, forma mais perigosa |
| SC2164 | `cd` sem tratar falha; use `cd x \|\| exit` |
| SC2155 | `local v=$(cmd)` — o status é o do `local`, sempre 0, e a falha de `cmd` **some**. Declare numa linha, atribua na seguinte |
| SC2181 | `if [ $? -eq 0 ]` é frágil: qualquer comando entre as duas linhas destrói o `$?`. Teste o comando direto: `if grep -q p arq; then` |
| SC2115 | `rm -rf "$DIR/build"` sem garantia de que `DIR` tem valor. Use `"${DIR:?DIR nao definida}"` |
| SC2034 | variável atribuída e nunca usada — quase sempre erro de digitação |
| SC2064 | aspas **duplas** no `trap`: a expansão acontece no registro, não no disparo |
| SC1090 | `source "$arq"` com caminho variável não pode ser analisado |

### SILENCIAR COM RESPONSABILIDADE

A diretiva vai **na linha imediatamente anterior** e **com justificativa**: `# shellcheck disable=SC2086  # a expansão sem aspas é intencional: OPCOES é lista de flags`. No topo do arquivo ela vale para o arquivo inteiro — raramente o que se quer. `disable` **sem comentário é dívida:** seis meses depois ninguém sabe se era intencional ou se alguém só queria calar o analisador. Úteis também: `# shellcheck shell=bash` (biblioteca sem shebang) e `# shellcheck source=lib/comum.sh` (resolve o `SC1090`).

### PLUGUE NO FLUXO

`shellcheck -S warning $(find . -name '*.sh')` varre o repositório; `-f gcc` dá `arquivo:linha:coluna:` para a integração contínua; e o gancho de pré-commit é `git diff --cached --name-only --diff-filter=ACM | grep '\.sh$' | xargs -r shellcheck`. Companheiro natural é o `shfmt -d -i 4 -ci` (mostra o diff) / `-w` (aplica): **com indentação regular, um `fi` no lugar errado salta aos olhos**. E antes de qualquer script que apague, formate ou reinicie algo, `bash -n` — um `fi` faltando em 300 linhas só aparece quando a execução chega lá, e aí metade do estrago já foi feita.

**O `SC2115` é o aviso que evita apagar o sistema.** `rm -rf "$DIR/build"` parece seguro por ter aspas — mas se `DIR` estiver vazia (nome digitado errado, ou variável definida só num ramo do `if`), o comando executado é `rm -rf "/build"`, e com privilégio suficiente ele apaga o que encontrar na raiz. A forma que **falha em vez de destruir** é `rm -rf "${DIR:?DIR nao definida}/build"`. Antes de rodar qualquer script novo que apague coisas, use as três camadas baratas na ordem: `shellcheck`, `bash -n` e **o rastro com o `rm` trocado por `echo rm`** — o `set -x` mostra o caminho **já expandido**, que é exatamente a informação que faltava. E ensaie em VM descartável, nunca na máquina de trabalho.

## RESUMO RÁPIDO

- ✓ **A causa mais comum de "funciona na minha máquina" é o ambiente**, não o código: `PATH` mínimo, nenhum perfil lido, sem TTY e localidade `C` sob `cron` e systemd.
- ✓ **Reproduza antes de acusar o script:** `env -i HOME="$HOME" PATH=/usr/bin:/bin /bin/bash --noprofile --norc ./script.sh` .
- ✓ **O ShellCheck é a ferramenta de melhor retorno por segundo investido.** `SC2086` , `SC2155` , `SC2181` e `SC2115` descrevem defeitos reais, não estilo.
- ✓ `local x=$(cmd)` **esconde a falha de** `cmd` — o status é o do `local` . Declare numa linha, atribua na seguinte.
- ✓ `set -x` **mostra o comando depois das expansões**: é assim que se enxerga o argumento partido pela falta de aspas. Em volta do trecho suspeito, não no script inteiro.
- ✓ `PS4` **transforma o rastro em diagnóstico** — arquivo, linha e função em cada linha — e `BASH_XTRACEFD` tira o rastro da saída, mantendo o `stdout` limpo para quem consome o resultado.
- ✓ **Silenciar aviso exige justificativa escrita** na linha anterior; `disable` sem comentário é dívida.
- ✓ **Suba de camada só quando a de baixo não resolveu** — estática, rastro, registro, isolamento — e escreva o teste que reproduz o defeito antes de corrigi-lo.

## NO KALI

**Ferramenta de segurança em script quase sempre roda sob** `cron` **ou temporizador do systemd** — coleta periódica, varredura agendada, verificação de integridade caem exatamente na armadilha da abertura. **Declarar** `PATH` **no topo do script é obrigatório no Kali**, onde muita ferramenta vive em `/usr/share/<ferramenta>` com um invólucro em `/usr/bin` , e o que se instala à mão fica em `/opt` . **Cuidado: o rastro do** `set -x` **vaza segredo.** Se o script manipula senha, token ou chave, o rastro imprime o **valor já expandido** — e se ele vai para log persistente, a credencial fica gravada em claro. Desligue em volta do trecho sensível ( `set +x` antes, `set -x` depois) e nunca passe segredo por linha de comando, onde ele também apareceria no `ps` . **Um** `SC2086` **num script que monta a lista de alvos é a diferença entre varrer** `10.0.0.1` **e varrer dois hosts** porque a variável trazia dois valores: estourar o escopo por falta de aspas não é atenuante nenhum diante da Lei 12.737/2012, e a conferência estática é barata. `[[ -t 1 ]]` **antes de colorir:** ferramenta do Kali produz muita saída colorida, e um script que a captura com `$( )` recebe as sequências de escape junto — a comparação de string falha por um motivo invisível. Desligue a cor na origem ( `--no-color` , `NO_COLOR=1` ) ou limpe com `sed -r 's/\x1b\[[0-9;]*m//g'` . E **log com marca de tempo é evidência**: num teste autorizado, o registro de quando cada comando foi disparado é o que separa a sua atividade da de terceiros no mesmo período — `PS4` com `\D{%F %T}` e rastro em arquivo dedicado viram auditoria da própria execução. **Família RHEL:** o `shellcheck` está no EPEL ( `dnf install epel-release` antes); o `cron` do usuário pode rodar com `SHELL=/bin/sh` , fazendo `[[ ]]` e arrays falharem com mensagens confusas (declare `SHELL=/bin/bash` no topo do `crontab` ); e o SELinux em `enforcing` produz "Permission denied" com permissões POSIX corretas — esse diagnóstico está no `ausearch -m avc -ts recent` , não no `set -x` .

GUIA VISUAL 073

# ARQUIVOS, DESCRITORES E PROCESSOS EM SCRIPTS

*O analista queria só deixar a lista de alvos ordenada no lugar: `sort alvos.txt > alvos.txt`. O arquivo ficou com **zero bytes** — sem mensagem de erro, sem confirmação, e o backup mais recente era de três dias antes. A explicação não está no `sort`: está na **ordem em que o shell faz as coisas**. Ele prepara os redirecionamentos **antes** de executar o comando, e `> arquivo` quer dizer "abra truncando". Quando o `sort` foi ler, o arquivo já estava vazio havia milissegundos.*

**073**
GUIAS LINUX

## A TABELA DE DESCRITORES · E A ORDEM QUE ESVAZIOU O ARQUIVO

**SEU SCRIPT**
bash backup.sh
PID 4821

| | | | |
|---|---|---|---|
| 0 | stdin | → | /dev/pts/1 o terminal |
| 1 | stdout | → | saida.txt exec 1> saida.txt |
| 2 | stderr | → | /dev/pts/1 o terminal |
| 3 | livre | → | dados.csv exec 3< dados.csv |
| 9 | livre | → | rastro.log BASH_XTRACEFD=9 |
| 200 | livre | → | meu.lock flock -n 200 |

### AS FORMAS DE REDIRECIONAR

| | |
|---|---|
| `> arq` | abre para escrita, **truncando** |
| `>> arq` | escreve acrescentando ao fim |
| `< arq` | abre para leitura |
| `<> arq` | leitura **e** escrita, sem truncar |
| `>\| arq` | escreve mesmo com `noclobber` ligado |
| `2>&1` | o 2 passa a apontar para onde o 1 aponta **naquele momento** |
| `&> arq` | atalho do Bash para `> arq 2>&1` |
| `n> arq` | redireciona o descritor n; `n>&m` duplica; `n>&-` fecha |

`sort a.txt > a.txt`

| 1 | 2 | 3 | 4 |
|---|---|---|---|
| shell prepara o redirecionamento | a.txt **truncado** | sort só agora começa a rodar | lê 0 bytes, grava 0 bytes — **sem aviso** |

`sort a > tmp && mv tmp a`

| 1 | 2 | 3 | 4 |
|---|---|---|---|
| mktemp **no mesmo diretório** | sort lê o original, grava no temporário | o **&&** só segue se deu certo | mv renomeia **atomicamente** |

**A regra vale para qualquer transformação de arquivo, não só o `sort`: leia de um lugar, escreva em outro, troque no fim.** O ponto de atomicidade é o `mv` **dentro do mesmo sistema de arquivos** — entre sistemas diferentes ele vira cópia mais remoção, e o intervalo entre as duas é exatamente a janela que se queria eliminar. Por isso o temporário nasce ao lado do original, e não em `/tmp`. Se o `sort` falhar, o `&&` impede o `mv` e o original fica intacto. Alternativas: `sponge` (do `moreutils`) absorve toda a entrada antes de abrir o destino — `sort alvos.txt | sponge alvos.txt` — ao custo de manter tudo em memória; e `sed -i` / `gawk -i inplace` resolvem, mas **não são atômicos de verdade**: gravam num temporário e renomeiam, o que troca o inode e **quebra hard links**. Como rede extra, `set -o noclobber` transforma sobrescrita acidental em erro, e `>|` é a permissão explícita. E a pegadinha da ordem: `cmd > log 2>&1` manda os dois para o arquivo, mas `cmd 2>&1 > log` manda o **stderr para o terminal** — quando o `2>&1` é processado, o 1 ainda aponta para lá. Leia da esquerda para a direita, como uma sequência de atribuições.

## LER LINHA A LINHA · EXEC · FLOCK

kali@kali: ~/lab-descritores

```
# o laço canônico — cada peça corrige um defeito
$ while IFS= read -r linha || [[ -n "$linha" ]]; do
    printf '[%s]\n' "$linha"
  done < entrada.txt
# comando de dentro que lê stdin engole o resto: use outro fd
$ while IFS= read -r -u 3 host; do ssh "$host" uptime; done 3< hosts.txt
# carregar tudo num array de uma vez
$ mapfile -t alvos < <(grep -v '^#' escopo.txt)
$ printf 'total: %d\n' "${#alvos[@]}"
total: 42
# registrar TUDO sem perder a tela — uma linha no topo do script
$ exec > >(tee -a /var/log/meuscript.log) 2>&1
# salvar e restaurar o stdout
$ exec 3>&1; exec 1> /tmp/relatorio.txt; echo linha; exec 1>&3 3>&-
# trava: nada de arquivo .pid — o kernel solta até em kill -9
$ exec 200> /var/lock/meu.lock
$ flock -n 200 || { echo "ja ha uma execucao" >&2; exit 1; }
# dois consumidores da mesma saída, e um teste de porta sem nc
$ gerar | tee >(gzip > r.gz) >(sha256sum > r.sha) > r.txt
$ timeout 2 bash -c 'exec 3<>/dev/tcp/127.0.0.1/22' && echo "22 aberta"
22 aberta
```

**Higiene do temporário, em quatro linhas:** `umask 077` **antes** de criar; respeite o `TMPDIR` (em serviço systemd com `PrivateTmp=yes` ele aponta para um diretório isolado); `TRABALHO="$(mktemp -d -t meuscript.XXXXXXXX)"`, com molde nomeado para o diretório ser identificável no `ls /tmp` quando algo der errado; e `trap 'rm -rf -- "$TRABALHO"' EXIT`. Para segredo que não deve tocar o disco, `mktemp -p /dev/shm` — memória, sem rastro em disco, mas **ainda legível por quem tem privilégio** na máquina.

### O LAÇO CANÔNICO, PEÇA POR PEÇA

| | |
|---|---|
| `IFS=` | vazio, impede que **espaços e tabulações das bordas** sejam removidos |
| `-r` | não interpreta a barra invertida — sem ele, `C:\novo` vira `C:ovo` |
| `\|\| [[ -n ]]` | salva a **última linha sem quebra final**: sem isso ela é lida mas o `read` devolve 1 e o laço a descarta em silêncio |
| `< arq no done` | e nunca `cat arq \|`: o pipe cria subshell e **toda variável modificada no laço se perde**. Para saída de comando, `done < <(cmd)` |

### EXEC E FLOCK

| | |
|---|---|
| `exec` | sem comando, aplica o redirecionamento ao **shell inteiro** — todos os comandos seguintes herdam |
| `flock -n` | falha na hora em vez de esperar; sem `-n`, a segunda cópia fica na fila. **Não precisa apagar o lock:** o kernel libera até em `SIGKILL` — é a vantagem sobre o arquivo `.pid`, que deixa órfão em toda morte anormal |
| `<( ... )` | vira **caminho de arquivo** (`/dev/fd/63`), o que alimenta programa que não aceita pipe: `diff <(sort a) <(sort b)` |
| `>( ... )` | o shell **não espera** o processo de dentro terminar — ler o arquivo na linha seguinte pode pegá-lo incompleto. Use `wait` |
| `/dev/tcp` | socket implementado **pelo próprio Bash**: não existe no sistema de arquivos e não funciona em **dash** nem **sh** |

### O SCRIPT COMO PROCESSO

| | |
|---|---|
| `$$` | PID do shell atual — **não muda** em subshell |
| `BASHPID` | PID real, que **muda** em subshell |
| `$! · PPID` | último posto em segundo plano · o pai |
| `BASH_SOURCE` | `[[ "${BASH_SOURCE[0]}" == "$0" ]]` distingue **executado** de **carregado com `source`** — é o que deixa o mesmo arquivo ser biblioteca e programa |

**Todo redirecionamento com `>` é uma destruição silenciosa em potencial.** Não há confirmação, não há mensagem, e o conteúdo anterior não é recuperável: `sort a.txt > a.txt`, `cmd > $VAZIA` e um `>` digitado onde se queria `>>` apagam arquivo de escopo, de resultado ou de log sem deixar rastro. As três defesas, em ordem: **(1)** nunca escreva no arquivo que está lendo — `tmp="$(mktemp "${arq}.XXXXXX")"; cmd "$arq" > "$tmp" && mv -- "$tmp" "$arq"`; **(2)** `set -o noclobber` no topo do script, que transforma sobrescrita em erro e exige `>|` para o caso intencional; **(3)** `set -u`, para que `> "$DESTINO"` com a variável vazia falhe em vez de virar erro de sintaxe ou arquivo perdido. Ensaie qualquer script novo em cópia descartável do diretório, nunca no artefato original.

## RESUMO RÁPIDO

- ✓ **O shell prepara os redirecionamentos antes de executar o comando** — por isso `sort a > a` esvazia o arquivo. Leia de um lugar, escreva em outro, troque com `mv` no fim.
- ✓ **Redirecionar é apontar um descritor para outro destino.** `2>&1` copia o destino **atual** do 1: `cmd > log 2>&1` e `cmd 2>&1 > log` fazem coisas diferentes.
- ✓ **`exec` sem comando redireciona o shell inteiro**: `exec 3>&1` salva a saída; `exec > >(tee -a log) 2>&1` registra tudo sem perder a tela.
- ✓ **O laço canônico é `while IFS= read -r linha || [[ -n "$linha" ]]; do … done < arquivo`** — cada peça corrige um defeito: bordas, barra invertida, última linha e o subshell do pipe.
- ✓ **Comando dentro do laço que lê `stdin` engole o resto do arquivo.** Use `-u 3` com `3< arquivo`, ou `< /dev/null` no comando de dentro.
- ✓ **`mktemp` no mesmo diretório do original** é o que torna o `mv` final atômico; entre sistemas de arquivos diferentes, deixa de ser.
- ✓ **`flock` sobre um descritor é a trava correta**: o kernel a libera mesmo em `SIGKILL`, ao contrário do arquivo `.pid` criado à mão.
- ✓ **`<(…)` e `>(…)` viram caminhos de arquivo**, alimentando programa que não aceita pipe — lembrando que o shell não espera o processo de `>(…)`.
- ✓ **`umask 077` antes de `mktemp`, `/dev/shm` para segredo** e `trap … EXIT` para a remoção. Nome previsível em diretório público é vulnerabilidade, não descuido.

## NO KALI

**Coleta que grava enquanto lê é o desastre da abertura em escala.** Um script que refina a lista de alvos a cada rodada e a grava por cima do próprio arquivo **perde o escopo inteiro** na primeira falha — e a lista de escopo é o artefato mais importante de um teste autorizado. Reconstruí-la de memória é como escopo vira excesso, com o enquadramento da Lei 12.737/2012. Temporário no mesmo diretório e `mv` no fim. **`flock` em varredura agendada não é preciosismo:** uma varredura que demora mais que o intervalo do agendamento acaba com **duas cópias disparando pacotes ao mesmo tempo contra o mesmo alvo** — o dobro da taxa combinada no contrato, sem que ninguém tenha pedido. `exec 200> /var/lock/varredura.lock; flock -n 200 || exit 0` é uma linha e resolve. **`exec > >(tee -a registro.log) 2>&1` no topo do script de coleta** dá registro completo do que foi executado com a saída ainda visível: num teste autorizado esse arquivo é a prova de que a atividade ficou dentro do combinado — sem ele, a defesa é a sua memória. **`/dev/shm` para credencial, `/tmp` nunca:** hash, token e chave temporária pertencem à memória, com `umask 077` antes e remoção garantida por `trap` — e em máquina compartilhada o `/dev/shm` reduz o rastro em disco, não elimina a exposição. **`/dev/tcp` verifica porta sem instalar nada** numa máquina onde você tem acesso legítimo mas não pode instalar pacotes; é recurso de diagnóstico, e usá-lo para varrer faixa fora de escopo autorizado é a mesma infração que fazê-lo com `nmap`. **Família RHEL:** serviços com `PrivateTmp=yes` ganham um `/tmp` privado — um script que grava lá esperando que outro processo leia simplesmente não se encontra, e o `TMPDIR` é o caminho de coordenação; e o SELinux rotula arquivos, então criar lock à mão em `/var/lock` pode dar "Permission denied" com permissões POSIX corretas — o idiomático é `/run/lock` com `RuntimeDirectory=` na unidade.

# SUBSHELLS, AGRUPAMENTO E HERE-DOCUMENTS

*O script de implantação tinha uma função com `cd "src/$1"` e `make`. O módulo **rede** compilou; **parser** e **interface** falharam com "diretório não encontrado". O `cd` dentro da função mudou o diretório do **script inteiro**, e a segunda chamada tentou entrar em `src/rede/src/parser`. A correção cabe em **dois caracteres**: `( cd "src/$1" && make )`. Saber o que atravessa a fronteira de um subshell — e em que direção — separa quem escreve shell por hábito de quem escreve com intenção.*

**074**
GUIAS LINUX

## A FRONTEIRA DO SUBSHELL · E O AGRUPAMENTO QUE NÃO TEM FRONTEIRA NENHUMA

**SHELL PAI**

```
PWD=/projeto
contador=0
umask 022
trap limpar EXIT
opções: set -e
```

herda tudo na entrada →

← só o código de saída volta

**SUBSHELL ( ... )**

recebe **cópia** de tudo
`cd /tmp` → só aqui dentro
`contador=99` → só aqui dentro
`exit 1` → encerra só o subshell
traps de sinal voltam ao padrão

**( COMANDO1; COMANDO2 ) — ISOLA**

Use para **isolar `cd`**, mudar `umask` ou opção do shell temporariamente, ou rodar algo em paralelo com `&`. Cada um é um **`fork`**: custa.

**{ COMANDO1; COMANDO2; } — AGRUPA**

Mesmo processo: `cd` e atribuição **persistem** e `exit` derruba o script. Exige **espaço depois do `{` e `;` antes do `}`** — são palavras reservadas, não operadores.

**Subshells nascem sem que você escreva parênteses:** em **cada estágio** de um pipeline (`a | b | c` são três), em toda substituição de comando `$(…)`, em todo comando posto em segundo plano com `&` e em toda substituição de processo `<(…)` / `>(…)`. É a explicação de `n=0; seq 3 | while read x; do ((n++)); done; echo $n` imprimir `0`: o `while` rodou num processo que já morreu levando o `n` junto. **O uso legítimo do `{ }` é aplicar um redirecionamento, um `|` ou um `&&` a vários comandos de uma vez** — sem ele seria preciso repetir `>> relatorio.txt` em cada linha, e a primeira teria de ser `>` para truncar, exatamente o tipo de assimetria que se erra. **E antes de abrir um subshell, verifique se existe uma expansão que faz o mesmo:** `$(<arquivo)` é implementado internamente pelo Bash e não cria processo nenhum (contra dois em `$(cat arquivo)`); `${f%.log}` substitui `$(basename "$f" .log)`; `wc -l < "$f"` substitui `cat "$f" | wc -l`. Num laço de dez mil iterações com três substituições cada, são **trinta mil processos** de diferença.

## HERE-DOCUMENTS E HERE-STRINGS: AS QUATRO VARIANTES

**`<<EOF`** — EXPANDE
**expande `$var`, `$(cmd)`, `\`**
Quando o bloco precisa de **valores do script**. Cuidado: um `$` literal no texto vira expansão sem aviso.

**`<< 'EOF'`** — LITERAL
**não expande nada**
**O padrão** para gerar script, configuração e código. As aspas no delimitador são a única diferença visível.

**`<<-EOF`** — INDENTÁVEL
**remove as tabulações iniciais**
Permite indentar o bloco dentro de um `if`. Só **TAB**, nunca espaço — editor que converte TAB quebra em silêncio.

**`<<< "texto"`** — LINHA ÚNICA
**here-string, sem pipe**
`read -r a b <<< "$linha"` substitui `echo | read` e **evita o subshell**: as variáveis sobrevivem.

kali@kali: ~/lab-subshell

```
# redirecionamento aplicado ao GRUPO: o uso legítimo do { }
$ { echo "=== $(date +%F) ==="; df -h; echo "=== fim ==="; } > rel.txt
# gerar arquivo: aspas no delimitador, SEMPRE, por padrão
$ cat > /usr/local/bin/limpar.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
for f in /var/tmp/*; do rm -rf -- "$f"; done
EOF
# sem as aspas, $f expandiria AGORA: o disco receberia rm -rf -- ""
# escrever onde exige privilégio — sudo echo > NÃO funciona
$ sudo tee /etc/systemd/system/coleta.service >/dev/null <<'EOF'
[Service]
Type=oneshot
ExecStart=/usr/local/bin/coletar.sh
EOF
# here-string: sem pipe, sem subshell — o read sobrevive
$ IFS=: read -r usuario _ uid _ <<< "kali:x:1000:1000"
$ echo "$usuario $uid"
kali 1000
# isolar o ambiente de UMA invocação
$ ( export HTTP_PROXY=http://127.0.0.1:8080; ferramenta )
```

### AQUI OU LÁ? O HEREDOC SOBRE SSH

| | |
|---|---|
| `<<EOF` | o bloco é expandido **localmente**: `$PARTICAO` chega já resolvido, e só o que você escapar com `\$` sobra para o remoto |
| `<<'EOF'` | o bloco vai **literal**: `$HOSTNAME` é resolvido **no remoto** |
| `teste antes` | troque `ssh servidor` por `cat` e **olhe o bloco** — um `$(…)` que você achou que rodaria aqui e roda lá é execução não pretendida na máquina de outra pessoa |
| `ssh -n` | em laço, o **`ssh` engole o resto da entrada** e o laço acaba na primeira volta. `ssh -n` ou `< /dev/null` |

### DETALHES QUE TRAVAM O SCRIPT

| | |
|---|---|
| `delimitador` | sozinho na linha, **sem espaço antes nem depois**. Um espaço invisível depois do fechamento é a causa nº 1 de **`unexpected end of file`**: o shell lê o resto do arquivo procurando um fim que não chega |
| `<<-'EOF'` | existe e junta as duas coisas — **literal e indentável**. É a forma de escolha para gerar arquivo de dentro de um bloco aninhado |
| `em pipeline` | o heredoc alimenta o **primeiro** comando: `sort <<'EOF' \| uniq -c` |
| `coproc` | subshell com quem se conversa, dois pipes já ligados. Nicho: mais que uma troca de linhas **trava em impasse** quando o buffer enche |

**Gerar script com `<<EOF` sem aspas grava o comando destrutivo já expandido no disco.** No bloco `for f in /var/tmp/*; do rm -rf -- "$f"; done`, se o delimitador não estiver entre aspas o `$f` é resolvido **na hora de gerar o arquivo** — e como `f` não existe no script gerador, o que fica gravado é `rm -rf -- ""`. O defeito só aparece quando o script gerado roda, com o privilégio que ele tiver. **Use `<<'EOF'` por padrão** e tire as aspas apenas quando quiser mesmo a expansão. Depois de gerar, **leia o arquivo antes de torná-lo executável** (`cat`, `shellcheck`, e o `rm` trocado por `echo rm` num ensaio). Vale o mesmo cuidado em `/etc/sudoers.d`: valide com `visudo -cf arquivo` **antes** de movê-lo para o lugar definitivo — um arquivo malformado ali derruba o `sudo` da máquina inteira.

## RESUMO RÁPIDO

- ✓ **`( )` cria processo, `{ }` não.** Dentro de `( )`, `cd`, atribuição e mudança de opção morrem na saída; só o código de retorno atravessa a fronteira.
- ✓ **`{ }` exige espaço depois da chave e `;` antes do fechamento** — são palavras reservadas, não operadores.
- ✓ **Subshells nascem sem parênteses:** em cada estágio de pipeline, em toda `$(…)`, em todo `&` e em `<(…)`. É a explicação do contador que continua zero depois do laço.
- ✓ **`( cd dir && comando )` é o idioma para isolar mudança de diretório** dentro de uma função — o defeito da abertura, corrigido com dois caracteres.
- ✓ **Cada subshell é um `fork`.** `$(<arquivo)`, `${var%sufixo}` e `wc -l < arquivo` fazem o mesmo trabalho sem criar processo.
- ✓ **`<<'EOF'` é o padrão para gerar arquivos**; `<<EOF` é a exceção para quando o bloco precisa das variáveis do script. As aspas decidem se o `$` do texto é literal.
- ✓ **`<<-` remove tabulação, nunca espaço.** Editor que converte TAB em espaço quebra o bloco de forma silenciosa.
- ✓ **`<<<` substitui `echo x | comando`** e evita o subshell que faria as variáveis do `read` se perderem.
- ✓ **`sudo echo x > /etc/arquivo` não funciona** — quem redireciona é o seu shell, antes do `sudo` entrar em cena. A forma correta é `sudo tee arquivo <<'EOF'`.

## NO KALI

**Gerar arquivo de configuração de ferramenta é caso de `<<'EOF'`, sem exceção.** Modelos de `proxychains`, `nuclei` e `msfconsole` são cheios de `$`, `%` e `\`: sem as aspas no delimitador o shell come metade da sintaxe e o arquivo sai **sutilmente errado** — o tipo de defeito que só se manifesta quando a ferramenta se comporta de um jeito inesperado no meio de um teste. **Subshell para isolar o ambiente de uma ferramenta:** `( export HTTP_PROXY=http://127.0.0.1:8080; export HTTPS_PROXY=$HTTP_PROXY; ferramenta )` faz o proxy valer **só naquela invocação** — sem os parênteses, o resto do script continua atravessando o proxy, inclusive comandos que você não pretendia interceptar. **No `ssh` com here-document, a pergunta "aqui ou lá?" tem consequência jurídica:** um `$(…)` que você achou que rodaria localmente e roda no remoto é **execução não pretendida numa máquina de outra pessoa** — em ambiente autorizado é um erro a corrigir; fora dele, é acesso não autorizado nos termos da Lei 12.737/2012. Teste o bloco com `cat` no lugar do `ssh` antes de disparar. **`sudo tee` para instalar unidade ou regra** — unidade do systemd, regra de `udev`, arquivo em `/etc/sudoers.d` — e sempre `visudo -cf` antes de mover. **E o custo de `fork` aparece justo na triagem de resultado:** um laço sobre dez mil linhas com três substituições por iteração são trinta mil processos; trocar `$(cat "$f")` por `$(<"$f")` e `$(basename "$f" .log)` por `${f%.log}` derruba o tempo de minutos para segundos. **Família RHEL:** subshell e heredoc são idênticos, mas em unidade do systemd com `ExecStart=` não há shell intermediário em nenhuma das duas famílias — redirecionamento e here-document escritos direto no `ExecStart=` não funcionam; aponte para um script ou use `/bin/bash -c '…'` explicitamente. E com SELinux em `enforcing`, script gerado em `/tmp` e executado dali costuma ser bloqueado por rótulo: gere em `/usr/local/bin` e rode `restorecon`.

# SINAIS, PROCESSOS FILHOS E EXECUÇÃO EM PARALELO

*O operador apertou **Ctrl-C**. O script parou de imprimir e o prompt voltou. **Trinta segundos depois, o roteador do laboratório caiu.** O script havia lançado doze processos com `&`; o `SIGINT` foi ao grupo do primeiro plano e o shell morreu, mas alguns filhos já haviam trocado de grupo e outros ignoravam `SIGINT`. O tráfego continuou saindo por meia hora até alguém rodar `pkill`. **Um script que cria processos assume a responsabilidade por eles** — "parece que parou" não é "parou".*

**075**
GUIAS LINUX

## CICLO DE VIDA DO FILHO · OS SINAIS QUE IMPORTAM · O TETO DE CONCORRÊNCIA

**PAI LANÇA** — comando &
**FILHO EXECUTANDO** — $! guarda o PID
**FILHO TERMINOU** — devolveu um status
**PAI FEZ WAIT** — status colhido

pai morreu ↓ → **ÓRFÃO**: continua rodando, adotado pelo `init` — **FORA DO ALCANCE DO SEU CTRL-C**

**ZUMBI**: terminou, mas ninguém leu o status: ocupa entrada na tabela de processos

↑ sem wait

Num laço que lança milhares de filhos sem `wait`, os zumbis **esgotam o limite de processos do usuário**. As duas curas são a mesma disciplina: **todo filho lançado tem um `wait` correspondente**, e todo script que lança filhos tem um `trap` que os mata antes de sair.

### SINAIS EM SCRIPT · INTERCEPTÁVEL?

| | | |
|---|---|---|
| `SIGINT` | 2 | sim — `Ctrl-C` no terminal |
| `SIGTERM` | 15 | sim — **o pedido educado**: `kill` sem argumento, `systemctl stop` |
| `SIGKILL` | 9 | **NÃO** — nenhum trap roda, nenhuma limpeza acontece |
| `SIGHUP` | 1 | sim — terminal fechado; recarga de configuração |
| `SIGPIPE` | 13 | sim — escreveu num pipe sem leitor |
| `SIGQUIT` | 3 | sim — `Ctrl-\` |
| `SIGCHLD` | 17 | sim — um filho terminou (padrão: ignora) |

**Morte por sinal devolve `128 + N`: 130** é Ctrl-C, **143** é `SIGTERM`, **137** é `SIGKILL`. Reconhecer esses números no log poupa investigação.

`for …; do cmd & done` — p1 p2 p3 p4 p5 p6 p7 p8 p9 p10 p11 p12 — 12 processos disputando 4 núcleos e uma placa de rede

`xargs -P 4 · wait -n` — p1 p2 p3 p4 ◀ p5 p6 p7 p8 p9 p10 p11 p12 — 4 executando · 8 na fila, entram quando um sai

**O teto certo vem do gargalo, não da sua vontade.** Trabalho de **CPU**: perto do número de núcleos (`nproc`). Trabalho de **disco** em HD mecânico: 1 ou 2 — mais que isso faz o cabeçote passar o tempo se deslocando, e fica **mais lento que sequencial**. Trabalho de **rede**: o número combinado com quem opera o alvo, e não o que a sua máquina aguenta. E a etapa final de agregação é **sempre sequencial** — é o gargalo que sobra depois de paralelizar tudo o que dava. As três formas de impor o teto: `xargs -P 4 -n 1` (nada a instalar, mas **intercala a saída** — cada processo escreve no próprio arquivo, a junção vem no fim); `parallel -j 4 --halt now,fail=1 --joblog t.log` (serializa a saída, registra, retoma o que falhou, para no primeiro erro — custa uma dependência); e a fila em Bash puro, `while (( $(jobs -rp | wc -l) ≥ MAX )); do wait -n; done`, que não depende de nada além do Bash 5.1.

## LANÇAR, COLHER O STATUS, E MATAR JUNTO

kali@kali: ~/lab-paralelo

```
# o esqueleto de qualquer paralelismo em Bash: PID → unidade
$ declare -A pid_de
$ for alvo in "${alvos[@]}"; do
    processar "$alvo" > "saida-$alvo.txt" &
    pid_de[$!]="$alvo"   # $! muda a cada &: capture já
  done
$ for pid in "${!pid_de[@]}"; do
    wait "$pid" || { echo "falhou: ${pid_de[$pid]}" >&2; ((falhas++)); }
  done
# o trap que faz o Ctrl-C parar DE VERDADE
$ encerrar() {
    trap - INT TERM      # evita reentrar no 2º Ctrl-C
    kill -TERM "${filhos[@]}" 2>/dev/null || true
    sleep 2
    kill -KILL "${filhos[@]}" 2>/dev/null || true
    wait 2>/dev/null; exit 130
  }
$ trap encerrar INT TERM
# prazo: 124 = estourou; -k mata quem ignora o TERM
$ timeout -k 5 30 varrer "$alvo"; echo $?
124
$ pgrep -af 'varrer' || echo "nenhum orfao -- correto"
nenhum orfao -- correto
```

### O TRAP QUE NÃO DISPARA

Se o script espera um **comando externo em primeiro plano**, o trap **fica pendente até esse comando retornar**: um script parado em `sleep 300` que recebe `SIGTERM` só executa o handler depois dos 300 segundos. A cura é pôr o comando longo em segundo plano e usar `wait`, que **é** interrompível — `sleep 300 & filho=$!; wait "$filho"`. É exatamente a diferença entre um serviço que para no `systemctl stop` e um que estoura o `TimeoutStopSec` e leva `SIGKILL`.

### MATAR OS FILHOS: TRÊS FORMAS

| | |
|---|---|
| `filhos=()` | **registre os PIDs e mate no trap** — previsível, resolve a esmagadora maioria dos casos. É a que se deve preferir |
| `kill -TERM -$$` | PID **negativo** = o grupo inteiro. Só funciona se o script for líder do próprio grupo; chamado de dentro de outro script, **mata o chamador junto**. Confira com `ps -o pid,pgid,comm -p $$` |
| `setsid --wait` | lança num grupo novo e mata só esse grupo — isolamento garantido, ao custo de uma camada a mais |
| `TERM → KILL` | nunca **-9** de primeira: com `SIGKILL` o filho **não remove os próprios temporários** nem fecha os arquivos |

### SOBREVIVER AO TERMINAL

| | |
|---|---|
| `nohup` | imune ao `SIGHUP` desde o lançamento |
| `disown -h` | protege um job **já lançado** |
| `setsid` | sessão própria, desligado do terminal |
| `systemd` | **a resposta certa para trabalho longo:** registro no `journald`, reinício automático e `KillMode=control-group`, que mata o cgroup inteiro — a garantia que o script sozinho não tem. `tmux` quando for preciso **voltar a interagir** |

### A ARMADILHA DO >( ) COM WAIT

O shell **não considera** o processo de `>(…)` um filho a esperar. Em `gerar | tee >(lento > r.txt) >/dev/null`, o `r.txt` pode estar vazio na linha seguinte. Quando a ordem importa, use PID explícito: `lento < <(gerar) > r.txt & pid=$!; wait "$pid"`.

**Um script que lança filhos e não tem `trap INT TERM` continua trabalhando depois que o operador mandou parar.** É o pior caso deste assunto: a tela para, o prompt volta, e o tráfego continua saindo — foi assim que o roteador da abertura caiu meia hora depois do `Ctrl-C`. Registre os PIDs ( `filhos+=("$!")` ), mate com `TERM`, espere, e só então `KILL` no que sobrou. **Cuidado redobrado com `kill -TERM -$$`** (PID negativo = grupo inteiro): se o script não for líder do próprio grupo, ele mata **o shell que o chamou e tudo o que estiver junto** — confirme com `ps -o pid,pgid,comm -p $$` antes, e prefira a lista explícita de PIDs. E **nunca use `pkill -f` com padrão curto** para limpar a bagunça: `pkill -f python` derruba serviços que nada tinham a ver. Ensaie o encerramento com `echo kill` no lugar do `kill`, em VM descartável, antes de confiar no handler.

## RESUMO RÁPIDO

- ✓ `SIGKILL` **não pode ser interceptado** — nenhum trap roda, nenhuma limpeza acontece. Sempre `TERM`, pausa, e `KILL` só no que sobrou.
- ✓ **Morte por sinal devolve `128 + N`:** 130 é Ctrl-C, 143 é `SIGTERM`, 137 é `SIGKILL`.
- ✓ **Trap não dispara enquanto um comando externo está em primeiro plano.** Ponha o comando longo em segundo plano e use `wait`, que é interrompível.
- ✓ **Capture `$!` na linha seguinte ao `&`**, porque ele muda a cada lançamento. Um array associativo de PID para unidade de trabalho é o esqueleto de qualquer paralelismo em Bash.
- ✓ `wait` **sozinho não revela qual filho falhou** — colha os status um a um se o resultado importar. Num relatório de cobertura, sempre importa.
- ✓ **Todo script que lança filhos precisa de um `trap INT TERM` que os mate.** Sem isso, o Ctrl-C para a tela e não para o trabalho.
- ✓ **Imponha teto de concorrência** com `xargs -P`, `parallel -j` ou fila com `wait -n`. O número vem do gargalo: núcleos para CPU, 1 ou 2 para disco mecânico, o combinado para rede.
- ✓ `timeout -k` **em toda ferramenta externa.** Sem `-k`, quem ignora o `TERM` sobrevive ao prazo; o status **124** identifica o estouro.
- ✓ **Para trabalho longo de verdade, uma unidade do systemd vence o `nohup`:** registro, reinício e encerramento do cgroup inteiro.

## NO KALI

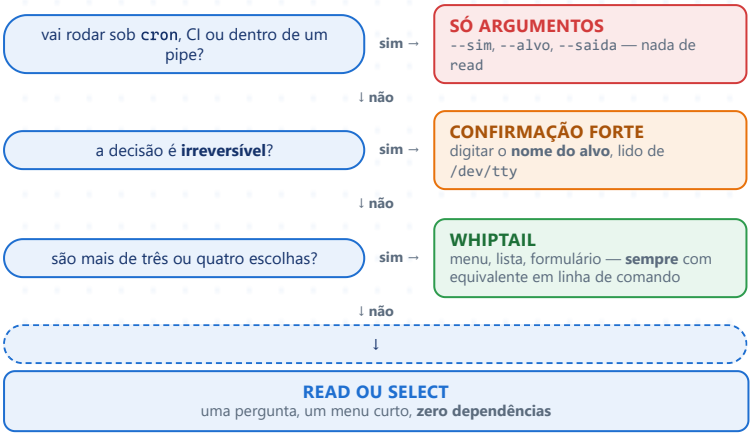
**Paralelismo contra alvo externo é decisão de contrato, não de máquina.** `-P 8` multiplica por oito o tráfego que você gera: o teto vem do que foi combinado com quem opera o alvo — taxa máxima, janela de execução, limite de conexões simultâneas. Estourá-lo por acidente é **indisponibilidade provocada**, e a autorização para testar não cobre negar o serviço. `parallel --delay 0.5` e `--jobs` explícito documentam a intenção no próprio comando. **O Ctrl-C tem de parar de verdade** — é o ponto mais importante deste assunto em contexto ofensivo: sem o `trap` que mata os filhos, o script "para" na tela e **continua atacando por baixo**, o que num teste com janela combinada coloca a atividade **fora da janela autorizada**, com o enquadramento da Lei 12.737/2012. `timeout -k` **em toda ferramenta externa:** varredura contra host que descarta pacotes fica pendurada indefinidamente, e uma fila de oito processos com quatro travados não anda — prazo explícito, `-k` para o caso de a ferramenta ignorar o `TERM`, e status 124 tratado como "sem resposta", não como "erro". **Registre PID e alvo, e confira o status um a um:** o `pid_de[$!]="$alvo"` é o que permite dizer no relatório quais alvos foram efetivamente testados e quais falharam — um `wait` sozinho devolve zero e produz relatório que **afirma cobertura que não houve**, o pior tipo de erro num teste de segurança. E **interrupção deixa evidência parcial:** ao encerrar por sinal, marque no arquivo de resultado que a execução foi interrompida e em que ponto — arquivo truncado sem marca parece completo, e alguém vai concluir que os alvos restantes estavam limpos. **Família RHEL:** sinais e controle de processos são do kernel e idênticos; a diferença é de operação — `KillMode=control-group` nas unidades do systemd mata todos os processos do cgroup no `systemctl stop`, que é justamente a garantia que o script sozinho não tem, mais um argumento para empacotar trabalho longo como serviço. E o `parallel` exige EPEL no RHEL, enquanto `xargs -P` e `timeout` estão nas duas famílias sem instalar nada.

# INTERAÇÃO: MENUS, CORES E CAIXAS DE DIÁLOGO

*O script de limpeza era cuidadoso: mostrava o tamanho e pedia `read -p "Confirma? [s/N] "` antes do `rm -rf`. Funcionou por meses — até alguém escrever `gerar_lista.sh | limpar.sh`. O `read` leu a **primeira linha da lista** em vez do teclado. Era `srv-web-01`, não bateu com `s`, e foi só sorte. **O defeito não é o `read`: é supor que existe alguém do outro lado.** Script bem escrito verifica se há interação e se comporta de forma segura quando não há.*

**076**
GUIAS LINUX

## QUE INTERFACE USAR · E A ANATOMIA DA COR




ANATOMIA DE UM ESCAPE ANSI

`0` normal · `1` negrito · `4` sublinhado
`31` vermelho · `32` verde · `33` amarelo
`34` azul · `36` ciano · `37` branco · `41`–`47` as mesmas no fundo
`\e[0m` **sempre** no fim — sem ele a cor vaza para o resto do terminal

**Prefira `tput` a escrever a sequência à mão.** `tput setaf 1`, `tput bold` e `tput sgr0` consultam o terminfo do terminal em uso e devolvem **string vazia** quando ele não suporta cor — a proteção acontece sozinha. Escrever `\e[1;31m` direto funciona em "quase" todo lugar, e esse "quase" inclui o arquivo de log, o `cron` e o terminal de quem usa leitor de tela.

**A pergunta do topo da árvore é a que mais economiza retrabalho:** se o script pode acabar num pipe ou num agendador — e quase todo script acaba —, a interatividade tem de ser **opcional**. Escreva primeiro a versão com argumentos e só depois acrescente o modo interativo que preenche os mesmos valores; o contrário, começar pelo menu e tentar automatizar depois, custa uma reescrita. **As três guardas antes de emitir cor** são o destino ser um terminal (`[[ -t 1 ]]`), a variável `NO_COLOR` não estar definida e o `TERM` não ser `dumb`. Se qualquer uma falhar, as variáveis de cor viram string vazia e **o resto do script não muda em nada** — a cor é opcional em **tempo de execução**, não em tempo de escrita. O mesmo `[[ -t 1 ]]` guarda a barra de progresso: sem ele, um log de `cron` acumula milhares de linhas de `\r`. Para progresso de um **fluxo de dados**, o `pv` já faz tudo isso sem uma linha de código.

## READ, SELECT E WHIPTAIL NA PRÁTICA

kali@kali: ~/lab-interacao

```
# leia do TERMINAL, não do stdin — e falhe alto se não houver
$ read -rp 'Confirma? [s/N] ' r < /dev/tty
$ interativo() { [[ -t 0 && -t 1 ]]; }
# sem terminal: RECUSE, não assuma. --sim libera o modo automático
$ printf 's\n' | ./seguro.sh; echo "status: $?"
erro: exige confirmacao; use --sim para modo nao interativo
status: 1
# o irreversível não aceita "s": exige o nome por extenso
$ read -rp "Digite o nome do alvo para confirmar: " c < /dev/tty
$ [[ "$c" == "$ALVO" ]] || { echo 'cancelado.' >&2; exit 1; }
# cor sob guarda tripla: desligada, tudo vira string vazia
$ if [[ -t 1 && -z "${NO_COLOR:-}" && "${TERM:-dumb}" != dumb ]]; then
    C_RESET=$(tput sgr0); C_VERM=$(tput setaf 1)
  else C_RESET='' C_VERM=''; fi
$ erro() { printf '%s[ERRO ]%s %s\n' "$C_VERM" "$C_RESET" "$*" >&2; }
# menu sem dependência nenhuma; case sobre $REPLY, não sobre o texto
$ PS3='Escolha uma opção: '
$ select o in "Listar" "Reiniciar" "Sair"; do case "$REPLY" in …
# whiptail desenha no stdout e devolve pelo stderr: troque os três
$ escolha=$(whiptail --menu "Tarefa:" 15 60 4 \
      "1" "Atualizar" "2" "Limpar" 3>&1 1>&2 2>&3); echo $?
0   # 0 confirmou · 1 cancelou · 255 pressionou Esc
```

READ: AS OPÇÕES QUE IMPORTAM

| | |
|---|---|
| `-r` | não interpreta a barra invertida — **sempre** |
| `-p 'txt'` | prompt antes de ler; sai no `stderr`, para não poluir o `stdout` que outro programa pode consumir |
| `-s` | não ecoa o digitado (senha) |
| `-n N` | retorna após N caracteres, sem esperar Enter |
| `-t N` | desiste após N segundos, status > 128 — **obrigatório** em prompt que possa travar script automatizado |
| `-e -i val` | edição de linha com valor inicial editável |
| `-a arr · -u N` | lê para um array · lê do descritor N |

WHIPTAIL: AS CAIXAS

| | |
|---|---|
| `--yesno` | pergunta; status **0** sim, **1** não, **255** Esc — tratar os três é o que evita que um Esc vire confirmação |
| `--inputbox` | uma linha de texto · `--passwordbox` sem eco |
| `--checklist` | múltipla escolha com `ON`/`OFF` |
| `--gauge` | barra alimentada por percentuais na entrada |
| `--stdout` | dispensa a ginástica `3>&1 1>&2 2>&3` nas versões recentes |

O CHECKLIST DA INTERAÇÃO

- ✓ prompt e diagnóstico em `stderr`, **resultado em `stdout`** — é o que faz `r=$(script.sh)` funcionar
- ✓ confirmação lida de `/dev/tty`, nunca do `stdin`
- ✓ **Enter vazio nunca confirma** operação destrutiva
- ✓ modo não interativo **explícito** (`--sim`, `--batch`) e recusa clara sem terminal
- ✓ toda pergunta tem equivalente em **opção de linha de comando**

**A "confirmação fantasma" é o modo mais silencioso de disparar um comando destrutivo.** Num pipe, num `cron` ou num contêiner sem TTY, `read -p "Confirma? [s/N]"` não espera ninguém: consome a próxima linha do `stdin`. Se ela for `s`, o `rm -rf`, o `mkfs` ou o `cryptsetup luksFormat` rodam **sem que ninguém tenha digitado nada**. As defesas: ler de `< /dev/tty` (que **falha** quando não há terminal, em vez de aceitar dados como resposta); `[[ -t 0 && -t 1 ]]` antes de perguntar, recusando com mensagem em vez de assumir; padrão sempre "não"; e, para o irreversível, exigir **o nome do alvo por extenso** — com tamanho e modelo mostrados na pergunta, para dar para conferir. **E nunca receba senha por argumento:** `argv` aparece no `ps` para todos os usuários da máquina e no histórico do shell. Use `read -rs`, arquivo com permissão `0600` ou variável de ambiente, nessa ordem.

## RESUMO RÁPIDO

- ✓ **`read` sem terminal lê do pipe, não do teclado.** Confirmação sempre com `< /dev/tty`, que falha alto quando não há terminal em vez de aceitar uma linha de dados como resposta.
- ✓ **`[[ -t 0 ]]` e `[[ -t 1 ]]` detectam interatividade.** Sem ela, recuse e exija uma opção explícita (`--sim`, `ASSUMIR_SIM=1`).
- ✓ **O padrão nunca é o destrutivo.** Enter vazio cancela; para o irreversível, exija o nome do alvo por extenso.
- ✓ **Prompt e diagnóstico em `stderr`, resultado em `stdout`** — é o que permite capturar a saída do script com `$( )`.
- ✓ **`select` resolve o menu simples sem dependência**, com `PS3` como prompt e `case` sobre `$REPLY`, não sobre o texto.
- ✓ **Cor é opcional em tempo de execução:** `tput` em vez de escape à mão, sob a guarda de `[[ -t 1 ]]`, `NO_COLOR` e `TERM` ≠ `dumb`. Desligada, as variáveis viram vazias e nada mais muda.
- ✓ **`whiptail` e `dialog` escrevem a interface no `stdout` e o resultado no `stderr`.** A troca `3>&1 1>&2 2>&3` (ou `--stdout`) captura a escolha; os status 0, 1 e 255 distinguem confirmar, cancelar e `Esc`.
- ✓ **Toda pergunta precisa de equivalente em opção de linha de comando.** Script que só funciona com menu não pode ser agendado, testado nem reproduzido.

## NO KALI

**Confirmação forte em tudo que é irreversível.** `cryptsetup luksFormat`, `mkfs`, `dd if=… of=/dev/sdX` e a limpeza de resultados de teste apagam o que não volta: exigir que a pessoa **digite o nome do dispositivo**, e não `s`, é a diferença entre um erro de digitação e um disco perdido — e o alvo tem de aparecer **por extenso na pergunta**, com tamanho e modelo, para que dê para conferir. **O `whiptail` é o que o `kali-tweaks` usa** — e é o mesmo motor do `dpkg-reconfigure`, então já está instalado em qualquer Debian ou Kali sem acrescentar dependência; se você escreve utilitário para a equipe, seguir a convenção deixa a experiência coerente com o resto do sistema. **Nunca peça senha com eco ligado nem a receba por argumento:** `read -rs` para digitar, arquivo com permissão `0600`, ou variável de ambiente, nessa ordem de preferência — senha em `argv` aparece no `ps` para todos os usuários e no histórico do shell, e num teste autorizado isso é **vazamento de credencial do cliente**. **Cor no relatório é armadilha:** saída de ferramenta capturada com `$( )` traz as sequências de escape junto e a comparação de string falha por um motivo invisível — desligue a cor na origem (`--no-color`, `NO_COLOR=1`) ou limpe com `sed -r 's/\x1b\[[0-9;]*m//g'` antes de processar. **E script de ataque tem de ser automatizável:** um menu bonito que exige teclado impede que a ação seja repetida de forma idêntica, e a **reprodutibilidade** é o que sustenta o relatório de um teste autorizado — se não dá para rodar o mesmo comando outra vez e obter o mesmo resultado, não dá para provar o que foi feito nem para o cliente verificar a correção depois. **Família RHEL:** o `whiptail` (pacote `newt`) costuma existir nas duas famílias, mas o utilitário de referência ali é o `dialog`, e `--gauge` e `--checklist` têm diferenças de sintaxe — detecte com `command -v whiptail || command -v dialog`. O `tput` funciona igual, com a ressalva de que o console do RHEL em modo texto pode oferecer menos cores: consultar `tput colors` antes de assumir 256 é a checagem correta.

# ESTRUTURANDO SCRIPTS GRANDES E BIBLIOTECAS

*O `deploy.sh` tinha 1.400 linhas. Ninguém o escreveu: ele **cresceu** — trinta linhas em 2021 e um trecho novo a cada incidente, sempre colado no fim, sempre por alguém com pressa. Quatro funções de log com nomes diferentes, dezenove `curl` ligeiramente distintos, e no meio uma linha que ninguém ousava tocar: `# NAO REMOVER -- quebra a producao (nao sabemos por que)`. **Cada acréscimo isolado era razoável; ninguém parou para reorganizar.***

**077**
GUIAS LINUX

## AS OITO CAMADAS DE UM SCRIPT SÉRIO · E A DIREÇÃO DA DEPENDÊNCIA

| | | |
|---|---|---|
| 1 | `shebang + cabeçalho de uso` | a ajuda sai daqui por `sed` — nunca dessincroniza |
| 2 | `set -Eeuo pipefail` `shopt -s inherit_errexit` | antes de qualquer comando que possa falhar — nem a leitura da biblioteca escapa |
| 3 | `constantes readonly` | caminhos, códigos de saída, versão. Atribuição acidental vira erro |
| 4 | `estado global mutável` | o mínimo possível: tudo o mais é `local` dentro de função |
| 5 | `source das bibliotecas` | **depois** das constantes — a biblioteca pode usá-las |
| 6 | `funções: genéricas → específicas` | uma responsabilidade cada, nome `verbo_substantivo` |
| 7 | `principal()` | só orquestra; não contém lógica de negócio |
| 8 | `guarda do ponto de entrada` | `[[ "${BASH_SOURCE[0]}" == "$0" ]]` — é o que torna o arquivo testável |

**`bin/`**
`implantar.sh`
`reverter.sh`
`coletar.sh`
**executáveis**

`source` →

**`lib/`**
`log.sh`
`http.sh`
`validacao.sh`
**só definem funções**

**`tests/`**
`log.bats` · `validacao.bats`
**carregam a lib e executam o bin com `run`**

**A dependência aponta numa direção só:** `bin/` usa `lib/`; `lib/` não conhece `bin/`. Uma biblioteca que chama função definida no executável cria um **ciclo** que impede testar qualquer um dos dois isoladamente — e é o começo do arquivo de 1.400 linhas, porque a única forma de fazer funcionar passa a ser juntar tudo.

A ordem das camadas não é estética: **cada uma depende só das anteriores.** E o `RAIZ` merece atenção própria — `RAIZ="$(cd -- "$(dirname -- "$(readlink -f -- "${BASH_SOURCE[0]}")")/.." && pwd)"`. O `BASH_SOURCE` é o caminho **do arquivo**, não o do processo, então funciona igual quando o script é chamado por caminho relativo, por link simbólico no `PATH` ou por outro script; o `readlink -f` resolve o link. Usar `$0` ou `$PWD` aqui é o defeito que só aparece quando alguém chama de outro diretório — e aí a mensagem é obscura. **As cinco propriedades de uma biblioteca bem comportada:** (1) **não executa nada** ao ser carregada — `source` que dispara ação é armadilha, porque quem carrega não espera efeito colateral; (2) protege-se contra **carregamento duplo** com `[[ -n "${_LIB_X_CARREGADA:-}" ]] && return 0`, sem o que duas bibliotecas que carregam uma terceira redefinem tudo e o `readonly` explode; (3) **prefixa os nomes** (`log_info`, `http_get`), porque o Bash tem um espaço de nomes só; (4) **não tem shebang**, e sim `# shellcheck shell=bash` para o analisador saber o dialeto; (5) é carregada por **caminho absoluto calculado**, com verificação explícita de existência.

## FUNÇÕES, BIBLIOTECAS E CONFIGURAÇÃO NA PRÁTICA

kali@kali: ~/lab-estrutura

```
# a guarda que faz o mesmo arquivo ser programa E biblioteca
$ if [[ "${BASH_SOURCE[0]}" == "${0}" ]]; then principal "$@"; fi
# a ajuda sai do próprio cabeçalho — nunca dessincroniza
$ uso() { sed -n '2,9p' "${BASH_SOURCE[0]}" | sed 's/^# \?//'; }
# carregar com mensagem clara em vez de erro obscuro do set -e
$ carregar() {
    local arq="$RAIZ/lib/$1"
    [[ -r "$arq" ]] || { printf '%s: biblioteca ausente: %s\n' \
        "$PROGRAMA" "$arq" >&2; exit 70; }
    # shellcheck source=/dev/null
    source "$arq"
  }
$ head -3 lib/log.sh
# shellcheck shell=bash
[[ -n "${_LIB_LOG_CARREGADA:-}" ]] && return 0
readonly _LIB_LOG_CARREGADA=1
# devolver ESTRUTURA sem subshell e sem serializar em texto
$ carregar_config() { local -n _ref=$1; _ref[host]='exemplo.local'; }
$ declare -A cfg; carregar_config cfg; echo "${cfg[host]}"
exemplo.local
# o teste carrega a lib e executa o bin — a seta da figura, na prática
$ bats tests/
6 tests, 0 failures
```

### FUNÇÕES: AS REGRAS

| | |
|---|---|
| `local` | **toda** variável de dentro. Sem isso o `i` do laço interno destrói o `i` do chamador, e o sintoma aparece **longe da causa**. `local -r` constante, `-a` array, `-i` inteiro |
| `_prefixo` | função interna de biblioteca — sinaliza que não é interface pública |
| `status` | devolver **sim ou não**: `eh_valido "$v" \|\| die …` |
| `stdout` | devolver **um valor**, capturado com `$( )` |
| `local -n` | devolver **estrutura**: cria referência ao array do chamador, sem `fork`. Prefixe o parâmetro (`_ref`) ou colide com o nome de fora |
| `contrato` | três linhas de comentário: assinatura, o que faz, e **quais status devolve** |

### CONFIGURAÇÃO: DA MENOR À MAIOR PRIORIDADE

1. padrões no próprio script
2. `/etc/meuprog/config`
3. `${XDG_CONFIG_HOME:-$HOME/.config}/meuprog/config`
4. variáveis de ambiente (`MEUPROG_DESTINO`)
5. **opções de linha de comando** — analisadas por último, sobrepõem tudo

**Saiba quando parar de usar shell — é o ponto que teria salvado o `deploy.sh`.** O shell é excelente em orquestrar processos, encadear ferramentas e manipular arquivos, e ruim em tudo o mais. Os sinais de que o limite foi ultrapassado: **estrutura de dados aninhada** (simular objeto dentro de lista com nomes de variáveis concatenados é o sinal mais claro de todos); **aritmética não inteira**, que obriga a chamar `bc` ou `awk` a cada operação; **tratamento de erro por item** com retentativa e registro, que sai com três vezes o tamanho do equivalente em Python; **concorrência com estado compartilhado**; **passar de 300 ou 400 linhas** sem que a maior parte seja invocação de comando externo; e **precisar de testes de unidade extensos**. A migração não é total: mantenha o shell como **casca** — analisar opções, verificar dependências, orquestrar — e delegue a lógica com `exec python3 "$RAIZ/lib/processar.py" --ambiente "$AMBIENTE"`. O `exec` **substitui** o processo do shell, então sinais e códigos de saída se propagam sem uma camada no meio. **E se você herdou o arquivo de 1.400 linhas, refatorar vence reescrever**, nesta ordem: (1) controle de versão; (2) `shellcheck`, corrigindo só a severidade alta; (3) acrescentar `set -Eeuo pipefail` e rodar — **cada quebra é um bug que já existia e estava sendo engolido**; (4) escrever um **teste de caracterização** em `bats` que registre a saída atual, correta ou não, como rede de segurança; (5) extrair funções **uma por vez**, rodando o teste a cada extração; (6) só então unificar as quatro funções de log e descobrir o que o `sleep 3` escondia (quase sempre uma condição de corrida que deveria ser um laço de espera com verificação). **Reescrever do zero não funciona:** o arquivo contém anos de correções de casos particulares que ninguém documentou, e a reescrita as perde todas — para redescobri-las uma a uma, em produção.

**`source` de arquivo de configuração é execução de código arbitrário.** Um `config` gravável por outro usuário — ou que veio do cliente por e-mail — roda **com o privilégio de quem executa o script**, no instante do `source`, e nada no arquivo precisa parecer suspeito para isso acontecer. Se a origem não é totalmente confiável, ou verifique dono e permissão antes — `[[ "$(stat -c '%U %a' "$arq")" == "root 600" ]] || die 70 "config com permissao insegura: $arq"` — ou **leia como dados**, com `jq` ou um laço de `chave=valor` validado, em vez de `source`. Pelo mesmo motivo, marque os caminhos de saída como `readonly` logo depois da análise de opções: com `SAIDA` mutável, um trecho posterior pode sobrescrevê-la e o relatório vai parar no lugar errado — ou **por cima do de outro alvo**.

## RESUMO RÁPIDO

- ✓ **Script grande tem forma reconhecível:** cabeçalho de uso, `set -Eeuo pipefail`, constantes `readonly`, estado mínimo, `source` das bibliotecas, funções, `principal()` e a guarda do ponto de entrada.
- ✓ `RAIZ` **calculado a partir de** `BASH_SOURCE`, não de `$0` nem de `pwd` — e com `readlink -f` se o script puder ser chamado por link simbólico.
- ✓ **A guarda** `[[ "${BASH_SOURCE[0]}" == "$0" ]]` **é o que torna o arquivo testável:** executado roda o programa, carregado só define as funções.
- ✓ **Toda variável de função é** `local`**.** Sem isso o `i` de dentro destrói o `i` de fora, e o sintoma aparece longe da causa.
- ✓ **Três formas de devolver valor:** status para sim ou não, `stdout` capturado para um valor, `local -n` para estrutura sem subshell.
- ✓ **Biblioteca não executa nada ao ser carregada**, protege-se contra carregamento duplo, prefixa os nomes e é carregada por caminho absoluto com verificação explícita.
- ✓ **A dependência aponta numa direção só:** `bin/` usa `lib/`, nunca o contrário. O ciclo é o começo do arquivo de 1.400 linhas.
- ✓ **Saiba quando parar:** estrutura aninhada, ponto flutuante, erro por item, mais de 300 linhas de lógica própria. A saída é o shell como casca e o motor em outra linguagem, entregue com `exec`.
- ✓ **Refatorar vence reescrever** — versão, `shellcheck`, `set -Eeuo pipefail`, teste de caracterização, extração função a função, nessa ordem.

## NO KALI

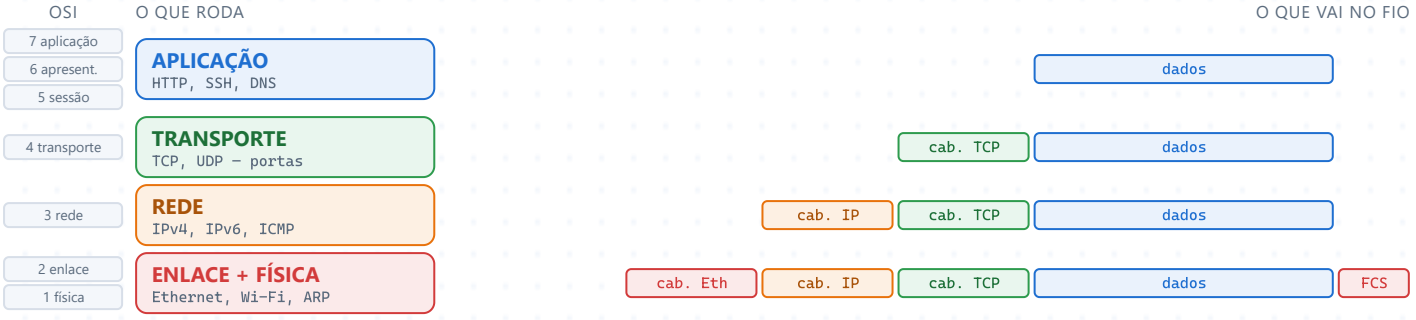
**A caixa de ferramentas da equipe é uma biblioteca, não vinte scripts com o mesmo** `log()` **copiado.** Padronizar `lib/log.sh`, `lib/escopo.sh` e `lib/relatorio.sh` faz com que a verificação de escopo e o formato do relatório sejam idênticos em todo utilitário — e uma correção na verificação passa a valer **para todos de uma vez**, em vez de ser replicada em vinte arquivos, onde inevitavelmente um fica para trás. **A validação de escopo pertence à biblioteca e é chamada por todo executável:** uma função `escopo_validar "$alvo" || die "$EX_USO" "fora do escopo"` centralizada e testada com `bats` contra casos de borda (CIDR, nome de host, IPv6) é o que evita que um utilitário novo escrito às pressas dispare contra alvo não autorizado — o que caracteriza acesso não autorizado nos termos da Lei 12.737/2012 **independentemente de haver contrato para os demais alvos.** `source` **de configuração é execução de código:** arquivo que veio do cliente por e-mail, ou que está em diretório gravável por outros, executa com o seu privilégio — verifique dono e permissão, ou leia como dados. `readonly` **nos caminhos de saída** num utilitário que grava evidência: deixar `SAIDA` mutável permite que um trecho posterior a sobrescreva e o relatório vá para o lugar errado, ou por cima do de outro alvo. **E a divisão que funciona é casca de shell, motor em Python:** correlacionar resultados de varredura, deduplicar achados e gerar relatório são exatamente o trabalho com estrutura aninhada que o shell faz mal — shell para invocar as ferramentas e organizar os arquivos, Python para ler os resultados e produzir o documento final. **Família RHEL:** a estrutura de projeto independe de distribuição, e `/usr/local/bin` + `/usr/local/lib/<programa>/` + `/etc/<programa>/` valem nas duas famílias — mas com SELinux o executável fora de `/usr/bin` precisa do rótulo correto (`restorecon -Rv /usr/local/bin`), sob pena de funcionar chamado à mão e falhar quando invocado por um serviço. Nas duas famílias, empacotar como `.deb` ou `.rpm` é preferível a copiar arquivos: o gerenciador registra o que está instalado e permite remover sem deixar resto.

# FUNDAMENTOS DE REDE: INTERFACES, IP E A PILHA

*"A internet caiu." Antes de reiniciar o roteador, três comandos:* `ping -c1 8.8.8.8` *responde em 14 ms,* `ping google.com` *devolve Temporary failure in name resolution,* `ip route` *mostra a rota padrão intacta.* ***Nada caiu*** *— quebrou a tradução de nome para endereço, uma camada acima.*

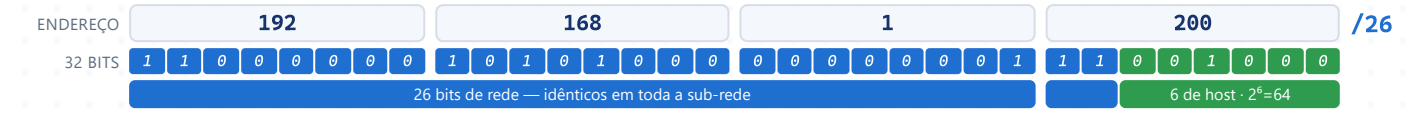
**078**
GUIAS LINUX

## A PILHA DE CAMADAS · E O QUE DE FATO VAI NO FIO




**Cada camada envelopa a de cima e acrescenta o próprio cabeçalho** — 14 (Eth) + 20 (IPv4) + 20 (TCP) = **54 bytes de overhead por pacote**. No destino, o inverso. A unidade muda de nome em cada andar (segmento, datagrama, pacote, quadro) e o quadro tem tamanho máximo — a **MTU**, 1500 em Ethernet. Pacote IP que chega mas não sobe é **porta ou firewall**, nunca cabo.

## A MÁSCARA: A ÚNICA PERGUNTA QUE ELA RESPONDE




| | | | |
|---|---|---|---|
| `192.168.1.192` | **`rede`** — bits de host em 0, não se atribui | `192.168.1.193` | **`1º válido`** — costuma ser o gateway, por convenção |
| `192.168.1.254` | **`último válido`** — 62 utilizáveis, não 64 | `192.168.1.255` | **`broadcast`** — bits de host em 1 |

**Prefixos iguais, entrega direta por ARP; diferentes, vai para o gateway.** É só isso — e por isso máscara errada produz o clássico "pingo o roteador mas não pingo o vizinho". **Cada bit a menos dobra a faixa:** /24 = 256, /25 = 128, /26 = 64, /27 = 32, /30 = 4, /32 = um. O corte não respeita octeto: `10.20.30.40/12` → rede `10.16.0.0`. **Privadas (RFC 1918):** `10.0.0.0/8`, `172.16.0.0/12` (Docker: `172.17.0.0/16`), `192.168.0.0/16`; e `169.254.0.0/16` link-local, cujo aparecimento significa **o DHCP não respondeu**. **IPv6** troca ARP e broadcast por NDP e multicast, usa `/64` na rede de acesso, exige zona no link-local (`ping fe80::1%enp0s3`) e **não tem NAT** — o firewall de host deixa de ser opcional.

## NA PRÁTICA: SEPARAR AS CAMADAS ANTES DE MEXER EM QUALQUER COISA

```
kali@kali: ~
$ ip -brief link show
lo       UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
enp0s3   UP      08:00:27:a1:b2:c3 <BROADCAST,MULTICAST,UP,LOWER_UP>
wlan0    DOWN    dc:a6:32:11:22:33 <NO-CARRIER,BROADCAST,MULTICAST,UP>
# UP = você habilitou · LOWER_UP = há sinal do outro lado
$ ip route get 1.1.1.1   # a decisão, sem enviar pacote
1.1.1.1 via 192.168.1.1 dev enp0s3 src 192.168.1.10
# responde 3 perguntas: gateway, interface e IP de ORIGEM
$ ip neigh show
192.168.1.1 dev enp0s3 lladdr c0:56:27:aa:bb:cc REACHABLE
# INCOMPLETE persistente no gateway = cabo/switch/VLAN, jamais DNS
$ ping -c1 8.8.8.8 # camadas 3 e 4 → 64 bytes ... 14 ms
$ dig +short google.com # resolução → (vazio)
# um passa, o outro falha: é DNS e só DNS
```

### ONDE O PROBLEMA MORA

| | |
|---|---|
| `camada 1–2` | **`ip link`**, **`ip neigh`** — falta **`LOWER_UP`** ou ARP **`INCOMPLETE`**. O OUI do MAC entrega o virtualizador |
| `camada 3` | **`ip route get`** — **prefixo mais longo vence**; **`default`** = **`0.0.0.0/0`**, o mais curto, logo o último recurso. **`metric`** desempata |
| `camada 4` | **`ss -tulpn`** — a **5-tupla** identifica a conexão; por isso mil abas falam com a mesma :443 |
| `nomes` | **`/etc/nsswitch.conf`** manda: **`files`** (o **`/etc/hosts`**, sempre vencedor) antes de **`dns`** (**`/etc/resolv.conf`**, **gerado**) |

### TCP × UDP

| | |
|---|---|
| **TCP** | handshake, entrega confirmada e ordenada, 20 B — HTTP, SSH |
| **UDP** | envia e esquece, sem ordem nem garantia, 8 B — DNS, DHCP, QUIC |
| porta | < 1024 exige privilégio; o nº é **convenção**, nada põe SSH na 22 |

## RESUMO RÁPIDO

- ✓ **A rede é uma pilha de camadas independentes.** `ping` num IP testa 3 e 4; `dig` testa nomes. Falham por motivos sem relação — identifique o andar **antes** de mexer.
- ✓ `UP` **sem** `LOWER_UP` é placa habilitada sem sinal do outro lado: cabo solto ou Wi-Fi não associado.
- ✓ **A máscara responde a uma pergunta só:** o destino está na minha sub-rede? Se sim, ARP e entrega direta; se não, gateway.
- ✓ **Prefixo mais longo vence** na tabela de rotas. `ip route get` mostra gateway, interface e IP de origem sem enviar pacote.
- ✓ **ARP não tem autenticação** — por isso filtro por MAC não é controle de acesso, é inventário.

## NO KALI

**O modo de rede da VM decide o que o laboratório consegue fazer.** Em **NAT** a VM alcança a internet mas ninguém alcança a VM — mata todo exercício com conexão de retorno; em **bridge** ela pega IP do DHCP real e fica visível para todos, inclusive para quem você não pretendia; **host-only** é o modo certo. `ip route get 1.1.1.1` com gateway `10.0.2.2` é o NAT do VirtualBox. **Kali não sobe serviço de rede, de propósito** — `ss -tulpn` em instalação limpa devolve quase nada, e o que estiver ali você habilitou. **Modo monitor não é interface IP:** a placa recebe todo quadro 802.11 sem se associar e derruba a conexão — esperado, não defeito. **MAC muda em um comando** ( `ip link set enp0s3 address …` ): serve para testar o *seu* controle de acesso; contornar o de rede alheia é crime (Lei 12.737/2012). **Privado ≠ protegido:** dentro do perímetro raramente há firewall entre pares. **RHEL:** mesmo `iproute2` , mas `rp_filter` estrito (descarta rota assimétrica em silêncio) e `firewalld` ativo de fábrica.

# CONFIGURAÇÃO DE REDE: IP, NETWORKMANAGER E ARQUIVOS

*O servidor precisava de IP fixo. Dois comandos — `ip addr add` e `ip route add` — e funcionou na hora. Depois da queda de energia, o IP voltou a ser o do DHCP; alguém anotou os comandos num arquivo "rodar isso depois de todo boot". **Esse arquivo é o sintoma:** configurar e persistir são operações separadas.*

**079**
GUIAS LINUX

## QUEM REALMENTE MANDA NA INTERFACE — E POR QUE A DE CIMA SEMPRE VENCE

**GERENCIADOR DE REDE**
`NetworkManager · systemd-networkd · ifupdown`

**Persistente.** Lê perfis em disco e os **reaplica**: no boot, ao conectar o cabo, ao renovar o DHCP, ao acordar da suspensão. No Kali é o NetworkManager: um `.nmconnection` por perfil em `/etc/NetworkManager/system-connections/`, modo `600`.

↓ aplica no boot e a cada evento de link

**COMANDO IP**
`altera o estado agora, não grava nada`

**Efêmero.** Escreve direto no kernel. Some no reboot — e some **antes disso**, na próxima vez que o gerenciador agir sobre a interface. Lugar legítimo: teste, diagnóstico, recuperação, contêiner sem gerenciador.

↓ netlink

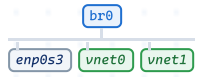
**KERNEL**
`/proc/net · /sys/class/net`

**O estado real** — é o que `ip addr` lê. Não guarda **intenção**: não sabe dizer se um endereço veio de perfil, de DHCP ou da sua mão. Por isso "quem configurou isto?" não se responde olhando o kernel.

**A regra que evita o problema:** configurou com `ip` numa máquina com gerenciador ativo? A configuração **tem prazo de validade** — ou marque a interface como não gerenciada, ou mude pelo gerenciador. Quem manda sai de `nmcli device status`: `connected` = o NM assumiu e reaplica o perfil quando quiser; `unmanaged` = e só aí o manual sobrevive.

## INTERFACES VIRTUAIS: TODAS NASCEM DE `IP LINK ADD`, TODAS SÃO EFÊMERAS

### BRIDGE




```
ip link add br0 type bridge
```

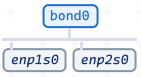
Switch em software: placa física e VMs no mesmo segmento.

### VLAN

```
ip link add link enp0s3 name
enp0s3.10 type vlan id 10
```

Uma placa, várias redes lógicas marcadas com 802.1Q.

### BOND




```
ip link add bond0 type bond mode
active-backup
```

Duas placas, um endereço: redundância ou soma de banda.

### VETH + NETNS

```
ip netns add lab; ip link set veth1
netns lab
```

Pilha de rede própria — rotas e firewall separados. É o que o Docker usa por baixo.

**Todas somem no reboot** — para persistir, `nmcli connection add type bridge|vlan|bond`. Faltam `dummy` e `tun/tap` (VPN).

## NA PRÁTICA: O IP FIXO QUE SOBREVIVE AO BOOT

kali@kali: ~

```
$ nmcli device status
enp0s3  ethernet  connected  Wired connection 1
$ sudo nmcli con add type ethernet con-name lab-fixo \
    ifname enp0s3 ipv4.method manual ipv4.gateway 10.0.0.1 \
    ipv4.addresses 10.0.0.50/24 ipv4.dns "1.1.1.1 9.9.9.9" \
    ipv6.method disabled connection.autoconnect yes
$ sudo nmcli con up lab-fixo   # só agora vale
$ sudo nmcli con mod lab-fixo ipv4.method auto \
    ipv4.gateway "" ipv4.addresses "" ipv4.dns ""   # "" APAGA
$ ls -l /etc/resolv.conf   # quem regrava o DNS?
... -> /run/systemd/resolve/stub-resolv.conf
# 127.0.0.53 = stub do resolved: servidor real em `resolvectl
# status`, e POR INTERFACE — daí a VPN quebrar só alguns nomes
```

### OS CINCO DETALHES QUE MORDEM

1. **`ip addr add` soma, não troca** — use `ip addr replace`
2. **Máscara não é opcional:** sem `/24` vira `/32` e a máquina não fala nem com o vizinho
3. **`ipv4.method manual` substitui** o DHCP; com `auto` o endereço é somado ao dele
4. **`connection.autoconnect yes`** faz subir no boot — sem ele o perfil existe e "não pega"
5. editar o `.nmconnection` à mão exige `con reload`

### NET-TOOLS → IPROUTE2

| | |
|---|---|
| `ifconfig · route -n` | `ip addr · ip route` |
| `arp -a · netstat -tulpn` | `ip neigh · ss -tulpn` |

**Mover o IP da placa para uma bridge derruba a conexão no instante em que o endereço sai da placa** — numa máquina remota, é ficar trancado do lado de fora. Faça pelo console, ou num script com rollback agendado por `at`. Idem `ip addr flush dev enp0s3`, que remove **todos** os endereços e mata o SSH em curso; e `chattr +i /etc/resolv.conf`, que funciona mas **quebra o gerenciador em silêncio**.

## RESUMO RÁPIDO

- ✓ **Configurar e persistir são operações distintas:** `ip` fala com o kernel e é efêmero; persistir é do gerenciador.
- ✓ **Com NetworkManager ativo, o manual tem prazo** — e o prazo não é o reboot, é o próximo toque na interface.
- ✓ **`ip addr add` soma e IP sem máscara vira `/32`:** os dois erros mais frequentes.
- ✓ **`ipv4.method manual` + `autoconnect yes`** faz o IP fixo pegar e subir no boot.
- ✓ **NM e `ifupdown` na mesma interface** = máquina que se reconfigura sozinha, com cara de defeito de hardware.

## NO KALI

**Namespace de rede é o laboratório mais barato que existe.** Dois `ip netns` ligados por um par `veth` reproduzem cliente, servidor e roteador numa máquina só, com firewall independente, sem VM e **sem risco de tocar em rede alheia** — o ambiente certo para praticar firewall e captura. **Modo monitor não é configuração de rede:** a placa perde IP e o NM tenta reassumi-la — daí o `airmon-ng check kill`; depois, `airmon-ng stop wlan0mon` e `systemctl restart NetworkManager` devolvem o normal. **MAC aleatório é recurso do próprio NM** (`wifi.cloned-mac-address random|stable|permanent`): privacidade legítima em Wi-Fi público — usá-lo para burlar controle de acesso alheio é acesso não autorizado (Lei 12.737/2012). **Perfil guarda segredo em claro:** os `.nmconnection` têm PSK de Wi-Fi e credencial de VPN protegidos só pelo modo `600` — em máquina emprestada, entram na limpeza junto com `~/.ssh`. **RHEL:** o 9 removeu os `ifcfg-*`; script antigo falha em silêncio.

# DIAGNÓSTICO DE REDE: PING, TRACEROUTE, SS, DIG E MTR

*"A rede está lenta" é o chamado mais comum e o mais inútil: pode ser o cabo, o DNS que demora dois segundos, 3% de perda num salto do meio — ou o disco do banco.* ***O que separa cinco minutos de uma tarde perdida é ter um método****: perguntas em ordem fixa, cada uma isolando uma camada.*

**080**
GUIAS LINUX

## A ESCADA: SEIS PERGUNTAS EM ORDEM FIXA

| | | | |
|---|---|---|---|
| 1 | o link existe? | `ip -br link show`<br>`sudo ethtool enp0s3` | Cabo, porta do switch, Wi-Fi não associado. **Nada acima disso importa.** `Duplex: Half` ou `100Mb/s` em placa gigabit = autonegociação ou par rompido. |
| 2 | tenho endereço e rota? | `ip -br addr show`<br>`ip route get 1.1.1.1` | DHCP, perfil do gerenciador, máscara errada, rota ausente. Um IP `169.254.x.x` **é o DHCP que não respondeu**. |
| 3 | alcanço o gateway? | `ping -c3 "$(ip route \| awk '/^default/{print $3}')"` | Camada 2 local — ARP, VLAN, filtro no switch. Confirme com `ip neigh`: `INCOMPLETE` no gateway fecha o caso. |
| 4 | alcanço a internet por IP? | `ping -c3 1.1.1.1`<br>`mtr -rwc50 1.1.1.1` | Roteamento, NAT, firewall de saída, MTU. É onde entram o `traceroute` e o `mtr`. |
| 5 | resolvo nomes? | `dig +short exemplo.com`<br>`resolvectl query exemplo.com` | Falhou aqui — e **só** aqui. IP funciona, nome não: é o `resolv.conf`, e nenhum reboot de roteador resolve. |
| 6 | a porta responde? | `ss -tulpn`<br>`nc -zv exemplo.com 443` | Serviço parado, escutando no endereço errado, firewall de destino, TLS. |

**A regra que faz o método funcionar: não pule degraus e não volte.** Cada teste que passa elimina **definitivamente** tudo o que está abaixo — quem começa pelo degrau 6 ("o site não abre") investiga TLS num problema de cabo.

## LER A SAÍDA: LATÊNCIA, JITTER E A PERDA QUE NÃO É PERDA

kali@kali: ~

```
$ ping -c4 1.1.1.1
64 bytes from 1.1.1.1: icmp_seq=1 ttl=57 time=12.4 ms
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max/mdev = 12.1/12.9/13.8/0.6 ms
$ mtr -rwc50 1.1.1.1
HOST                 Loss%  Snt  Last   Avg  Best  Wrst StDev
1. 192.168.1.1        0.0%   50   1.2   1.4   1.0   3.1   0.3
2. 100.64.0.1         0.0%   50  11.8  12.5  11.2  22.4   1.8
3. core-rj.exemplo 18.0%   50  13.1  13.9  12.4  28.7   2.6
4. 1.1.1.1            0.0%   50  12.9  13.2  12.6  15.1   0.4
# perda num salto que SOME nos seguintes NÃO é perda: é rate-
# limit de ICMP. Perda real vai até a ÚLTIMA linha.
```

### AS QUATRO LEITURAS DO PING

| | |
|---|---|
| `time` | RTT ida e volta: LAN < 1 ms · mesmo país 5–30 ms · intercontinental 100–250 ms |
| `mdev` | o **jitter**. Alto com média baixa arruína VoIP sem afetar download nenhum — é o "a internet está boa mas a chamada trava" |
| `ttl` | pista de distância: sistemas partem de 64, 128 ou 255; **`ttl=57`** sugere 7 saltos |
| `loss` | **1–2% já degrada TCP**: cada perda dispara retransmissão e corta a janela |

`ping` **mudo não prova host fora do ar** — bloquear ICMP é padrão em borda e em nuvem. E responder não prova aplicação de pé: é camada 3, e só.

## DEGRAU 6: QUEM ESCUTA, QUEM RESPONDE E O BURACO NEGRO DE MTU

kali@kali: ~

```
$ ss -tulpn
tcp LISTEN 127.0.0.1:5432  users:(("postgres",pid=812))
tcp LISTEN 0.0.0.0:22      users:(("sshd",pid=744))
# 127.0.0.1 = SÓ loopback: nenhum remoto conecta, por mais
# liberado que o firewall esteja.
$ dig exemplo.com   # `ss -tin` dá rtt/cwnd/retrans
;; ->>HEADER<<- opcode: QUERY, status: NOERROR
exemplo.com.  280 IN A 93.184.215.14
;; Query time: 24 msec ;; SERVER: 127.0.0.53#53
$ diff <(dig +short site) <(dig @1.1.1.1 +short site)
# difere: cache velho ou interceptação. Igual: não é o DNS.
```

### O QUE O DIG RESPONDE

| | |
|---|---|
| `status` | `NOERROR` sem resposta = domínio existe, esse registro não · `NXDOMAIN` = não existe · `SERVFAIL` = servidor ou DNSSEC |
| `TTL` | o número caindo a cada consulta prova que veio do **cache** |

### ESTADOS QUE DENUNCIAM

| | |
|---|---|
| `TIME-WAIT` | aos milhares: a aplicação abre **conexão nova por requisição** |
| `CLOSE-WAIT` | acumulando: **bug na aplicação**, que não chama `close()`. Nenhum ajuste de kernel conserta |
| `recusada` | chegou e ninguém escuta — **o host está vivo** |
| `esgotada` | nada respondeu: firewall descartando em silêncio ou rota quebrada |

**O buraco negro de MTU atravessa a escada inteira sem ser pego por degrau nenhum:** o `ping` passa, o DNS resolve, o TCP abre — e a transferência trava quando os pacotes crescem (o SSH conecta e congela ao listar um diretório). Algum enlace aceita menos que 1500 bytes e o ICMP que avisaria está bloqueado. Teste: `ping -M do -s 1472 alvo` falha e `-s 1400` passa.

## RESUMO RÁPIDO

- ✓ **O método é a escada:** link → endereço e rota → gateway → IP → nomes → porta. Não pule degraus e não volte.
- ✓ `ping` **mede camada 3 e só:** mudo não é host morto; respondendo não é aplicação no ar.
- ✓ **No** `mtr`**, perda que some depois é rate-limit de ICMP** — vale a última linha.
- ✓ `127.0.0.1:porta` **no** `ss` explica metade dos "roda mas não conecto"; `CLOSE-WAIT` acumulando é bug da aplicação, não da rede.
- ✓ **Recusada × esgotada:** recusada = host vivo sem quem escute; esgotada = firewall calado ou rota quebrada.

## NO KALI

**Diagnóstico e reconhecimento são separados pela autorização, não pela ferramenta.** Os mesmos `ping`, `dig` e `nc -z` contra a própria infraestrutura são rotina e, contra faixas de terceiros em volume, são a etapa inicial de acesso não autorizado (Lei 12.737/2012). **Ping bloqueado não é host fora do ar — e isso define levantamento:** varredura ICMP acha só uma fração dos hosts vivos, porque servidor Windows e nuvem não respondem por padrão; é para isso que existe o `nmap -Pn`. `dig` **é o reconhecimento mais educado que existe:** `NS`, `MX` e `TXT` leem dados publicados de propósito, sem tocar o alvo — já a transferência de zona (`dig axfr`) entrega o domínio inteiro: contra a própria zona é auditoria, contra zona alheia é acesso indevido. **Meça o caminho antes de varrer:** sobre enlace com 5% de perda o `nmap` reporta como fechadas portas abertas — o SYN se perdeu e o silêncio virou "filtrada". **RHEL:** `dig` vem em `bind-utils`, `mtr` costuma faltar, e o `firewalld` ativo dá timeout num `nc -zv` externo com o serviço no ar.

# ACESSO REMOTO: SSH, SCP E RSYNC

*Um servidor ficou exposto com SSH na porta 22 e senha habilitada. No primeiro dia,* `journalctl -u ssh | grep -c "Failed password"` *devolveu* ***41027*** *— centenas de endereços testando* `root`, `admin`, `git`. *Nenhuma acertou, mas a conta é simples:* ***bastava uma****.*

**081**
GUIAS LINUX

## O SEGREDO QUE TRAFEGA × O SEGREDO QUE FICA PARADO

### AUTENTICAÇÃO POR SENHA

CLIENTE → a senha atravessa o canal cifrado → SERVIDOR (compara com o hash)

**Qualquer um** pode tentar adivinhar, indefinidamente.

### AUTENTICAÇÃO POR CHAVE PÚBLICA

CLIENTE (`~/.ssh/id_ed25519` — **privada — nunca sai daqui**)

← 1. desafio aleatório

2. assinatura do desafio →

SERVIDOR (`~/.ssh/authorized_keys` — pública — pode ser divulgada)

3. O servidor verifica a assinatura com a chave pública. Confere? Entra.

**Por que isso encerra a força bruta:** não há segredo compartilhado para adivinhar — o atacante teria de **forjar uma assinatura** sem a chave privada. **Antes disso vem o TOFU:** na 1ª conexão o SSH mostra a impressão digital e a grava em `known_hosts`; digitar `yes` sem conferir por outro canal (`ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`) abre a janela do ataque de intermediário.

## OS TRÊS TÚNEIS: `-L` PUXA, `-R` EMPURRA, `-D` ABRE A REDE

**-L local: trago um serviço remoto para cá**
`ssh -L 8080:127.0.0.1:80 servidor`

sua máquina `:8080` — túnel SSH — servidor SSH — conexão local — serviço `127.0.0.1:80`

**-R remoto: exponho um serviço meu lá**
`ssh -R 9000:127.0.0.1:3000 servidor`

serviço local `127.0.0.1:3000` — túnel SSH — servidor SSH `:9000` — conecta na 9000 — quem usa o servidor

**-D dinâmico: proxy SOCKS pela rede do servidor**
`ssh -D 1080 servidor`

navegador `SOCKS :1080` — túnel SSH — servidor SSH — sai pelo servidor — qualquer destino

**A porta que você digita primeiro é sempre a que passa a existir**; o `host:porta` seguinte é resolvido **na perspectiva do outro lado** — daí `127.0.0.1:80` em `-L` ser o loopback *do servidor*, não o seu. O `-R` atravessa NAT e firewall de entrada e por padrão só escuta no loopback remoto: `GatewayPorts yes` abre para a rede — e aí você publicou um serviço interno. Túnel sem sessão: `ssh -fN -L …`.

## NA PRÁTICA: CHAVE, CONFIG, ENDURECIMENTO E SINCRONIZAÇÃO

kali@kali: ~

```
$ ssh-keygen -t ed25519 -C "vitor@notebook-2026"
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub vitor@servidor
# ele ajusta a permissão — causa nº1 de "copiei e ainda pede
# senha": o sshd recusa em SILÊNCIO; o motivo sai em `ssh -vvv`
$ cat ~/.ssh/config
Host lab
    HostName 10.0.0.50
    IdentityFile ~/.ssh/id_ed25519_lab
    IdentitiesOnly yes    # ofereça SÓ esta chave
Host interno
    ProxyJump lab         # substituto seguro do -A
$ sudo sshd -t && sudo systemctl reload ssh
# PasswordAuthentication no + PermitRootLogin no = 41027 → 0
```

### RSYNC: AS OPÇÕES DE TODO COMANDO

| | |
|---|---|
| `-a` | modo arquivo: recursivo, preserva permissão, dono, tempo e links |
| `-z` · `-P` | comprime · barra de progresso e **retomada** do interrompido |
| `-n` | **ensaio**: mostra o que faria, sem fazer |
| `--delete` | apaga no destino o que sumiu da origem |
| `--link-dest` | backup incremental com hard links: cada dia parece cópia completa, só o novo ocupa espaço |

### POR QUE RSYNC E NÃO SCP

| | |
|---|---|
| **diferença** | transfere **só o que mudou**: 10 GB com 1 MB alterado movem ~1 MB |
| **scp** | aposentado (CVE-2018-20685); sem os `:` copia local→local **em silêncio** |

**A barra final na origem muda tudo:** `rsync -a /origem/ /destino/` copia o **conteúdo**; sem a barra, cria `/destino/origem/`. Com `--delete`, errar a barra **apaga o conteúdo legítimo do destino** — rode com `-n` antes. E **nunca feche a sessão atual depois de mexer no** `sshd_config`: o `sshd -t` pega sintaxe, não lógica — um `AllowUsers` com typo tranca você para fora.

## RESUMO RÁPIDO

- ✓ **A chave privada nunca trafega:** o servidor manda um desafio, o cliente assina, o servidor confere com a pública.
- ✓ **Permissão errada = "ainda pede senha":** `700` no `~/.ssh`, `600` na chave e no `authorized_keys`. A recusa é silenciosa.
- ✓ `IdentitiesOnly yes` evita a desconexão por excesso de tentativas quando o agente oferece todas as chaves.
- ✓ `ProxyJump` **substitui o** `-A`**:** quem tem root no salto se apropria do agente encaminhado.
- ✓ **Trocar a porta 22 é conforto de log, não segurança;** o que protege é chave em vez de senha.

## NO KALI

O `ssh` **do Kali vem desabilitado de propósito.** Habilitá-lo numa máquina que vive em Wi-Fi público, sem antes desligar a senha, é o cenário da abertura. E em imagens antigas as **chaves de host vinham idênticas em toda instalação**: `rm /etc/ssh/ssh_host_* && dpkg-reconfigure openssh-server` é passo obrigatório em imagem que você não gerou. `ProxyJump` **é a forma limpa de trabalhar em rede segmentada:** com um salto acordado em contrato, `ssh -J salto interno` chega à rede interna **sem deixar chave nem agente no intermediário**. `-D 1080` + `proxychains` **é o pivô mais simples que existe** — e o ponto em que "alcancei um host" vira "alcancei a rede dele": atravessar para sub-rede fora do escopo ultrapassa a autorização (Lei 12.737/2012). **Chave privada é o segredo mais valioso da máquina:** use *passphrase*, prefira `ssh-add -t`, e trate `authorized_keys` como inventário — quase sempre há uma chave que devia ter saído há meses. `rsync` **é a exfiltração mais discreta que existe**, na porta 22, no meio do tráfego administrativo. **RHEL:** o serviço é `sshd` e trocar a porta exige `semanage port` (SELinux).

# HTTP NA LINHA DE COMANDO: CURL, WGET E NETCAT

*Um webhook parou. A aplicação jurava enviar; o parceiro jurava não receber. Dois dias de e-mail até alguém rodar `curl -v` e ver um **301** apontando para o endpoint novo — que a biblioteca HTTP, por padrão, **não segue em POST**. **O navegador esconde tudo; na linha de comando você vê o protocolo cru.***

**082**
GUIAS LINUX

## TRÊS FERRAMENTAS, TRÊS TERRITÓRIOS — E A SAÍDA QUE RESOLVE O CASO

**curl**
DIAGNÓSTICO E API
Fala uma requisição e **mostra tudo**: qualquer método, cabeçalho e corpo. Interagir com API é aqui.

**wget**
DOWNLOAD E ESPELHO
Grava em arquivo por padrão, **retoma** o que caiu (`-c`) e insiste sozinho. ISO de 4 GB é aqui.

**nc**
CANO DE TCP/UDP
Não entende protocolo nenhum: move bytes entre o socket e o terminal. "Tem alguém nesta porta?" é aqui.

```
$ curl -v -X POST https://api.parceiro.com/webhook \
    -H 'Content-Type: application/json' -d '{"evento":"teste"}'
*  Connected to api.parceiro.com · TLS handshake ok
>  POST /webhook HTTP/2
>  content-type: application/json
<  HTTP/2 301
<  location: https://api.parceiro.com/v2/webhook
```

`*` nota do curl: DNS, TCP, TLS, reuso de conexão  `>` linha **enviada** — a sua requisição  `<` linha **recebida** — a resposta do servidor

## CURL: AS OPÇÕES QUE APARECEM EM TODO COMANDO

```
kali@kali: ~
$ curl -s -H 'Authorization: Bearer TOKEN' \
    https://api.exemplo.com/v1/itens | jq -r '.[].nome'
$ curl -s -X POST https://api.exemplo.com/v1/itens \
    -H 'Content-Type: application/json' \
    --data @corpo.json
# -d implica POST e tipo FORMULÁRIO: sem o Content-Type
# explícito, a API recebe JSON rotulado errado e rejeita.
# -d remove quebras de linha; --data-binary @arq preserva.
$ curl -s -o /dev/null -w 'dns:%{time_namelookup} \
    tls:%{time_appconnect} ttfb:%{time_starttransfer}\n' URL
# o mtr do HTTP: ttfb alto com o resto baixo = o servidor
# está demorando para processar, e não a rede.
```

### O BÁSICO

| | |
|---|---|
| `-sS` | silencioso, **mas ainda mostra erro** — o par certo em script |
| `-L` | segue redirecionamento (o que faltava no webhook) |
| `-I` · `-o` · `-O` | só cabeçalhos · grava com o nome que você der · com o nome remoto |
| `-m` · `--retry` | timeout total · tenta de novo em falha transitória |

### A PEGADINHA QUE QUEBRA SCRIPT

| | |
|---|---|
| `-f` | **Sem ele, o curl sai com 0 mesmo num 404 ou 500** — receber a resposta já é sucesso para ele. Sob `set -e`, um `500` passa como se tivesse dado certo. O `-f` faz o status HTTP virar status de saída: **obrigatório em automação** |

## NETCAT: O MESMO BINÁRIO ESCUTA E CONECTA

### MODO CLIENTE — CONECTA A UMA PORTA

```
nc -zv exemplo.com 443 · nc -v smtp.exemplo.com 25
```

seu teclado e sua tela → stdin/stdout → nc cliente → TCP → serviço remoto :443

### MODO SERVIDOR — ESCUTA NUMA PORTA

```
nc -lvnp 9000 > recebido.tar
```

quem conecta → bytes que chegam → nc -l à escuta → stdout → seu terminal ou um arquivo

**Como o `nc` não formata nada, você fala o protocolo à mão:** `printf 'GET / HTTP/1.1\r\nHost: exemplo.com\r\n\r\n' | nc exemplo.com 80`. E `-z` não envia dado: abre, confirma e fecha.

### WGET: BAIXAR E ESPELHAR

| | |
|---|---|
| `-c` | continua o download interrompido — a razão de usar `wget` para arquivo grande |
| `--mirror` | com `--convert-links --page-requisites --no-parent`: site offline sem subir na árvore por acidente |
| `--wait=1` | com `--random-wait`: espelhar sem parecer — nem funcionar como — negação de serviço |

### TLS SEM O CADEADO

| | |
|---|---|
| `s_client` | `echo \| openssl s_client -connect host:443 -servername host \| openssl x509 -noout -subject -dates` |
| `-servername` | **obrigatório com SNI**: sem ele você inspeciona o certificado errado e diagnostica um problema que não existe |

`nc` **escutando com** `-e /bin/bash` **entrega um shell a quem conectar** — é o *reverse shell* canônico, e por isso a `netcat-openbsd` do Kali **removeu o** `-e`. Use só em laboratório próprio e isolado; o `nc` não tem cifra nem autenticação. E `curl -k` **é marcador temporário, nunca configuração**: em produção, apaga o alerta de intermediário que você quer ouvir.

## RESUMO RÁPIDO

- ✓ **O `-v` mostra requisição ( `>` ) e resposta ( `<` ) lado a lado** — é o que transforma "não funciona" em "responde 301 para cá".
- ✓ `curl` **inspeciona e automatiza API;** `wget` **baixa e espelha;** `nc` **é o cano de TCP.**
- ✓ `curl` **sai com 0 mesmo em 404 e 500:** `-f` é obrigatório em automação, e `-sS` mantém a saída limpa mostrando só erro.
- ✓ `-d` **implica POST e formato de formulário** — para JSON, o `Content-Type` explícito não é opcional.
- ✓ `-w` **decompõe a latência** em DNS, TCP, TLS e primeiro byte, sem instalar nada.
- ✓ **O mesmo** `nc` **escuta e conecta** — simetria que serve ao diagnóstico e ao shell reverso.

## NO KALI

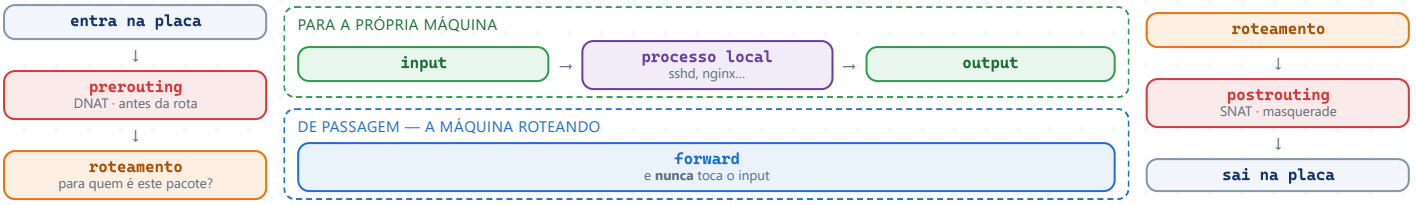
`curl` e `wget` **são o motor de metade das ferramentas ofensivas, e a mesma requisição muda de natureza pelo alvo:** buscar a release de uma ferramenta é trivial; automatizar milhares de requisições contra servidor alheio para mapear endpoints é ataque a aplicação web, com o enquadramento da Lei 12.737/2012 quando não há autorização. O `-A` (User-Agent) e os cookies ( `-b` / `-c` ) mantêm sessão num teste — e denunciam automação mal disfarçada num log. `nc` **é a ferramenta didática do shell reverso, e entendê-la é defesa:** o padrão a detectar é uma conexão de **saída**, iniciada pelo próprio servidor, para porta alta incomum — por isso firewall de saída importa tanto quanto o de entrada. ***Banner grabbing*** **é o reconhecimento mais barato e educado** ( `nc -v host porta` colhe o que o serviço publica de propósito); vira problema ao escalar para faixas inteiras. Do lado defensivo, reduzir banners ( `ServerTokens Prod` ) diminui o que um levantamento colhe de graça. `socat` **é o** `nc` **com esteroides:** túnel cifrado e *relay* — pivô limpo num teste autorizado, e o mesmo canal por onde dados saem quando o firewall de saída deixa. **RHEL:** o `nc` costuma ser o `ncat` do Nmap, com o `-e` atrás de `--sh-exec`.

# FIREWALL NO LINUX: NFTABLES E IPTABLES

*O administrador digitou de memória `iptables -A INPUT -p tcp --dport 80 -j ACCEPT` e a porta continuou fechada. A primeira regra da cadeia, posta meses antes por outra pessoa, era um `-j DROP` genérico: a liberação entrou **depois** do descarte, num ponto que o pacote nunca alcançava.*

**083**
GUIAS LINUX

## UM MOTOR (NETFILTER), E O CAMINHO QUE O PACOTE FAZ POR ELE




**A decisão de rota parte o fluxo em dois, e é ela que determina qual gancho o pacote vê** — o erro clássico de quem monta um gateway ou VPN é escrever a regra em `input` quando o pacote está em `forward`. **Um motor, duas interfaces:** o `iptables` (1998) é o que você vai **encontrar** em sistema legado e em tutorial; o `nftables` é o que você vai **escrever** — e no Kali atual o próprio `iptables` já é um invólucro que fala `nftables` por baixo.

## A ORDEM DECIDE · E O ESTADO É O QUE TORNA O FIREWALL USÁVEL

1. `-j DROP` — posta meses antes por outra pessoa — **o pacote para aqui**
2. `-p tcp --dport 80 -j ACCEPT` — acrescentada com **`-A`**, ao fim: **nunca alcançada**

**Lista ordenada, avaliada de cima para baixo, e a primeira que casa decide.** Não há "a regra mais específica vence": há **posição na fila**. Quase todo erro de firewall é erro de ordem, não de sintaxe — e o diagnóstico é uma linha: `iptables -L INPUT -n -v --line-numbers`. A coluna `pkts` é ouro: regra que devia pegar tráfego e mostra **0 packets** não está sendo alcançada.

### ESTADO DA CONEXÃO (CONNTRACK)

| | |
|---|---|
| `new` | o primeiro pacote, tentando abrir conexão — **é só isto que você precisa decidir** |
| `established` | pertence a uma conexão já aceita: **a resposta vem de graça** |
| `related` | nova, mas ligada a outra (erro ICMP, canal de dados do FTP) |
| `invalid` | não casa com estado nenhum — quase sempre lixo, a descartar |

**Sem estado, permitir navegação exigiria abrir todas as portas altas por onde as respostas voltam** — impossível de prever, e abriria a máquina inteira. E `iif lo accept` não é opcional: incontáveis serviços conversam consigo mesmos por `127.0.0.1`.

## NA PRÁTICA: A BASE QUE TODA POLÍTICA SENSATA TEM

kali@kali: /etc/nftables.conf

```
flush ruleset   # após o #!/usr/sbin/nft -f; idempotente
table inet filter {    # inet = IPv4 e IPv6 juntos
  chain input {
    type filter hook input priority 0; policy drop;
    ct state established,related accept
    ct state invalid drop
    iif lo accept
    ip protocol icmp accept
    ip6 nexthdr icmpv6 accept   # NDP é essencial
    tcp dport 22 ct state new limit rate 4/minute accept
    tcp dport { 80, 443 } ct state new accept
  }
  chain forward { type filter hook forward priority 0; policy drop; }
}
$ sudo nft -f /etc/nftables.conf   # troca TUDO, atômico
$ sudo systemctl enable nftables   # e só isto persiste
```

### IPTABLES: ONDE CADA OPÇÃO MORDE

| | |
|---|---|
| `-A` | **A**crescenta ao fim — vai **depois** de um `DROP` existente e nunca é alcançada |
| `-I [n]` | **I**nsere no topo (ou na posição **n**): a forma de pôr uma regra antes das outras |
| `-P` | define a **política padrão** da cadeia |
| `-F` | esvazia a cadeia — com política `DROP`, corta todo o acesso na hora |

### GATEWAY: ENCAMINHAR E MASCARAR

```
1  sysctl -w net.ipv4.ip_forward=1
2  nft add rule ip nat postrouting oifname "enp0s3" masquerade
```

`masquerade` — o SNAT que o roteador doméstico faz

**Política padrão `DROP` na cadeia `INPUT` corta a sessão SSH atual no mesmo instante**, se a regra que libera o SSH ainda não estiver no lugar. Em máquina remota a ordem é inegociável: insira `established,related` e o SSH, **confirme numa segunda conexão**, e só então mude a política. Melhor ainda, agende o resgate antes: `echo 'nft flush ruleset' | sudo at now + 5 minutes` — `atrm` cancela se tudo der certo. **E persistir é passo separado, sempre.**

## RESUMO RÁPIDO

- ✓ **Lista ordenada, primeira que casa decide.** Não existe "mais específica vence" — existe posição na fila.
- ✓ **A decisão de rota escolhe o gancho:** para a máquina é `input`, de passagem é `forward`, e um nunca vê o outro.
- ✓ **Com `conntrack` você só decide o que é `new`** — a resposta entra por `established,related`. E `iif lo accept` não é opcional.
- ✓ **`-A` acrescenta ao fim, `-I` insere no topo** — e `-L -n -v --line-numbers` mostra a ordem real e os contadores.
- ✓ **Regra na memória some no reboot:** `nftables.service` ou `iptables-persistent`.

## NO KALI

**O Kali não vem com firewall carregado, de propósito:** `nft list ruleset` numa instalação nova volta vazio e as cadeias do `iptables` estão em `ACCEPT`. Não é falha — uma máquina de teste precisa que o tráfego flua sem obstáculo para não confundir "o firewall bloqueou" com "o alvo não respondeu". A contrapartida é que **a proteção é responsabilidade sua. Firewall de saída é o que contém o shell reverso, e quase ninguém configura:** todo mundo filtra o que entra, poucos filtram o que sai — mas o padrão de um comprometimento é uma conexão de **saída**, iniciada pela máquina invadida, para porta alta num servidor de controle. Uma política `output` restrita transforma isso numa tentativa frustrada e visível no log: melhor custo-benefício, e a mais negligenciada. `conntrack -L` **é análise, não só firewall:** lista o que o kernel rastreia agora — mais difícil de falsear que o `ss`, porque vive no kernel. **Limitar conexão nova contém força bruta na origem** (`limit rate 4/minute`), antes de o `sshd` gastar um ciclo. **Regra escrita não é regra testada:** dispare `nc -zv` e `ping` de um namespace e confira os contadores. **RHEL:** o `firewalld` vem ativo e restritivo, com **zonas** e `firewall-cmd --add-service=https --permanent` — mexer no `nft` por baixo dele gera conflito.

# CAPTURA DE TRÁFEGO: TCPDUMP E TSHARK

*O log da aplicação dizia "falha na requisição"; o da API não registrava requisição alguma.* ***As duas equipes estavam certas*** *— o problema estava no fio. Um* `tcpdump` *mostrou três* `[S]` *saindo e nenhum* `[S.]` *voltando:* ***quando os logs de ponta discordam, o tráfego é o árbitro que não mente.***

**084**
GUIAS LINUX

## O QUE A CAPTURA MOSTRA — E A FRONTEIRA EXATA ONDE O TLS COMEÇA

| | |
|---|---|
| `cab. Ethernet` — MAC de origem e destino | VISÍVEL |
| `cab. IP` — **quem fala com quem**, endereços de origem e destino | VISÍVEL |
| `cab. TCP` — portas, flags, número de sequência | VISÍVEL |
| `TLS ClientHello` — **SNI: o nome do site visitado**, ainda em claro | VISÍVEL metadado |
| `dados HTTP` — URL, cabeçalhos, cookies, corpo, senha | CIFRADO |

**O TLS protege o conteúdo, não a existência da conversa.** Sem quebrar cifra nenhuma, uma captura revela **quem** falou com **quem**, **quando**, **quanto** e — pelo SNI — **qual site**. Para mapear infraestrutura, detectar exfiltração ou identificar comunicação com um servidor de controle, o metadado basta. Conteúdo em claro só aparece em protocolo sem TLS (HTTP simples, FTP, Telnet, DNS tradicional) — daí o valor num diagnóstico e o risco numa rede hostil.

## LER AS FLAGS: O HANDSHAKE E OS DOIS PADRÕES DE FALHA

`[S]` SYN · abrindo | `[S.]` SYN-ACK · aceitando | `[.]` ACK · confirmação | `[P.]` PSH-ACK · dado entregue | `[F.]` FIN-ACK · fecha ordenado | `[R]` RST · recusa ou aborto

| | |
|---|---|
| **SAUDÁVEL** | `[S]` → `[S.]` → `[.]` — o handshake de três vias completo. Depois vem `[P.]` (dados) e `[F.]` no fim. Handshake completo seguido de silêncio já **não é problema de rede**: é a aplicação. |
| **SÓ SYN, REPETIDO** | `[S]` `[S]` `[S]` sem nenhum `[S.]` — **ninguém respondeu**. Firewall descartando em silêncio ou destino fora do ar. Foi o caso da abertura. |
| **RESET IMEDIATO** | `[S]` → `[R.]` — **recusa ativa**: a máquina está viva e respondeu dizendo que ninguém escuta naquela porta. Silêncio é firewall; `[R]` é porta fechada em host vivo. |

## TCPDUMP CAPTURA, TSHARK ANALISA

kali@kali: ~

```
$ sudo tcpdump -i any -n 'host 10.0.0.80 and port 443'
10.0.0.20.52014 > 10.0.0.80.443: Flags [S], seq 998...
10.0.0.20.52014 > 10.0.0.80.443: Flags [S], seq 998...
10.0.0.20.52014 > 10.0.0.80.443: Flags [S], seq 998...
# -n NÃO resolve nomes: a resolução reversa gera tráfego DNS
# que polui a própria captura e a deixa lentíssima.
$ sudo tcpdump -i any -n -w caso.pcap host 10.0.0.80
# grava o tráfego bruto; analise depois, com calma
$ tshark -r caso.pcap -Y 'http.request.method == "POST"' \
    -T fields -e ip.src -e http.host -e http.request.uri
$ tshark -r caso.pcap -q -z conv,tcp   # conversas
$ tshark -r caso.pcap -q -z io,phs     # protocolos
# capturar no servidor e ver no desktop, num cano só:
$ ssh srv 'sudo tcpdump -i any -U -w - -n not port 22' \
    | wireshark -k -i -
```

BPF: O FILTRO QUE SALVA A CAPTURA

| | |
|---|---|
| eixos | `host`/`net` · `src`/`dst` · `tcp`/`udp` `port`, combinados com **`and`**, **`or`**, **`not`** |
| exemplo | `'tcp and not port 22'` — tudo menos o seu próprio SSH |
| só SYN | `'tcp[tcpflags] & (tcp-syn\|tcp-ack) == tcp-syn'`: quem tenta conectar aonde |
| só RST | `'tcp[tcpflags] & tcp-rst != 0'`: conexões recusadas |

**Filtro de captura (BPF) e filtro de exibição são linguagens diferentes.** O BPF (`tcp port 443`, no `tcpdump` e no `tshark -f`) decide o que **entra** na captura, e roda no kernel — barato mesmo sob carga. O de exibição (`http.request.method == "POST"`, no `-Y`) filtra o que **já foi** capturado e é muito mais expressivo. Trocar um pelo outro é o erro mais comum, e a mensagem de erro não deixa óbvio qual foi.

**Capture apenas em rede própria ou com autorização escrita.** Em rede de terceiros, o mesmo comando é interceptação de comunicação alheia — com o enquadramento da Lei 12.737/2012 e da legislação de sigilo das comunicações. A diferença entre auditoria e crime não está no binário, está na autorização. **E um `.pcap` é dado sensível de altíssimo valor:** tráfego sem TLS carrega senha, cookie de sessão e conteúdo privado em claro. Trate-o como um dump de credenciais — `umask 077`, cifra em repouso, `shred -u` ao descartar. Um `.pcap` esquecido em `/tmp` é vazamento.

## RESUMO RÁPIDO

- ✓ **Quando os logs das duas pontas discordam, a captura é o árbitro** — ela mostra o que de fato passou no fio.
- ✓ **O TLS esconde o conteúdo, não a conversa:** endereços, portas, tamanhos, tempos e o SNI ficam visíveis.
- ✓ `[S]` **sem** `[S.]` **é firewall ou host morto;** `[R]` **é recusa ativa** num host vivo. Reconhecer os dois resolve a maioria dos casos.
- ✓ `-n` **sempre:** resolver nomes polui a captura com o próprio DNS e a deixa lentíssima.
- ✓ `tcpdump -w` **captura, Wireshark/** `tshark` **analisa** — e `-T fields -e` transforma a captura numa fonte tabular para `awk` e `sort`.
- ✓ **Filtro estreito desde o início:** arquivo pequeno, análise focada e nada capturado além do necessário.

## NO KALI

**A captura é reconhecimento passivo puro:** em rede autorizada, o `tcpdump` mapeia hosts, serviços e padrões de comunicação **sem enviar um único pacote** — invisível para o alvo. A legalidade é definida pela rede, não pela ferramenta. **Modo promíscuo não vence a comutação:** numa rede com switch a interface só vê o que é dela e o broadcast, então capturar o tráfego de outra máquina exige espelhamento de porta, um *tap* ou um ataque ativo de camada 2 (ARP spoofing) — que é intrusivo e já não é passivo. Em Wi-Fi aberto o modo monitor vê tudo do ar; em Wi-Fi cifrado, só metadados de rádio. **A captura é a prova contra o host que mente:** um `ss` ou um `iptables -L` rodados numa máquina comprometida podem ter sido adulterados por um rootkit de espaço de usuário — a captura feita **de fora** vê o tráfego real, incluindo a conexão de saída para o servidor de controle que o host esconde. É por isso que investigação séria de comprometimento não confia só no que a máquina relata sobre si. **Filtro estreito é discrição, não só conveniência:** capturar tudo num engajamento gera arquivo gigante cheio de dados de terceiros possivelmente fora do escopo — um `host <alvo> and port <serviço>` coleta só o que o contrato permite. **RHEL:** o `tshark` vem em `wireshark-cli`, a captura sem root é gerida pelo grupo `wireshark`, e o SELinux pode negar a escrita do `.pcap` fora de `/tmp` ou do home sem dizer que é SELinux.

# HARDENING DO LINUX: SUPERFÍCIE DE ATAQUE

*Kernel do dia anterior, nenhum CVE aberto, nenhum dia zero — e a máquina foi invadida assim mesmo. O que havia era um banco subido para um teste em janeiro, escutando em `0.0.0.0` com a senha da documentação.* ***Ninguém quebrou nada: entraram pela porta que ninguém lembrava de ter deixado aberta.***

**085**
GUIAS LINUX

## AS CINCO CAMADAS — E O COMANDO QUE MEDE CADA UMA

**REDE** — portas em escuta
quem alcança a máquina sem credencial nenhuma
`ss -tulpn`

**SERVIÇOS** — daemons ativos
cada um é código rodando e uma configuração a errar
`systemctl list-units --state=running`

**CONTAS** — shell utilizável, chaves, sudo
quem já está dentro, e até onde consegue ir
`getent passwd`
`cat /etc/sudoers.d/*`

**BINÁRIOS PRIVILEGIADOS** — SUID, SGID, capabilities
o caminho de usuário local até root
`find / -perm -4000`
`getcap -r /`

**KERNEL** — módulos, `/proc`, `/sys`, syscalls
a última fronteira; falha aqui derruba tudo acima
`lsmod`
`sysctl -a`

**Cada camada só é alcançável depois da anterior — reduzir a de cima vale mais:** serviço que não roda não tem CVE que importe, nem patch, nem perfil a escrever. E a diferença que decidiu o caso da abertura: `127.0.0.1:5432` e `0.0.0.0:5432` é o mesmo banco, o mesmo firewall, duas superfícies opostas — **e é opção do serviço, não do firewall**.

## CINCO PRINCÍPIOS QUE SUBSTITUEM QUALQUER LISTA DE RECEITAS

### MENOR PRIVILÉGIO
Cada processo recebe só o que precisa. Não é desconfiar de gente — é limitar o estrago. `nginx` como `www-data` ou como `root` servem a mesma página; a diferença aparece no dia em que ele é explorado.

### DEFESA EM PROFUNDIDADE
A pergunta útil não é "esta medida impede o ataque?", e sim **"QUANDO ELA FALHAR, O QUE AINDA ESTÁ DE PÉ?"**
— camadas independentes, nunca alternativas entre si.

### FALHAR FECHADO
Diante de erro ou ambiguidade, negue: a política `drop`, a regra de `sudo` que lista o que *pode*, o serviço que recusa configuração inválida.

### REDUZIR ANTES DE PROTEGER
Software não instalado não precisa ser configurado, atualizado, monitorado nem confinado. Antes de escrever um perfil, pergunte se o serviço precisa existir — a resposta é "não" com frequência surpreendente.

### AUDITABILIDADE
Controle que não deixa registro é controle que você não sabe se funciona — e cuja falha só se descobre quando alguém avisa de fora.

### O PADRÃO DE FÁBRICA NÃO É O PADRÃO SEGURO
Distribuições otimizam para "funciona na primeira tentativa", não para "resiste a um adversário". É uma escolha legítima delas e um problema seu.

## MEDIR, ENDURECER — E PAGAR O PREÇO COM CONSCIÊNCIA

```
kali@kali: ~
# 1. o retrato ANTES — sem medição, hardening é fé
$ sudo ss -tulpn | grep -v '127.0.0.1\|::1'
tcp LISTEN 0 128 0.0.0.0:5432 (("postgres"...
#    ^ o esquecido de janeiro, exposto à rede inteira
$ systemctl list-unit-files --state=enabled
$ sudo find / -xdev -perm -4000 -type f | sort
# 2. desligar é sempre melhor do que configurar
$ sudo systemctl disable --now cups avahi-daemon
$ sudo systemctl mask bluetooth   # nem por dep.
# 3. kernel — /etc/sysctl.d/99-hardening.conf
kernel.kptr_restrict = 2     kernel.dmesg_restrict = 1
kernel.yama.ptrace_scope = 1 fs.suid_dumpable = 0
$ sudo sysctl --system
# 4. SSH: validar ANTES de recarregar. Sempre.
$ sudo sshd -t && sudo systemctl reload ssh
$ sudo lynis audit system --quick # o placar
```

| MEDIDA | O QUE ELA COBRA |
|---|---|
| `PasswordAuthentication no` | perder a chave privada é perder o acesso |
| política `drop` no firewall | cada serviço novo exige regra explícita |
| `noexec` em `/tmp` | instaladores que usam `/tmp` param |
| `ptrace_scope = 1` | `gdb` e `strace` passam a exigir `sudo` |
| confinamento estrito | perfil mal ajustado quebra o serviço de forma opaca |

**Hardening genérico não existe** — só para uma máquina com propósito declarado. Ignorar o custo produz o servidor "seguro" que ninguém administra e que acumula gambiarras que anulam tudo.

`sudo sshd -t` **antes de todo** `reload`**, sem exceção** — erro de sintaxe faz o serviço recusar-se a subir, e numa máquina remota isso é perder o acesso em definitivo. **Mantenha a sessão aberta**, abra uma **segunda** para testar, e só feche a primeira quando a segunda funcionar. E `PasswordAuthentication no` sem chave pública instalada **e testada** tranca você para fora na hora.

## RESUMO RÁPIDO

- ✓ **A maioria das invasões explora configuração, não vulnerabilidade:** o serviço esquecido em `0.0.0.0` com senha padrão, não o dia zero.
- ✓ **Superfície de ataque é lista finita e enumerável** — rede, serviços, contas, binários privilegiados, kernel — e cada camada tem seu comando.
- ✓ **Fechar a camada de cima vale mais que corrigir tudo abaixo.** Trabalhe do `ss -tulpn` para baixo.
- ✓ **Cinco princípios decidem o que a lista não previu:** menor privilégio, defesa em profundidade, falhar fechado, reduzir antes de proteger, auditabilidade.
- ✓ **Atualizar é a medida de maior retorno** (no Kali, `full-upgrade`). E guarde o "antes": relatório honesto é inventário inicial, final e o `diff`.

## NO KALI

**O Kali é deliberadamente o oposto de um sistema endurecido, e por bom motivo:** ele hospeda ferramentas feitas para gerar tráfego incomum, abrir sockets crus, injetar pacotes e carregar módulos — firewall restritivo, confinamento estrito e `ptrace_scope` alto atrapalhariam isso. Endurecer aqui é **decisão sua, consciente e reversível. Mas ele não é o descuidado que a fama sugere:** desde 2020 usa o usuário não privilegiado `kali` em vez de `root`, e **nenhum serviço de rede sobe habilitado** — `ssh`, `postgresql` e `apache2` vêm desativados, e um `ss -tulpn` novo sai quase vazio. O risco real é o `systemctl enable ssh` que você deu num laboratório há três meses e nunca desfez. `kali-tweaks` **centraliza as decisões:** a seção *Hardening* configura o SSH deste capítulo e inclui *aceitar algoritmos legados* — as duas direções a um menu de distância. **VM clonada herda a chave de host do SSH**, e máquinas com a mesma identidade anulam a checagem de *fingerprint*: `sudo rm /etc/ssh/ssh_host_*` e `dpkg-reconfigure openssh-server`. **Fechar o ciclo:** `nmap -sV -p- 127.0.0.1` e uma varredura de outra VM sua mostram a superfície **como um atacante a vê** — a máquina que você endureceu é o único alvo cuja autorização é indiscutível; contra outro é crime (Lei 12.737/2012). **RHEL** já parte de `firewalld` ativo e SELinux *enforcing*, e troca o `lynis` pelo **OpenSCAP**.

# GERENCIAMENTO DE SEGREDOS: CHAVES SSH E GPG

*Um robô que varre o GitHub achou, num commit de dois anos antes, uma `deploy_key` privada sem senha. O commit fora revertido no dia seguinte — mas **reverter não apaga o objeto do histórico**, e a chave ficou recuperável por 730 dias. Ninguém rotacionou nada porque ninguém sabia que havia algo a rotacionar.*

**086**
GUIAS LINUX

## POR QUE A CHAVE VENCE A SENHA: A PRIVADA ASSINA, E NUNCA TRAFEGA

**CLIENTE `~/.ssh/id_ed25519`**
a chave **PRIVADA**, protegida por senha — nunca sai desta máquina

**SERVIDOR `~/.ssh/authorized_keys`**
guarda **SÓ A PÚBLICA**; comprometê-lo não entrega sua credencial

| | | |
|---|---|---|
| 1 | O cliente anuncia **qual** chave pública pretende usar. | cliente → |
| 2 | O servidor confere se ela consta do **`authorized_keys`**. | ← servidor |
| 3 | O servidor devolve um **desafio aleatório**, novo a cada conexão. | ← servidor |
| 4 | O cliente **assina o desafio** com a privada — e envia a assinatura, não a chave. | cliente → |
| 5 | O servidor verifica a assinatura com a pública. **Sessão aberta.** | servidor |

**Tudo muda no passo 4.** A senha precisa ser *entregue* para ser verificada: quem verifica passa a conhecê-la, ela trafega e é reutilizável em outro serviço. A privada só **assina**, e é a assinatura que viaja — o servidor nunca vê o segredo, e um desafio capturado não serve para a conexão seguinte.

## DOIS FORMATOS, DOIS PROBLEMAS DIFERENTES

### SSH — AUTENTICAR UMA SESSÃO

"sou eu que estou conectando, agora"

- **`ssh-keygen -t ed25519 -C "vitor@estacao"`** é o padrão hoje: curta, rápida, sem parâmetro para errar. RSA 4096 só para servidor velho; **DSA nunca**.
- **Senha na chave não é opcional** — sem ela o arquivo *é* o acesso. O **`ssh-agent`** pede uma vez por sessão; **`ssh-add -t 3600`** faz ele esquecer.
- **Chave de automação vem restrita:** **`restrict`** desliga tudo, **`command=`** fixa o programa, **`from=`** fixa a origem. Mesmo vazada, faz uma coisa só.
- **O comentário `-C` é para o futuro:** um dia você olha um **`authorized_keys`** de nove linhas e precisa decidir quais apagar.

### GPG — PROVAR PROCEDÊNCIA DO DADO

"este arquivo é o que o projeto publicou"

- **Primária × subchaves:** a primária (**`sec`**) é a identidade e fica offline; as subchaves (**`ssb`**) trabalham e são revogáveis uma a uma — uma comprometida não derruba sua rede de confiança.
- **O primeiro comando depois de gerar a chave** é **`gpg --gen-revoke`**: o certificado de revogação não pode ser criado *depois* que a chave se perdeu.
- **`Good signature` prova que o dado não mudou** — não que a chave é de quem você pensa. O aviso *"not certified"* é o passo que falta.
- **Confira a impressão digital por canal independente.** Chave e assinatura vindas do mesmo servidor protegem contra corrupção, não contra adversário.

## NA PRÁTICA — E ONDE O SEGREDO FICA GUARDADO

kali@kali: ~

```
$ ssh-keygen -t ed25519 -C "vitor@estacao-2026"
$ ssh-copy-id -i ~/.ssh/id_ed25519.pub vitor@srv
# ~/.ssh 700 e authorized_keys 600, ou o servidor IGNORA
# em silêncio: a causa nº 1 de "ainda pede senha"
$ ssh -v vitor@srv 2>&1 | grep -i 'offering\|accepted'
$ ssh-add -t 3600 ~/.ssh/id_ed25519 # esquece em 1h
# ~/.ssh/config — resolve metade dos problemas
Host interno
    ProxyJump bastion      # salta sem expor agente
    IdentityFile ~/.ssh/id_infra
    IdentitiesOnly yes     # não oferece TODAS as chaves
# authorized_keys: credencial de menor privilégio
restrict,command="/usr/local/bin/receber",
from="10.0.5.0/24" ssh-ed25519 AAAAC3Nza... deploy@ci
$ gpg --armor --detach-sign pacote.tar.gz
$ gpg --verify pacote.tar.gz.asc pacote.tar.gz
```

| ONDE GUARDAR | O QUE ISSO CUSTA |
|---|---|
| variável de ambiente | vaza em `/proc/PID/environ` e em `ps e` |
| arquivo `0600` no disco | texto claro se o disco não for cifrado |
| `pass` (GPG + Git) | versionável: o conteúdo já é cifrado |
| cofre de rede (Vault) | token curto, rotação e auditoria — mas é infra |
| **dentro do repositório Git** | **nunca, em hipótese alguma** |

**Segredo commitado é segredo público** — reverter não apaga o objeto, alcançável pelo hash e em todo clone. Rotacione **primeiro**; limpar o histórico vem depois, e o **`gitleaks`** vem antes.

**`ForwardAgent yes` entrega suas chaves ao `root` da máquina do meio.** quem controla o servidor intermediário usa o socket do seu agente enquanto a sessão está aberta e autentica-se **como você** em todo servidor que aceita sua chave. Para saltar por um *bastion*, use `ProxyJump` ( `ssh -J bastion vitor@interno` ): a conexão vai de ponta a ponta a partir da sua máquina e nenhum agente fica exposto no caminho.

## RESUMO RÁPIDO

- ✓ **A privada assina um desafio e nunca sai da sua máquina** — comprometer o servidor não entrega a credencial. É a diferença estrutural para a senha.
- ✓ **`ed25519` , com senha e com comentário identificável.** Chave sem senha é acesso imediato para quem copiar o arquivo.
- ✓ **`~/.ssh` em 700, `authorized_keys` em 600** — fora disso o servidor ignora o arquivo sem dizer nada.
- ✓ **`ForwardAgent` não; `ProxyJump` sim.** E `IdentitiesOnly yes` evita o "too many authentication failures".
- ✓ **`Good signature` ≠ chave confiável** — falta conferir a impressão digital por canal independente.
- ✓ **Gere o certificado de revogação no minuto seguinte** ao de gerar a chave GPG — depois de perdê-la, ele já não pode ser criado.

## NO KALI

**Verificar a ISO do Kali é o exercício de GPG mais útil que existe**, porque é real: baixe `SHA256SUMS` e `SHA256SUMS.gpg` , importe a chave do projeto, **confira a impressão digital contra o site oficial por HTTPS** e só então verifique a assinatura das somas. Cada elo omitido é uma confiança assumida sem perceber — espelhos são operados por terceiros. **A chave do repositório mantém o `apt` honesto:** todo `apt update` confere a assinatura do `Release` contra o `kali-archive-keyring` ; quando ela expira ( `NO_PUBKEY` , `EXPKEYSIG` ) a tentação universal é `--allow-unauthenticated` ou `[trusted=yes]` , e isso **desliga a única proteção contra um espelho hostil** — reinstale o chaveiro da fonte oficial. **Senha fraca de chave não resiste às *wordlists*:** o formato antigo do OpenSSH derivava a chave com pouquíssimas iterações, e `ssh-keygen -p -o -a 100 -f ~/.ssh/id_rsa` reencripta sem trocar o par — vale em toda chave anterior a 2018. **A máquina Kali acumula credencial de terceiro** — capturas, *hashes*, chaves de laboratório —, o que a torna alvo de valor desproporcional: disco cifrado e senha na chave GPG são requisitos práticos, não zelo. Material de teste autorizado tem cláusula de custódia; perdê-lo é incidente contratual. **Chave de laboratório não entra em produção**, nem o contrário: pares por contexto ( `id_lab` , `id_infra` ) com `IdentitiesOnly yes` eliminam a categoria inteira de erro. **RHEL:** mesmos comandos, mas o SELinux exige o contexto `ssh_home_t` — `authorized_keys` copiado de outro lugar é ignorado com permissões perfeitas; `restorecon -Rv ~/.ssh` conserta.

# CAPABILITIES, ISOLAMENTO E MENOR PRIVILÉGIO

*Por décadas o `ping` foi SUID: rodava **inteiro** como `root` — com direito a ler `/etc/shadow`, matar qualquer processo e montar sistemas de arquivos — só para mandar 64 bytes de ICMP. Hoje ele não tem SUID nenhum: tem `cap_net_raw`, e mais nada. **Menor privilégio deixou de ser conselho e virou engenharia.***

**087**
GUIAS LINUX

## O PODER MONOLÍTICO DE ROOT, QUEBRADO EM PEÇAS NOMEADAS

**UID 0**
uma única verificação no kernel inteiro:
**pode tudo**
. Sem gradação entre "pode um pouco" e "pode tudo".

→

| | |
|---|---|
| `CAP_NET_RAW` | socket cru e pacote forjado — `ping`, `tcpdump`, `nmap -sS` |
| `CAP_NET_BIND_SERVICE` | escutar em porta abaixo de 1024 **sem ser `root`** |
| `CAP_CHOWN` | trocar o dono de um arquivo — instaladores |
| `CAP_KILL` | sinalizar processo de outro dono — supervisores |
| `CAP_SYS_ADMIN` | montar, criar *namespace*, dezenas de outras — **equivale a root** |

**Cada peça é verificada de forma independente:** o `ping` tem `CAP_NET_RAW` e nada mais, então uma falha nele rende forjar pacotes — não a máquina. **Mas a quebra não é uniforme:** `CAP_SYS_ADMIN` acumulou tantas operações que concedê-la é conceder `root` por um caminho mais longo. Capability só reduz privilégio **quando a peça é pequena**.

## A LISTA QUE VALE LER ANTES DE CONCEDER QUALQUER COISA

### PEÇAS PEQUENAS — REDUÇÃO REAL

fazem uma coisa, e a falha para nelas

| | |
|---|---|
| `cap_net_raw` | socket cru: `ping`, `dumpcap`, `mtr-packet` |
| `cap_net_bind_service` | porta < 1024 sem root |
| `cap_net_admin` | interfaces, rotas, firewall (`ip`, `nft`, VPN) |
| `cap_chown` | dono de arquivo alheio — **com cautela: reescreve a rota até `root`** |
| `cap_kill` | sinalizar processo de outro usuário |

### PEÇAS QUE SÃO ROOT COM OUTRO NOME

conceder é entregar a máquina, mais devagar

| | |
|---|---|
| `cap_sys_admin` | montar, *namespaces*, dezenas de operações |
| `cap_sys_module` | carregar módulo de kernel = **controle total** |
| `cap_dac_override` | ignora **toda** checagem de permissão: `/etc/shadow`, chaves privadas |
| `cap_setuid / setgid` | assumir outra identidade — escalada direta |
| `cap_sys_ptrace` | ler a memória de qualquer processo |

## NA PRÁTICA: LER, CONCEDER — E DEIXAR O SYSTEMD CONFINAR

kali@kali: ~

```
$ getcap /usr/bin/ping
/usr/bin/ping cap_net_raw=ep
$ sudo getcap -r / 2>/dev/null # o inventário inteiro
# lista curta e explicável, ou há pergunta a fazer
$ capsh --print | head -3        # do processo atual
$ capsh --decode=0000000000003000 # hex → nomes
$ sudo setcap cap_net_bind_service=+ep ./meuservidor
# e/p/i = efetivo (ativo agora) / permitido (o teto) /
# herdável. O conjunto LIMITANTE é o teto absoluto:
$ sudo capsh --drop=cap_net_raw -- -c 'ping -c1 1.1.1.1'
ping: socktype: SOCK_RAW  (falha até sob sudo)
# no_new_privs: nem eu nem meus filhos ganham privilégio
$ setpriv --no-new-privs sudo id
sudo: efetivo uid não é 0 — a escalada por SUID acabou
$ systemd-analyze security          # placar geral
$ systemd-analyze security meuservico.service
```

### O SANDBOX QUE CABE NUM ARQUIVO DE UNIDADE

```
[Service]
User=meuservico
NoNewPrivileges=yes            # fecha o SUID
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
AmbientCapabilities=CAP_NET_BIND_SERVICE
ProtectSystem=strict           # tudo só-leitura
ProtectHome=yes                # /home e /root somem
PrivateTmp=yes                 # /tmp só dele
StateDirectory=meuservico      # o único gravável
ProtectKernelTunables=yes      # /proc/sys travado
ProtectKernelModules=yes       # não carrega módulo
PrivateDevices=yes             # só /dev/null e cia.
ProtectProc=invisible          # não vê outro PID
RestrictNamespaces=yes
MemoryDenyWriteExecute=yes
SystemCallFilter=@system-service
```

> Por baixo são *namespaces*, cgroups e seccomp — os mecanismos do contêiner, **em forma declarativa e sem tocar no código**. **Aperte o parafuso aos poucos, com `journalctl -fu` aberto:** essas diretivas quebram serviço de forma opaca.

**Nunca conceda capability (nem SUID, nem regra de `sudo`) a um interpretador.** Um `setcap cap_setuid=+ep /usr/bin/python3` transforma o Python numa máquina de escalada de duas linhas: `import os; os.setuid(0); os.system("/bin/bash")` — e qualquer usuário vira `root`. Vale para `perl`, `node`, `gdb` e todo programa com escape para shell: `NOPASSWD: /usr/bin/find` parece restrito e não é, porque `find . -exec /bin/sh \;` é um shell como `root`. **Privilégio dado a quem executa código arbitrário é privilégio irrestrito** — metade da escalada local que existe cabe nessa frase.

## RESUMO RÁPIDO

- ✓ **O kernel quebrou `root` em ~40 peças verificadas independentemente.** O `ping` moderno tem `cap_net_raw` e mais nada.
- ✓ **A quebra não é uniforme:** `cap_sys_admin`, `cap_sys_module`, `cap_dac_override` e `cap_setuid` são `root` por caminho mais longo.
- ✓ **Capability vai em binário que faz uma coisa só**, com entrada controlada — nunca em interpretador.
- ✓ **`no_new_privs` apaga a escalada por SUID** em toda a árvore de processos, e é de mão única. É a base de todo sandbox moderno.
- ✓ **Copiar um arquivo não copia sua capability** — nem o bit SUID. Bom lembrete ao empacotar ou restaurar backup.
- ✓ **`systemd-analyze security` dá nota a cada serviço** e converte "deveríamos endurecer isso" em lista priorizada.

## NO KALI

**O Kali é um museu vivo de capability aplicada certo**, porque as ferramentas de rede precisam de privilégio e não podem ser SUID: `sudo getcap -r /usr` mostra o `dumpcap` com `cap_net_raw,cap_net_admin=eip` — a peça que deixa o Wireshark capturar com o usuário `kali`. `sudo dpkg-reconfigure wireshark-common` **é a decisão certa** para capturar sem `sudo`: cria o grupo `wireshark`, dá a capability só ao `dumpcap` e restringe a execução ao grupo. Rodar o Wireshark inteiro como `root` exporia um decodificador gigantesco — generoso em CVEs — a tráfego arbitrário, com privilégio total. É o exemplo didático perfeito: privilégio mínimo, no menor componente, pelo menor tempo. **O `nmap` moderno não é SUID, e isso muda o resultado:** `nmap -sS` precisa de socket cru e, sem `CAP_NET_RAW` ou `sudo`, cai **em silêncio** para `-sT` (`connect()`) — mais lento, mais barulhento no log do alvo e com resultado diferente. Ver o `nmap` "funcionando" e devolvendo coisa estranha quase sempre é isto. **Enumerar capability e SUID é etapa padrão de pós-exploração — por isso enumere primeiro na sua máquina:** `sudo getcap -r /` e `sudo find / -perm -4000 -type f` levam dez segundos e respondem a pergunta que importa; é o que o `linpeas` faz. Contra sistema de terceiro sem autorização escrita, é crime pela Lei 12.737/2012. **`systemd-analyze security` na sua VM é uma revelação desconfortável** — vários serviços comuns passam de 8. Num laboratório nem todos precisam ser corrigidos, mas o exercício ensina a ler o placar, e o mesmo comando em produção vira lista de tarefas. **RHEL:** capabilities e diretivas do systemd são idênticas (são do kernel), mas o SELinux empilha por cima — um `httpd_t` com todas as capabilities do mundo continua sem ler `/home` se a política não permitir. Duas verificações desacopladas: o invasor precisa das duas.

# CONFINAMENTO: APPARMOR, SELINUX E SANDBOXING

*Às 3h47 o invasor executou código dentro do `nginx`. O roteiro é sempre o mesmo: ler `/etc/passwd`, achar chave privada em `/home`, abrir conexão de saída. **As permissões Unix teriam deixado passar** — o processo era o dono. Barrou um perfil que listava, arquivo por arquivo, o que aquele programa podia tocar.*

**088**
GUIAS LINUX

## DOIS PORTÕES EM SÉRIE — E BASTA UM NEGAR

**PROCESSO**
o pedido ao kernel

```
open("/home/x/.ssh/id_rsa")
```

→

**PORTÃO DAC**
dono, grupo, outros · ACL · SUID.
**DISCRICIONÁRIO:**
o dono decide, e `chmod 777` abre.
*"quem você é?"*
`nega → EACCES`

→

**PORTÃO MAC**
perfil AppArmor ou tipo SELinux.
**OBRIGATÓRIO:**
política do administrador; nem `root` afrouxa em execução.
*"o que você é?"*
`nega → EACCES + registro`

→

**RECURSO**
arquivo, socket, capability

**Os dois portões são verificados em série e de forma independente:** a chamada só chega ao recurso se **ambos** permitirem. Afrouxar o DAC não abre o MAC, e ser `root` não abre nenhum dos dois quando há política. O invasor que ganhou execução no processo herda a identidade dele — e portanto o DAC —, mas **não escapa do perfil: o perfil descreve o programa, não o usuário**. E a negação do MAC **deixa registro** — é bloqueio e detecção ao mesmo tempo.

## A MESMA IDEIA, DUAS ESCOLHAS OPOSTAS DE PROJETO

| ASPECTO | APPARMOR | SELINUX |
|---|---|---|
| IDENTIFICA POR | **caminho** do arquivo | **rótulo** no atributo estendido |
| PADRÃO EM | Debian, Ubuntu, **Kali**, SUSE | RHEL, Fedora, CentOS, Android |
| POLÍTICA EM | texto legível, parecido com shell | linguagem própria, compilada |
| APRENDER LEVA | horas | semanas |
| PONTO FORTE | confinar um programa, rápido | política do sistema inteiro |
| PONTO FRACO | cópia ou link com outro caminho escapa | diagnóstico difícil no início |

## NA PRÁTICA — E COMO SE LÊ UMA NEGAÇÃO

kali@kali: ~

```
# AppArmor — no Kali pode estar instalado e NÃO ativo
$ sudo aa-status
apparmor module is not loaded  ← perfil em disco
não é política em vigor
$ sudo aa-genprof /usr/local/bin/servico # esqueleto
$ sudo aa-complain ...; sudo aa-logprof # aprende
$ sudo aa-enforce ...   # só então, aplica
$ sudo dmesg | grep 'apparmor="DENIED"'
# SELinux (RHEL) — o mesmo, porém por rótulo
$ getenforce; ls -Z /var/www/html; id -Z
$ sudo ausearch -m AVC -ts recent   # as negações
$ sudo restorecon -Rv /var/www/html # rótulo certo
$ firejail --net=none --private ./binario-desconhecido
```

### A LINHA QUE DENUNCIOU O INCIDENTE DAS 3H47

```
apparmor="DENIED" operation="open" profile="/usr/sbin/nginx" name="/home/deploy/.ssh/id_rsa" requested_mask="r" pid=4711 comm="nginx"
```

| | |
|---|---|
| `profile` | **quem** foi barrado — o **programa**, não o usuário |
| `name` | **o que** ele tentou tocar |
| `requested_mask` | com **qual** permissão (`r`, `w`, `x`) |

**Um `nginx` não tem motivo nenhum para procurar chave privada em `/home`.** Perfil bem construído gera **zero** negações em operação normal — o que torna *qualquer* negação um evento a investigar. Perfil ruidoso treina o operador a ignorar o log.

**Sandbox não é ambiente de análise de *malware*.** `firejail`, `bubblewrap` e contêineres **compartilham o kernel** com o hospedeiro, e uma falha de kernel escapa de todos eles. Sandbox serve para reduzir o dano de um programa que você *espera* ser bem-intencionado; para executar código que você **sabe** ser hostil, o isolamento correto é **VM descartável, em rede isolada, com snapshot e sem pasta compartilhada**. Confundir os dois transforma uma análise de amostra num incidente na sua estação.

## RESUMO RÁPIDO

- ✓ **DAC: o dono decide** — e o processo comprometido herda toda a autoridade do usuário. **MAC: o administrador decide** o que cada *programa* pode fazer, e nem `root` afrouxa isso em execução.
- ✓ **Os dois portões são independentes:** `chmod 777` não abre o MAC. O invasor precisa vencer os dois.
- ✓ **AppArmor identifica por caminho; SELinux, por rótulo.** Caminho é fácil de escrever e contornável por cópia; rótulo viaja com o arquivo, ao custo de uma curva bem maior.
- ✓ **Modo `complain` constrói perfil sem quebrar serviço** — registra tudo, bloqueia nada. O caminho de código que você não exercitou vai quebrar depois, às 3h da manhã.
- ✓ **`Permission denied` com permissões corretas** é o sintoma clássico: `dmesg | grep DENIED` no AppArmor, `ausearch -m AVC` no SELinux.
- ✓ **Sandbox reduz dano de programa falho; VM isola código hostil.** Não são intercambiáveis.

## NO KALI

**O AppArmor vem instalado no Kali, mas nem sempre ativo — confira antes de supor.** Um `aa-status` pode responder `apparmor module is not loaded`; ativar exige `apparmor=1 security=apparmor` em `GRUB_CMDLINE_LINUX_DEFAULT`, `update-grub` e reboot. Sem isso, os perfis em `/etc/apparmor.d/` estão em disco e não valem nada — **a situação mais enganosa possível, porque a configuração parece feita. Confinar a ferramenta ofensiva é tão legítimo quanto confinar o serviço:** boa parte do que roda num Kali processa entrada não confiável da rede — `tcpdump` decodifica pacote de terceiro, o Wireshark tem uma superfície enorme de decodificadores, `sqlmap` e `nikto` consomem resposta de servidor alheio —, fontes historicamente generosas de CVEs. O perfil `usr.sbin.tcpdump` já vem pronto e mostra por que perfil + capability é a defesa completa: a capability dá só o privilégio necessário, o perfil limita o alcance se o processo for explorado. `aa-complain` **é a resposta certa** para "o AppArmor está quebrando meu serviço" — mantém o registro e para de bloquear, permitindo diagnosticar sem desligar a proteção do resto do sistema; `aa-disable` por um perfil problemático é jogar a camada fora por um caso pontual. **RHEL inverte o padrão:** SELinux em *Enforcing* desde a instalação, e quem vem do Debian tropeça uma dezena de vezes no mesmo sintoma — serviço com permissão perfeita e comportamento inexplicável — antes de criar o reflexo: **antes de investigar permissão,** `ausearch -m AVC -ts recent`. Havendo AVC, o problema é rótulo, e `restorecon -Rv` ou um booleano resolve numa linha. `setenforce 0` "só para testar" vale se vier seguido de `setenforce 1`; `SELINUX=disabled` no arquivo, nunca. E `audit2allow` é diagnóstico, não solução: transforma **toda** negação em permissão — inclusive a que era o ataque.

# AUDITORIA, INTEGRIDADE E DETECÇÃO DE ALTERAÇÕES

*A pergunta do cliente era simples e ninguém soube responder: **"esse servidor ainda é o que vocês instalaram?"** Quatro anos no ar, oito administradores, correções de madrugada e scripts baixados da internet rodados como `root`. `/usr/bin` tinha 2.400 arquivos — e nenhum ponto de comparação.*

**089**
GUIAS LINUX

## O PIPELINE: SÓ A ETAPA 1 NÃO PODE SER FEITA DEPOIS

**1 — LINHA DE BASE**
Hash de cada arquivo **no momento em que o sistema estava íntegro**.

→

**2 — COLETA**
Estado atual: hash, dono, permissão, `mtime`, tamanho.

→

**3 — COMPARAÇÃO**
O que mudou, o que sumiu, o que apareceu do nada.

→

**4 — TRIAGEM**
Mudança esperada — ou **alerta**? É aqui que o esforço sobrevive ou morre.

**CUSTÓDIA EXTERNA**
mídia só-leitura, host de gestão separado ou repositório assinado

**A base guardada na própria máquina é editável por quem comprometeu a máquina.** O invasor com `root` troca o binário, roda `aide --update`, e a verificação seguinte dá tudo em ordem. A base tem de viver fora do alcance do sistema monitorado — e a verificação ideal roda **de fora dele**, porque um *rootkit* de kernel mente para qualquer programa que rode ali dentro.

**Monitore o que muda pouco e vale muito:** binários, `/etc`, módulos de kernel, pontos de arranque automático (unidades, `cron`, `~/.bashrc`) e credenciais (`shadow`, `sudoers`, `authorized_keys`). **Deixe de fora o volátil por natureza** — `/var/log`, `/var/cache`, `/tmp`, `/proc` —, senão o ruído afoga o sinal.

## DOIS REGISTROS QUE RESPONDEM PERGUNTAS DIFERENTES

### AUDITD — O QUE O KERNEL OBSERVOU

evidência: independe da boa vontade do programa

- **Vigiar caminho:** `-w /etc/sudoers -p wa -k escalada` (`w`=escrita, `a`=atributo, `-k`=chave de consulta).
- **Vigiar chamada:** `-a always,exit -S execve -F euid=0`. Meça o volume antes de deixar fixa.
- **Persistir:** `/etc/audit/rules.d/*.rules` + `augenrules --load`. Via `auditctl`, só na memória.
- `-e 2` **torna o conjunto imutável até o boot** — nem `root` altera as regras. É o que vira evidência.

### JOURNALD — O QUE O PROGRAMA CONTOU

diagnóstico: rico, mas voluntário

- Registra o que a aplicação **decidiu** registrar — se ela não conta, não há linha.
- Ótimo para entender **por que** um serviço falhou; fraco como prova do que alguém fez.
- **Log que vive só na máquina monitorada é prova sob custódia do suspeito.** Envie para fora: `systemd-journal-remote`, `audisp-remote`.
- `chattr +a` num log impede reescrever e truncar — a limpeza seletiva de rastro deixa de funcionar.

`uid=0 auid=1001`

**O campo mais valioso de um evento.** O `auid` é fixado no *login* e **imutável dali em diante**: esta linha diz que foi o usuário **1001** quem, via sudo, fez aquilo como root. Numa máquina com cinco administradores, é a diferença entre "alguém apagou" e "fulano apagou".

## NA PRÁTICA — E O QUE CADA CAMADA REALMENTE COBRE

kali@kali: ~

```
# 1. os hashes que o apt já mantém — 30 segundos
$ sudo debsums -c
/usr/bin/whoami            ← um byte a mais e ele aparece
# 2. auditoria: regras, e depois torná-las imutáveis
$ sudo auditctl -w /etc/shadow -p wa -k identidade
$ sudo augenrules --load && sudo auditctl -l
$ sudo auditctl -e 2   # imutável até o reboot
$ sudo ausearch -k identidade -ts today -i
$ sudo ausearch -m USER_LOGIN -sv no # logins que falharam
# 3. linha de base explícita — e a custódia dela
$ sudo aideinit && sudo aide --check
$ sudo sha256sum /var/lib/aide/aide.db  # leve para fora
# 4. o que nem root altera sem dar um passo auditável
$ sudo chattr +i /etc/passwd    # imutável
$ sudo chattr +a /var/log/importante # só acréscimo
```

| CAMADA | COBRE — E O BURACO QUE DEIXA |
|---|---|
| `debsums -c` | tudo que veio de pacote, sem configurar nada. Não vê o que foi instalado fora do `apt` |
| `aide --check` | o que você declarar, com hash e metadados. Custa manter a base fora da máquina |
| `auditd` | **quem** fez, com qual chamada. Volume alto se a regra for genérica |
| `chattr +i` / `+a` | não impede `root`: obriga um passo a mais, e auditável |
| log remoto | tira a prova da custódia do suspeito — exige um coletor de pé |

**Quando o alerta é real: não desligue** (memória e conexões morrem junto), isole da rede sem apagar nada, colete antes de mexer (`ss -tunap`, `ps auxf`, `lsof -n`, `last`), preserve com hash — e **não conserte por cima**: sistema comprometido em nível de `root` só volta a ser confiável reconstruído de mídia limpa.

`rkhunter` e `chkrootkit` **rodam sobre o sistema que investigam.** Um *rootkit* em nível de kernel responde o que quiser a quem perguntar — e essas ferramentas perguntam. E só detectam o **conhecido**. Diante de suspeita séria, a análise correta é feita **de fora**: disco montado em outra máquina, ou o *live* USB do Kali em **modo forense**, que não monta nada automaticamente nem toca na área de troca.

## RESUMO RÁPIDO

- ✓ **Detecção exige uma linha de base registrada quando o sistema estava íntegro.** É a única etapa que não se faz em retrospecto.
- ✓ **Base guardada na máquina monitorada é adulterável por quem a comprometeu** — a custódia externa é o que separa o método do teatro.
- ✓ `auditd` **registra o que o kernel observou;** `journald`**, o que os programas decidiram contar.** O `auid` responde "quem fez", não só "com qual UID".
- ✓ `-e 2` **torna as regras imutáveis até o reboot** — e o reboot é ele próprio um evento visível.
- ✓ `debsums -c` **é a verificação de trinta segundos**; o AIDE cobre o resto, ao custo de manter a base.
- ✓ **Log que só existe na máquina monitorada não é prova.** Envie para fora no momento em que é gerado.

## NO KALI

**O Kali *rolling* é hostil a uma linha de base ingênua — é a lição mais prática do capítulo aqui.** Num Debian estável `/usr/bin` fica parado por meses e uma mudança é notável; no Kali, um `full-upgrade` semanal reescreve centenas de binários e o `aide --check` seguinte devolve milhares de linhas — em duas semanas o operador aprende a ignorar o relatório, que é **o modo mais comum de um sistema de detecção falhar**. O método que funciona: `aide --update` logo após cada atualização planejada, **com a base nova recustodiada fora da máquina**. Assim qualquer mudança fora da janela de atualização vira anomalia por definição. **A máquina Kali é a mais interessante de auditar da sua rede e não é a mais monitorada:** concentra credencial de laboratório, captura e resultado de teste autorizado — e ao mesmo tempo um `auditd` mal configurado nela gera ruído gigantesco. Comece pelas regras de identidade e escalada e deixe o `execve` genérico de fora até medir o volume. `debsums -c` **é o teste que você pode rodar hoje:** a resposta esperada é lista vazia, e qualquer binário de `/usr/bin` que apareça merece explicação imediata. **Perfil de MAC bem construído é sensor:** um perfil que gera zero negações em operação normal transforma *qualquer* `apparmor="DENIED"` em evento — cruzado com o `auditd`, dá duas fontes independentes. **RHEL:** o `auditd` é nativo e já vem habilitado, com conjuntos alinhados a normas em `/usr/share/audit/sample-rules/` — `30-stig.rules` dá um ponto de partida auditado. A verificação por pacote é `rpm -Va`.

# O ECOSSISTEMA DE FERRAMENTAS DO KALI

*Quem instala o Kali abre o menu, encontra centenas de ferramentas em treze categorias e escolhe a de nome mais chamativo para apontar a algum alvo. É a forma errada de usar a distribuição — e a mais provável de cometer um crime sem entender o que se fez.* ***As ferramentas não são o assunto: o fluxo é.***

**090**
GUIAS LINUX

## O CICLO DE UM TESTE — E ONDE CADA CATEGORIA ENTRA

**1 RECONHECIMENTO**
Mapear **sem tocar**: fontes públicas, DNS, certificados, metadados.
`whois · dig · theHarvester · recon-ng`

→

**2 VARREDURA E ENUMERAÇÃO**
O que existe e responde. **Aqui começa a interação — e a necessidade de autorização.**
`nmap · masscan · gobuster · enum4linux`

→

**3 ANÁLISE DE VULNERABILIDADE**
Separa "esta porta está aberta" de "esta versão tem falha conhecida".
`nikto · nuclei · searchsploit · wpscan`

**4 EXPLORAÇÃO**
A prova de conceito, controlada — a menor fração do tempo real.
`metasploit · sqlmap · hydra`

←

**5 PÓS-EXPLORAÇÃO**
Até onde o acesso leva: escalar, pivotar. **Reinicia o ciclo por dentro.**
`linpeas · BloodHound`

←

**6 RELATÓRIO**
**O único entregável.** Achado, severidade, reprodução, correção.
`faraday · dradis · CherryTree`

**O ciclo é iterativo, não linear:** a pós-exploração revela hosts e serviços internos invisíveis de fora, e cada descoberta reinicia a varredura — agora de um ponto de vista interno. **As três primeiras fases consomem a maior parte do tempo**, e a fase 6 é a de menos ferramenta e mais valor: **o cliente paga pelo relatório, não pelo acesso**. Um acesso obtido e não documentado de forma reproduzível não vale nada.

## ANTES DE QUALQUER FERRAMENTA: O ESCOPO ASSINADO

**QUAIS ALVOS**
IPs, domínios e faixas **exatos**. O IP vizinho do mesmo bloco não entra por proximidade.

**QUAIS TÉCNICAS**
Negação de serviço, engenharia social e exploração ativa costumam ficar de fora.

**QUAL JANELA**
Datas e horários. Testar fora deles é agir **sem autorização**.

**QUEM ASSINA**
Quem tem poder legal sobre os sistemas — o administrador pode não ser essa pessoa.

**CONTATO DE EMERGÊNCIA**
Para quando algo sai do previsto. Às vezes sai.

**A ferramenta é idêntica; o que muda é o papel.** `nmap`, `hydra`, `sqlmap` e `metasploit` fazem exatamente a mesma coisa nas mãos de um *pentester* contratado e nas de um criminoso — a lei não distingue os dois pela ferramenta, e sim pela existência de **autorização escrita, específica e prévia** para aquele alvo, naquela janela, com aquelas técnicas. No Brasil, a **Lei 12.737/2012** tipifica a invasão de dispositivo informático alheio com pena de detenção: *alheio* e *indevida* são as palavras que importam, e valem independentemente de ter havido dano, da intenção declarada ou de a ferramenta ser pública. Varrer as portas de um servidor de terceiro já é o começo do que a lei descreve; a **LGPD** acrescenta responsabilidade sobre qualquer dado pessoal que o teste venha a tocar.

## INSTALAR MENOS, NÃO MAIS — E TER ONDE PRATICAR

```
kali@kali: ~
# metapacotes: conjuntos coerentes, não ferramenta a ferramenta
$ apt search '^kali-tools-' | head -30
$ sudo apt install kali-tools-web
$ kali-tweaks  # o painel: metapacotes, hardening, shell
# wordlists — a rockyou vem compactada na 1ª vez
$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz
# o banco que dá memória ao Metasploit entre sessões
$ sudo msfdb init  # e desative quando não usar
# o fluxo, contra alvo de treino em rede isolada:
$ whois exemplo.com; dig +short exemplo.com  # fase 1
$ nmap -sV -sC -p- 10.10.10.5                # fase 2
$ gobuster dir -u http://10.10.10.5 -w common.txt
$ searchsploit apache 2.4; nikto -h http://10.10.10.5
$ msfconsole                                 # fase 4
```

| METAPACOTE | O QUE TRAZ |
|---|---|
| `kali-linux-headless` | o núcleo, sem interface gráfica |
| `kali-linux-default` | o conjunto padrão da imagem — **comece aqui** |
| `kali-tools-web` | ferramental de aplicação web |
| `kali-tools-passwords` | quebra de senha e força bruta |
| `kali-tools-wireless` | ataque e auditoria de Wi-Fi |
| `kali-linux-everything` | literalmente tudo — **e não é o que você quer** |

**Instale menos, não mais:** uma máquina com `everything` é superfície de ataque imensa, atualização lenta e centenas de programas que você nunca abrirá. E **a regra de ouro do laboratório é o isolamento de rede** — uma VM deliberadamente vulnerável (Metasploitable, DVWA, *Juice Shop*) é, por definição, uma máquina que qualquer um compromete em minutos: ela vai em rede *host-only* ou interna do hipervisor, **jamais em ponte com a rede real**.

## RESUMO RÁPIDO

- ✓ **A ferramenta é a mesma nas duas mãos; o que muda é a autorização** — escrita, específica e prévia, com alvos, técnicas, janela e quem assina.
- ✓ **O assunto não são as ferramentas, é o fluxo:** reconhecimento → varredura → análise → exploração → pós-exploração → relatório.
- ✓ **O ciclo é iterativo:** a pós-exploração revela alvos internos e reinicia a varredura de dentro.
- ✓ **A exploração é a menor fração do tempo; o relatório é o entregável.** Acesso não documentado não vale nada.
- ✓ **Metapacotes organizam as centenas de ferramentas** — e a recomendação contraria a tentação: instale menos.
- ✓ **Prática legal só contra a própria máquina ou alvo de treino**, sempre em rede isolada.

## NO KALI

`kali-tweaks` **é o painel de controle da distribuição e a primeira coisa a abrir numa instalação nova:** gerencia metapacotes, rede, comportamento do shell, persistência da imagem *live* e o *hardening* — tudo num menu navegável e reversível. Muita gente instala ferramenta na mão por não saber que ele existe. **O modo *undercover* tem motivo prático e legítimo:** `kali-undercover` transforma a área de trabalho num clone visual do Windows 10 — num teste físico autorizado, ou trabalhando num café, a tela inconfundível do Kali chama atenção que o profissional não quer. É disfarce da interface, **não das ferramentas nem da responsabilidade legal.** `msfdb init` **na primeira execução vale a pena:** cria a base PostgreSQL onde o Metasploit guarda hosts, serviços e achados, que é justamente o que alimenta a fase de relatório — mas é também um serviço a mais escutando na sua máquina, então ative quando for usar. **As *wordlists* em** `/usr/share/wordlists` **são o argumento vivo a favor de chave em vez de senha:** tudo que está na `rockyou.txt` é quebrável em segundos. **A ferramenta do Kali é dual por natureza, e essa é a lição que fecha o volume:** o `nmap` que enumera o alvo é o mesmo que audita a própria superfície; o `linpeas` que mapeia escalada na pós-exploração é o inventário de SUID e capabilities do guia 087 lido do outro lado; o `nikto` que acha má configuração no alvo acha a mesma coisa no seu servidor. **Aprender o Kali é aprender a olhar o próprio sistema com os olhos de quem vai atacá-lo** — e é para isso, legalmente, que ele serve na esmagadora maioria das mãos. **Outras distribuições ofensivas: Parrot OS Security** (foco em privacidade, mais leve), **BlackArch** (milhares de ferramentas sobre Arch) e **Commando VM** (o análogo para Windows) partilham a mesma natureza dual e a mesma exigência de autorização — a distribuição muda a embalagem, não a lei. Do lado defensivo, o **Security Onion** monta o outro polo: monitoramento, detecção e análise em vez de ataque.

# ANONIMATO E PRIVACIDADE: PROXYCHAINS, TOR E VPN

*O jornalista rodava tudo por VPN, navegava em aba anônima e tinha fita sobre a webcam. A fonte foi identificada assim mesmo: no meio de uma sessão cuidadosa, o cliente de e-mail fez **uma** consulta DNS por fora do túnel, e ela ligou o horário ao alvo. Todo o resto da cadeia era sólido. **Bastou um elo vazar.***

091
GUIAS LINUX

## O MODELO DE AMEAÇA DECIDE TUDO: QUEM OBSERVA QUAL TRECHO

| VOCÊ | REDE LOCAL | PROVEDOR | TRÂNSITO | DESTINO |
|---|---|---|---|---|
| a origem | Wi-Fi, roteador | o seu ISP | a internet | o servidor |
| | vizinho de Wi-Fi, dono do roteador | ISP, governo local | trânsito global, estado-nação | dono do site, e quem o intima |

**VPN** — Cobre de você até o servidor da VPN: tira a rede local e o ISP da jogada — e **entrega tudo ao provedor da VPN**. Você não removeu o observador; **trocou-o por um que escolheu confiar**. Inútil se o adversário é o próprio provedor, ou quem pode intimá-lo.

**TOR** — Separa **quem-você-é** de **para-onde-vai** por três saltos independentes. Mas não esconde do seu ISP *que você usa Tor* — para isso existem as *bridges* e o `obfs4`.

**Três palavras que não são sinônimas. Privacidade:** o *conteúdo* é secreto (é o que o HTTPS entrega). **Anonimato:** a *identidade* é secreta — há tráfego, mas ninguém o liga a você. **Não-rastreabilidade:** suas ações não podem ser ligadas *entre si*. Uma VPN dá privacidade perante o ISP e **desloca a confiança**; não dá anonimato, porque o provedor sabe exatamente quem você é. Confundir os dois é o erro conceitual mais caro da área — e a razão de "use uma VPN e fique anônimo" ser publicidade, não engenharia.

## COMO O TOR FUNCIONA: NENHUM NÓ TEM AS DUAS PONTAS

**1 · ENTRADA (GUARD)**
- ✓ sabe **quem você é** — vê o seu IP
- ✗ não sabe o destino: recebe tudo cifrado em três camadas

**2 · MEIO**
- ✗ não sabe quem você é
- ✗ não sabe para onde vai — **só repassa de um relé ao outro**

**3 · SAÍDA (EXIT)**
- ✓ sabe o **destino** e **vê o tráfego** como ele entra na internet
- ✗ não sabe quem você é: recebe do nó do meio

**É a separação, não a cifragem, que dá o anonimato** — nenhum nó sozinho tem as duas pontas. Daí duas consequências que se ignora por conta própria: **o nó de saída lê o que não estiver cifrado de ponta a ponta** (uma senha por HTTP simples é lida em claro por quem opera aquele nó — que pode ser alguém que o subiu só para isso), então **HTTPS continua obrigatório dentro do Tor**; e a **correlação de tempo** é o limite teórico de todo sistema de baixa latência — quem observa as duas pontas liga entrada e saída pelo horário e pelo volume, e contra esse adversário global o próprio projeto não promete proteção.

## NA PRÁTICA — E OS VAZAMENTOS QUE ANULAM TUDO

kali@kali: ~

```
$ sudo systemctl start tor  # SOCKS5 em :9050
$ curl -s https://check.torproject.org/api/ip
{"IsTor":false,"IP":"189.x.x.x"}   ← o seu, real
# --socks5-hostname manda TAMBÉM o DNS pelo proxy.
# Com --socks5 apenas, o curl resolve o nome ANTES —
# e essa consulta vaza. Foi o erro do caso da abertura.
$ curl -s --socks5-hostname 127.0.0.1:9050 \
      https://check.torproject.org/api/ip
{"IsTor":true,"IP":"185.x.x.x"}
# proxychains força QUALQUER programa pela cadeia
/etc/proxychains4.conf:
  dynamic_chain
  proxy_dns              # CRUCIAL: DNS pela cadeia
  socks5  127.0.0.1 9050
$ proxychains4 -q nmap -sT -Pn alvo-autorizado
# -sT obrigatório: SYN cru não passa por SOCKS
$ mat2 --show foto.jpg # GPS, câmera, autor
$ mat2 foto.jpg        # gera foto.cleaned.jpg
```

| VAZAMENTO | O QUE REVELA · COMO FECHAR |
|---|---|
| **DNS** | os domínios que você visita · `proxy_dns` , `--socks5-hostname` |
| **WebRTC** | seu IP real, mesmo com VPN · desabilitar no navegador |
| **IPv6** | escapa de VPN só-IPv4 · desligar ou cobrir no túnel |
| **ICMP / SYN cru** | a origem real, fora do `proxychains` · não usar |
| **Metadados** | autor, GPS, software no arquivo · `mat2` antes de publicar |
| ***Fingerprint*** | identifica sem cookie · Tor Browser, que uniformiza |
| **Login** | **destrói tudo: você se identificou** · não misturar identidades |

`proxychains` **cobre conexões TCP via** `connect()` **, e nada mais.** Ele intercepta o que passa pela `libc` dinâmica: binário estático escapa e — o mais perigoso — **tráfego fora da pilha TCP normal vaza a origem real**. O `ping` (ICMP) e o `nmap -sS` montam pacotes por baixo do SOCKS e saem do seu IP verdadeiro; daí `proxychains nmap` só fazer sentido com `-sT` . Supor cobertura total é o erro que expõe quem se acha protegido. **E anonimato não é escudo legal:** rotear por Tor uma ação não autorizada é o mesmo crime da Lei 12.737/2012, agora com a agravante de tentativa de ocultação. Estas ferramentas existem para proteger comunicação legítima — a fonte, o jornalista, o *pentester* que respeita seu escopo —, não para dar cobertura a quem age fora da lei.

## RESUMO RÁPIDO

- ✓ **Anonimato é propriedade da cadeia inteira**, e ela tem a força do elo mais fraco. Uma consulta DNS bastou.
- ✓ **Privacidade ≠ anonimato ≠ não-rastreabilidade.** VPN desloca a confiança do ISP para o provedor da VPN — não anonimiza.
- ✓ **O modelo de ameaça precede a ferramenta:** nomeie o adversário e o que ele observa antes de escolher qualquer coisa.
- ✓ **O Tor separa quem-você-é de para-onde-vai em três saltos**, mas o nó de saída vê o tráfego não cifrado — HTTPS continua obrigatório.
- ✓ `--socks5-hostname` **, não** `--socks5` ; e `proxy_dns` no `proxychains` . É a palavra que separa a cadeia fechada da vazada.
- ✓ **Testar o próprio vazamento importa mais que configurar a ferramenta.** Ferramenta que parece pronta e vaza é pior que nenhuma.

## NO KALI

**O Kali traz o ferramental, mas para anonimato de verdade a ferramenta é o Tails, não o Kali.** O Tails roda da RAM, força **todo** o tráfego pelo Tor, não deixa rastro no disco e esquece tudo ao desligar — foi projetado para o modelo de ameaça do jornalista da abertura. O Kali foi projetado para *testar* segurança: não força o tráfego pelo Tor e guarda estado no disco, o oposto do que anonimato exige. `proxychains` **na frente de ferramenta ofensiva é rotina em teste autorizado — e tem pegadinha:** rotear uma varredura por proxy tem uso legítimo (ver como o alvo enxerga tráfego de outra origem, cumprir regra de engajamento que exige origem específica), mas o `nmap -sS` , o mais comum, **não passa** e vaza sua origem real. Em teste autorizado isso falseia o resultado; fora dele, é a diferença entre achar-se anônimo e não estar. Sempre `-sT` , e sempre confira o IP de saída antes de confiar na cadeia. **Testar o próprio vazamento é o exercício que importa mais do que configurar:** `check.torproject.org` , um teste de vazamento de DNS e um de WebRTC dizem se a configuração realmente funciona — e a resposta surpreende quem confiava nela. Uma ferramenta que parece configurada e vaza DNS é **pior do que nenhuma**, porque induz o comportamento arriscado que a pessoa não teria sem a falsa confiança. **O modo *undercover* é privacidade de outra natureza:** privacidade *visual*, contra o observador humano ao seu lado, não contra o observador de rede. Disfarçar a tela não protege o tráfego, e proteger o tráfego não disfarça a tela — modelos de ameaça diferentes, camadas ortogonais, e é útil não confundi-los. **Fora do Debian:** Tor, WireGuard e `proxychains` são software de rede e funcionam igual em qualquer distribuição — o WireGuard é nativo no kernel moderno e o NetworkManager integra ambos pela interface gráfica nas duas famílias. O raciocínio de modelo de ameaça e a lista de vazamentos independem por completo do sistema operacional: são propriedades da rede e da criptografia.

# PERSONALIZANDO O SHELL: .BASHRC, ALIASES E FUNÇÕES

*Digitou `alias ll='ls -la'`, gostou, pôs no `~/.bashrc`: funcionou em toda aba nova. Escreveu um script com `ll`, agendou no `cron` — e às três da manhã chegou um e-mail com `ll: command not found`. Nada quebrou.* ***O shell só leu outro arquivo.***

**092**
GUIAS LINUX

## COMO O SHELL FOI INICIADO DECIDE QUAIS ARQUIVOS ELE LÊ

### LOGIN
tty, ssh, `su -`, `bash -l` — nasce da autenticação

`/etc/profile`

↓

`~/.bash_profile`
só o 1º que existir entre ele, `.bash_login` e `.profile`

**só se pedir**

`~/.bashrc`
via `source` explícito

### INTERATIVO, SEM LOGIN
aba nova do terminal — você já estava autenticado

`/etc/bash.bashrc`

↓

`~/.bashrc`
o único que roda em **toda** aba nova

nenhum perfil
ambiente vem herdado

### NÃO-INTERATIVO
script, `cron`, `ssh host 'cmd'` — sem prompt

`$BASH_ENV`
e só se a variável existir

↓

nenhum arquivo

nenhum alias
o e-mail das 3h nasce aqui

**A seta tracejada é a chave de tudo.** O `~/.bashrc` **não** é lido por um shell de login — ele só entra porque a distribuição pôs um `source ~/.bashrc` dentro do `~/.bash_profile`. Apague a linha e o SSH perde os aliases; o script não lê nenhum dos dois. Os testes: `shopt -q login_shell` e `[[ $- == *i* ]]` — a letra **i** em `$-` só existe em shell interativo, daí todo `.bashrc` de distribuição abrir com `case $- in *i*) ;; *) return;; esac`: sem a guarda, um banner ali entra no meio da saída e quebra `scp` e `rsync`.

## ONDE VAI CADA COISA — E ALIAS × FUNÇÃO

| ARQUIVO | QUANDO RODA · O QUE COLOCAR |
|---|---|
| `~/.bash_profile` | seu login · **ambiente**: `export`, `PATH`, agente SSH |
| `~/.bashrc` | toda aba sua · **comportamento**: alias, função, prompt, histórico |
| `~/.bash_logout` | fim do login · limpeza: cache, travar tela |
| `~/.zshenv` | **todo** Zsh, até script · o lugar certo do `PATH` no Kali |
| `~/.zshrc` | Zsh interativo · o equivalente ao `.bashrc` |

### ALIAS — TROCA A 1ª PALAVRA

- Substituição textual, **só da primeira palavra**, antes de executar.
- Argumento sempre no **fim**: `alias x='cmd --opt'` não põe nada no meio.
- `sudo ll` falha — contorne com `alias sudo='sudo '` (com espaço).
- **Não vale em script**: expansão vem desligada.
- Escape pontual: `\grep` ou `command ls` — o alias de cor suja o pipeline.

### FUNÇÃO — COMANDO DE VERDADE

- Tem parâmetros, lógica e **código de saída**. Faz tudo que o alias faz.
- `mkcd(){ mkdir -p -- "$1" && cd -- "$1"; }` — impossível como alias.
- Robustez: `--`, aspas em toda expansão, `local`, erro em `stderr`.
- `export -f` alcança subshells Bash, e só eles.
- O que precisa valer **em script** vem de biblioteca com `source`, não do `.bashrc`.

## NA PRÁTICA — OS AJUSTES DE MAIOR RETORNO DIÁRIO

kali@kali: ~ — em qual shell eu estou?

```
$ ps -p $$ -o comm= ; echo $SHELL
zsh                         /usr/bin/zsh
# editar ~/.bashrc aqui nao muda NADA
$ type ll          # alias? funcao? binario?
ll is an alias for ls -alF --color=auto
$ bash -n ~/.bashrc && echo "sintaxe ok"
# checa SEM executar. Um erro ali te deixa
# sem shell utilizavel no proximo login.
$ shellcheck ~/.bashrc ; bash -lic 'exit'
```

| LINHA | O INCÔMODO QUE ELA RESOLVE |
|---|---|
| `shopt -s histappend` | a última aba fechada **sobrescrevia** o histórico de todas |
| `HISTCONTROL=ignoreboth` | duplicatas somem; e linha iniciada por **espaço** não é gravada — a forma certa de digitar uma senha |
| `export LESS='-R -F -X -i'` | cor preservada, sai se couber, não limpa a tela, busca sem caixa |
| `~/.bashrc.d/*.sh` | um laço com `source` fragmenta o monstro de 300 linhas |

**Nunca ponha `.` (o diretório atual) no `PATH`, muito menos no início.** Com ele lá, entrar num diretório escrito por outra pessoa e digitar `ls` executa o `ls` plantado ali: escalada de privilégio clássica, trivial de explorar, inteiramente evitável — use `./programa`, a barra torna a intenção inequívoca. **E pense antes de** `alias rm='rm -i'`: ele treina o reflexo de confirmar sem ler, e no servidor, no contêiner ou dentro de um script — todos sem o alias — o `rm` apaga sem perguntar. Prefira `alias rmi='rm -i'` ou uma função-lixeira, deixando o `rm` real com o comportamento real.

## RESUMO RÁPIDO

- ✓ **Login, interativo e nenhum dos dois lêem arquivos diferentes.** O script e o `cron` não lêem nem `.bash_profile` nem `.bashrc` — é a causa do *command not found* das três da manhã.
- ✓ **Ambiente no `.bash_profile`, comportamento interativo no `.bashrc`.** O `.bashrc` roda a cada aba: tem de ser rápido e **silencioso** — banner ali quebra `scp` e `rsync`.
- ✓ **O Bash lê só o primeiro** entre `.bash_profile`, `.bash_login` e `.profile`. Criar um `.bash_profile` onde só havia `.profile` apaga o `PATH` em silêncio.
- ✓ **Alias troca a primeira palavra e morre no script;** quando precisa de argumento no meio, de lógica ou de código de saída, é função.
- ✓ `histappend` + `HISTCONTROL=ignoreboth` corrigem os dois piores padrões do histórico: sobrescrita pela última aba e duplicata.
- ✓ `bash -n` **antes de confiar.** Um erro de sintaxe no arquivo de inicialização é a via mais rápida para ficar sem shell utilizável.

## NO KALI

**O shell padrão é o Zsh desde o Kali 2020.4 — e é aqui que quase todo mundo tropeça:** o usuário `kali` tem `/usr/bin/zsh` no `/etc/passwd`, com um `~/.zshrc` de mais de cem linhas mantido pelo pacote `kali-defaults`. O `~/.bashrc` continua lá, íntegro, para os contextos que usam Bash — então **o alias que você acrescentou ao `~/.bashrc` simplesmente não aparece no terminal**, e nenhum erro indica isso. Confira com `ps -p $$` e `echo $SHELL` *antes* de escolher o arquivo. O Zsh tem cinco arquivos onde o Bash tem dois, e o `~/.zshenv` é o único dos dois shells lido **sempre**, inclusive por script: o lugar certo do `PATH` e o pior possível para qualquer coisa que imprima. **O `kali-tweaks` troca o prompt sem editar arquivo**, mas escreve em arquivos do pacote — atualizar o `kali-defaults` sobrescreve edições manuais em `/usr/share/kali-*`; personalização sua vai no `~/.zshrc`. **Trocar de shell é** `chsh -s /usr/bin/bash`, nunca editar o `/etc/passwd` na mão — e vale só no *próximo* login, o que faz muita gente achar que falhou. **Root tem `/root/.zshrc` próprio:** seus aliases não valem sob `sudo -i`, e assim deve ser. **Fora do Debian:** Bash é Bash, mas o Fedora/RHEL chama `/etc/bash.bashrc` de `/etc/bashrc` e usa `/etc/profile.d/*.sh` intensamente — dotfile portável testa a existência antes do `source`.

# ZSH, PLUGINS E O PROMPT DO KALI

*Quatro abas abertas: duas locais, uma no laboratório, uma no servidor de produção que ninguém deveria estar tocando. O prompt das quatro dizia `$`. A pessoa digitou `systemctl stop nginx`, apertou Enter — e **só então** olhou o título da janela. **O prompt não é enfeite: é a instrumentação permanente da sessão.***

093
GUIAS LINUX

## ANATOMIA DO PROMPT: A LINHA MAIS LIDA DO TERMINAL

```
kali @ kali : ~/lab-zsh $  … o comando que você digita …        1  12:04
```

- **`%n` usuário** — quem você é (importa muda sob `sudo -i`: é o primeiro campo a conferir antes de um comando destrutivo.
- **`%m` máquina** — nome curto do host. É o campo que teria evitado o `systemctl stop` na máquina errada.
- **`%~` diretório atual**, com o home como til. `%(4~|.../%3~|%~)` trunca caminho longo.
- **`%#` privilégio** — `%` para usuário comum, `#` para root. Sinal de que dali em diante não há rede de proteção.
- **`RPROMPT` à direita, e some sozinho** quando o comando cresce: `%?` (código de saída) e `%*` (hora). O `%(?..)` só o mostra quando algo falhou.

**Dois detalhes decidem se o prompt funciona. 1.** `setopt prompt_subst` reavalia a expansão a cada redesenho — sem ele, o código de saída e o ramo do Git *congelam* no valor que tinham quando o `.zshrc` foi lido. **2.** Todo byte não imprimível precisa ser marcado como tal: `%{ %}` no Zsh, `\[ \]` no Bash. Sem ela o shell conta os bytes invisíveis da cor como largura, erra o cursor e **a linha se sobrescreve** quando você navega no histórico — a causa nº 1 do "meu prompt embaralha na seta para cima". Com `%F{cor}/%f` em vez de escape cru, o Zsh cuida disso sozinho.

## O QUE O ZSH ACRESCENTA — E QUANTO CUSTA CADA CAMINHO

### COMPLETAÇÃO COM MENU

O `compsys` lê a página de manual: entende subcomando, navega com as setas e descreve cada candidato. `autoload -Uz compinit && compinit` + `zstyle ':completion:*' menu select`.

### QUALIFICADORES DE GLOB

`**/*.log(.)` só arquivos regulares · `(.mh-2)` últimas 2 h · `(.Lm+10)` maiores que 10 MB · `*(/)` só diretórios. Substitui muito `find` **no uso interativo**; em script, `find` continua a escolha portável.

### HISTÓRICO DE VERDADE COMPARTILHADO

`setopt share_history` — o comando digitado numa aba já está na seta-para-cima de outra aba **aberta**. Junto: `hist_ignore_all_dups`, `hist_ignore_space`, `hist_verify`, `extended_history`.

### OS DOIS PLUGINS QUE VALEM SOZINHOS

`zsh-autosuggestions` completa em cinza pelo histórico (→ aceita). `zsh-syntax-highlighting` colore **enquanto você digita**: comando inexistente em vermelho *antes* do Enter.

| CAMINHO | O QUE DÁ · O QUE CUSTA |
|---|---|
| **o .zshrc do Kali** | prompt, menu, histórico e os 2 plugins · **nada: é o padrão** |
| **2 plugins por apt** | autosuggestions + highlighting · duas linhas de `source` |
| **zinit / antidote** | gerenciador com carga preguiçosa · aprender a ferramenta |
| **Oh My Zsh** | ecossistema pronto · **inicialização lenta**, muito que não se usa |
| **starship** | um prompt só para Bash, Zsh e Fish · binário externo |

**Meça antes de otimizar:** `time zsh -i -c exit`. Abaixo de **200 ms** ninguém percebe; acima de **500 ms** incomoda, multiplicado por dezenas de aberturas por dia. O culpado costuma ser `compinit` sem cache — ou um prompt que roda `git status` a cada Enter, o que trava o terminal em repositório grande.

## NA PRÁTICA — O PROMPT DO KALI E A MIGRAÇÃO DO BASH

kali@kali: ~ — ~/.zshrc

```
# o Kali nao usa framework: o prompt vem do
# pacote kali-defaults, ligado por uma variavel
PROMPT_ALTERNATIVE=twoline   # oneline | backtrack
NEWLINE_BEFORE_PROMPT=yes
# prompt proprio, com codigo de saida a direita
setopt prompt_subst
PROMPT='%F{cyan}%n%f@%F{green}%m%f:%F{yellow}%~%f %# '
RPROMPT='%(?..%F{red}[%?]%f )%F{8}%*%f'
# a ORDEM importa: highlighting SEMPRE por ultimo
source /usr/share/zsh-autosuggestions/…zsh
source /usr/share/zsh-syntax-highlighting/…zsh
$ exec zsh   # recarrega sem abrir processo novo
```

| VARIANTE / ARMADILHA | O QUE MUDA DE FATO |
|---|---|
| `twoline` | padrão do Kali: o comando **sempre começa na coluna 1** — caminho longo não empurra o cursor. As outras: `oneline` e `backtrack` |
| **migrar do Bash** | o `.bashrc` não é lido: alias e função vão **movidos**; `shopt` vira `setopt` e array começa em **1**, não em 0 |
| **Nerd Font** | o ícone de ramo/pasta só existe nela — e instala-se no **cliente**, não no servidor |

**O `zsh-syntax-highlighting` tem de ser o último `source` do arquivo.** Ele instala ganchos no widget de edição de linha do Zsh, e qualquer plugin carregado *depois* pode substituí-los, desligando o realce **em silêncio**: não há mensagem de erro, a cor simplesmente não aparece e você passa meia hora procurando um pacote que já estava instalado. **E o shell interativo não afeta script: o shebang manda.** Um arquivo com `#!/bin/bash` roda em Bash mesmo que o seu terminal seja Zsh — desde que você não confie em nada do `.zshrc` lá dentro.

## RESUMO RÁPIDO

- ✓ **O prompt é instrumentação, não decoração.** `%n %m %~ %#` respondem quem-você-é, onde-está e com-qual-privilégio — as três perguntas cuja resposta errada custa caro.
- ✓ **O código de saída no `RPROMPT` é a informação mais útil que o prompt padrão omite** — e o `%(?..)` só a mostra quando algo falhou.
- ✓ **Escape de cor não marcado embaralha a linha:** `%F{}` / `%f` no Zsh, `\[ \]` no Bash. E `prompt_subst`, ou o valor congela.
- ✓ **Dois plugins valem mais que qualquer framework** — e o `syntax-highlighting` sempre como **último** `source`, ou o realce some sem aviso.
- ✓ **Framework tem custo mensurável:** `time zsh -i -c exit`. Acima de 500 ms incomoda; o culpado é `compinit` sem cache ou prompt que consulta o Git a cada Enter.
- ✓ **O shebang manda no script.** Mas alias e função do `.bashrc` não migram sozinhos, e array no Zsh começa em 1.

## NO KALI

**O Zsh é padrão desde o Kali 2020.4, e a migração foi feita com cuidado:** o `~/.zshrc` do `kali-defaults` já traz completação com menu, histórico compartilhado, os dois plugins essenciais e o prompt de duas linhas — você não precisa configurar *nada* para ter uma boa experiência. Vale lê-lo inteiro uma vez: é um exemplo bem escrito de configuração defensiva. **O `kali-tweaks` é o caminho oficial para trocar o prompt**, alterando `PROMPT_ALTERNATIVE` nos arquivos gerenciados pelo pacote; edição manual ali é sobrescrita na próxima atualização — o lugar seguro para o que é seu é o fim do seu `~/.zshrc`. **O prompt de root é visualmente distinto de propósito, e isso é decisão de segurança:** sob `sudo -i` o `%#` vira `#` e a cor muda. Num sistema em que quase toda ferramenta roda com privilégio, esse sinal permanente é a diferença entre apagar *o seu* diretório de testes e apagar *o do sistema* — não remova a distinção ao personalizar. **Em trabalho autorizado, informação vale mais que estética:** com várias sessões simultâneas — máquina local, pivô, alvo — `%m` e o código de saída no `RPROMPT` são o que impede confundir uma com a outra; pôr o nome do engajamento no prompt é o raciocínio de rastreabilidade da auditoria aplicado à própria sessão, e gravar tudo com `script` ou pelo `tmux` é hábito da casa. **Fora do Debian:** Zsh é Zsh (`dnf install zsh`); muda o caminho dos plugins — `/usr/share/zsh-<plugin>/` aqui, `/usr/share/zsh/site-functions/` no Fedora —, então dotfile portável testa `[[ -r caminho ]]` antes do `source`.

# AUTOMATIZANDO TAREFAS COM CRON E SCRIPTS AGENDADOS

*O backup rodava às duas da manhã havia oito meses. No dia em que o disco morreu, o destino tinha **um** arquivo: o primeiro. O script quebrara na segunda noite — e as 240 execuções seguintes falharam caladas, porque o `>/dev/null 2>&1` que evitava o incômodo diário descartava também o erro.*

**094**
GUIAS LINUX

## SEIS ETAPAS, SEIS MANEIRAS DE FALHAR EM SILÊNCIO

| ETAPA | COMO FALHA EM SILÊNCIO | O QUE FECHA A BRECHA |
|---|---|---|
| **DISPARO** | o serviço `cron` não está ativo, ou a linha do `crontab` não tem o `\n` final | `systemctl is-active cron`; editar sempre por `crontab -e` |
| **AMBIENTE** | `PATH` mínimo, sem `.bashrc`, `HOME` diferente: `command not found` | caminho absoluto; `PATH`, `LC_ALL` e `cd` declarados no topo do script |
| **CONCORRÊNCIA** | a execução anterior ainda roda; duas cópias corrompem o destino | `flock -n 9` num arquivo de trava |
| **EXECUÇÃO** | erro no meio e as linhas seguintes rodam mesmo assim — o script ainda diz "concluído" | `set -euo pipefail` e `trap ... EXIT` de limpeza |
| **REGISTRO** | `>/dev/null 2>&1` descarta justamente a mensagem de erro | `logger -t` para o journald; nunca descartar o `stderr` |
| **NOTIFICAÇÃO** | ninguém é avisado; a falha só aparece no dia em que se precisa do resultado | `MAILTO`, ou um sinal de vida que alerta pela **ausência** |

**O caso da abertura atravessou a tabela inteira:** o erro nasceu na execução, foi descartado no registro e ninguém foi notificado. **Qualquer uma das três defesas, sozinha, teria bastado.** Um comando digitado tem três coisas que a tarefa agendada não tem — você olhando a saída, um ambiente completo e uma execução por vez. Cada ausência dessas é um modo de falha.

## O AMBIENTE MÍNIMO DO CRON — E O ESQUELETO QUE RESISTE A ELE

`/usr/local/sbin/backup-diario.sh`

```
#!/usr/bin/env bash
set -euo pipefail                # para no 1º erro
export PATH=/usr/local/bin:/usr/bin:/bin
export LC_ALL=C                  # saída previsível
cd "$(dirname "$0")"
log() { logger -t "$TAG" -p user.info -- "$*"; }
exec 9>"$TRAVA"                  # execução única
flock -n 9 || { log "já rodando"; exit 0; }
tmp=$(mktemp -d)                 # limpeza sempre
trap 'rc=$?; rm -rf "$tmp"; exit $rc' EXIT
[[ -d $ORIGEM ]] || { erro "sem origem"; exit 1; }
rsync -a --link-dest="$DEST/atual" "$ORIGEM/" "$tmp/"
mv -- "$tmp" "$DEST/$(date +%F)"  # quase atômico$ env -i HOME=$HOME
PATH=/usr/bin:/bin ./backup.sh
# imita o cron: se passa aqui, o ambiente não é a causa
```

| O QUE O CRON ENTREGA | POR QUE QUEBRA |
|---|---|
| `PATH=/usr/bin:/bin` | sem `/usr/local/bin` nem `~/.local/bin`: ferramenta de `pipx` ou compilada some |
| `SHELL=/bin/sh` | é o **dash** no Debian/Kali: `[[ ]]`, arrays e `<( )` dão erro de sintaxe. E nenhum arquivo de inicialização é lido: alias e função do `.bashrc` não existem |

**Idempotência não é opcional** — a tarefa *vai* rodar duas vezes. Teste o estado antes de agir: `grep -qxF "linha" arq || echo "linha" >> arq`. E fuja do `|| true`, que engole "já existia" e "faltou permissão" com a mesma cara.

## SER AVISADO — E ESCOLHER ENTRE CRON E TIMER

**1 · MAILTO**
O `cron` manda a saída por e-mail local — funciona **se** a máquina entrega e-mail, e quase nenhuma entrega. Frágil.

**2 · ALERTA NO ERRO**
O script avisa quando falha. Melhor — mas com um furo grave: **se nunca chegar a rodar, também nunca alerta**.

**3 · SINAL DE VIDA**
`curl -fsS "$URL" || true` na **última** linha; o observador externo alerta quando o sinal **não chega**. É o único que pega a tarefa que **deixou de existir**.

| SITUAÇÃO | ESCOLHA |
|---|---|
| **tarefa simples e portável; sobreposição resolvida com `flock`** | `cron` |
| **máquina desligada no horário, tarefa não pode ser pulada** | `anacron` ou timer com `Persistent=true` |
| **depende de outro serviço, precisa de limite de CPU/memória ou de registro estruturado** | timer do systemd: herda as diretivas da unit; a saída já vai ao journal |

**Registrar não é ser avisado.** Ninguém lê o journal de uma máquina que funciona. Padronize `logger -t tarefa` desde a primeira linha e consulte com `journalctl -t tarefa --since today`. Se preferir arquivo: `>> /var/log/tarefa.log 2>&1` — **`>>`, não `>`**, e o **`2>&1` depois**, ou o `stderr` some; e arquivo próprio pede `logrotate`, ou você trocou o silêncio pelo disco cheio. Antes de agendar: `bash -n`, `shellcheck`, `env -i` e um `*/5 * * * *` por meia hora.

**Nunca escreva `>/dev/null 2>&1` numa tarefa agendada.** Para silenciar a saída normal e manter os erros, o correto é `>/dev/null` sozinho: o `stderr` continua chegando e o `cron` avisa. Descartar o `stderr` é decidir de antemão não saber do dia em que a tarefa quebrar — foi a linha que custou oito meses de backup. **E a tarefa roda com o privilégio de quem a agendou:** script no `crontab` do root que seja gravável por usuário comum entrega root a ele na próxima execução. Script de root fica em `/usr/local/sbin`, dono `root`, permissão `755`, nunca onde outro escreve.

## RESUMO RÁPIDO

- ✓ **O silêncio é o padrão.** Disparo, ambiente, concorrência, execução, registro e notificação: cada etapa falha sem sintoma até o dia em que você precisa do resultado.
- ✓ **O ambiente do `cron` é mínimo:** `PATH=/usr/bin:/bin`, `SHELL=/bin/sh` (o *dash*) e nenhum arquivo de inicialização. Descubra o real com `* * * * * /usr/bin/env > /tmp/amb.txt`.
- ✓ `set -euo pipefail` + `flock` + `trap ... EXIT` é o esqueleto mínimo: parar no 1º erro, execução única, limpeza **inclusive** no caminho de falha.
- ✓ **Idempotência não é opcional** — teste o estado antes de agir, e nunca mascare erro com `|| true`.
- ✓ **O alerta que pega o pior caso funciona pela ausência:** só o sinal de vida detecta a tarefa que *deixou de existir* — `crontab` apagado, serviço desativado, máquina desligada.
- ✓ **Teste com `env -i` antes de agendar.** Um script que você só viu funcionar no seu terminal não foi testado para o `cron`.

## NO KALI

**O Kali costuma ser uma VM que fica desligada — e isso quebra o `cron` por inteiro:** ele só dispara se a máquina estiver ligada no minuto exato, então a tarefa das duas da manhã *nunca* roda numa VM que você liga às nove, e nenhum erro avisa. Para manutenção e atualização de bases de ferramentas, o que você quer é o `anacron` — que executa o que foi perdido assim que a máquina liga — ou um timer com `Persistent=true`. **Automatizar `apt update` é seguro; `apt full-upgrade` sem supervisão, não:** num repositório *rolling*, uma atualização pode trocar kernel, driver gráfico ou biblioteca no meio da noite e deixar a VM sem interface pela manhã. Índices sozinhos, upgrades com você olhando. **Como praticamente nenhum Kali tem MTA, o `MAILTO` não entrega nada** e a saída se perde: `logger -t` desde a primeira linha resolve sem instalar nada, e `journalctl -t tarefa --since today` vira a consulta única. **Coleta periódica em teste autorizado é rotina — e o escopo tem de estar dentro do script**, porque a tarefa *continua rodando depois que o engajamento termina*: um scan esquecido no `crontab`, batendo num alvo após o fim do contrato, deixa de ser autorizado, e uso não autorizado é crime (Lei 12.737/2012). Grave a data de término no script e remova a entrada no encerramento. **Auditar o `crontab` do root e o `/etc/cron.d/` é parada obrigatória** em revisão de sistema comprometido: é um dos lugares mais comuns de persistência de invasor. **Fora do Debian:** o Fedora/RHEL usa o `cronie`, mesma sintaxe, `anacron` embutido; lá o `/bin/sh` é o Bash em modo POSIX, então *bashismo* passa e falha de volta no Debian (teste no `dash`), e o SELinux bloqueia sem sintoma: `ausearch -m avc -ts recent` quando a tarefa "não faz nada".

# CONTROLE DE VERSÃO COM GIT NA LINHA DE COMANDO

*Uma frase que situa o problema — como o subtítulo do FTTH.*

095
GUIAS LINUX

1 **ETAPA** O que acontece aqui. → 2 **ETAPA** O que acontece aqui. → 3 **ETAPA** O que acontece aqui. → 4 **ETAPA** O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

kali@kali: ~

```
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# EDITORES NO TERMINAL: VIM E NANO

*Uma frase que situa o problema — como o subtítulo do FTTH.*

096
GUIAS LINUX

1 **ETAPA** O que acontece aqui. → 2 **ETAPA** O que acontece aqui. → 3 **ETAPA** O que acontece aqui. → 4 **ETAPA** O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

```
kali@kali: ~
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# DOTFILES E AMBIENTES REPRODUZÍVEIS

*Uma frase que situa o problema — como o subtítulo do FTTH.*

097
GUIAS LINUX

1 **ETAPA** O que acontece aqui. → 2 **ETAPA** O que acontece aqui. → 3 **ETAPA** O que acontece aqui. → 4 **ETAPA** O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

**ITEM** Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

```
kali@kali: ~
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# O KERNEL LINUX: MÓDULOS, /PROC E /SYS

*Uma frase que situa o problema — como o subtítulo do FTTH.*

098
GUIAS LINUX

**1** **ETAPA**
O que acontece aqui.

→

**2** **ETAPA**
O que acontece aqui.

→

**3** **ETAPA**
O que acontece aqui.

→

**4** **ETAPA**
O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

```
kali@kali: ~
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# CONTÊINERES POR DENTRO: NAMESPACES, CGROUPS E DOCKER

*Uma frase que situa o problema — como o subtítulo do FTTH.*

099
GUIAS LINUX

**1** **ETAPA**
O que acontece aqui.

→

**2** **ETAPA**
O que acontece aqui.

→

**3** **ETAPA**
O que acontece aqui.

→

**4** **ETAPA**
O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

```
kali@kali: ~
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# VIRTUALIZAÇÃO: KVM, QEMU E O LABORATÓRIO DE ESTUDOS

*Uma frase que situa o problema — como o subtítulo do FTTH.*

100
GUIAS LINUX

**1** **ETAPA**
O que acontece aqui.

→

**2** **ETAPA**
O que acontece aqui.

→

**3** **ETAPA**
O que acontece aqui.

→

**4** **ETAPA**
O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

kali@kali: ~

```
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# COMPILAÇÃO, TOOLCHAINS E O ECOSSISTEMA DE DEV

*Uma frase que situa o problema — como o subtítulo do FTTH.*

101
GUIAS LINUX

**1** **ETAPA**
O que acontece aqui.

→

**2** **ETAPA**
O que acontece aqui.

→

**3** **ETAPA**
O que acontece aqui.

→

**4** **ETAPA**
O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

```
kali@kali: ~

$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# OBSERVABILIDADE E DESEMPENHO: STRACE, LTRACE E PERF



*Uma frase que situa o problema — como o subtítulo do FTTH.*

**1** **ETAPA**
O que acontece aqui.

→

**2** **ETAPA**
O que acontece aqui.

→

**3** **ETAPA**
O que acontece aqui.

→

**4** **ETAPA**
O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

kali@kali: ~

```
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

## RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

## ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.

# PARA ONDE IR DEPOIS: CERTIFICAÇÕES, COMUNIDADE E PRÁTICA

103
GUIAS LINUX

*Uma frase que situa o problema — como o subtítulo do FTTH.*

**1** **ETAPA**
O que acontece aqui.

→

**2** **ETAPA**
O que acontece aqui.

→

**3** **ETAPA**
O que acontece aqui.

→

**4** **ETAPA**
O que acontece aqui.

## CONCEITOS-CHAVE

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

**ITEM**
Descrição curta e direta.

## CONCEITO A × CONCEITO B

### CONCEITO A

- ✓ Ponto.
- ✓ Ponto.

### CONCEITO B

- ✓ Ponto.
- ✓ Ponto.

## NA PRÁTICA

kali@kali: ~

```
$ comando --opcao alvo
saída esperada do comando
# comentário explicando o que observar
```

| ASPECTO | OPÇÃO A | OPÇÃO B |
|---|---|---|
| Finalidade | ... | ... |
| Exemplo | `cmd a` | `cmd b` |

### RESUMO RÁPIDO

- ✓ Ponto essencial 1.
- ✓ Ponto essencial 2.
- ✓ Ponto essencial 3.

### ◆ DICA DE OURO

O atalho, a pegadinha ou o **No Kali** que só se aprende usando.